GRADUATE TEXTS IN MINECRAFT

An Introduction to Technical Minecraft

Locale: English

Generated: 2026-05-14

TABLE OF CONTENTS

总序

Block Update

更新的概念与不同类型的更新

特殊的更新行为

连续的方块更新及其分析方法

Components & Features

Chunk Savestate 原理分析

活塞

铁轨

Loading Tickets

加载票系统中的类型

加载票系统的运作细节

中英术语表

辅助mod

Masa Mod

Masa Gadget

投影模组教程

Micro Timing

初窥刻内时序

刻与刻间时序

方块事件

方块实体

计划刻与计划刻元件

molforte

实战篇：入门机器的快速上手

沉淀篇：基础模块的举例讲解

预备篇：环境配置和基础知识

0a-附录

Tree Farm

为什么你的树场卡——无粉布线简介

为什么你的树场慢——高速树场入门

从零开始设计一台最简单的树场

关于4gt树场的一切

前置知识与树场的基本结构

双核、三核、四核树场

地狱菌场

大云杉树场

完美时序，与樱花全树种

尝试设计一台全树种树场

树苗生长机制：树形与催熟

深色橡木树场

生炸树场

Appendix

术语表

栈的概念

CODE_OF_CONDUCT

CODE_OF_CONDUCT_CN

REVIEWERS

REVIEWERS_CN

ROADMAP

1 TITLE: 总序
TITLE-EN: PREFACE
SLUG: PREFACE

Graduate Texts in Minecraft (GTMC) 是一个社区驱动的 Minecraft 技术和红石教程项目。GTMC 可以作为新手入门的指南，也可以作为经验丰富的玩家深入理解 Minecraft 内部机制的文档。

编辑组将致力于编写提供清晰、准确且易于理解的内容。同时，社区贡献者也可以通过提交 Pull Request 的方式参与到本项目的编写和完善中来。

2 解释说明

文档中的每一篇都可以分为**基础部分**和**进阶部分**，分别面向萌新和熟练者。

部分**基础部分**的内容可能由于较为罕见 / 不常用，会在章节标题以 * 标注。

为便于理解，**基础解释**部分可能会存在一些概括性的表述，这些表述并不绝对严谨，但不影响通常情况下的分析。这些表述会以上标[1]注明，并在**进阶**部分或脚注中进一步解释。

文档默认读者对 Minecraft 以及各种红石元件有**比较基础**的认识。若在阅读过程中存在问题，可自行查阅 Wiki 解决。同样，一些具有特殊性且并不典型的情况不在本篇的说明范围内，这些情况通常在 Wiki 已有归档罗列。当某一概念存在这些情况时，将会以上标[2]注明。

文档中采用的代码使用`1.20.1-yarn`反编译，若有特殊情况会以上标[3]注明。

可以来 QQ 裙吹水：1031431170

3 著作权声明与授权信息

Copyright © 2024-2026 GTMC Wiki.

本文档原创内容采用CC BY-NC-SA 4.0授权。

Minecraft ® 是 Mojang AB 的注册商标。
Minecraft © Mojang AB / Microsoft Corporation. All Rights Reserved.

本文档引用的游戏截图、纹理、源码等内容受著作权法保护，在符合Minecraft EULA和Minecraft 使用准则的前提下进行有限引用。

其中，本文档引用的源码来源于对 Minecraft 的反编译研究，使用了Yarn 映射表进行可读化反编译，该映射表采用CC0 1.0授权。

本文档不构成对 Minecraft 游戏代码的再分发或任何形式的商业使用，也不提供可直接运行的完整游戏代码或可执行文件。所有对游戏内部机制的描述均限于学习和说明目的。

联系邮箱: tanh_heng@outlook.com, fannbryan@gmail.com, yuhan2680@qq.com，suyang233@hotmail.com，Molforte@outlook.com

GTMC Team:

- 写手 / ABxl_lly, [@BFladderbean], [@Molforte], [@Ryan100c], [@tanh_Heng], [@Twisuki], [@xhbsh], [@yuhan2680]
- 开发 / [@BFladderbean], [@4rcadia]
- 协作 / Petris, [@void], XJH_Jorhai
- 鸣谢 / [@Fallen_Breath], 五羊飞 kaniol

本文档并非 Minecraft 官方文档，与 Mojang 和微软亦无从属关系。

This is not an official documentation. We are not affiliated with Mojang or Microsoft by any means.

FOOTNOTES

1. 像这样 ↩
2. 像像这这样样 ↩
3. 像像像这这这样样样样 ↩

CHAPTER 1

BLOCK UPDATE

---

slug: update-concepts
title: 更新的概念与不同类型的更新
title-en: null
author: SDU-Little-River
co-authors:

- Arcadi4
- BFladderbeanawa
- MorningMC
- tanhHeng
- hotpad100c
- Petris
  date: '2024-12-08T18:28:55+08:00'
  lastmod: '2026-04-06T18:56:51+08:00'
  index: 1
  is-advanced: false
  banner:
  src: ./img/Updates.jpg
  alt: 缺少更新的非法状态方块们

本篇是更新理论的入门。

基础篇

- 认识更新
- 不同类型的更新
- 方块自检行为

进阶篇（源码分析）

- 简要的 NC 更新与 PP 更新的调用与响应
- 方块自检的调用

1 广义更新

在 Minecraft 中，方块与方块通过互相 “通知” 来建立相互的联系。

有一天，你突发奇想 —— 在地上放置了一个音符盒。然后，你又紧邻着音符盒放置了一个红石块，音符盒发出了悦耳的声音。那么，音符盒是怎么知道自己应该恰好在这个时候发出声音的吗？难道音符盒会一直给玩家发消息：我要发出声音吗？我要发出声音吗？我要发出声音吗…… 这显然是不合理的。实际上，音符盒能够发出声音，是因为红石块在被放置时，“通知” 了音符盒：嘿，这里有点变化，你注意一下。

这种**通知**行为就称为广义上的**更新**。

需要注意的是，红石块并不会通知 “我是一个红石块” 或者 “这里从空气变成了红石块”，而是非常简单地通知：这里有点变化。也就是说，在 Minecraft 中，更新行为并不包含行为的具体信息。

2 更新的类型

在 Minecraft 中，更新有不止一种类型。

让我们继续上文中的例子。这天，你又灵光一现 —— 在地上先放置了一个红石块，然后间隔一个方块放置了一个栅栏门，在红石块和栅栏门的中间放置了音符盒。你突然发现，音符盒怎么没有发出声音呢？于是，你右键栅栏门，把栅栏门打开了，但音符盒还是没有发出声音。按理来说，上文中的 “放置红石块” 和 “栅栏门打开” 都是某种变化，它们都应该 “通知” 音符盒，为什么音符盒没有如愿地发出声音呢？带着疑惑，你直接把栅栏门破坏掉了，此时音符盒又发出声音了。

—— 这是为什么呢？

你于是猜想：“放置红石块” 和 “栅栏门打开” 的 “通知” 也许有什么差别。

实际上，这两个通知的确是不同的。现如今我们分别称之为**NC 更新**和**PP 更新**。

此时你又思考：为什么音符盒被放置的时候，不会立即发出声音呢？它难道在 “出生” 的时候不会看一看 “自己要不要说两句话” 吗？—— 没错，音符盒比较懒，它确实不会 “检查自己要不要发出声音”。

这种在被放置等情况下检查自身的行为，被称为**自检**。**自检**可以被视作一种**特殊的更新**。而音符盒不会进行自检。

你在先前的学习中又了解到：比较器是一种能够检测容器中物品数量的红石元件，根据容器中物品数量的多少输出 0-15 的不同强度高低的红石信号。你想到：当容器中物品数量发生变化时，容器并没有必要发出**NC 更新**或**PP 更新**，毕竟只是内部物品多少改变了，并不会对周围方块产生什么太大影响。但你知道比较器却能够及时地改变自身输出的红石信号强度，说明比较器还是能够收到这种变化的 “通知”。这是因为容器在容器中物品数量发生变化时，也会发出一种特殊的通知。

这样的通知被称为**比较器更新**。

3 NC 更新

3.1 NC 更新的概念与行为

方块在被放置、破坏、或发生能够显著影响周围方块的变化时，会发出**NC 更新**。例如：

- 方块的放置与破坏
- 红石粉能量等级发生变化
- 活塞的推拉 *
- ...

> *官方提供的反混淆中 NC 更新为 `neighborChanged`，与 mcp 反混淆相同。yarn 反混淆中则称为 `updateNeighbors`*

但一些变化不会发出**NC 更新**，最为常见的有：

- 可连接方块的连接状态改变，例如玻璃板与其他方块连接、各种墙与其他方块连接
- 红石粉的连接状态发生变化，例如一个与它南北侧红石粉相连的红石粉又与东侧的一个红石粉相连接
- 活板门、栅栏门的开关
- 发射器、投掷器的激活状态改变（**！重要**）
- 漏斗的激活状态改变
- ...

> *活塞的具体行为不在本篇讨论范围内。*

一个发出 NC 更新的 “方块” 称为**更新核**[1]，大部分方块发出更新时，更新核是它本身。

更新核发出 NC 更新时，按照**西东下上北南**的顺序，依次对更新核西侧紧邻、东侧紧邻…… 的方块发出 NC 更新。这样的更新就是通常意义上的更新。

部分方块在发出 NC 更新时不只有一个更新核，它们具有特殊的更新核与范围：

- 红石粉的能量等级变化为二阶毗邻更新[2]。
- 带有指向的红石元件，例如红石中继器、比较器、侦测器，它们在激活状态发生变化时先单独更新它们输出端指向的方块，再以输出端指向的方块为更新核发出 NC 更新，并且在 NC 更新时不会更新它们自身方块。
- 平放的动力铁轨切换激活状态时，先以自身为更新核产生 NC 更新，再以下方方块为更新核产生 NC 更新。
- 斜放的动力铁轨切换激活状态时，更新顺序为上、中、下、中、下、上（以自身为更新核产生 NC 更新简写为**中**、以上方方块为更新核简写为**上**、以下方方块为更新核简写为**下**，详细解读参见 [斜向铁轨的更新行为](./特殊的更新行为.md#3.2-[!ADVANCED] 斜侧铁轨的更新行为)）。

以上是常见且较为典型的例子。更多的特殊情况可以参见Wiki - 方块更新

3.2 NC 更新的性质、QC 激活、BUD 装置

NC 更新最典型的性质是能够使**BUD 装置**响应。

一个方块当前的状态与它**本应该的状态**不同时，这个方块就可以称作是一个**BUD 装置**，即**方块更新检测器（Block Update Detector）**。这种方块状态也可以称作是**BUD 态**。

最典型的 BUD 装置是一个被充能（受到红石信号）但未被激活（未伸出）的活塞。活塞具有特殊的充能范围，当活塞本身和活塞上方的一个方块（可以为空气）被充能时，活塞就可以被认为被充能。通过充能活塞上方的空气充能活塞，这种充能方式就称为**QC 充能**。这样被充能的活塞在受到 NC 更新后伸出，就称作**QC 激活**。

QC (quasi connectivity) 称为**半连接性**，活塞、粘性活塞、投掷器、发射器都具有这种性质。

想象一个活塞的斜上方放置了一个红石块，此时这个活塞被 QC 充能。而红石块的放置只会更新它毗邻的六个方块，无法更新到活塞，所以没有任何方块通知活塞 “这里有一个红石信号”。此时这个活塞本应伸出，但却因为没有受到 NC 更新而没有伸出。也就是*当前的状态与它本应该的状态不同*，即此时这个活塞处于**BUD 态**，该活塞就构成了一个**BUD 装置**。

此时，如果在活塞边上放置一个方块，方块放置就会发出 NC 更新。活塞受到 NC 更新，解除 BUD 态。

一个最简单常用的可以自行复位的 BUD 装置如下：

图中的红石块**QC 充能**下方粘性活塞，当粘性活塞受到 NC 更新后伸出，红石块不再充能，而后粘性活塞收回，红石块再次 QC 充能粘性活塞。[3]

4 PP 更新

4.1 PP 更新的概念与行为

几乎所有的变化都会发出**PP 更新**。除了：

- 使用命令放置方块
- 使用调试棒改变方块状态

等极其特殊的情况；但还是有一个特例：

- 粘性活塞推动正在激活的侦测器，到位时侦测器不发出 PP 更新。

> *官方提供的反混淆中 PP 更新为 `updateShape`，即更新形状（例如，各种墙的形状、活板门的开关、红石元件的激活状体改变，都可以称作这里的“形状”），该反混淆名更有益于理解 PP 更新。PP 更新的命名来源于 mcp 反混淆中是 `postPlacement` 方法。yarn 反混淆中则称为 `getStateForNeighborUpdate`*

和 NC 更新不同，PP 更新的顺序为**西东北南下上**。

在上文中提到的 * 不会发出**NC 更新 ***的行为，都是较为典型的发出**PP 更新**但不发出**NC 更新**的行为。同样，上文中的**BUD 装置**无法用来检测**PP 更新**。

例如这里的栅栏门状态发生变化，发出 pp 更新，但并不会被活塞 BUD 装置检测到

需要注意的是，除了放置与破坏方块外，大部分红石元件的 NC 更新范围和 PP 更新范围是不一样的。例如中继器的 NC 更新范围是输出端及输出端的毗邻（除中继器自身），而其 PP 更新范围是自身的毗邻。

PP 更新可以理解为**自身发生变化**的更新，而 NC 更新可以理解为**通知可能需要改变状态的方块**的更新。

同样以中继器为例，中继器的亮灭属于**自身发生了变化**，通过 PP 更新通知自身毗邻的方块。而中继器亮起会充能它输出端的方块，如果输出端的方块能够传递红石信号，这个方块还会再向它的毗邻传递红石信号；所以说中继器的亮起有可能会影响它输出端的方块以及输出端毗邻的方块，因此中继器通过 NC 更新**通知可能受到影响的方块检查是否要改变状态**。

又例如，当投掷器被激活，投掷物品时，投掷物品这一行为并不会对周围方块造成影响[4]，因此投掷器激活并不发出 NC 更新。但是投掷器的确改变了激活状态，这种状态改变使得投掷器发出了 PP 更新。

这样，PP 更新与 NC 更新的差异以及它们更新范围的不同就易于理解了。

4.2 PP 更新的性质

既然上文中的 BUD 装置无法检测 PP 更新，那么什么装置能够检测（响应）PP 更新呢?

答案是 —— 侦测器。侦测器长得像脸的一面为检测 PP 更新的一面，有红色小点的一面为输出红石信号的一面。

侦测器响应、且仅响应 PP 更新。[5]但由于大多数情况 NC 更新总是伴随 PP 更新的，所以这里的 “仅” 可能体现的并不明显。不过我们还是可以通过一些方法来体现：当侦测器直接面向中继器时，中继器亮灭会激活侦测器；但如果侦测器面向中继器输出端指向的方块（空气），中继器的亮灭则不会激活侦测器。这里也很好理解：当中继器指向一个空气时，空气不会有任何变化，因此侦测器自然也不会响应了。

通过 BUD 装置与侦测器的配合使用，我们可以很方便的在不翻阅源码的情况下测出各个红石元件的 NC 更新范围和 PP 更新范围。

5 比较器更新

5.1 比较器更新的概念与行为

顾名思义，比较器更新是专用于更新比较器的更新。

以下行为会发出比较器更新：

- 一般的容器，如箱子、木桶、漏斗等中的物品数量发生变化。信号强度计算方式见下。
- 更广义上的容器，有 “容量” 概念的方块例如堆肥桶（信号强度与堆肥等级相等，范围为 0~9）、炼药锅（信号强度与装液体的等级相等，范围为 0~3）。
- 物品展示框，物品展示框输出的比较器信号强度取决于展示物品的旋转角度，仅输出强度为 1~8 的信号。不放置物品时信号强度为 0。
- 讲台，信号强度计算方式见下。
- 探测铁轨上方有掉落物。
- 探测铁轨上有带有容器的矿车，如运输矿车（箱子矿车）、漏斗矿车时，矿车内物品数量发生变化时会在 20gt 后发出比较器更新。信号强度计算方式与一般容器相同。
- 雕纹书架（1.20+），信号强度与最后一次放置或取出的书的位置相等，第一排为 1~3，第二排为 2~6。

比较器既可以直接检测容器容量，也可以间隔一个可传递红石信号的方块检测容器容量。因此比较器更新的范围为容器水平方向的毗邻范围内的比较器，或二阶毗邻范围内间隔一个可传递红石信号的方块的比较器。比较器更新不会对比较器以外的方块造成更新。

5.2 比较器信号强度计算

对一般的容器：

- 输出信号强度 = 对 `每个物品槽位填充比例的平均值*14` 向下取整，如果容器不是空的，那么输出信号强度再 `+1`。即：当容器为空，输出 0。当容器不为空，输出容器的平均填充比例到 1~15 的映射并向下取整。

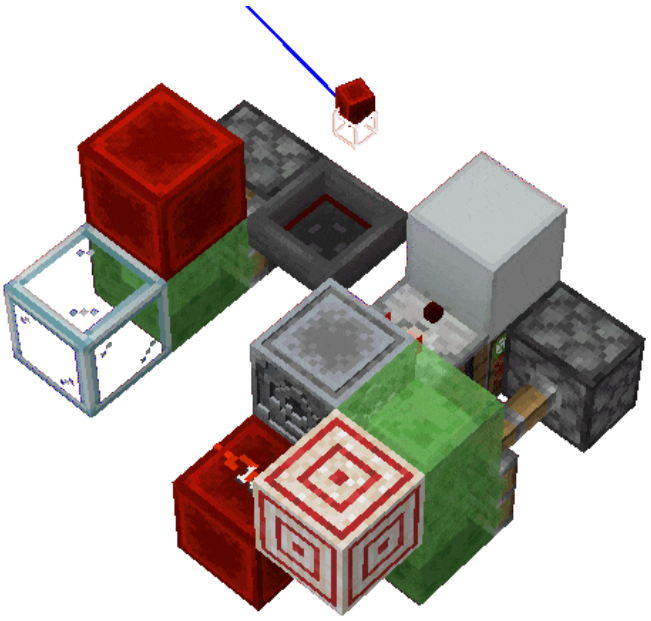
对讲台：

- 当讲台上不放置书时，输出信号强度为 0。
- 当讲台上放置书且书只有一页时，输出信号强度为 15。
- 当讲台上放置书且书不只有一页时，输出信号强度为 `当前页码/总页码*14` 向下取整再 `+1`。
- 即：当讲台为空，输出 0。当讲台不为空，输出当前页码占总页码的比例到 1~15 的映射并向下取整。

5.3 比较器更新检测器（CUD）

一个比较器当前的状态与它**本应该的状态**不同时，这个比较器就构成了一个比较器更新检测器（CUD，Comparator Update Detector）。一个最简单的例子是用比较器间隔一个方块检测一个有填充的堆肥桶，然后用活塞将堆肥桶推走。此时比较器依然有输出信号。

由于其他可复位的 CUD 结构与涉及原理较为复杂，在此处仅作展示，不作展开。



图为基于红石粉粉转向的 CUD。

可以看到 BUD 并未启动，而 CUD 检测到了漏斗发出的比较器更新。

6 方块自检

红石元件在被放置时会进行**自检**，即检查自身是否应当改变状态。例如放置一个红石块，然后放置一个中继器，并且这个中继器的输入端紧邻着红石块。此时中继器没有受到更新，但还是亮起了，这就是因为中继器在被放置时进行了**自检**，发现自己应该亮起，于是亮起。

活塞自身到位也可以视作一种 “被放置”，因此活塞自身在伸出后到位和拉回后到位也都会进行一次**自检**。

7 NC 更新与 PP 更新的调用与响应

7.1 `setBlockState` 函数

`setBlockState` 通过一个 `bitflags` 即 `FLAG` 控制更新行为，在 `World.java` 的 `World` 类中：

```
public boolean setBlockState(BlockPos pos, BlockState state, int flags, int maxUpdateDepth) {        ...        BlockState
blockState = worldChunk.setBlockState(pos, state, (flags & Block.MOVED) != 0);        if (blockState != null) {
BlockState blockState2 = this.getBlockState(pos);            if (blockState2 == state) {                ...
// 由 FLAG 控制的 NC 更新的调用                if ((flags & Block.NOTIFY_NEIGHBORS) != 0) {
this.updateNeighbors(pos, blockState.getBlock());                    if (!this.isClient && state.hasComparatorOutput()) {
this.updateComparators(pos, block);                        }                }                // 由 FLAG 控制的 PP 更新的调用
if ((flags & Block.FORCE_STATE) == 0 && maxUpdateDepth > 0) {                    int i = flags & ~(Block.NOTIFY_NEIGHBORS |
Block.SKIP_DROPS);                    blockState.prepare(this, pos, i, maxUpdateDepth - 1);
state.updateNeighbors(this, pos, i, maxUpdateDepth - 1);                    state.prepare(this, pos, i, maxUpdateDepth - 1);
}                ...            }            return true;        }        return false;    }
```

7.2 NC 更新的调用与响应

在源码中，NC 更新主要通过两种方式调用：

- `updateNeighbor` 及其衍生
- `flag` 中 `bit0 = 1` 的 `setBlockState`

`updateNeighbor` 及其衍生，即 `updateNeighbor` `updateNeighborsExcept` `updateNeighborsAlways`。

其中 `updateNeighbor` 是最根本的方法，传入 `pos, sourceBlock, sourcePos` 三个参数，分别为要更新的方块坐标 `pos`，发出更新的方块类型 `sourceBlock` 和发出更新的方块坐标 `sourcePos`。

`updateNeighborsAlways` 和 `updateNeighborsExcept` 通过调用 `NeighborUpdater.updateNeighbors` 进行更新，在 `NeighborUpdater.java` 的 `NeighborUpdater` 实例中：

```
public static final Direction[] UPDATE_ORDER = new Direction[]{Direction.WEST, Direction.EAST, Direction.DOWN, Direction.UP,
Direction.NORTH, Direction.SOUTH};default public void updateNeighbors(BlockPos pos, Block sourceBlock, @Nullable Direction
except) {    for (Direction direction : UPDATE_ORDER) {        if (direction == except) continue;
this.updateNeighbor(pos.offset(direction), sourceBlock, pos);    }}
```

因此 NC 更新顺序为西东下上北南。

其中在 `Block.java` 中定义：

```
// 向周围方块广播方块更新事件public static final int NOTIFY_NEIGHBORS = 1;// 提醒需要对此方块改变做出反应的监听器和客户端public static
final int NOTIFY_LISTENERS = 2;// 默认的方块变更时行为：与一起启用上述 NOTIFY_NEIGHBORS 和 NOTIFY_LISTENERS 同效。public static
final int NOTIFY_ALL = 3;// 跳过更新，强制设置方块状态。public static final int FORCE_STATE = 16;
```

对于以上的调用方法，只有调用 `updateNeighborsAlways` 和 `updateNeighborsExcept` 的 `sourcePOS` 和 `setBlockState` 的 `pos` 可以称作 “更新核”，即以一个方块为中心对毗邻六个方块（或除去某个方向的方块）进行更新的 “核”。而 `updateNeighbor` 是简单直接的 “更新”。

仅 PP 更新时，`FLAG` 通常为 `NOTIFY_LISTENERS`；同时发出 PP 更新和 NC 更新时，`FLAG` 通常为 `NOTIFY_ALL`。部分覆写了 `onBlockAdded` 方法的方块具有不同的调用方式，见下文

在各个方块中，NC 更新由方块的 `neighborUpdate` 函数响应。

7.3 PP 更新的调用

在源码中，PP 更新通过 `setBlockState` 调用。当 `FLAG` 的 `bit4 = 0` 时会产生 PP 更新。

`state.updateNeighbors` 定义在 `AbstractBlock.java` 的 `AbstractBlockState` 类中：

```
public final void updateNeighbors(WorldAccess world, BlockPos pos, int flags, int maxUpdateDepth) {    BlockPos.Mutable
mutable = new BlockPos.Mutable();    for (Direction direction : DIRECTIONS) {        mutable.set((Vec3i)pos, direction);
world.replaceWithStateForNeighborUpdate(direction.getOpposite(), this.asBlockState(), mutable, pos, flags, maxUpdateDepth);
}}
```

其中在 `AbstractBlock` 中有：

```
protected static final Direction[] DIRECTIONS = new Direction[]{Direction.WEST, Direction.EAST, Direction.NORTH,
Direction.SOUTH, Direction.DOWN, Direction.UP};
```

因此 PP 更新为西东北南下上。

其中`world.replaceWithStateForNeighborUpdate`：

```
default public void replaceWithStateForNeighborUpdate(Direction direction, BlockState neighborState, BlockPos pos, BlockPos
neighborPos, int flags, int maxUpdateDepth) {    NeighborUpdater.replaceWithStateForNeighborUpdate(this, direction,
neighborState, pos, neighborPos, flags, maxUpdateDepth - 1);}
```

调用了`NeighborUpdater`中的`replaceWithStateForNeighborUpdate`方法：

```
public static void replaceWithStateForNeighborUpdate(WorldAccess world, Direction direction, BlockState neighborState,
BlockPos pos, BlockPos neighborPos, int flags, int maxUpdateDepth) {    BlockState blockState = world.getBlockState(pos);
BlockState blockState2 = blockState.getStateForNeighborUpdate(direction, neighborState, world, pos, neighborPos);
Block.replace(blockState, blockState2, world, pos, flags, maxUpdateDepth);}
```

最后调用了各个方块`blockState`的`getStateForNeighborUpdate`方法。也就是方块响应 PP 更新的方法。

7.4 红石元件的 NC 更新与 PP 更新的调用

而对于大多数的红石元件，它们在调用`setBlockState`时通常传入`FLAG=2`，NC 更新实际是由`setBlockState`中的`worldChunk.setBlockState`方法调用覆写`@Override`后的`onBlockAdded`方法发出 NC 更新：

在`worldChunk.setBlockState`中：

```
public BlockState setBlockState(BlockPos pos, BlockState state, boolean moved) {    ...    if (!this.world.isClient) {
state.onBlockAdded(this.world, pos, blockState, moved);    }    ...    return blockState;}
```

而 PP 更新受`FLAG`控制正常由`setBlockState`调用。

以红石中继器为例，在红石中继器继承的`AbstractRedstoneGateBlock.java`的`AbstractRedstoneGateBlock`中：

```
public void scheduledTick(BlockState state, ServerWorld world, BlockPos pos, Random random) {    if (this.isLocked(world,
pos, state)) {        return;    }    boolean bl = state.get(POWERED);    boolean bl2 = this.hasPower(world, pos, state);
if (bl && !bl2) {        world.setBlockState(pos, (BlockState)state.with(POWERED, false), Block.NOTIFY_LISTENERS);    } else
if (!bl) {        world.setBlockState(pos, (BlockState)state.with(POWERED, true), Block.NOTIFY_LISTENERS);        if (!bl2)
{            world.scheduleBlockTick(pos, this, this.getUpdateDelayInternal(state), TickPriority.VERY_HIGH);        }
}}@Overridepublic void onBlockAdded(BlockState state, World world, BlockPos pos, BlockState oldState, boolean notify) {
this.updateTarget(world, pos, state);}    protected void updateTarget(World world, BlockPos pos, BlockState state) {
Direction direction = state.get(FACING);    BlockPos blockPos = pos.offset(direction.getOpposite());
world.updateNeighbor(blockPos, this, pos);    world.updateNeighborsExcept(blockPos, this, direction);}
```

`scheduledTick`为中继器执行计划刻时调用的方法。在中继器执行计划刻时，它会调用`setBlockState`函数，而后由`setBlockState`调用`worldChunk.setBlockState`，在`worldChunk.setBlockState`调用了`@Override`的`onBlockAdded`，对中继器输出端指向的方块及指向方块的毗邻（除中继器自身）产生了 NC 更新。接着由`setBlockState`调用`state.updateNeighbors`产生了 PP 更新。

7.5 NC 更新与 PP 更新的顺序

综上，对于大部分通过`setBlockState`发出更新的方块，都满足先 NC 更新、后 PP 更新的顺序。这一点极其重要，它影响了在后续更新中 NC 更新与 PP 更新的顺序问题。本篇不对 “连续的方块更新” 作展开。

7.6 侦测器的 NC 与 PP 更新

侦测器是一个特例。它并没有调用`onBlockAdded`方法中的 NC 更新，而是直接在`scheduledTick`中先调用`setBlockState`发出 PP 更新，再调用`updateNeighbors`发出 NC 更新。因此，侦测器在亮起和熄灭是遵循先 PP 后 NC 的顺序。

在侦测器的`onBlockAdded`中：

```
@Overridepublic void onBlockAdded(BlockState state, World world, BlockPos pos, BlockState oldState, boolean notify) {    if
(state.isOf(oldState.getBlock())) {        return;    }    if (!world.isClient() && state.get(POWERED).booleanValue() &&
!world.getBlockTickScheduler().isQueued(pos, this)) {        BlockState blockState = (BlockState)state.with(POWERED, false);
world.setBlockState(pos, blockState, Block.NOTIFY_LISTENERS | Block.FORCE_STATE);        this.updateNeighbors(world, pos,
blockState);    }}
```

> 其中第一个 `if` 中的 `state.isOf(oldState.getBlock())` 用于判断当前位置的方块在被改变状态之前是否还是当前的方块。例如，如果活塞推动了一个侦测器，那么侦测器到位的位置在改变前就是 `air`，改变后就是 `observer`，此时该函数返回 `true`。而一个普通的侦测器在调用这一函数时总是返回 `false`。
>
> 所以，侦测器的 `onBlockAdded` 方法在通常情况下都不会执行后续的内容，也就不会由 `onBlockAdded` 发出 NC 更新，而是如上文中所说，“直接在 `scheduledTick` 中先调用 `setBlockState` 发出 PP 更新，再调用 `updateNeighbors` 发出 NC 更新。”

8 方块自检的调用

方块在被放置时会被调用`onPlaced`函数，红石元件在该函数中完成自检。例如红石中继器：

```
@Overridepublic void onPlaced(World world, BlockPos pos, BlockState state, LivingEntity placer, ItemStack itemStack) {    if (this.hasPower(world, pos, state)) {        world.scheduleBlockTick(pos, this, 1);    }}
```

活塞自检的源码不在本篇讨论范围内。

9 更新行为的控制

ServerWorld.setBlockState 方法中，一个整数参数被传入：flag
这个 flag 实际上被视为一个 9 位二进制码，这 9 位中，每一位都控制了不同的行为

10 FLAG 的 9 位标志定义

- **Bit 0**: 发出邻居更新。
- **Bit 1**: 通知监听器（服务器和客户端）。
- **Bit 2**: 抑制客户端渲染。
- **Bit 3**: 强制客户端同步重绘。
- **Bit 4**: 强制状态存储。
- **Bit 5**: 阻止物品掉落。
- **Bit 6**: 标记活塞移动。
- **Bit 7**: 光照更新。
- **Bit 8**: 出发行为副作用。

11 常量解释

Block.java 中定义了多个更新行为，他们分别是

常量	值	二进制（9 位）	行为	用途
`NOTIFY_NEIGHBORS`	1	`000000001`	触发周围方块更新	俗称 NC 更新
`NOTIFY_LISTENERS`	2	`000000010`	通知服务器和客户端监听器	同步状态
`NO_REDRAW`	4	`000000100`	阻止客户端重绘（与 `NOTIFY_LISTENERS` 配合）	不可见更新
`REDRAW_ON_MAIN_THREAD`	8	`000001000`	强制客户端同步重绘	方块外观变化
`FORCE_STATE`	16	`000010000`	绕过虚拟状态，直接存储指定状态	强制更换方块
`SKIP_DROPS`	32	`000100000`	阻止容器方块破坏时掉落物品	命令替换等
`MOVED`	64	`001000000`	标记方块由活塞移动	活塞推动 / 拉动逻辑 触发
`SKIP_REDSTONE_WIRE_STATE_REPLACEMENT`	128	`010000000`	跳过红石线状态替换	-
`SKIP_BLOCK_ENTITY_REPLACED_CALLBACK`	256	`100000000`	跳过方块实体替换回调	-
`SKIP_BLOCK_ADDED_CALLBACK`	512	`1000000000`	跳过自检	不触发 `onBlockAdded`
`SKIP_REDRAW_AND_BLOCK_ENTITY_REPLACED_CALLBACK`	260	`100000100`	组合 `NO_REDRAW` 和 `SKIP_BLOCK_ENTITY_REPLACED_CALLBACK`	最小化更新，跳过渲染和实体回调
`NOTIFY_ALL`	3	`000000011`	组合 `NOTIFY_NEIGHBORS` 和 `NOTIFY_LISTENERS`	默认完整服务器端更新
`NOTIFY_ALL_AND_REDRAW`	11	`000001011`	组合 `NOTIFY_ALL` 和 `REDRAW_ON_MAIN_THREAD`	服务器更新并强制客户端重绘
`FORCE_STATE_AND_SKIP_CALLBACKS_AND_DROPS`	816	`110011000`	组合 `FORCE_STATE`，`SKIP_DROPS`，`MOVED`，`SKIP_BLOCK_ADDED_CALLBACK`	强行变更，跳过自检和掉落

我不知道啊，阿卡迪亚要我测的

FOOTNOTES

1. 有关**更新核**的理解在基础部分不作过多解释，且并非所有 NC 更新都存在更新核。参见进阶部分。 ↩
2. 红石粉具有更加复杂的更新行为，本篇中不作展开。~~可能后续会追加红石粉专题 DLC~~ ↩
3. 此处对 1.3.2 中的 BUD 装置涉及活塞原理部分作简单展开，读者仅作了解即可。粘性活塞在伸出时没有受到任何 NC 更新，因此不会中途丢下方块。粘性活塞伸出到位后触发自检，发现没有被充能于是执行收回。收回时由于粘性活塞的方块到位顺序，粘性活塞先到位，此时触发自检但红石块还未到位（处于方块实体状态），红石块后到位。因此粘性活塞不会由于自检而再次伸出。 ↩
4. 不考虑投掷的物品被漏斗吸取。漏斗吸取属于漏斗自身的行为，不受更新控制。 ↩
5. 消歧义：此处仅从更新类型的角度考虑，侦测器仅在受到 PP 更新时判断自身是否应当发生变化并执行特定的行为。若认为 “响应” 指广义的 “侦测器发生变化”，那么除侦测器的计划刻行为外，当侦测器被活塞推到位时，它会调用自身的 `onBlockAdded` 方法，仅当到位位置无侦测器计划刻，并且侦测器原先的状态为亮起时，侦测器会继续在其中调用 `setBlockstate` 方法，最终状态变为熄灭（不存在计划刻），而原先状态为熄灭的侦测器则不产生任何变化。熄灭的侦测器被活塞到位时添加的计划刻来自于 TE 对侦测器的 “更新” `postProcessState`，实际上最终仍旧调用的是侦测器响应 PP 的函数 `getStateForNeighborUpdate`。 ↩

---

slug: special-updates
title: 特殊的更新行为
title-en: null
author: hotpad100c
co-authors:

- Arcadi4
- BFladderbeanawa
- tanhHeng
- SDU-Little-River
  date: '2024-12-30T23:25:16+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 3
  is-advanced: false
  banner:
  src: ./img/SpecialUpdates.jpg
  alt: 部分特殊的更新范围

本部分将介绍一些方块的特殊的更新行为

1 红石粉的更新行为

1.1 二阶毗邻更新的概念

在#01中，我们引入了 NC 更新的 “更新源” 的概念，最普通常见的方块更新：例如放置、破坏方块产生的更新，都是以变化方块的位置为更新源，向六个方向发出 NC 更新的，这种就可以称作 “一阶毗邻更新”。

而红石粉则不然。举例来说，在红石粉的能量等级发生变化时，会发出 “二阶毗邻更新”，指的是：红石粉以自身位置和自身毗邻的六个方块分别为更新源，分别向六个方向发出 NC 更新。

“一阶” 与 “二阶” 的命名，是由最远受到 NC 更新的方块距离发出更新方块的曼哈顿距离[1]得来的。一阶毗邻更新即最远受到 NC 更新的方块距离发出更新方块的曼哈顿距离为 1，二阶毗邻更新即该距离为 2。

1.2 红石粉二阶毗邻更新的更新源顺序（红石粉更新的位置性）

红石粉在发出二阶毗邻更新时，7 个更新源的先后顺序是基于红石粉坐标的哈希信息随机排列的，其顺序见下表。这也就是红石粉更新具有位置性的由来。这些更新源有 97% 的概率被分为三组（`-Y, +Z, +X`、`O`、`+Y, -Z, -X`），并以如下顺序发出更新：

注：表中 O 表示源红石粉，-X 等表示更新源相对于源红石粉的方位

先	然后	最后	这种顺序的概率
-Y, +Z, +X	O	+Y, -Z, -X	24.267%
+Y, -Z, -X	O	-Y, +Z, +X	24.267%
O	-Y, +Z, +X	+Y, -Z, -X	12.133%
O	+Y, -Z, -X	-Y, +Z, +X	12.133%
-Y, +Z, +X	+Y, -Z, -X	O	12.133%
+Y, -Z, -X	-Y, +Z, +X	O	12.133%
	其他各项		<0.2%

各组别内的更新顺序是固定的，但组别排列顺序随机。除此之外，还有一些其他的概率极低的排列选项。

2 斜侧铁轨的更新行为

2.1 简述

斜放的动力铁轨（`PoweredRailBlock`）的激活状态（`powered=true|false`）变化时，它会按如下顺序依次发出两组更新：

1. 第一组：依次以上方方块、自身、下方方块为更新核发出 NC 更新。
2. 第二组：依次以自身、以下方方块、以上方方块为更新核发出 NC 更新。

2.2 代码走读

2.2.1 调用栈如下

```
PoweredRailBlock.updateBlockState(...)├── world.setBlockState(pos, ..., 3) // pos, flags=3│   ├──
worldChunk.setBlockState(...)│   │   └── PoweredRailBlock.onStateReplaced(...)│   │   ├──
world.updateNeighborsAlways(pos.up(), this)   // 1.1 上方│   │   ├── world.updateNeighborsAlways(pos, this)
// 1.2 自身│   │   └── world.updateNeighborsAlways(pos.down(), this) // 1.3 下方│   └── world.updateNeighbors()
// 2.1 自身├── world.updateNeighborsAlways(pos.down(), this)   // 2.2 下方└── world.updateNeighborsAlways(pos.up(), this)
// 2.3 上方
```

2.2.2 源码简析如下

```
protected void updateBlockState(BlockState state, World world, BlockPos pos, Block neighbor) {    boolean bl =
(Boolean)state.get(POWERED);    boolean bl2 = world.isReceivingRedstonePower(pos) || this.isPoweredByOtherRails(world, pos,
state, true, 0) || this.isPoweredByOtherRails(world, pos, state, false, 0);    if (bl2 != bl) {
world.setBlockState(pos, (BlockState)state.with(POWERED, bl2), 3); // 2.1 自身        world.updateNeighborsAlways(pos.down(),
this); // 2.2 下方        if (((RailShape)state.get(SHAPE)).isAscending()) {            world.updateNeighborsAlways(pos.up(),
this); // 2.3 仅斜铁轨 上方        }    }}// 见调用栈，由于 setBlockState 先调用到 onStateReplaced 然后再 updateNeighbors// 所以先执
行这一组 NC 更新，然后再以自身为更新源发出 NC 更新，即上文中标注的 2.1public void onStateReplaced(BlockState state, World world,
BlockPos pos, BlockState newState, boolean moved) {    if (!moved) {        super.onStateReplaced(state, world, pos,
newState, moved);        if (((RailShape)state.get(this.getShapeProperty())).isAscending()) {
world.updateNeighborsAlways(pos.up(), this); // 1.1 仅斜铁轨 上方        }        if (this.forbidCurves) { // 所有不能弯曲的铁轨
也就是除了普通铁轨            world.updateNeighborsAlways(pos, this); // 1.2 自身
world.updateNeighborsAlways(pos.down(), this); // 1.3 下方        }    }}
```

3 红石粉的更新及其代码解析

4 铁轨链的递归检查行为

4.1 简述

铁轨在接受到 NC 更新时会检测自身充能状态。当满足以下两个条件之一时，它将会判断自己被激活 `powered=true`：

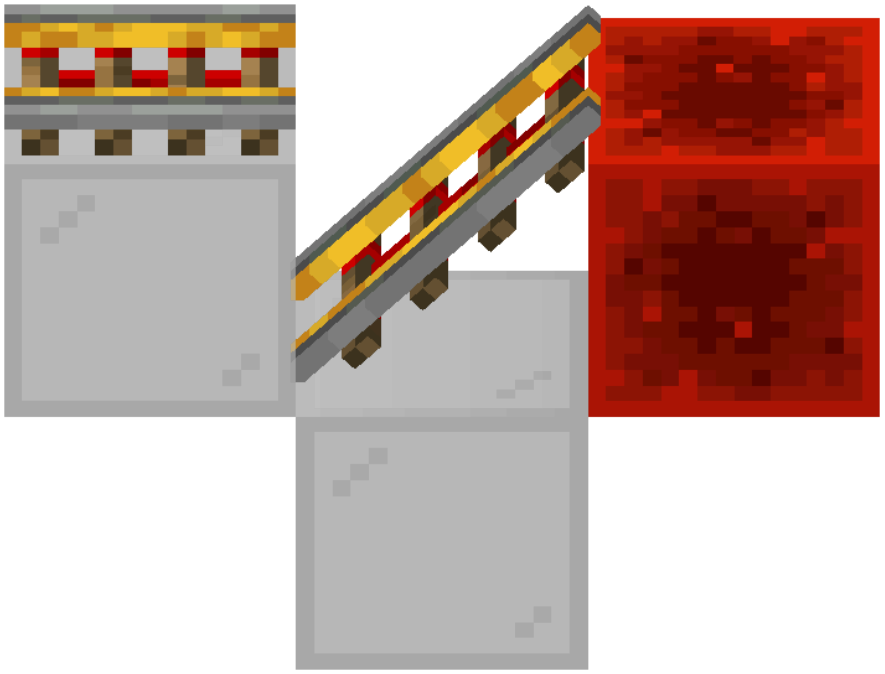
1. 直接接收到红石信号，即被直接充能
2. 由相连的其他铁轨充能，即被间接充能

铁轨检查自身是否被间接充能并不是检测相连铁轨是否被充能，而是递归寻找整条铁轨链上距离自身 8 个铁轨及以内的 *(若认为直接相连的两个铁轨的距离为 1 个铁轨)*、被直接充能的铁轨，过程如下：

1. 初始时距离变量为 `0`。若 `距离=8`，则终止检查并返回 `False`，即不被间接充能。
2. 铁轨具有两个方向，当开始检查时，铁轨先检查哪一个方向由一个布尔值控制。当布尔值为 `True` 时，且当铁轨形状为南北/东西朝向时，它检查南/西侧铁轨；当铁轨形状是上下斜向 `shape=ascending_*` 时，它检查南侧上方 / 西侧上方的铁轨。当布尔值为 `False` 时反之。此布尔值在开始检查时先为 `True`，后为 `False`。且由于逻辑运算短路[2]：若第一次布尔值为 `True` 时检查成功，则不会进行第二次布尔值为 `False` 的检查。在递归检查过程中 *(见步骤 6)* 此布尔值不会改变。
3. 移动坐标 `(i, j, k)` 到接下来要检查的铁轨位置。
4. 判断当前铁轨能否可能在当前检查方向上与当前方向的下方的一个铁轨相连。若当前铁轨为向东/西/南/北侧向上，且检查方向为东/西/南/北，则认为不能。此步骤是为步骤 8 准备的。例如：当前铁轨为北侧向上，此时在向南侧检查，则此铁轨可能与南侧下方的铁轨相连；当前铁轨为北侧向上，此时在向北侧检查，则此铁轨不可能与南侧下方的铁轨相连。
5. 若当前铁轨形状为南侧向上/北侧向上或东侧向上/西侧向上，则认为当前铁轨形状为南北朝向或东西朝向 —— 这一形状更改是为下一步骤 *(6.b.)* 准备的。
6. 检查坐标 `(i, j, k)`：
    1. 检查当前位置是否是一个铁轨，如果不是，则返回 `False`，结束当前步骤检查。
    2. 检查当前位置是否与原铁轨相连。此步骤并不检查 “相连”，而是检查 “不相连”：若原铁轨形状为东西朝向，则若当前铁轨形状为南北/北侧向上/南侧向上，则不相连，返回 `False`，结束步骤 6 检查。若原铁轨为南北朝向，则若当前铁轨形状为东西/东侧向上/西侧向上，则不相连，返回 `False`，结束当前步骤检查。
    3. 步骤 i.ii. 实际都是 iii.iv. 的前置条件。此步骤检查当前位置的铁轨是否被直接充能，如果是，则返回 `True`，结束当前步骤检查。
    4. 跳至步骤 1，距离增加 `1`，并按照开始检查的布尔值进行下一个铁轨的检查 *(递归)*。
7. 若步骤 6 返回 `True`，则终止检查并返回 `True`。
8. *(见步骤 4 判断)* 若当前铁轨可能在当前检查方向上与当前方向的下方的一个铁轨相连，那么检查坐标 `(i, j, k)` 下方的方块，即跳至步骤 6，但检查的坐标为 `(i, j-1, k)`。在步骤 4 的例子中，若步骤 3 返回 `False`，那么此时将检查原铁轨南侧下方的铁轨。

4.2 代码走读

```
// 这是起始点@Overrideprotected void updateBlockState(BlockState state, World world, BlockPos pos, Block neighbor) {
boolean bl2;    boolean bl = state.get(POWERED);    boolean bl3 = bl2 = world.isReceivingRedstonePower(pos) ||
this.isPoweredByOtherRails(world, pos, state, true, 0) || this.isPoweredByOtherRails(world, pos, state, false, 0);    // 逻辑
运算短路说的是这里 bl3。    // 如果被直接充能，那么直接结束。否则先 bl = true，后 false。见后    if (bl2 != bl) {
world.setBlockState(pos, (BlockState)state.with(POWERED, bl2), Block.NOTIFY_ALL);
world.updateNeighborsAlways(pos.down(), this);        if (state.get(SHAPE).isAscending()) {
world.updateNeighborsAlways(pos.up(), this);        }    }}// 便于区分，这块函数标记为 isPoweredByOtherRails（01）protected
boolean isPoweredByOtherRails(World world, BlockPos pos, BlockState state, boolean bl, int distance) {    // bl，即上文中控制
“先检查哪一方向”的布尔值    if (distance >= 8) {        return false;    }    int i = pos.getX();    int j = pos.getY();    int
k = pos.getZ();    boolean bl2 = true;    RailShape railShape = state.get(SHAPE);    switch (railShape) { // 根据当前铁轨形状移
位检查，由 bl 可知铁轨递归检查总是先检查 - x（西）或 + z（南）方向        case NORTH_SOUTH: {            if (bl) {
++k; // 铁轨南北向，则先 z 增（向南）                break;            }            --k; // 若南侧检查失败则 z 减（回过来向北），若南侧
成功则熔断不再向北            break;        }        case EAST_WEST: {            if (bl) { // 同上，铁轨东西向，则先 x 减（向西）
--i;                break;            }            ++i; // 同上，x 增（向东）            break;        }        case
ASCENDING_EAST: { // 铁轨东侧上升，即东高西低的斜铁轨            if (bl) {                --i; // 因为西低，所以 - x 西侧可能存在同 y
的铁轨连接            } else {                ++i; // 因为东高，所以 + x 东侧不可能存在同 y 的铁轨连接                ++j; // 所以 y
+ 1，东侧若有连接那么铁轨必然高 1 格                bl2 = false; // 上文中步骤 4 的“当前铁轨能否可能在当前检查方向上与当前方向的下方的一个
铁轨相连”的判断，这一判断默认是成立的。                // 举例来说，一个 y = 1 的平放铁轨，如果边上有一个下降的铁轨与它相连，那么这个铁轨一定
比它低一格。                // 而此时检查东侧，因为东高，所以不可能东侧有低一格的铁轨相连。判断变为 false            }
railShape = RailShape.EAST_WEST; // 上文种步骤 5 的形状更改            break;        }        case ASCENDING_WEST: { // 同理
if (bl) {                --i;                ++j;                bl2 = false;            } else {                ++i;
}            railShape = RailShape.EAST_WEST;            break;        }        case ASCENDING_NORTH: { // 同理            if
(bl) {                ++k;            } else {                --k;                ++j;                bl2 = false;
}            railShape = RailShape.NORTH_SOUTH;            break;        }        case ASCENDING_SOUTH: { // 还是同理
if (bl) {                ++k;                ++j;                bl2 = false;            } else {                --k;
}            railShape = RailShape.NORTH_SOUTH;        }    }    if (this.isPoweredByOtherRails(world, new BlockPos(i, j,
k), bl, distance, railShape)) {        return true;        // 注意这里调用的不是当前代码块的这一函数。java 有通过入参类型匹配不同函数的
性质。        // 这里调用的是下文中的 isPoweredByOtherRails（02）    }    return bl2 && this.isPoweredByOtherRails(world, new
BlockPos(i, j - 1, k), bl, distance, railShape);    // 同上，调用的也是（02）    // 这里的 bl2 也就对应上文中步骤 8。j-1 也就是 y 坐
标向下移了一格。}// 便于区分，这块函数标记为 isPoweredByOtherRails（02）protected boolean isPoweredByOtherRails(World world,
BlockPos pos, boolean bl, int distance, RailShape shape) {    // 这里 RailShape shape 是调用这一函数的铁轨的 shape，也就是上一个铁
轨的 shape    BlockState blockState = world.getBlockState(pos);    if (!blockState.isOf(this)) { // 略        return false;
}    RailShape railShape = blockState.get(SHAPE); // 略    if (shape == RailShape.EAST_WEST && (railShape ==
RailShape.NORTH_SOUTH || railShape == RailShape.ASCENDING_NORTH || railShape == RailShape.ASCENDING_SOUTH)) {        return
false; // 上一个铁轨与当前铁轨不相连，那么不可能被简介充能。对应步骤 6.ii.        // 在 isPoweredByOtherRails（01）中也就是上文步骤 5 有形
状更改，形状更改的影响在这里就体现了        // 盲猜本意是为了便于检查，但是这导致斜铁轨由于被改变形状了，导致一个平铁轨可能与看似不相连的斜铁轨判断
为相连        // 而且这种判断是单向的，因为铁轨递归判断不存在 + y 的行为。例子见后图    }    if (shape == RailShape.NORTH_SOUTH &&
(railShape == RailShape.EAST_WEST || railShape == RailShape.ASCENDING_EAST || railShape == RailShape.ASCENDING_WEST)) {
return false; // 同上    }    if (blockState.get(POWERED).booleanValue()) { // 如果铁轨是亮的（这里不是“被直接/间接”充能的判断，而
是只要铁轨是亮的就行（nbt 中 powered = true）        if (world.isReceivingRedstonePower(pos)) {            return true; // 一直到
找到被直接充能的铁轨才结束。这里就是整个递归判断为 true 的返回点        }        return this.isPoweredByOtherRails(world, pos,
blockState, bl, distance + 1);        // 这里回到 isPoweredByOtherRails（01），distance + 1，递归。    }    return false;}
```



一个平铁轨可能与看似不相连的斜铁轨判断为相连。同样的结构若红石块直接充能平铁轨则斜铁轨不会被充能

5 亮起侦测器到位无毗邻 NC 更新

当活塞推动一个亮起的侦测器到位、且到位位置无侦测器计划刻时，它不会对毗邻方块（除输出端指向的方块）发出常规意义上由活塞推动触发的 NC 更新（*指通常情况下活塞推动一般方块到位后，会在方块到位位置发出由 `flags=3` 的 `setBlockState` 触发的 NC 更新*），原理如下：

1. 到位分两种情况：
    1. 一般到位：调用 `PistonBlockEntity.tick`，其中调用 `setBlockState`，`flags=67`
    2. 强制到位：调用 `PistonBlockEntity.finish`，其中调用 `setBlockState`，`flags=3`
2. 在 `setBlockState` 中调用 `worldChunk.setBlockState`，然后调用 `onBlockAdded`
3. 在 `ObserverBlock.onBlockAdded` 中：
    1. 判断当前位置的旧方块是否为侦测器，由于旧方块是 b36，判断 `true`，下一步
    2. 判断自身是否激活且当前位置无侦测器计划刻，由于自身亮起 `powered=true` 且当前位置无侦测器计划刻，判断 `true`，下一步
    3. `setBlockState`，`flags=18`，将自身激活状态变为 `powered=true -> powered=false`，且发出 PP 更新
    4. 发出常规的侦测器的 NC 更新，即对输出端指向的方块及指向方块的毗邻方块发出 NC 更新
4. 由于在步骤 3.iii. 中，侦测器 `powered` 属性改变，此时继续步骤 1 的 `setBlockState`，但由于 `blockState2 == state` 即前后状态是否一致为 `false`，`setBlockState` 不再发出常规的 NC 更新，而是直接结束

调用栈如下：

```
PistonBlockEntity.tick(...) | PistonBlockEntity.finish(...)└── world.setBlockState(pos, state, flags=67) |
world.setBlockState(pos, state, flags=3)    ├── // state: Block{minecraft:observer}[facing=..., powered=true]    ├──
worldChunk.setBlockState(pos, state, (flags & 64) != 0)    |   ├── BlockState blockState = chunkSection.setBlockState(j, k,
l, state); // Block{minecraft:moving_piston}[..]    |   └── state.onBlockAdded(this.world, pos, blockState, moved)    |
└── ObserverBlock.onBlockAdded(state, this.world, pos, oldState, moved)    |           ├── if
(!state.isOf(oldState.getBlock())): // true    |           |   // state: Block{minecraft:observer}[...], oldState:
Block{minecraft:moving_piston}[..]    |           |   └── if (!world.isClient() && (Boolean)state.get(POWERED) &&
!world.getBlockTickScheduler().isQueued(pos, this)): // true    |           |       ├── world.setBlockState(pos,
state.with(powered=false), flags=18)    |           |       |   ├── 状态变为 powered = false    |           |       |   └── 发
出 PP 更新    |           |       |   └── this.updateNeighbors(world, pos, blockState) // 对输出端指向及指向的毗邻发出 NC 更新
|           |   return    ├── BlockState blockState2 = getBlockState(pos) // Block{minecraft:observer}[facing=...,
powered=false]    └── if (blockState2 == state): // false, powered 不同        └── 跳过 NC 更新和 PP 更新，结束
```

FOOTNOTES

1. 曼哈顿距离，即两坐标在各坐标轴上的距离的和，在 Minecraft 的三维空间中有 `d=|x|+|y|+|z|` ↩
2. 逻辑运算短路，计算机进行逻辑判断语句时的一种 “熔断” 机制。例如 `A and B` 中，若 A 不成立，则逻辑判断熔断，返回 `False`，不再判断 B 是否成立；`A or B` 中，若 A 成立，则逻辑判断熔断，返回 `True`，不再判断 B 是否成立。 ↩

---

slug: continuous-updates
title: 连续的方块更新及其分析方法
title-en: null
author: hotpad100c
co-authors:

- Arcadi4
- BFladderbeanawa
- Gudu-Z
- tanhHeng
- SDU-Little-River
  date: '2024-12-14T20:58:16+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 2
  is-advanced: false
  banner:
  src: ./img/UpdateOrder.jpg
  alt: 一条奇怪的更新链

本部分将教学 Minecraft 中更深层次的更新机制，以及如何分析更新顺序

1 方块的一般更新行为

一般情况下，方块总是以**先 NC 更新、后 PP 更新**的顺序发出更新。要深入理解更新理论，必须清楚一个概念：**更新不是一个单一的事件，而是一段 “过程”**。

示意图如下：

```
方块 A: ├── 发出 NC 更新 |    ├── NC 更新触发了一些事件 |    |    └── 这些事件又触发了一些事件 |    └── 这些事件运行结束了 |    * NC 更新完成 * └── 发出 PP 更新    ├── PP 更新触发了一些事件    └── 这些事件运行结束了    * PP 更新完成 *
```

例如，当方块 A 发出 NC 更新后，更新到方块 B，此时 B 改变状态，又发出 NC 更新。更新过程如下：

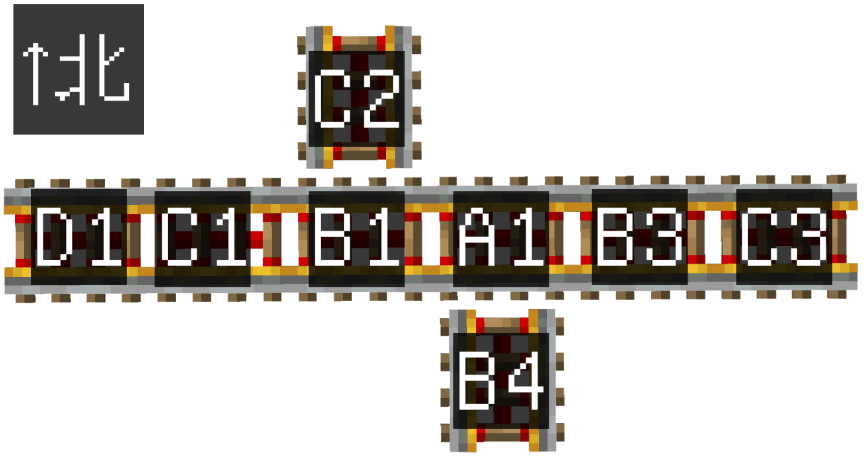
```
├── 方块 A 发出 NC 更新 |    ├── 方块 B 受到 NC 更新 |    ├── 方块 B 状态改变 |    ├── 方块 B 发出 NC 更新 |    |    └── 什么都没有发生 |    |    * 方块 B NC 更新完成 * |    └── 方块 B 发出 PP 更新 |    └── 什么都没有发生 |    * 方块 B PP 更新完成 * |    * 方块 A NC 更新完成 * └── 方块 A 发出 PP 更新    └── 什么都没有发生    * 方块 A PP 更新完成 *
```

一般讨论的**NC 更新顺序**指的是**发出 NC 更新的先后次序**，**PP 更新顺序**同理，也是**发出 PP 更新的顺序**。从上文中的示意和例子我们可以看出，在较为复杂的更新中，PP 更新顺序也可以近似认为是**方块 NC 更新过程结束的先后次序**。

2 MINECRAFT 中的更新方法

我们将通过一些生动形象的简单例子，让你循序渐进地理解更新这一复杂过程是如何发生的。

例如下面这串 bud 铁轨为例，更新其中的 A1 会怎样发生什么样的事情呢



2.1 更新顺序的分析方法

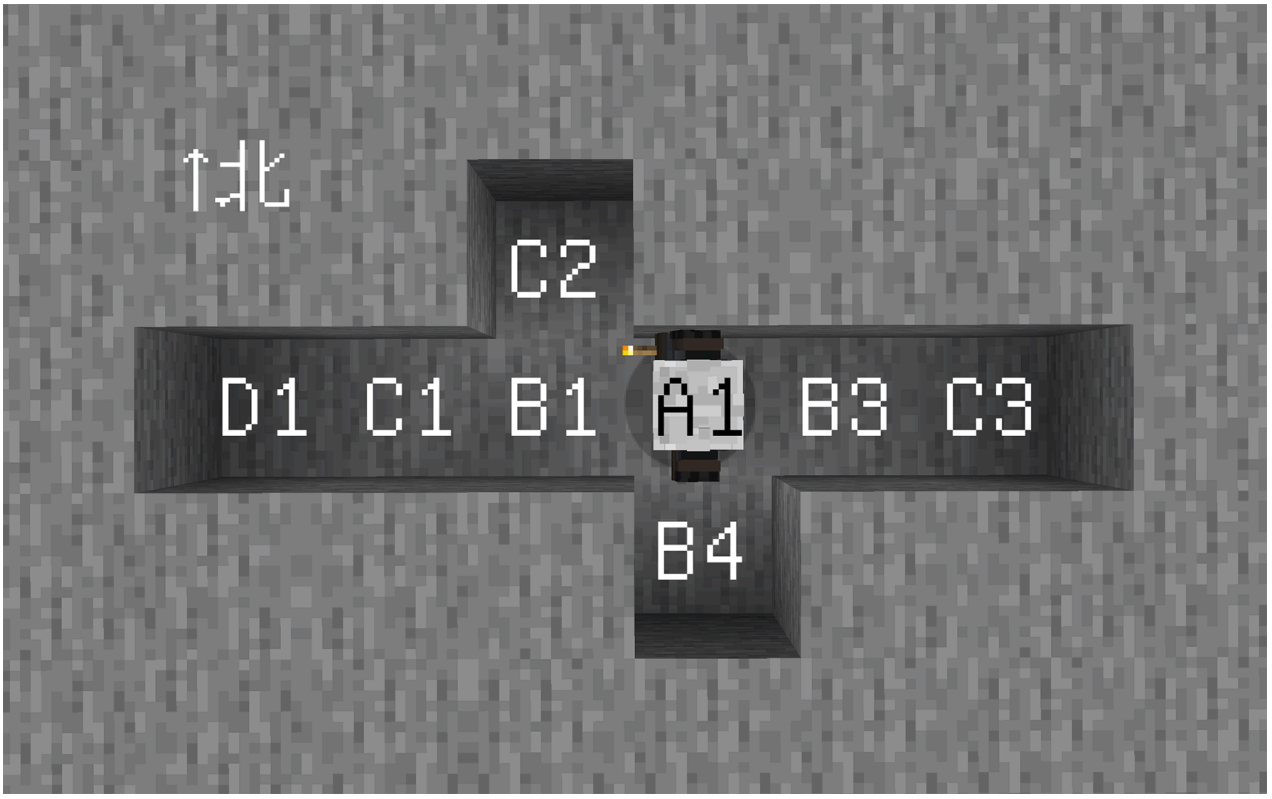
~~想要理解更新，我们就需要先抛开更新不谈~~

想象一下，你是一个强迫症，此时此刻你正拿着许多火把在一个矿洞。

由于你是强迫症，你将完全遵循以下规则：

1. 你走到每一格都会插一根火把
2. 每走一个格就按照**西东下上北南**的顺序寻找有没有格子
3. 如果有格子可走，就立刻进入
4. 无路可走的时候就往回走
5. 走回头路的时候会捡回放下火把
6. 走回岔路口的时候如果还有**其他方向**没检查完就继续检查

按照上面的规则，我们来看一下下面这个矿洞你会如何探索

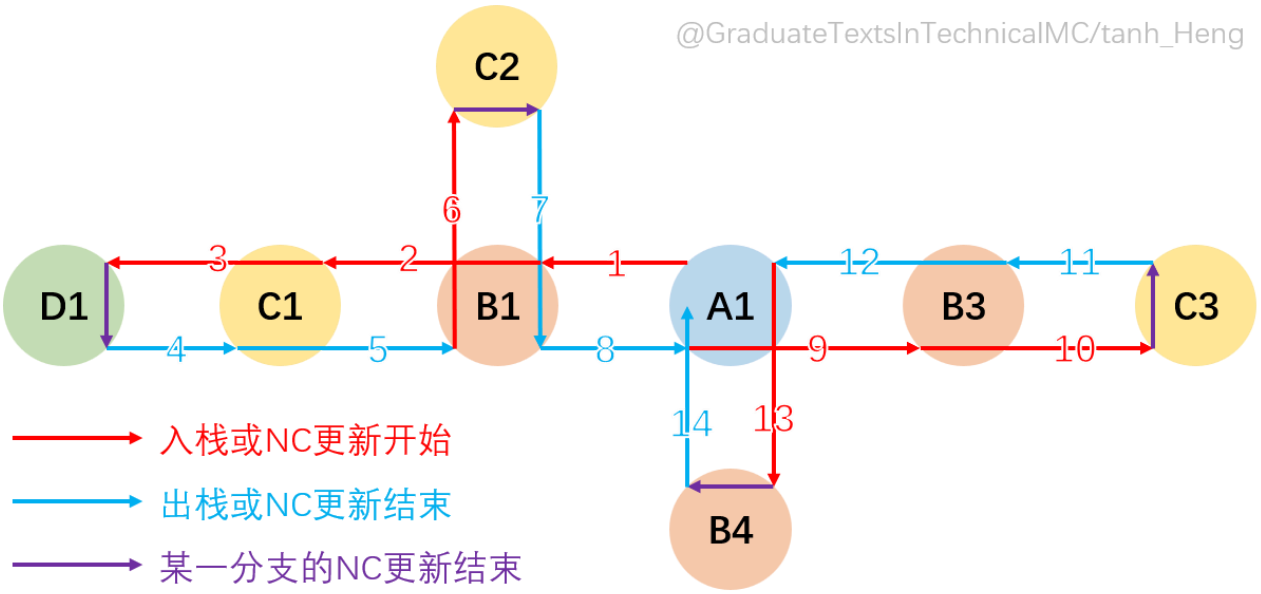


1. 最开始，你位于在 A1，于是你在 A1插下火把
2. 你开始按照**西东下上北南**的顺序寻找岔路，你发现西方有一格可走
3. 你进入这个格子（B1），插下火把，继续检查→发现西方有路，进入 C1，插火把，检查→西方有路，进入 D1，插火把
4. 哎，这个时候你按照西东下上北南的顺序检查完发现没路了，于是你收起火把往回走（进入 C1）
5. 西你检查过了，于是检查东下上北南，发现没路，收火把，回到 B1
6. 东下上都没路可走，但是当你检查到北的时候你发现有路，于是进入（C2），插火把
7. 检查完发现没路，收火把回到 B1
8. B1 检查完没路，收火把回到 A1, 你开始检查其他方向
9. ！ 东方有路，进入 B3 插火把→东方有路，进入 C3 插火把
10. 无路可走了，收火把回 B3 →没路，收火把回 A1
11. 检查剩余方向，南方有路，进入 B4，插火把
12. 无路，收火把回 A1
13. A1 无剩余岔路，收起火把，探索完毕

这个探索矿洞的过程就是更新的过程，其中“插火把”是发出 NC 更新（入栈），收火把是发出 PP 更新（出栈）。至于这个 “栈” 是什么意思，你可以参阅附录 - 专有名词解释→栈与调用栈,而这种更新方法就是深度优先搜索（DFS）

推荐搭配 BiliBili-tanh_Heng：NC 更新 × 深度优先搜索.exe 食用

让我们用一种更加形象的方式理解例子中的更新过程。我们可以将 NC 更新的入栈看作 “逐步深入”，NC 更新的出栈，也就是 PP 更新看作 “逐步返回”。那么，让我们将上文中的图画得更加形象一些，并且再多加入一些节点：



图中为 A1 发出更新后的更新链。上方为北侧。

从图中就能够很好地看出：

NC 更新顺序：`A1->B1->C1->D1->C2->B3->C3->B4`

PP 更新顺序：`D1->C1->C2->B1->C3->B3->B4->A1`

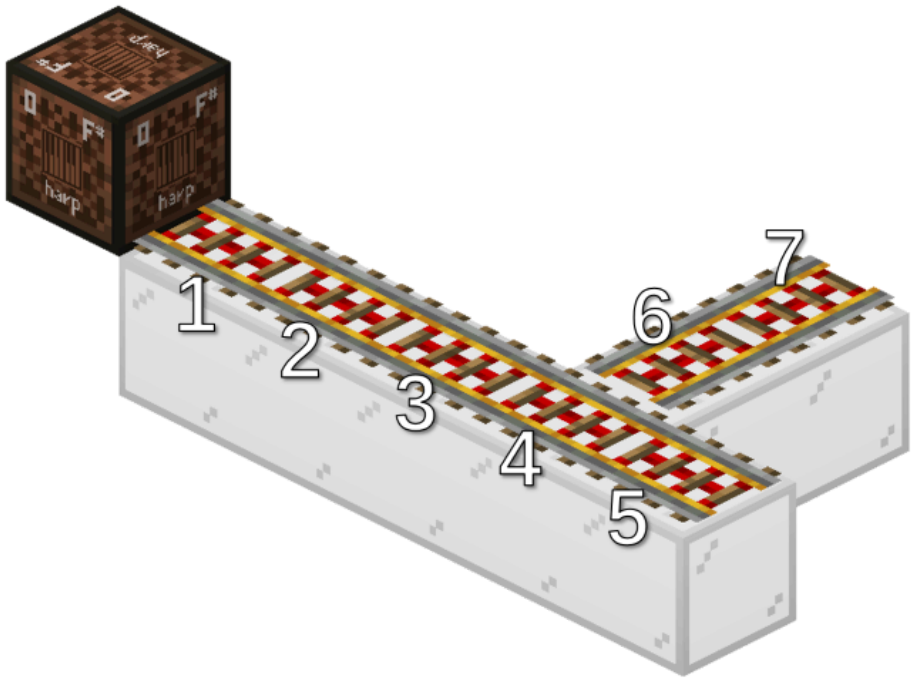
将更新行为想象为一个点，这个点从 A1 出发，按照一定的更新顺序移动，并且 “不撞南墙不回头”。那么，这个点的移动路径就构成了图示的箭头。逐步深入的过程就是入栈或 NC 更新，“回头” 的过程就是出栈或 PP 更新。

需要注意的是，这里的分析将 PP 更新简化了。因为通常情况下，PP 更新并不会导致**连续的方块更新**[1]。要分析 PP 更新，方法和 NC 更新是类似的，从栈的角度分析即可。

3 常见的更新顺序分析

4 BUD 铁轨链更新顺序分析

让我们以一个实例来尝试分析更新顺序



已知铁轨在状态改变时，先发出 NC 更新，后发出 PP 更新。图中是一串处于 BUD 状态的铁轨链，1-5 铁轨位于东西走向。问：**音符盒按下后，铁轨熄灭的 NC 更新和 PP 更新顺序分别是?**

当音符盒按下后，发出 NC 更新。毗邻的 1 号铁轨受到 NC 更新，改变自身状态，发出 NC 更新，更新到 2 号、再到 3 和 4。在 4 号铁轨处，4 号铁轨先对东西侧的 5 号铁轨更新。5 号铁轨受到更新后改变自身状态，发出 NC 更新，不再有后续 NC 更新，于是 5 号铁轨此时发出 PP 更新。退回到 4 号铁轨，4 号铁轨再更新南北侧 6、7 号铁轨。深入到 6 号铁轨，6 号铁轨发出 NC 更新；再深入到 7 号铁轨，7 号铁轨发出 NC 更新。7 号铁轨更新结束，发出 PP 更新。退回到 6 号铁轨，6 号铁轨发出 PP 更新。退回到 4 号铁轨，4 号铁轨发出 PP 更新。再退回到 3、2、1 号铁轨，3、2、1 号铁轨发出 PP 更新。

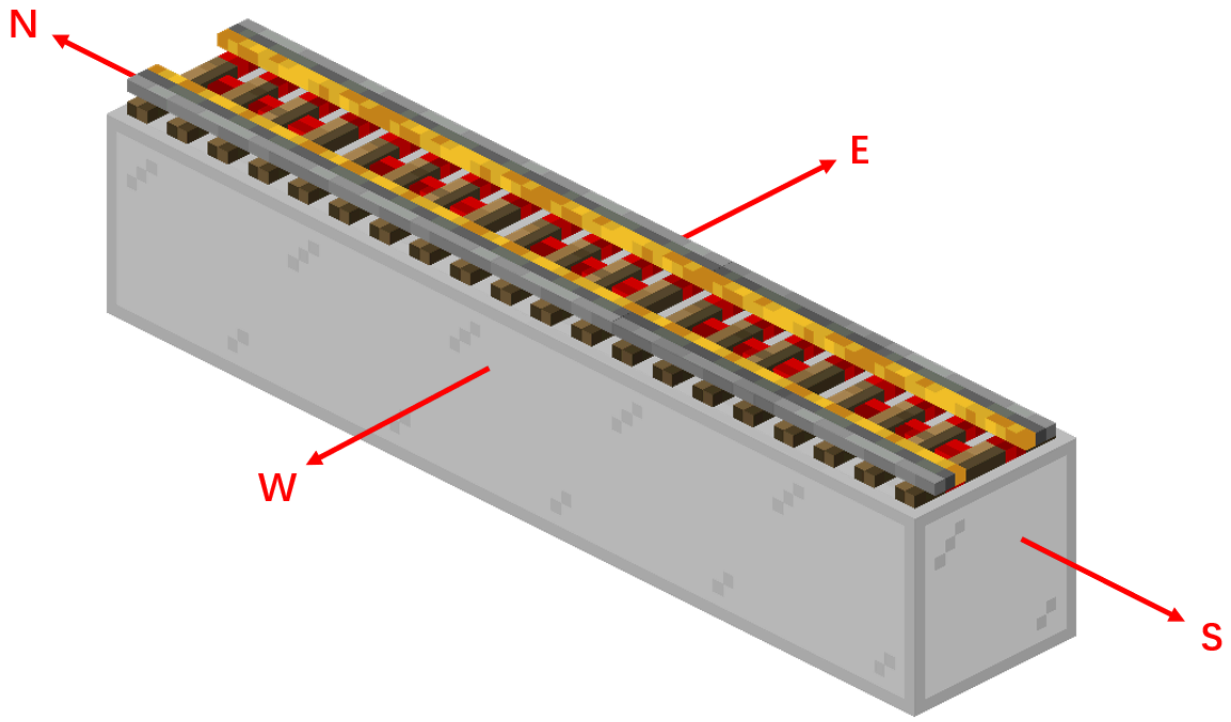
所以，

NC 更新顺序为：1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7

PP 更新顺序为：5 -> 7 -> 6 -> 4 -> 3 -> 2 -> 1

5 考虑 NC 更新方向的分析 *

此部分并不常用，读者可选择性跳过。



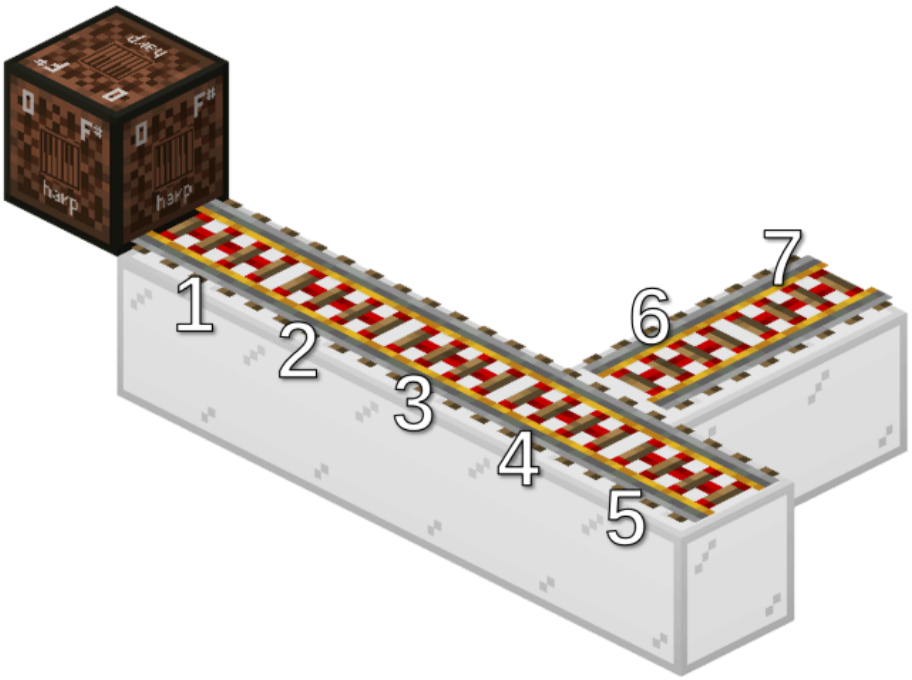
图中铁轨链方向如标注所示，铁轨链为南北向。我们以最北端的铁轨为例分析其不同方向的 NC 更新顺序。

NC 更新按照西东下上北南的方向顺序发出。当最北侧铁轨熄灭时，它先发出西侧、东侧更新，无事发生；再发出下侧、上侧更新，无事发生；然后发出北侧更新，无事发生；最后发出南侧更新，更新到南侧铁轨，后续更新即可按上文中的方法分析。

假如我们在该铁轨北侧制作一个 BUD 装置，那么这个 BUD 装置会先被更新，然后南侧铁轨再进行连续的 NC 更新。

而如果我们考虑**对六个方向的 NC 更新中，对上侧方块的 NC 更新**的顺序，那么它就是**自北向南**，即**由近到远**。

让我们再考虑到 2.3.1 中的例子



音符盒为 1-5 铁轨链的最西侧，仅考虑 1-5 铁轨链的 NC 更新。

当音符盒按下后，1 号铁轨受到更新而熄灭，先发出西侧更新，无事发生；再发出东侧更新，更新到 2 号铁轨，2 号铁轨开始更新…… 等到 2、3、4…… 铁轨更新结束后，1 号铁轨的**东侧 NC 更新**完成，1 号铁轨继续更新下侧、上侧、北侧、南侧。

此时如果我们考虑**对上侧方块的 NC 更新**的顺序，它就变成了 5、4、3、2、1，也就是**由远到近**。

那么读者可能就会想问：既然 NC 更新的顺序还要考虑方向性，我们学习 2.3.3 的发出 NC 更新顺序又有什么用呢?

用处有二：

1. 用于分析 PP 更新。通常我们使用铁轨链时，多用侦测器来观测，因此这实际上是为了帮助我们分析 PP 更新的。
2. NC 更新的方向性是在 2.3.3 的基础之上的，只有理解了 2.3.3 才能更好的理解方向性。

6 练习

上文中的第二个例子我们并没有考虑 6、7 号铁轨。读者可尝试分析考虑 6、7 号铁轨时，整个 BUD 铁轨链对**对上侧方块的 NC 更新**的顺序。

答案：5 -> 4 -> 6 -> 7 -> 3 -> 2 -> 1

FOOTNOTES

1. 墙、栅栏等元件在受到 PP 更新后改变状态，并发出 PP 更新，这类方块可能会触发由 PP 更新导致的连续的方块更新。但这不在通常需要分析的情况内。 ↩

CHAPTER 2

COMPONENTS & FEATURES

---

slug: chunk-savestate-analysis
title: Chunk Savestate 原理分析
title-en: Chunk Savestate Analysis
author: HackerRouter
co-authors:

- BFladderbeanawa
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-09T22:00:46+08:00'
  index: 3
  is-advanced: true

> *此文档由HackerRouter撰写，机制由 BFladderbean 发现。*

参考视频：

1 TL;DR

`TACS`（`ThreadedAnvilChunkStorage`）在普通 tick 的增量保存中保存了区块 A（旧状态）并设置 10 秒冷却 →
区块 A 和 B 的状态被改变 →
后续保存尝试中区块 A 因冷却被跳过、区块 B 正常保存 →
主线程卡死后 Watchdog 强退 →
重启后区块 A 是旧快照、区块 B 是新快照，物品同时存在于两处。

2 TACS

`TACS`（`ThreadedAnvilChunkStorage`），在 Mojang 映射中对应 `ChunkMap` 类，是服务端区块持久化的核心组件。
它负责决定哪些区块需要保存、何时保存、以及保存的节流控制。

`TACS` 持有所有已加载区块的 `ChunkHolder`（区块持有者，每个已加载区块在 `TACS` 中的包装对象）引用，
这些引用存储在 `visibleChunkMap`（一个 `Long2ObjectLinkedOpenHashMap<ChunkHolder>`，以区块坐标 long 编码为 key）中。
`TACS` 在每 tick 和自动保存中驱动区块的序列化与写盘流程。

下文涉及的所有保存逻辑（增量保存、自动保存、冷却节流）都在 `TACS` 内部完成。

3 区块保存的实际触发

区块保存不只发生在 6000 tick 一次的自动保存中。
每个 tick，服务器都在做增量保存。

调用链如下：

```
MinecraftServer.tickServer()
  → tickChildren()
    → serverLevel.tick()
      → chunkSource.tick()            // ServerChunkCache.tick()
        → chunkMap.tick()             // ChunkMap.tick()
          → processUnloads()
            → 遍历 visibleChunkMap，每 tick 最多保存 20 个区块
```

在 `ChunkMap.processUnloads()` 末尾（1.19.2 `ChunkMap.java`:502-509）：

```
int k = 0;ObjectIterator<ChunkHolder> objectIterator = this.visibleChunkMap.values().iterator();while (k < 20 &&
booleanSupplier.getAsBoolean() && objectIterator.hasNext()) {    if (this.saveChunkIfNeeded(objectIterator.next())) {
k++;    }}
```

每 tick 最多尝试保存 20 个区块。这个循环独立于自动保存，一直在后台运行。

4 秒保存冷却

10 秒保存冷却是 `TACS` 对单个区块保存频率的节流机制。
其目的是避免频繁变化的区块（如有持续红石活动的区块）在每个 tick 都被序列化和写盘，从而减少 I / O 开销。

具体实现在 `ChunkMap.saveChunkIfNeeded()`（1.19.2 `ChunkMap.java`:762-786）：

```
long m = this.chunkSaveCooldowns.getOrDefault(l, -1L);long n = System.currentTimeMillis();if (n < m) {    return false;    // 冷却期内，跳过保存}boolean bl = this.save(chunkAccess);if (bl) {    this.chunkSaveCooldowns.put(l, n + 10000L);  // 保存成功后设置 10 秒冷却}
```

`chunkSaveCooldowns` 是一个 `Long2LongOpenHashMap`，key 为区块位置的 long 编码，value 为下次允许保存的时间戳（ms）。
一个区块保存成功后，其冷却截止时间被设置为当前时间 + 10000ms。

在冷却期内，即使区块已经因为方块变化再次变脏（`needsSaving == true`，即区块的 dirty 标记被置位），`saveChunkIfNeeded()` 也会直接返回 `false` 跳过保存。
dirty 标记只是保存的必要条件，不是充分条件。冷却未过期时，dirty 也照样跳过。

5 序列化在主线程同步完成

`ChunkMap.save()`（1.19.2 `ChunkMap.java`:788-818）的关键顺序：

```
if (!chunkAccess.isUnsaved()) { return false; }chunkAccess.setUnsaved(false);                                    // 清除
dirtyCompoundTag compoundTag = ChunkSerializer.write(this.level, chunkAccess); // 主线程同步序列化this.write(chunkPos,
compoundTag);                                    // 提交给 IOWorker（负责异步写盘的后台工作线程）
```

`ChunkSerializer.write()` 在主线程内联执行，遍历 `chunk.getBlockEntityPositions()`，
对每个位置调用 `chunk.getPackedBlockEntityNbt()`。
序列化器看到的是调用那一瞬间 live chunk（内存中正在被游戏逻辑使用的活跃区块对象）的逻辑状态。
I / O 线程读取活跃区块内存的情况不存在。

6 异步写盘 —— 三层链路

序列化后的 NBT 经过三层才到达磁盘：

第一层：主线程 → `IOWorker.pendingWrites`（JVM 堆内存）

`ChunkStorage.write()` → `IOWorker.store()`，将 NBT 存入 `pendingWrites` 这个 `LinkedHashMap`。

第二层：`IOWorker` 后台线程 → `RegionFile`（OS 文件缓存）

`IOWorker.storePendingChunk()` 每次取一个 entry，调用 `RegionFile.write()`。
`RegionFile.write()` 调用 `FileChannel.write()` 将数据写入文件，但不调用 `force()` / `fsync()`（操作系统级的强制刷盘系统调用）。
数据进入 OS 的 page cache（操作系统在内存中缓存文件写入的区域，写入后数据暂存于此而非直接写入物理磁盘），尚未保证到达物理磁盘。

第三层：OS page cache → 物理磁盘

没有 `fsync`，刷盘取决于 OS 的 dirty page writeback 策略。
进程终止时，未刷盘的数据可能丢失。

`IOWorker` 是逐区块、逐 entry 写盘的，不是整个世界一次性原子提交。

7 自动保存不绕过冷却

每 6000 tick 触发的自动保存（`MinecraftServer.tickServer()` 1.19.2:828）：

```
if (this.tickCount % 6000 == 0) {    this.saveEverything(true, false, false);  // flush=false}
```

这里的 `flush` 参数决定保存行为的彻底程度：

- `flush=true`：强制序列化所有 dirty 区块（绕过 10 秒冷却），并等待 `IOWorker` 将所有排队数据写完磁盘后才返回。用于 `/save-all flush` 或关服时。
- `flush=false`：走普通的 `saveChunkIfNeeded()` 路径，受冷却限制，且提交给 `IOWorker` 后立即返回，不等待实际落盘完成。

自动保存使用的是 `flush=false`。

`saveEverything` → `saveAllChunks` → `serverLevel.save()` → `serverChunkCache.save(false)` → `ChunkMap.saveAllChunks(false)`。

`saveAllChunks(false)` 的非 flush 分支（1.19.2 `ChunkMap.java`:452-454）：

```
this.visibleChunkMap.values().stream()    .filter(ChunkHolder::wasAccessibleSinceLastSave)
.forEach(this::saveChunkIfNeeded);    // 复用 saveChunkIfNeeded，受冷却限制
```

自动保存调用的是同一个 `saveChunkIfNeeded()`。
如果某区块在之前的增量保存中刚被保存过，冷却期内自动保存也会跳过它。

8 区块变脏的时机

`WorldChunk.setBlockState()` 在主线程执行时（1.19.2）：

1. 修改 section 内部方块状态
2. 对旧状态调用 `onStateReplaced`
3. 必要时 `removeBlockEntity`
4. 最后设置 `needsSaving = true`

区块一旦发生方块变化就重新变脏。
但 dirty 状态和之前已排入 `IOWorker` 的旧 NBT 没有回写关系 —— 旧 NBT 已经是独立的数据对象。

9 WATCHDOG 强制退出

`DedicatedServerWatchdog` 在独立线程中监控主线程。
检测到超时后：

1. 写 crash report
2. 调用 `System.exit(1)`
3. 安排 10 秒后 `Runtime.halt(1)` 兜底

`System.exit(1)` 触发 JVM shutdown 流程。
在完全退出前，`IOWorker` 后台线程仍可能继续运行一小段时间，把 `pendingWrites` 中已排队的区块继续写出。
直到 JVM 正常退出或被 `halt(1)` 硬杀。

10 流程梳理

以发射器发射潜影盒到相邻区块，即 Comedy 视频中的演示为例：

T = 0：增量保存保存了区块 A

普通 tick 中，增量保存循环遍历到区块 A（含发射器 + 潜影盒），条件满足：

- 冷却已过期
- `needsSaving == true`

主线程同步执行 `ChunkSerializer.write()`，生成 `A_old_nbt`（发射器含潜影盒）。
提交给 `IOWorker`。
设置 `chunkSaveCooldowns[A] = now + 10000ms`。

此后区块 A 进入 10 秒冷却期。

T = 1：区块 A 和 B 被改变

发射器被红石激活，4 tick 后 `ServerTickList.tick()` 执行该计划刻，发射器触发。
潜影盒从区块 A（发射器）移动到区块 B（着陆位置）。
区块 A 变脏（发射器变空），区块 B 变脏（出现潜影盒方块）。

T = 2：自动保存 / 增量保存再次运行

自动保存触发，或增量保存循环遍历到这两个区块：

- 区块 A：`saveChunkIfNeeded()` → 冷却未过（仍在 10 秒内）→ **跳过**
- 区块 B：无冷却或冷却已过 → 保存成功 → 生成 `B_new_nbt`（含潜影盒）

此时持久化状态已经改变：

- 区块 A：磁盘上是 `A_old_nbt`（发射器含潜影盒）
- 区块 B：磁盘上是 `B_new_nbt`（着陆位置有潜影盒）

T = 3：主线程被红石卡死 + Watchdog 强退

主线程冻结后不再进行新的序列化。
`IOWorker` 后台线程继续将 `pendingWrites` 中的剩余 NBT 写出。
Watchdog 检测超时，`System.exit(1)`。

注意：状态分叉在 T = 2 阶段就已经形成 —— 区块 A 因冷却被跳过，区块 B 已被保存为新快照。
T = 3 的 Watchdog 强退只是阻止了后续可能修正这个分叉的保存机会（例如冷却过期后的下一次增量保存）。
此外，由于 `RegionFile.write()` 不调用 `fsync`（见第 5 节），JVM 退出时 OS page cache 中尚未刷到物理磁盘的数据也可能丢失，但这只是分叉的附加因素，不是核心原因。

重启后

区块 A 从磁盘加载：发射器含潜影盒（旧快照）。
区块 B 从磁盘加载：着陆位置有潜影盒（新快照）。
潜影盒被复制。

---

slug: pistons
title: 活塞
title-en: Pistons
author: Molforte
co-authors:

- Arcadi4
- BFladderbeanawa
- tanhHeng
- Twisuki

- hotpad100c
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-06T18:18:36+08:00'
  index: 2
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

1 机制

基础部分

- 活塞的组成
- 无头活塞的创建
- 活塞的自检触发方式与红石信号判定 (QC 半连接性)
- 活塞尝试移动方块的结构分析与通俗逻辑
- 活塞的更新顺序与方块事件响应

进阶部分

- 活塞自检与信号判定的源码调用
- b36 转化顺序与推拉结构分析
- 方块移动底层实现与哈希表运用
- b36 到位与实体运算

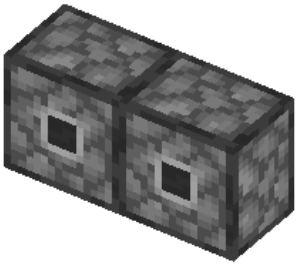
1.1 活塞的组成

完整意义上的活塞由两部分组成，分别是**活塞底座**与**活塞头**。

活塞底座完全控制活塞头，并且活塞由两种状态：`收回` 与 `伸出`。通常情况下，当活塞处于被激活状态时，其会同时更改活塞底座为伸出状态，并出现活塞头。当活塞处于收回状态时，其活塞头也随即消失。也有部分手段可以创造出只有活塞底座，没有活塞头的**无头活塞**；也可以通过某些手段创造出只有活塞头没有活塞底座的活塞。

1.2 无头活塞

1.2.1 无头活塞的创建



所有无头活塞的创建本质上只有两种办法，分别是：

1. 在活塞伸出时，活塞头处于 b36 状态时使用 TNT 等手段移除 b36
2. 通过特定手段让未伸出的活塞触发收回事件。本内容会在后续更详细的阐述

1.2.2 无头活塞的基本机制

活塞头其实在活塞底座看来依然存在，它就位于活塞底座的正前方。注意，活塞底座认为自己正前方是活塞头与自己正前方是否真的是活塞头无关。如果更新未被充能的无头活塞，无头活塞就会自检并收回。

1.3 活塞的自检机制

1.3.1 触发方式

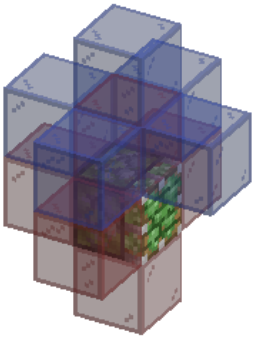
活塞当且仅当以下两种情况发生时触发自检：

1. 被放置 (包括被玩家放置，或被其他活塞推拉到位放置)
2. 受到 NC 更新

必须强调的是，活塞的自检是可以发生在任意阶段的。换言之，其自检是瞬时的。当上述情况发生时，活塞会立即自检并通过一定的逻辑尝试推出或收回。这里的 “尝试” 并不代表一定会成功执行。

1.3.2 红石信号的检查与 QC 充能

自检的本质是为了确认活塞是否需要动作，因此它会先判断自己是否被充能。



活塞检查信号的范围包括：

- 自身一阶毗邻 (除推动方向外) 的所有方块是否正在输出红石信号
- 自身**正上方一格方块为核心**的一阶毗邻方块是否正在输出红石信号

由于活塞会检查其上方方块的毗邻充能情况，这就导致了活塞可以通过上方的空间被 “隔空” 充能。这种充能无法直接对活塞发出 NC 更新，因此需要额外的 NC 更新来更新它，这就是活塞的半连接性 (Quasi-Connectivity, 简称 QC) 的来源。这也是为什么 BUD 只能检查 NC 更新。

1.3.3 自检逻辑通俗解释

活塞自检时，总是会遵循以下行为树：

1. 应伸出且未伸出：创建一个分析实例尝试分析前方的推动结构。如果分析失败 (超过推动上限，有不可推动方块阻挡等)，则不动作；如果分析成功，则给自己当前位置添加一个**伸出**的方块事件
2. 不应伸出且已伸出：活塞需要收回
    - 如果前方是一个正在伸出的同向移动的 "移动的活塞"(Moving_Piston, 或称 B36,36 号方块, 后续均简称 b36)，且当前刻不会到位 (进度 < 50%)，则触发**瞬推收回**，添加对应方块事件
    - 否则，执行**普通收回**，添加对应方块事件。

1.4 活塞的推拉与可推动方块

1.4.1 可推动方块判定

一个方块要是能被活塞推动，必须同时满足以下要求:

1. 在世界有效高度范围内
2. 不能是黑曜石，哭泣的黑曜石，重生锚，或强化深板岩等硬编码不可推动方块
3. 方块硬度不能为 - 1
4. 如果是带釉陶瓦，被推动方向必须与活塞推动方向一致

5. 该方块不能带有方块实体 (如箱子、熔炉、漏斗)

1.4.2 尝试移动方块与结构分析

活塞在尝试推动时，会动态维护两个列表：**推动方块列表**与**破坏方块列表**。

它会以一定的顺序去顶前方将要推动的方块和将要破坏的方块结构。活塞只会注意在推动轴方向的可破坏方块，若方块附着在推动结构的侧面，则会在稍后被更新并掉落，而不会进入破坏列表。

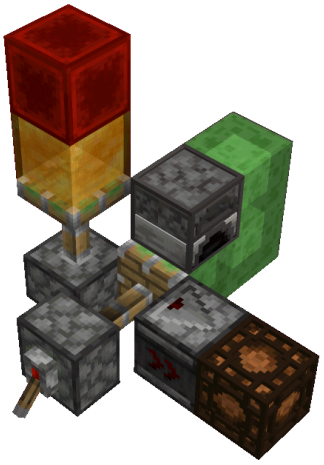
> *提示：由于结构分析极其复杂 (直线分析与分支分析交替入栈)，人工分析极为困难。强烈建议使用Fallen_Breath 的PistOrder 模组来直接在游戏内查看活塞尝试推动的方块到位顺序。*

1.5 活塞的更新顺序

活塞的更新顺序总的来说分为五种情况：

- 活塞 / 粘性活塞只伸出
    1. 同活塞 / 粘性活塞推动方块，只是移动方块为空
- 活塞 / 粘性活塞只收回
    1. 创建活塞底座 b36，发出 PP 更新和 NC 更新
    2. 移除活塞头，发出 PP 更新和 NC 更新
- 活塞 / 粘性活塞推动方块
    1. 移除破坏方块位置的方块
    2. 在移动的目标位置创建 b36，发出 PP 更新
    3. 创建活塞头 b36，发出红石粉 prepare 更新与 PP 更新
    4. 在破坏方块位置发出红石粉 prepare 更新和 NC 更新
    5. 在移动原位置发出 NC 更新
    6. 在活塞头位置发出 NC 更新
    7. 在活塞底座发出 PP 更新和 NC 更新。
- 粘性活塞收回方块
    1. 创建活塞底座 b36，发出 PP 更新和 NC 更新
    2. 移除破坏方块位置的方块
    3. 在移动目标位置创建 b36，发出 PP 更新
    4. 移除未被覆盖的方块，发出红石粉 prepare 更新，PP 更新
    5. 移动原位置，发出 NC 更新
- 粘性活塞收回失败
    1. 创建活塞底座 b36 方块，发出 PP 更新和 NC 更新

我们发现，这个行为中不具备任何在活塞头的更新，也就是我们常说的粘性收回方块失败无更新。这里给几个实例：



1.6 B36 与方块到位

1.6.1 常规到位

在每个游戏刻的方块时阶段，b36 会给自己的推动进度 + 0.5

- 0gt BE：开始推动，创建 b36，此时进度为 0
- 0gt TE：进度 + 0.5，进行实体位移运算
- 1gt TE：进度 + 0.5，进行实体位移运算，此事总进度达到 1.0
- 2gt TE：发现进度已经大于等于 1.0，b36 变回原来方块，发出最终的 PP 和 NC 更新。

若我们从 0gt AT 开始观测的话，在 3gt AT 才算完全完成方块到位，因此就有了所谓活塞 3gt 延迟的说法

1.6.2 瞬时到位

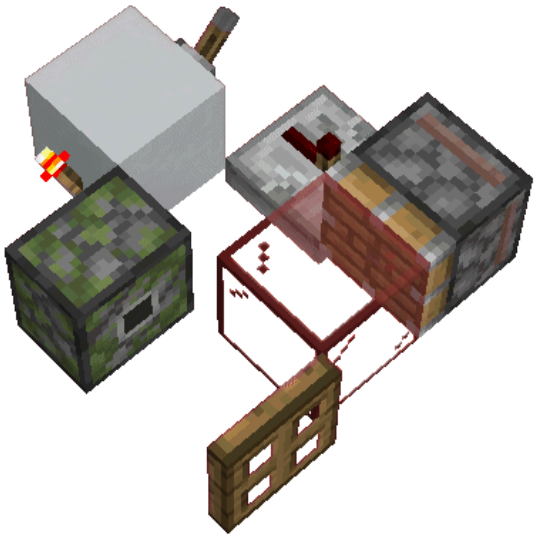
当活塞触发瞬推收回时，方块不会经理常规的 TE 阶段累加，而是直接强制到位。触发条件为：

- 收回事件，且活塞头位置当前方块时 b32.
- 收回事件，粘性活塞伸出方向一格外为 b36 且该 b36 正在朝活塞推动轴同方向移动。

这里举两个例子做演示：

活塞头位置为 b36

我们看如下的一个实例：

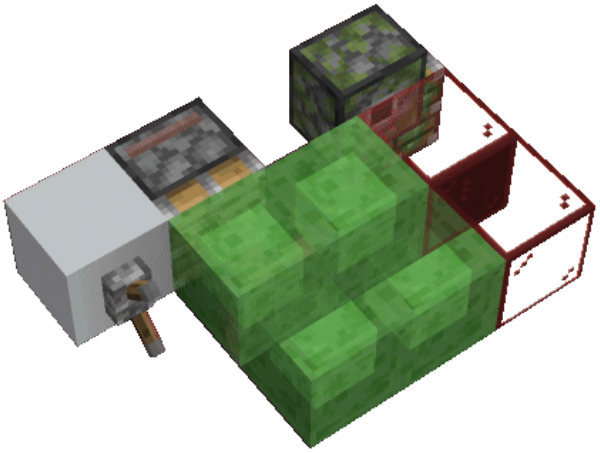


在这个实例中，红色玻璃被瞬间到位，这是从观测角度而言的，而从理论上，我们给出如下解释：

中继器的优先级高于红石火把，所以普通活塞先添加方块事件，无头粘性活塞再添加方块事件，尝试拉回更前方 1 格的方块（实际上此时 actionType = 1）

但是，在 BE 阶段，由于 b36 的插入，粘性活塞拉回受阻，并不会破坏这个 b36，此时活塞头位置为 b36，触发瞬时收回，红色玻璃瞬时到位。

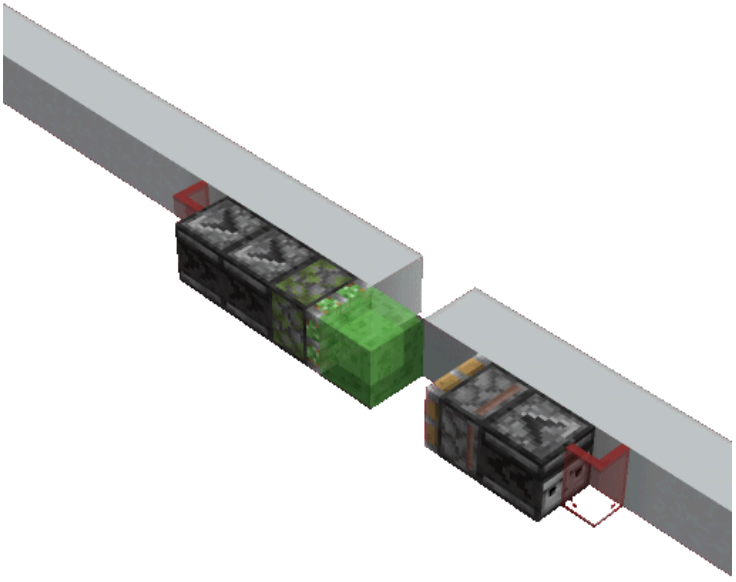
粘性活塞伸出方向一格外为 b36 且 b36 移动方向与活塞移动方向一致



在以上实例中，红色玻璃被瞬时到位，是因为：

活塞推动粘液棒子，红色玻璃被移动，发出 NC 更新，触发无头粘性活塞的自检。此时，粘性活塞可以成功拉回，并且在伸出方向一格外存在 b36，该 b36 推动方向与粘性活塞推动方向是一致的，因此被瞬时到位。

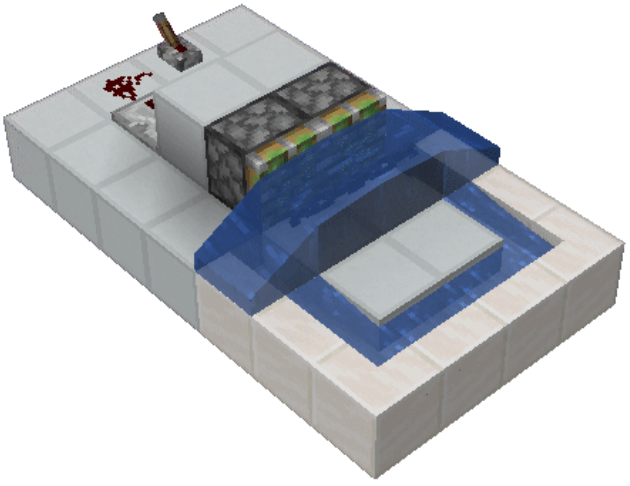
当然，值得注意的是，这种瞬时到位只会影响一格方块，也就是粘性活塞面前的方块，其余方块仍然会被转化为 b36，所以，我们可以利用这一点稍微解读一下一种在许多人看来是魔法的装置：



在这个装置中，当拉杆被拉下，粘性活塞会在方块到位 1gt 后移动它侧翼的方块，此时这个方块被粘液块带着向前推动，但由于粘性活塞是被侦测器激活的，该粘液块被提前到位，被活塞回推，而此时这个方块尚处于 b36 的状态，因此被留在了被带过去的位置，这样就实现了对方块流的延长。

瞬时到位会将进度直接设为 1.0，移除 b36 方块实体，直接替换为原本的方块并发出更新。值得注意的是，瞬推不会附带对周围实体的位移推拉运算。

值得注意的是，由于笨蛋麻将并没有在这部分写任何实体运算或对含水方块的特殊判断，所以瞬推不会影响实体，也不会消除含水状态。但是值得注意的是，如果在此前经历了 TE 阶段的实体运算，在此前的实体运算会被保留，仅有当前 gt 的 TE 阶段的实体运算会被忽略。



1.7 活塞自检逻辑与源码分析

1.7.1 活塞的自检函数

活塞的自检由以下方法触发，其核心在于它们只在服务（`!world.isClient`）端且满足特定条件时，调用统一的`trymove`方法:

```
//被玩家放置public void onPlaced(World world, BlockPos pos, BlockState state, LivingEntity placer, ItemStack itemStack) {
if (!world.isClient) {        this.tryMove(world, pos, state);    }}//被 NC 更新public void neighborUpdate(BlockState state,
World world, BlockPos pos, Block sourceBlock, BlockPos sourcePos, boolean notify) {    if (!world.isClient) {
this.tryMove(world, pos, state);    }}//被其他东西放置public void onBlockAdded(BlockState state, World world, BlockPos pos,
BlockState oldState, boolean notify) {    if (!oldState.isOf(state.getBlock())) {        if (!world.isClient &&
world.getBlockEntity(pos) == null) {            this.tryMove(world, pos, state);        }    }}
```

1.7.2 信号判定的源码`SHOULDEXTEND`

活塞检查两个核心位置的一阶毗邻。遍历顺序时严格先检查活塞本身再检查上方核心的一阶毗邻，检查毗邻的顺序严格符合 NC 更新顺序。

```
private boolean shouldExtend(RedstoneView world, BlockPos pos, Direction pistonFace) {    // 遍历所有方向（上、下、北、南、东、
西）    for (Direction direction : Direction.values()) {        if (direction != pistonFace &&
world.isEmittingRedstonePower(pos.offset(direction), direction)) {            return true;// 如果活塞所在的一格接受到红石信号，活
塞应当伸展        }    }    if (world.isEmittingRedstonePower(pos, Direction.DOWN)) {        return true;  // 如果下方有红石信
号，活塞应当伸展    } else {        // 如果下方没有红石信号，检查活塞上方的方块        BlockPos blockPos = pos.up(); // 获取活塞上方的方
块位置（这是为了 QC 充能准备的）        for (Direction direction2 : Direction.values()) {            // 如果当前方向不是向下（活塞本体
所在的位置），并且该方向的方块发出红石信号            if (direction2 != Direction.DOWN &&
world.isEmittingRedstonePower(blockPos.offset(direction2), direction2)) {                // 如果活塞上方的一格接受到红石信号，活塞
应当伸展                return true;            }        }        // 如果没有任何方向的方块发出红石信号，活塞不应伸展        return
false;    }}
```

1.7.3 尝试移动的源码`TRYMOVE`

`tryMove`决定了方块事件的种类（`actionType`），0 为伸出，1 为收回，2 为瞬推收回

```
private void tryMove(World world, BlockPos pos, BlockState state) {    // 获取活塞的朝向（通过状态中的 FACING 属性）    Direction facing = (Direction)state.get(FACING);    // 判断是否有红石信号    boolean shouldExtend = this.shouldExtend(world, pos, facing);    // 如果应该伸出且活塞当前没有伸出（EXTENDED 属性为 false）    if (shouldExtend && !(Boolean)state.get(EXTENDED)) {    // 创建一个 PistonHandler 对象来尝试推动方块        // 如果成功推动，则添加一个方块事件        if ((new PistonHandler(world, pos, facing, true)).calculatePush()) {            world.addSyncedBlockEvent(pos, this, 0, facing.getId());        }    }    // 如果不应该伸出且活塞当前处于伸出状态（EXTENDED 属性为 true）    else if (!shouldExtend && (Boolean)state.get(EXTENDED)) {        // 计算活塞头前方的位置（当前活塞位置向活塞推出朝向进行 2 个单位的偏移）        BlockPos pistonHeadFront = pos.offset(facing, 2);        // 获取目标位置的方块状态        BlockState headFrontState = world.getBlockState(pistonHeadFront);        // 初始化状态码（默认为 1）    // 状态码的代表状态:        // 0: 伸出        // 1: 收回        // 2: 瞬推收回        int actionType = 1;        if (headFrontState.isOf(Blocks.MOVING_PISTON) // 如果目标方块是一个正在移动的活塞                && headFrontState.get(FACING) == facing //并且其朝向与当前活塞相同                && world.getBlockEntity(pistonHeadFront)                        instanceof PistonBlockEntity pistonBlockEntity // 如果方块实体是移动的活塞（b36）                && pistonBlockEntity.isExtending()// 如果目标 b36 正在伸出                && (pistonBlockEntity.getProgress(0.0F) < 0.5F //如果伸出进度小于 50%                        || world.getTime() == pistonBlockEntity.getSavedWorldTime()                        || ((ServerWorld)world).isInBlockTick())        ) {            actionType = 2;// 应该瞬推收回        }        // 添加方块事件        world.addSyncedBlockEvent(pos, this, actionType, facing.getId());    }}
```

1.8 B36 转化与结构分析源码

> *温馨提示，这部分是本章节最为复杂的一部分之一，如果比较懒可以直接使用 pistOrder 模组而并不需要真正学习本部分在说的内容*

```
public boolean calculatePush() {    // 清空已移动的方块和已破坏的方块列表，以便重新计算    this.movedBlocks.clear();    this.brokenBlocks.clear();    // 获取活塞将要推动的起始点    BlockState pistonHeadFrontState = this.world.getBlockState(this.posTo);    // 如果起始点不能推动    if (!PistonBlock.isMovable(pistonHeadFrontState, this.world, this.posTo, this.motionDirection, false, this.pistonDirection)) {        // 如果方块不可推动且当前处于缩回状态，并且方块行为为 DESTROY，则将其标记为破坏        if (this.retracted && pistonHeadFrontState.getPistonBehavior() == PistonBehavior.DESTROY) {            this.brokenBlocks.add(this.posTo); // 添加到破坏列表            return true; // 活塞仍然可以进行操作（通过破坏方块）        } else {            return false; // 活塞无法推动且无法破坏目标方块        }    }    // 如果目标方块可推动，但无法添加直线结构，无法推动    else if (!this.tryMove(this.posTo, this.motionDirection)) {        return false;    }    else {        // 遍历所有被标记为需要移动的方块        for (int i = 0; i < this.movedBlocks.size(); ++i) {            BlockPos posToValidate = (BlockPos) this.movedBlocks.get(i);            // 如果当前方块是粘性方块，尝试移动它周围的粘连方块            if (isBlockSticky(this.world.getBlockState(posToValidate)) && !this.tryMoveAdjacentBlock(posToValidate)) {                return false; // 如果移动周围粘连方块失败，则整体推动失败            }        }        return true; // 所有方块都成功移动或处理    }}
```

当活塞需要推动时，会先执行以上代码，这段代码是用于分析移动结构的。这里需要更多更为详细的表述以辅助各位读者理解：`movedBlocks`和`brokenBlocks`实际是两个`arrayList`。没有了解过`Java`语言的读者可以简单理解为一个可以随意插入，取出的列表。这段代码本质在按照以下顺序执行操作：

1. 清空`movedBlocks`和`brokenBlocks`两个列表。
2. 检查其起始位置是否可以推动
   - 如果不可推动，则检查其对活塞行为的响应
     - 如果可破坏，则将该方块添加至`brokenBlocks`列表，推动成功
     - 如果不可破坏，则推动失败
   - 如果可推动，则尝试添加其直线结构（我们稍后会解析活塞是如何分析直线结构的）
     - 如果无法添加其直线结构，则推动失败
     - 如果成功添加其直线结构，则遍历所有需要推动的方块
       - 如果存在粘性方块（粘液块和蜂蜜块），则尝试添加分支
         - 若添加失败，则推动失败
     - 如果以上均没有失败，则推动成功

Mojang 在这里通过两个方法实现了对直线结构的分析和对分支结构的分析。分别称为`tryMove`[1]和`tryMoveAdjacentBlock`。

1.8.1 直线结构分析 `TRYMOVE`

首先先明确这个方法会在何种情况下被调用：

1. 这个方块直接被方块推动
2. 这个方块直接被粘性方块带动

值得注意的是，这两种方法是严格互相独立的，这意味着这两种情况之间不会有任何重复。

当因第一种情况被调用时，这个方块本身一定不是一个不可推动的方块，详情可以回看上文 5.4.2 部分。当因为第二种情况调用时，这个方块本身如果是一个不可推动的方块，考虑到尝试带动它的方块是一个粘性方块，则没有必要考虑对其进行分析。(实际上也就是下图中的这种情况)

在进行了以上分析后，我们就可以正式的看看本方法的第一部分代码。

```
BlockState stateToValidate = this.world.getBlockState(posToValidate);// 如果方块是空气方块，直接返回 true，因为不需要处理空气if (stateToValidate.isAir()) {    return true;}// 如果方块不可移动，根据当前条件直接返回 trueelse if (!PistonBlock.isMovable(stateToValidate, this.world, posToValidate, this.motionDirection, false, dir)) {    return true;}// 如果方块位置是活塞的初始位置，直接返回 trueelse if (posToValidate.equals(this.posFrom)) {    return true;}// 如果方块已经在已移动列表中，直接返回 true，避免重复计算else if (this.movedBlocks.contains(posToValidate)) {    return true;}
```

这段代码的逻辑如下：

1. 如果活塞在试图推动空气，那么可以推动
2. 如果这个方块不可移动，那么可以推动（看看上文我论证了这部分代码的合理性）
3. 如果这个方块就是活塞本身，那么可以推动
4. 如果这个方块已经在推了，那么可以推动

```
while (isBlockSticky(stateToValidate)) {// 获取黏性方块在相反方向上的下一个方块    BlockPos blockBehind = posToValidate.rearBlockCount(this.motionDirection.getOpposite(), rearBlockCount);    BlockState currentStateSaved = stateToValidate; // 当前方块状态    stateToValidate = this.world.getBlockState(blockBehind); // 下一个方块状态    // 如果下一个方块是空气、不粘连、不可移动或是活塞初始位置，结束循环    if (stateToValidate.isAir()    || !isAdjacentBlockStuck(currentStateSaved, stateToValidate)    || !PistonBlock.isMovable(stateToValidate, this.world, blockBehind, this.motionDirection, false, this.motionDirection.getOpposite())            || blockBehind.equals(this.posFrom))    {            break;    }++rearBlockCount; // 增加偏移距离    if (rearBlockCount + this.movedBlocks.size() > 12) {        // 如果移动的方块总数超过 12 个，返回 false        return false;    }}
```

接下来，游戏首先定义了一个变量`offset`，它实际上表示了活塞正在尝试推动的方块。如果此时`movedBlocks`列表的大小 + 1 已经大于最大推动数量上限，则推动失败。（算上这个方块就超出可推动大小的上限了）

游戏会从这个方块开始朝着推动方向的相反方向开始递推方块，寻找直线拉动中断的位置，当这个位置是空气或者与此粘块不互粘的或者无法移动的方块，或者是活塞本身的一列方块便是粘块拉动的部分 (这部分存在的意义是处理以下这种情况:)

如果寻找到新直线结构的拉动部分加上原来的`movedBlocks`列表大小已经大于 12，则推动失败。

接下来，将这部分拉动的部分添加至移动方块列表中，添加方向为推动方向的反向，即最前方的方块先添加，最后方的方块后添加。

```
while (true) {    BlockPos blockFront = posToValidate.offset(this.motionDirection, frontBlockCount); // 下一个方块的位置    int l = this.movedBlocks.indexOf(blockFront); // 检查方块是否已在移动列表中    if (l > -1) {        // 如果方块已经在移动列表中，调整移动列表的顺序        this.setMovedBlocks(succeededCount, l);        // 遍历调整后的列表，检查所有黏性方块并处理其连接方块for (int m = 0; m <= l + succeededCount; ++m) {        BlockPos stuckBlock = (BlockPos) this.movedBlocks.get(m);        if (isBlockSticky(this.world.getBlockState(stuckBlock))            && !this.tryMoveAdjacentBlock(stuckBlock)) {return false;            }        }        return true; // 所有相关方块处理成功    }    // 获取当前偏移位置的方块状态stateToValidate = this.world.getBlockState(blockFront);    // 如果当前方块是空气，推动成功    if (stateToValidate.isAir()) {            return true;            }            // 如果当前方块不可移动或是活塞的初始位置，返回 falseif (!PistonBlock.isMovable(stateToValidate, this.world, blockFront, this.motionDirection, true, this.motionDirection)|| blockFront.equals(this.posFrom)) {            return false;            }            // 如果当前方块是可破坏的，将其添加到破坏列表            if (stateToValidate.getPistonBehavior() == PistonBehavior.DESTROY) {this.brokenBlocks.add(blockFront);                        return true;                    }// 如果移动列表已达到 12 个方块，返回 false                    if (this.movedBlocks.size() >= 12) {            return false;            }        // 将当前方块添加到移动列表        this.movedBlocks.add(blockFront);                ++succeededCount;// 增加移动计数                ++frontBlockCount; // 增加偏移量}
```

在完成拉动部分的分析后，游戏会接着处理推动部分。从我们一开始开始分析的方块沿移动方向向前寻找会被推动的方块，如果被被遍历的方块已经在推动列表中时，意味着这个直线结构结束。再往前遍历会与前面已分析的结构发生碰撞，此时将重新排列推动方块列表。

遍历已经走完的方块（取决于添加顺序），如果其是粘性方块，进入分支结构的检查，寻找侧面粘动的方块，如果侧面粘动使其推动失败，返回推动失败的结果。

如果正前方移动的方块是空气，直线结构结束。如果正前方方块是不可移动方块或活塞本身，推动失败。如果正前方移动的方块是可破坏方块，将该方块加入可破坏方块列表，直线结构结束。

将所有方块加入移动方块列表，检查移动方块列表，若其大小大于 12，则达到推动上限，推动失败。

1.8.2 尝试添加分支 `TRYMOVEADJACENTBLOCK`

```
private boolean tryMoveAdjacentBlock(BlockPos pos) {    // 获取指定位置的方块状态    BlockState targetBlockState = this.world.getBlockState(pos);    // 遍历所有方向    for (Direction direction : Direction.values()) {        // 只处理与推动方向轴线不同的方向        if (direction.getAxis() != this.motionDirection.getAxis()) {            // 获取相邻位置的方块            BlockPos adjacentBlockPos = pos.offset(direction);            BlockState adjacentBlockState = this.world.getBlockState(adjacentBlockPos);            // 如果当前方块和相邻方块是粘性方块，尝试移动相邻方块            if (isAdjacentBlockStuck(adjacentBlockState, targetBlockState) && !this.tryMove(adjacentBlockPos, direction)) {                return false; // 如果相邻方块无法移动，返回 false            }        }    }    // 如果所有相邻方块都成功移动，返回 true    return true;}
```

这里的形参输入的方块总是可粘方块，因此执行以下逻辑：

1. 遍历其周围所有方块，这是通过遍历方向实现的，顺序和 NC 更新一致
2. 由于与推动轴方向相同方向的两个相邻方块的分析是不必要的，故在这里排除。侧面的方块如果与此方块互粘，则尝试分析其直线结构，如果直线结构添加失败，逐级返回失败结果。

1.8.3 移动方块列表处理

`movedBlocks` 和 `brokenBlocks` 直接倒过来就是 b36 到位顺序和破坏顺序。原因我们会在稍后详细解释。

1.8.4 B36 到位顺序的分析

我们来分析一下如下实例:

本图的推动轴是 + z 向 - z 方向

首先分析直线结构，活塞头前的两格粘液块最先被添入移动方块列表。

由于两格均为可粘方块，最贴近活塞的粘液块尝试分析其 - x 方向的普通方块，并将其添加进推动方块列表，再分析 + y 方向的粘液块，将其加入推动方块列表，其会有下一次调用，但此时这个调用任务会被压入系统栈。直线方块的第二格压栈分析其 - x 方向的粘液块，这个粘液块发现自己和后面的普通方块冲突了，于是处理碰撞，合成的新列表分别是普通方块下标为为 3，粘液块下标为 4。粘液块检查周围方块的任务被压入系统栈，等待下一轮调用。最后以此类推，最终可以得到整个结构的推动顺序。

当然你要是比较懒不如直接 pistorder。

1.9 活塞响应事件与移动方块源码

1.9.1 事件的统一入口 `onSyncedBlockEvent`

```
public boolean onSyncedBlockEvent(BlockState state, World world, BlockPos pos, int actionType, int directionData) {    // 获取块的方向（例如，活塞朝向的方向）     Direction facing = (Direction)state.get(FACING);    // 获取激活的方块状态（表示方块正在扩展）     BlockState extendedState = (BlockState)state.with(EXTENDED, true);    // 如果当前不是客户端，进行一些服务器端操作     if (!world.isClient) {        // 检查是否需要扩展活塞（是否有红石信号）         boolean shouldExtend = this.shouldExtend(world, pos, facing);        // 如果需要扩展并且事件类型是收缩，更新方块状态并返回         if (shouldExtend && (actionType == 1 || actionType == 2)) {            world.setBlockState(pos, extendedState, 2);            return false;        }        // 如果不需要扩展并且事件类型是 0，直接返回 false         if (!shouldExtend && actionType == 0) {            return false;        }    }    // 事件类型为 0：活塞伸展事件     if (actionType == 0) {        // 如果移动方块失败，返回 false         if (!this.move(world, pos, facing, true)) {            return false;        }        // 设置方块为扩展状态，并播放伸展声音         world.setBlockState(pos, extendedState, 67);        world.playSound((PlayerEntity)null, pos, SoundEvents.BLOCK_PISTON_EXTEND, SoundCategory.BLOCKS, 0.5F, world.random.nextFloat() * 0.25F + 0.6F);        world.emitGameEvent(GameEvent.BLOCK_ACTIVATE, pos, Emitter.of(extendedState));    }    // 事件类型为 1 或 2：活塞收缩事件     else if (actionType == 1 || actionType == 2) {        // 获取活塞前方的方块实体         BlockEntity blockEntityInFront = world.getBlockEntity(pos.offset(facing));        // 如果该位置有活塞方块实体，完成收缩操作         if (blockEntityInFront instanceof PistonBlockEntity) {            ((PistonBlockEntity)blockEntityInFront).finish();        }        // 创建移动的活塞方块状态         BlockState pistonHeadState = Blocks.MOVING_PISTON.getDefaultState().with(PistonExtensionBlock.FACING, direction)            .with(PistonExtensionBlock.TYPE, this.sticky ? PistonType.STICKY : PistonType.DEFAULT);        // 设置新的活塞方块状态，并创建新的方块实体         world.setBlockState(pos, pistonHeadState, 20);        world.addBlockEntity(PistonExtensionBlock.createBlockEntityPiston(pos, pistonHeadState, (BlockState)this.getDefaultState().with(FACING, Direction.byId(directionData & 7)), direction, false, true));        world.updateNeighbors(pos, pistonHeadState.getBlock());        pistonHeadState.updateNeighbors(world, pos, 2);        // 如果是粘性活塞，执行特殊逻辑         if (this.sticky) {            // 计算目标位置             BlockPos targetpos = pos.add(facing.getOffsetX() * 2, facing.getOffsetY() * 2, facing.getOffsetZ() * 2);            BlockState targetState = world.getBlockState(targetpos);            boolean hasMovingPistonInFront = false;            // 如果目标位置已有移动活塞方块，尝试完成它的扩展             if (targetState.isOf(Blocks.MOVING_PISTON)) {                BlockEntity blockEntity2 = world.getBlockEntity(targetpos);                if (blockEntity2 instanceof PistonBlockEntity) {                    PistonBlockEntity pistonBlockEntity = (PistonBlockEntity)blockEntity2;                    if (pistonBlockEntity.getFacing() == facing && pistonBlockEntity.isExtending()) {                        pistonBlockEntity.finish();                        hasMovingPistonInFront = true;                    }                }            }            // 如果没有特殊情况，移除目标方块或继续移动             if (!hasMovingPistonInFront) {                if (actionType != 1 || targetState.isAir() || !isMovable(targetState, world, targetpos, facing.getOpposite(), false, direction)                    || targetState.getPistonBehavior() != PistonBehavior.NORMAL && !targetState.isOf(Blocks.PISTON) && !targetState.isOf(Blocks.STICKY_PISTON)) {                    world.removeBlock(pos.offset(facing), false);                } else {                    this.move(world, pos, facing, false);                    }            }        }        // 如果不是粘性活塞，直接移除目标方块         else {            world.removeBlock(pos.offset(facing), false);        }        // 播放收缩声音         world.playSound(null, pos, SoundEvents.BLOCK_PISTON_CONTRACT, SoundCategory.BLOCKS, 0.5F, world.random.nextFloat() * 0.15F + 0.6F);        world.emitGameEvent(GameEvent.BLOCK_DEACTIVATE, pos, Emitter.of(pistonHeadState));    }    // 返回 true 表示事件成功处理     return true;}
```

服务端

如果不是客户端，那么会执行几个服务端特有的事件：

- 检查激活情况，如果激活，并且方块事件为收回或者瞬推收回，将活塞设置为伸出状态，并进行寻路更新，方块事件执行失败。
- 如果没有激活并且方块事件为伸出，方块事件执行失败。

伸出事件

> *注意，本部分无论客户端，服务端都必须执行*

活塞会尝试移动前方方块，这是通过 `move` 方法实现的，我稍后会补充这个方法的细节。如果移动失败，则活塞执行事件失败。

接下来，将活塞设置为伸出状态，此处 `bitflag` 为 `0b01000011` 放置移除更新，寻路更新，NC 更新

最后，播放活塞伸出的声音，方块事件执行成功

瞬间收回与收回事件

首先获取活塞头的方块实体，如果是 b36，那么使其瞬间到位。

接下来，将活塞设置为 b36，此处 `bitflag` 为 `0b00010100`，无 PP 更新，寻路更新

再将活塞设置成 b36 方块实体，先发出 NC 更新，再发出 PP 更新

如果这个活塞是一个粘性活塞，那么将活塞头伸出 1 格外的地方，如果它试图放置活塞头的位置的一格外是一个 b36，并且该 b36 的推动方向与活塞同向，那么让方块瞬间到位。

如果活塞行为不为瞬推收回，并且该方块不是空气，活塞可以移动前方一格的方块，而且这个方块可以被正常拉回或者这个方块是活塞或者粘性活塞，那么尝试拉回前方方块。如果前方一格的方块无法被拉动，那么删除活塞头位置的方块。

如果是普通活塞，直接删除活塞头位置的方块。

无论何种情况，播放活塞收回的声音。方块事件执行成功。

1.9.2 活塞移动方块 `MOVE`

```
// retract = 0: 收回// retract = 1: 伸出private boolean move(World world, BlockPos pos, Direction dir, boolean retract) {
// 活塞头的位置     BlockPos headPos = pos.offset(dir);     // 如果是收缩动作并且目标位置是活塞头     if (!retract &&
world.getBlockState(headPos).isOf(Blocks.PISTON_HEAD)) {         // 将活塞头设为空气         // flag = 0b00010100, 无 PP 更新, 寻路
更新 客户端         world.setBlockState(headPos, Blocks.AIR.getDefaultState(), 20);     }     // 创建 PistonHandler 实例, 用于计算推
动或拉回操作的合法性     PistonHandler pistonHandler = new PistonHandler(world, pos, dir, retract);     // 如果无法执行推动/拉回操作,
则无法移动     if (!pistonHandler.calculatePush()) {         return false;     } else {         // 使用哈希表存储需要移动的方块及其状
态, 便于后续处理         Map<BlockPos, BlockState> map = Maps.newHashMap();         // 创建移动方块列表并获取移动方块列表
List<BlockPos> blocksToMove = pistonHandler.getMovedBlocks();         // 用于记录需要移动的方块的原始状态         List<BlockState>
blocksOriginal = Lists.newArrayList();         // 遍历需要移动的方块, 将其原始状态保存到列表中, 并存储到 map 中         for (int i = 0;
i < blocksToMove.size(); ++i) {             BlockPos blockPos2 = (BlockPos) blocksToMove.get(i);             BlockState
blockState = world.getBlockState(blockPos2);             blocksOriginal.add(blockState);             map.put(blockPos2,
blockState);         }         // 获取所有需要破坏的方块位置列表         List<BlockPos> list3 = pistonHandler.getBrokenBlocks();
// 用于记录被破坏方块的状态         BlockState[] blockStates = new BlockState[blocksToMove.size() + list3.size()];         // 确定
移动方向         Direction direction = retract ? dir : dir.getOpposite();         // 处理被破坏的方块, 从后到前         int j = 0;
for (int k = list3.size() - 1; k >= 0; --k) {             BlockPos blockPos3 = (BlockPos) list3.get(k);             BlockState
blockState2 = world.getBlockState(blockPos3);             // 如果方块有方块实体, 则先处理掉落逻辑             BlockEntity
blockEntity = blockState2.hasBlockEntity() ? world.getBlockEntity(blockPos3) : null;             dropStacks(blockState2,
world, blockPos3, blockEntity);             // 将方块设置为空气并触发销毁事件             world.setBlockState(blockPos3,
Blocks.AIR.getDefaultState(), 18);             world.emitGameEvent(GameEvent.BLOCK_DESTROY, blockPos3,
Emitter.of(blockState2));             // 如果方块不是火类方块, 则添加破坏粒子效果             if (!blockState2.isIn(BlockTags.FIRE))
{                 world.addBlockBreakParticles(blockPos3, blockState2);             }             // 保存被破坏方块的状态
blockStates[j++] = blockState2;         }         // 处理需要移动的方块         for (int k = blocksToMove.size() - 1; k >= 0; --k)
{             BlockPos targetPos = (BlockPos) blocksToMove.get(k);             BlockState targetState =
world.getBlockState(targetPos);             // 移动到目标位置             targetPos = targetPos.offset(direction);
map.remove(targetPos);             // 设置目标位置为 MOVING_PISTON 状态, 用于动画处理             BlockState blockState3 =
(BlockState) Blocks.MOVING_PISTON.getDefaultState().with(FACING, dir);             world.setBlockState(targetPos,
blockState3, 68);             // 创建 b36 方块实体, 用于控制移动动画
world.addBlockEntity(PistonExtensionBlock.createBlockEntityPiston(targetPos, blockState3, (BlockState)
blocksOriginal.get(k), dir, retract, false));             blockStates[j++] = targetState;         }         // 如果是伸出操作, 添
加活塞头         if (retract) {             PistonType pistonType = this.sticky ? PistonType.STICKY : PistonType.DEFAULT;
BlockState blockState4 = (BlockState) ((BlockState) Blocks.PISTON_HEAD.getDefaultState().with(PistonHeadBlock.FACING,
dir)).with(PistonHeadBlock.TYPE, pistonType);             BlockState targetState = (BlockState) ((BlockState)
Blocks.MOVING_PISTON.getDefaultState().with(PistonExtensionBlock.FACING, dir)).with(PistonExtensionBlock.TYPE, this.sticky ?
PistonType.STICKY : PistonType.DEFAULT);             map.remove(headPos);             // 设置当前活塞位置为 MOVING_PISTON 状态
world.setBlockState(headPos, targetState, 68);             // 添加 b36 方块实体, 用于控制动画
world.addBlockEntity(PistonExtensionBlock.createBlockEntityPiston(headPos, targetState, blockState4, dir, true, true));
}         // 将 map 中剩余的方块设置为空气         BlockState blockState5 = Blocks.AIR.getDefaultState();         for (BlockPos
blockPos4 : map.keySet()) {             world.setBlockState(blockPos4, blockState5, 82);         }         // 更新邻居方块状态
for (Map.Entry<BlockPos, BlockState> entry : map.entrySet()) {             BlockPos blockPos5 = entry.getKey();
BlockState blockState6 = entry.getValue();             blockState2.prepare(world, blockPos5, 2);
blockState5.updateNeighbors(world, blockPos5, 2);             blockState5.prepare(world, blockPos5, 2);         }         // 更
新被破坏方块的邻居状态         for (int l = list3.size() - 1; l >= 0; --l) {             BlockState targetState =
blockStates[j++];             BlockPos blockPos5 = (BlockPos) list3.get(l);             targetState.prepare(world, blockPos5,
2);             world.updateNeighborsAlways(blockPos5, targetState.getBlock());         }         // 更新移动方块的邻居状态
for (int l = blocksToMove.size() - 1; l >= 0; --l) {             world.updateNeighborsAlways((BlockPos) blocksToMove.get(l),
blockStates[j++].getBlock());         }         // 如果是收缩动作, 更新活塞位置邻居         if (retract) {
world.updateNeighborsAlways(headPos, Blocks.PISTON_HEAD);         }         return true; // 如果执行成功, 返回 true     }}
```

活塞尝试移动方块时，会区分两种情况，伸出和收回。这里的`retract`布尔值代表了活塞的动作，当其值为`false`时，活塞收回，当其值为`true`时，活塞伸出，接下来，活塞会区分两种活塞动作，分别处理。

当活塞**尝试**收回时，游戏会首先检查**预期是**活塞头的位置是不是活塞头，如果是活塞头才执行收回。每次执行收回时，游戏都会先将活塞头的位置设置为空气（删除活塞头）。此时的`setBlockState`的传入`bitflag`为`0b00010100`，即无 PP 更新，和客户端的寻路更新。

在正式开始移动前，无论收回或伸出，游戏都会调用5.4.2部分分析的`calculatePush`方法，再次分析是否可以移动，如果不能成功移动，则活塞执行推动 / 拉动失败。

接下来，游戏会正式开始移动方块，它首先会逆序地处理破坏方块列表（`brokenBlocks`），如果被破坏的方块具备掉落物品的方块实体，那么掉落物品，再将被锚定破坏的方块（也就是当前正在处理的方块）设置为空气，此时无 PP 更新，发出寻路更新。

破坏了应该破坏的方块后，游戏会开始移动方块。从后往前遍历移动方块列表（`movedBlocks`），将该方块设置为 b36，`bitflag`为`0b01101000`，也就是放置移除更新，再向该方块添加 b36 的方块实体，并添加到`blockStates`列表中备用。

如果是伸出，就在活塞头位置创建 b36，放置移除更新，再向活塞头位置添加 b36 方块实体。

完成以上操作后，将未被覆盖到的方块全部设置为空气，并放置移除更新，寻路更新。此时无 PP 更新。再统一发出更新，对于哈希表中所有的方块，都先统一发出红石粉 prepare 更新，再给出方块更新（在方块原位置），如果是伸出，活塞头完成方块更新。

以上代码很好的解读了 b36 的添加为何是倒序与`movedBlocks`列表。另外由于哈希表是乱序的，所以这也解释了为什么在有些依赖 b36 到位顺序的机器卸载后会坏掉。（卸载后会丢失本来的索引）

1.10 到位的源码解析

1.10.1 普通到位`TICK`

> *这东西太简单了没什么好讲的，所以通俗解释都扔到前面去了(笑)，因此这里只放个代码供参考*

```
public static void tick(World world, BlockPos pos, BlockState state, PistonBlockEntity blockEntity) {        //记录当前 gt
blockEntity.savedWorldTime = world.getTime();    blockEntity.lastProgress = blockEntity.progress;    //如果推动已完成    if
(blockEntity.lastProgress >= 1.0F) {        //如果是客户端且当前 deathTick 没有超过 5        if (world.isClient && deathTicks <
5) {            //增加 deathTicks 计数            ++deathTicks;        } else {            //移除方块实体
world.removeBlockEntity(pos);            blockEntity.markRemoved();            //如果该方块为 b36            if
(world.getBlockState(pos).isOf(Blocks.MOVING_PISTON)) {                //给出 PP 更新                BlockState blockState =
Block.postProcessState(blockEntity.pushedBlock, world, pos);                //如果为空气                if
(blockState.isAir()) {                    //设置为对应方块，无更新，bitflag = 0b01010100;
world.setBlockState(pos, blockEntity.pushedBlock, 84);                    //更新或破坏该方块
Block.replace(blockEntity.pushedBlock, blockState, world, pos, 3);                } else {                    //如果是含水方块
if (blockState.contains(Properties.WATERLOGGED) && (Boolean)blockState.get(Properties.WATERLOGGED)) {
//移除含水方块中的水                    blockState = (BlockState)blockState.with(Properties.WATERLOGGED, false);
}                //设置为对应方块，给出 NC 更新，PP 更新，客户端更新，bitflag = 0b01000011
world.setBlockState(pos, blockState, 67);                    //自身受到 NC 更新                    world.updateNeighbor(pos,
blockState.getBlock(), pos);                }            }        }    }    //如果推动未完成    //推动进度 + 0.5    else {
float f = blockEntity.progress + 0.5F;        //进行实体运算        pushEntities(world, pos, f, blockEntity);
moveEntitiesInHoneyBlock(world, pos, f, blockEntity);        blockEntity.progress = f;        //如果推动进度 >=1，取为 1
if (blockEntity.progress >= 1.0F) {            blockEntity.progress = 1.0F;        }    }}
```

1.10.2 瞬时到位`FINISH`

```
public void finish() {    // 存在世界且进度未满或客户端运算    if (this.world != null && (this.lastProgress < 1.0F ||
this.world.isClient)) {        // 将进度设置为完成        this.progress = 1.0F;        this.lastProgress = this.progress;
// 移除方块实体        this.world.removeBlockEntity(this.pos);        this.markRemoved();        //如果此处为 b36 方块        if
(this.world.getBlockState(this.pos).isOf(Blocks.MOVING_PISTON)) {            // 如果是活塞臂            BlockState blockState;
if (this.source) {                // 设置为空气                blockState = Blocks.AIR.getDefaultState();            } else {
// 否则在 PP 更新后到位                blockState = Block.postProcessState(this.pushedBlock, this.world, this.pos);
}            // 设置方块，PP 更新，寻路更新，NC 更新            this.world.setBlockState(this.pos, blockState, 3);            //
自身受到方块更新            this.world.updateNeighbor(this.pos, blockState.getBlock(), this.pos);        }    }}
```

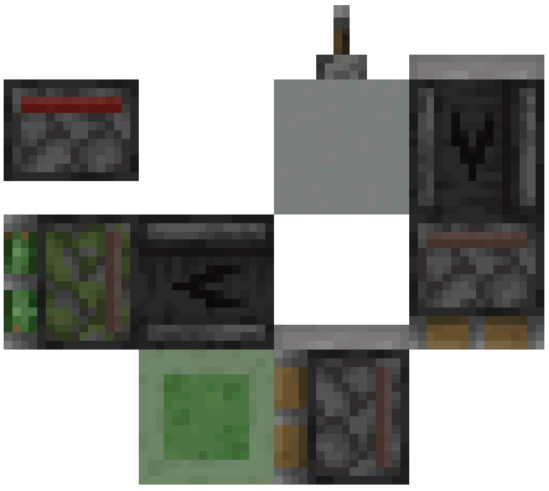
大部分内容前面也解释过了，所以这里直接略过。我们只看后半部分的 b36 特判看起。

在 1.20 中，只有活塞臂的`source`属性是`true`，其它都是`false`。因此当该方法被调用时，若该位置是目标方块为活塞臂的 b36，会直接变成空气。其余部分就不细说了，前面都讲过了。

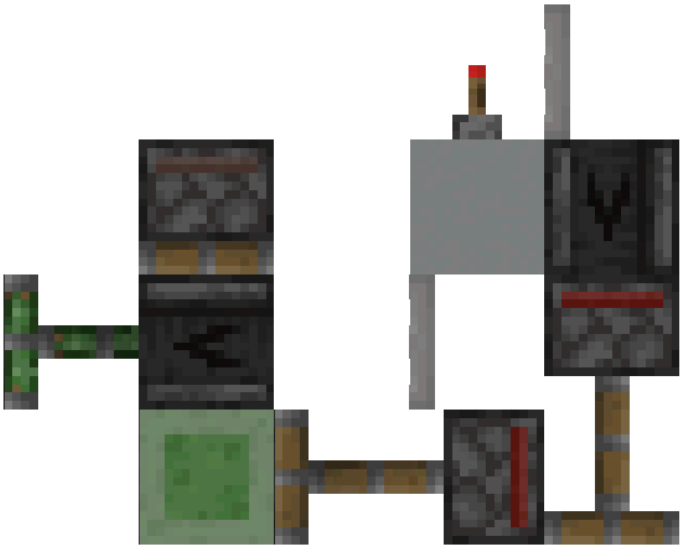
1.11 活塞头

> *本章其实没什么难度，但是由于除了前阵子Menggui233在PetrisAFE的视频中演示的装置外，未见得其它用途，故在这里放在了进阶的位置。*

我们首先观察如下装置：



现在，拉下拉杆



观察到活塞底座被换成了侦测器，而活塞头被保留。为什么?

我们首先回顾一下 b36 到位时发生了什么：

- 给出自身位置的 PP 更新
- 给出自身位置的一阶毗邻的 NC，PP 更新
- 给出自身位置的 NC 更新

因此，我们分别查看活塞头受到 PP 更新和 NC 更新会发生什么

1.11.1 活塞头存在的条件判定 `CANSURVIVE`

```
private boolean isFittingBase(BlockState baseState, BlockState extendedState) {    Block block = baseState.getValue(TYPE) == PistonType.DEFAULT ? Blocks.PISTON : Blocks.STICKY_PISTON;    return extendedState.is(block) && extendedState.getValue(PistonBaseBlock.EXTENDED) && extendedState.getValue(FACING) == baseState.getValue(FACING);}@Overridepublic boolean canSurvive(BlockState state, LevelReader level, BlockPos pos) {BlockState blockState = level.getBlockState(pos.relative(state.getValue(FACING).getOpposite()));    return this.isFittingBase(state, blockState) || blockState.is(Blocks.MOVING_PISTON) && blockState.getValue(FACING) == state.getValue(FACING);}
```

> `isFittingBase` 方法本质上说明，当且仅当一个活塞头的后方 (更确切地说，是推动方向的反方向 1 格，后续均以后方简称) 是正在伸展的活塞，且该活塞的伸出方向与活塞头一致活塞头可以被称之为 `适配`。而 `canSurvive` 则要求一个活塞头的存在是合法的，当且仅当该活塞头是 `适配` 的，或其后方是一个与活塞头推动方向相同的 b36。

1.11.2 活塞头的移除 `ONREMOVE`

```
@Overridepublic void onRemove(BlockState state, Level level, BlockPos pos, BlockState newState, boolean isMoving) {    if (!state.is(newState.getBlock())) {        super.onRemove(state, level, pos, newState, isMoving);        BlockPos blockPos = pos.relative(state.getValue(FACING).getOpposite());        if (this.isFittingBase(state, level.getBlockState(blockPos))) {            level.destroyBlock(blockPos, true);        }    }}
```

一句话总结：移除时如果状态为 `适配`，则连带着底座一起移除，并掉落活塞。

1.11.3 活塞头受到 `PP` 更新，`NC` 更新

```
@Overridepublic BlockState updateShape(BlockState state, Direction facing, BlockState facingState, LevelAccessor level, BlockPos currentPos, BlockPos facingPos) {    return facing.getOpposite() == state.getValue(FACING) && !state.canSurvive(level, currentPos)        ? Blocks.AIR.defaultBlockState()        : super.updateShape(state, facing, facingState, level, currentPos, facingPos);}@Overridepublic void neighborChanged(BlockState state, Level level, BlockPos pos, Block block, BlockPos fromPos, boolean isMoving) {    if (state.canSurvive(level, pos)) {        level.neighborChanged(pos.relative(state.getValue(FACING).getOpposite()), block, fromPos);    }}
```

PP 更新

活塞头实际上只关心来自后方的 PP 更新。当后方发生 PP 更新时，活塞头会调用 `canSurvive` 来检查自己是否合法，若不合法则立刻消失。

NC 更新

在任何情况下，活塞头均不会因受到 NC 更新而消失。当受到 NC 更新时，若自身合法，则将更新视为后方方块收到的 NC 更新。

1.11.4 案例分析

因此，对于在章首提到的案例：

- 0gt AT: 玩家拉下拉杆，活版门触发
- 2gt BE 深度 0: 粘性活塞被激活，更新无头活塞
- 2gt BE 深度 1: 无头活塞吃掉粘性活塞底座，粘液块 & 亮起侦测器被推动
- 4gt TE: 粘性活塞头先到位，此时检查后方是和自己同方向的 b36，自检通过。随后亮起侦测器 b36 到位，无更新。

因此侦测器成功取代了原本的活塞底座。

1.12 B36 相关的实体运算

```
//该方法用于处理活塞对实体的推动。它会获取 b36 在当前游戏刻的移动方向和距离，并找出路径上所有可能接触的实体。然后，它会精确计算每个实体与 b36 在移动方向上的最大重叠部分，并据此推动实体（同时增加微小位移 0.01 防卡入）。此过程还会特殊处理黏液块的弹射效果以及 b36 收回时对实体的影响。private static void pushEntities(World world, BlockPos pos, float f, PistonBlockEntity blockEntity) {     //获取 b36 移动的方向 Direction direction = blockEntity.getMovementDirection();     //计算在当前 tick，b36 需要移动的距离     double d = f - blockEntity.progress;     //获取 b36 的碰撞箱体素模型     VoxelShape voxelShape = blockEntity.getHeadBlockState().getCollisionShape(world, pos);     //检查 b36 是否有实际的碰撞体积。如果没有，就什么也不做     if (!voxelShape.isEmpty()) {     //获取 b36 该 tick 移动后的包围盒     Box box = offsetHeadBox(pos, voxelShape.getBoundingBox(), blockEntity);         //根据 b36 移动前后扫描过的区域，粗略筛选可能被推动的实体         List<Entity> list = world.getOtherEntities(null, Boxes.stretch(box, direction, d).union(box));         //如果扫描区域内有实体，则开始处理         if (!list.isEmpty()) {             //获取组成 b36 体素模型的所有子包围盒列表             List<Box> list2 = voxelShape.getBoundingBoxes();             //检查 b36 是否是黏液块             boolean bl = blockEntity.pushedBlockState.isOf(Blocks.SLIME_BLOCK);             Iterator var12 = list.iterator();             //遍历所有可能被影响的实体             while (true) {                 Entity entity;                 while (true) {                     if (!var12.hasNext()) {                         return;                     }                     entity = (Entity)var12.next();                     //检查实体是否忽略 b36 推动                     if (entity.getPistonBehavior() != PistonBehavior.IGNORE) {                         if (!bl) {                             break;                         }                         //如果是黏液块在推动，对于除服务端玩家外的实体，将其沿移动轴向的速度分量强制设为 1.0                         if (!(entity instanceof ServerPlayerEntity)) {                             Vec3d vec3d = entity.getVelocity();                             double e = vec3d.x;                             double g = vec3d.y;                             double h = vec3d.z;                             switch (direction.getAxis()) {                             case X:                                 e = direction.getOffsetX();                                 break;                             case Y:                                 g = direction.getOffsetY();                                 break;                             case Z:                                 h = direction.getOffsetZ();                             }                             entity.setVelocity(e, g, h);                             break;                         }                     }                 }                 //记录实体与 b36 在移动方向上的最大重叠距离                 double i = 0.0;                 //遍历 b36 模型的每一个子包围盒                 for (Box box2 : list2) {                     //检查 b36 的子包围盒的移动路径是否与实体的碰撞箱相交                     Box box3 = Boxes.stretch(offsetHeadBox(pos, box2, blockEntity), direction, d);                     Box box4 = entity.getBoundingBox();                     //如果相交，计算重叠的距离                     if (box3.intersects(box4)) {                         i = Math.max(i, getIntersectionSize(box3, direction, box4));                         //重叠的距离大于实际推动距离，这是最坏的情况，不需要进行多余检查了                         if (i >= d) {                             break;                         }                     }                 }                 //如果最终计算出的最大重叠距离大于 0，进行推动                 if (!(i <= 0.0)) {                     //确定最终的推动距离。这个距离不能超过 b36 本身在这一刻的移动距离                     //并额外增加 0.01 防止下一 tick 实体被卡在 b36 里                     i = Math.min(i, d) + 0.01;                     //沿移动方向，将实体推动计算出的距离                     moveEntity(direction, entity, i, direction);                     //如果是 b36 在收回，则特殊处理被其影响的实体                     if (!blockEntity.extending && blockEntity.source) {                         push(pos, entity, direction, d);                     }                 }             }         }     }}//此方法用于解决实体卡在 b36 内的问题。它会检查实体是否与 b36 所在位置的方块存在重叠，如果存在，则计算出沿反方向将实体推出的最小距离。通过一次精细的重叠深度比较后，它会校准并执行这个位移，确保实体被平滑地推离 b36。private static void push(BlockPos pos, Entity entity, Direction direction, double amount) {     //获取实体的包围盒     Box box = entity.getBoundingBox();     //创建一个位于 b36 位置的、标准大小（1x1x1）的方块包围盒     Box box2 = VoxelShapes.fullCube().getBoundingBox().offset(pos);     // 如果实体与方块的包围盒没有重叠，则说明实体未被卡住，直接返回     if (box.intersects(box2)) {         //获取原始推动的相反方向         Direction direction2 = direction.getOpposite();         //计算要使实体脱离方块所需的最小推出距离         double d = getIntersectionSize(box2, direction2, box) + 0.01;         //计算实体与方块实际重叠区域的深度         double e = getIntersectionSize(box2, direction2, box.intersection(box2)) + 0.01;         // 如果两次计算的推出距离几乎相等（表明实体只是轻微接触），则进行位置校准（float align）         if (Math.abs(d - e) < 0.01) {             d = Math.min(d, amount) + 0.01;             //沿相反方向，将实体推动计算出的距离             moveEntity(direction, entity, d, direction2);         }     }}//该方法用于实现移动的蜂蜜块对上方实体的粘滞效果，且仅在 b36 水平推动时生效。其核心逻辑是：在蜂蜜块表面上方定义一个“粘滞区域”，然后将该区域内所有符合条件的实体，与蜂蜜块 b36 进行同步移动。private static void moveEntitiesInHoneyBlock(World world, BlockPos pos, float f, PistonBlockEntity blockEntity) {     //整个逻辑只在 b36 是蜂蜜块时才会触发     if (blockEntity.isPushingHoneyBlock()) {         //获取 b36 移动的方向         Direction direction = blockEntity.getMovementDirection();         //蜂蜜块的粘滞效果仅在水平方向推动时生效         if (direction.getAxis().isHorizontal()) {             //获取蜂蜜块 b36 碰撞箱的实际顶部高度             double d = blockEntity.pushedBlockState.getCollisionShape(world, pos).getMax(Direction.Axis.Y);             //定义一个位于蜂蜜块表面上方的粘滞区域（从其顶部到 Y = 1.5），并使其随 b36 同步移动             Box box = offsetHeadBox(pos, new Box(0.0, d, 0.0, 1.0, 1.5000010000000001, 1.0), blockEntity);             //计算在当前 tick，蜂蜜块 b36 需要移动的距离             double e = f - blockEntity.progress;             //遍历粘滞区域内所有符合移动条件的实体             for (Entity entity : world.getOtherEntities((Entity)null, box, entityx -> canMoveEntity(box, entityx, pos))) {                 //沿移动方向，将所有符合条件的实体推动计算出的距离                 moveEntity(direction, entity, e, direction);             }         }     }}
```

`pushEntities`：处理活塞推动

该方法是活塞与实体交互的核心。它会获取 b36 在当前游戏刻的移动方向和距离，并找出路径上所有可能接触的实体。然后，它会精确计算每个实体与 b36 在移动方向上的最大重叠部分，并据此推动实体（同时增加微小位移 0.01 防卡入）。此过程还会特殊处理黏液块的弹射效果以及 b36 收回时对实体的影响。

其工作流程如下：

1. 确定移动参数：首先，获取活塞头在当前游戏刻（tick）的移动方向和精确距离。
2. 粗略筛选实体：通过计算活塞头在这一帧内扫过的空间，快速找出所有可能被推动的实体。
3. 精确计算重叠：遍历每一个可能被影响的实体，并将其碰撞箱与活塞头模型的每一个子碰撞箱进行比较。它会精确计算出在活塞移动方向上，实体与活塞头之间发生的最大重叠距离。
4. 执行推动：根据计算出的最大重叠距离，对实体施加一个位移。为了防止实体在下一帧被卡住，推动的距离会额外增加一个极小值（0.01）。

5. 特殊情况处理：
    - 黏液块：如果推动的方块是黏液块，它会对非玩家实体产生一个弹射效果，将其沿移动方向的速度设置为一个固定值。
    - 活塞收回：如果活塞正在收回，它会调用 push 方法来特殊处理可能被拉动的实体，确保它们不会卡在活塞臂消失后的空间里。

`push`：解决实体卡入问题

这个方法是一个辅助函数，专门用于解决实体意外卡入活塞臂所在方块的问题。
其工作流程如下：

1. 检查重叠：首先，检查实体的碰撞箱是否与活塞臂目标位置的方块碰撞箱有重叠。如果没有重叠，则说明实体没有卡住，方法直接结束。
2. 计算推出距离：如果检测到重叠，它会计算需要将实体沿反方向推出多远才能恰好分离。它会进行两次计算和比较，以确保推出的距离精确且最小，避免过度位移。
3. 执行位移：最后，它会沿活塞移动的相反方向，将实体推出计算好的距离，从而平滑地解决卡入问题。

`moveEntitiesInHoneyBlock`：实现蜂蜜块粘滞效果

这个方法专门用于实现移动中的蜂蜜块对上方实体的 “粘滞” 效果。
其工作流程如下：

1. 条件检查：该功能只在活塞推动的是蜂蜜块，并且是水平移动时才会触发。
2. 定义粘滞区域：它会在移动的蜂蜜块表面上方定义一个特定的 “粘滞区域”（从蜂蜜块顶部到 Y = 1.5 的高度）。
3. 同步移动：系统会找出所有位于这个粘滞区域内的实体。
4. 施加位移：最后，将这些实体与蜂蜜块一起，沿着相同的方向移动相同的距离，从而实现了实体被 “粘” 在蜂蜜块上一起移动的效果。

1.13 基于`EXTENDED=FALSE`的活塞收回事件的无头活塞制造办法

我们先召回前面提到的`onSyncedBlockEvent`。值得注意的是，当服务端执行该方法并`return false`时，并不会向客户端发包，因此客户端也不会执行任何动画，或将任何东西变成 b36。（也就是客户端并不会执行该方法）

实际上在前文提到的`setBlockState`的传入形参`flag`仅仅在通常意义下是寻路更新，这里需要更为详细的解释。

我们观察如下现象：

1. 放置一个任意方向的任意种类的活塞。并在其除活塞面向方向的一阶毗邻位置放置红石块。这一步会激活这个活塞
2. `/tick freeze`
3. 移除红石块
4. 将原本的活塞替换为任意方向的同一类型的活塞
5. 放回红石块，红石块的位置要求等同与第一步提到的位置要求
6. 在活塞面向方向放置一个不可推动方块
7. `/tick freeze`解除时停

如果操作完全没有错误，则该活塞变为无头活塞

这是因为在`onSyncedBlockEvent`中

```
if (!world.isClient) {    // 检查是否需要扩展活塞（是否有红石信号）     boolean shouldExtend = this.shouldExtend(world, pos,
facing);    // 如果需要扩展并且事件类型是收缩，更新方块状态并返回    if (shouldExtend && (actionType == 1 || actionType == 2)) {
world.setBlockState(pos, extendedState, 2);        return false;    }    // 如果不需要扩展并且事件类型是 0，直接返回 false    if
(!shouldExtend && actionType == 0) {        return false;    }}
```

方块事件在执行时只会校验方块类型和位置，不会校验这个方块的朝向，因此在这个活塞的位置，我们得到了一个可以校验通过并执行的，活塞状态处于 Extended = False 的活塞，并且该活塞将执行收回事件。在服务端，活塞会被校验是否应该伸出。此时活塞因为被红石块激活，确实应该伸出，所以进入第一个 if 分支条件。这个分支条件首先调用了`setBlockState`，而我们查看一下`setBlockState`在`flag=2`时做了什么：

```
public boolean setBlockState(BlockPos pos, BlockState state, int flags, int maxUpdateDepth) {    if
(this.isOutOfHeightLimit(pos)) {        return false;    } else if (!this.isClient && this.isDebugWorld()) {        return
false;    } else {        WorldChunk worldChunk = this.getWorldChunk(pos);        Block block = state.getBlock();
BlockState blockState = worldChunk.setBlockState(pos, state, (flags & 64) != 0);        if (blockState == null) {
return false;        } else {            BlockState blockState2 = this.getBlockState(pos);            if (blockState2 ==
state) {                if (blockState != blockState2) {                    this.scheduleBlockRerenderIfNeeded(pos,
blockState, blockState2);                }                if ((flags & 2) != 0 && (!this.isClient || (flags & 4) == 0) &&
(this.isClient || worldChunk.getLevelType() != null && worldChunk.getLevelType().isAfter(ChunkLevelType.BLOCK_TICKING))) {
this.updateListeners(pos, blockState, state, flags);                }                ...
this.onBlockChanged(pos, blockState, blockState2);            }            return true;        }    }}
```

我们发现在`if ((flags & 2) != 0 && (!this.isClient || (flags & 4) == 0) && (this.isClient || worldChunk.getLevelType() != null && worldChunk.getLevelType().isAfter(ChunkLevelType.BLOCK_TICKING)))`这个分支调用中，调用了一个方法`updateListeners`，我们会发现这个`updateListeners`有两个实现方法：

1. `ServerWorld`
2. `ClientWorld`

在`ServerWorld`中，这里就是普通的寻路更新。这里寻路更新会假定活塞臂存在，当然这是题外话了。

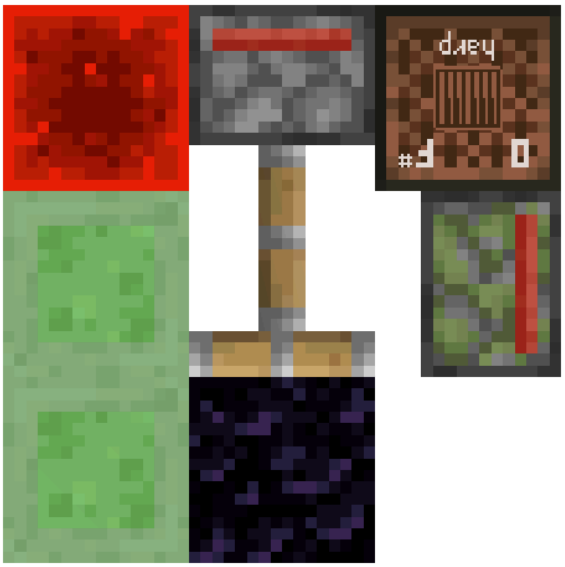
而在`ClientWorld`中，这个`updateListeners`负责了渲染。因此这里客户端抢跑了，直接渲染了伸出的活塞臂而没有渲染活塞头。

接下来是第二个，由于活塞自检引起的伸出事件。这个伸出事件在执行时会直接失败。无论客户端还是服务端，在这里不做进一步解释。

因此，这样就制造出了一个无头活塞。

1.14 案例分析

1.14.1 案例 1



敲击音符盒，会发生什么?

活塞掉落

粘性活塞受到 NC 更新，自检。调用`trymove`，添加收回方块事件，调用`onSyncedBlockEvent`。进行下述判断：

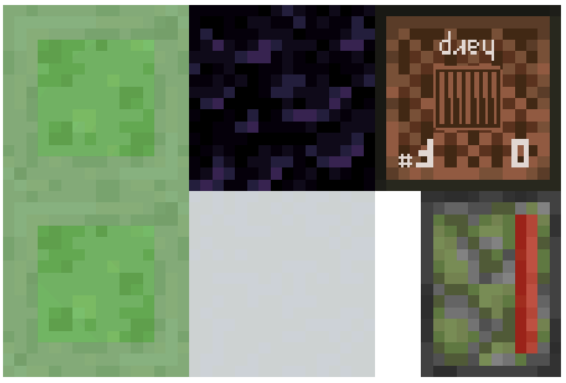
```
if (!hasMovingPistonInFront) {    if (actionType != 1 || targetState.isAir() || !isMovable(targetState, world, targetpos,
facing.getOpposite(), false, direction)            || targetState.getPistonBehavior() != PistonBehavior.NORMAL &&
!targetState.isOf(Blocks.PISTON) && !targetState.isOf(Blocks.STICKY_PISTON)) {        world.removeBlock(pos.offset(facing),
false);    } else {        this.move(world, pos, facing, false);    }}
```

稍做判断，进入`move`方法。

```
if (!retract && world.getBlockState(headPos).isOf(Blocks.PISTON_HEAD)) {    // 将活塞头设为空气    // flag = 0b00010100, 无 PP
更新, 寻路更新 客户端    world.setBlockState(headPos, Blocks.AIR.getDefaultState(), 20);}
```

由于目标位置是活塞头，且是收缩动作，将活塞头位置设为空气，触发活塞头的`onRemove`，活塞头连带着活塞底座掉落。

1.14.2 案例 2



敲击音符盒，会发生什么?

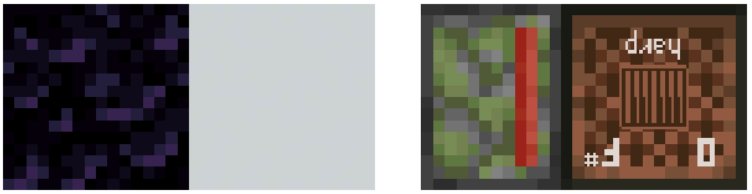
粘性活塞变回完整状态，除此之外无事发生

步骤如前一例所述，但这里进入`move`后不符合上文的移除活塞头的方法。于是进入下一步

```
if (!pistonHandler.calculatePush()) {    return false;}
```

由于面前结构无法拉回，则直接`return false`，无事发生。

1.14.3 案例 3



敲击音符盒，会发生什么?

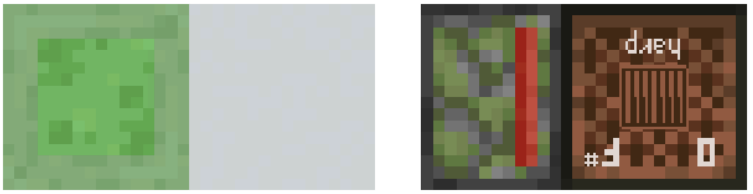
普通方块被移除

回到`onSyncedBlockEvent`，依旧是这个判断:

```
if (!hasMovingPistonInFront) {    if (actionType != 1 || targetState.isAir() || !isMovable(targetState, world, targetpos,
facing.getOpposite(), false, direction)            || targetState.getPistonBehavior() != PistonBehavior.NORMAL &&
!targetState.isOf(Blocks.PISTON) && !targetState.isOf(Blocks.STICKY_PISTON)) {        world.removeBlock(pos.offset(facing),
false);    } else {        this.move(world, pos, facing, false);    }}
```

但在此处，if 分支成立 (黑曜石不可移动,`!isMovable`是`true`)，因直接移除活塞面前的方块，普通方块被移除。

1.14.4 案例 4



敲击音符盒，会发生什么？

无事发生

同案例 2，进入 `move` 方法。当活塞尝试计算是否可拉回时，它把自己检查到了，而激活的活塞底座本身又是不可推动的，于是 `return false`，无事发生。

1.14.5 案例 5

拉下拉杆，会发生什么?

前面的活塞变为无头活塞，后面的活塞变回普通活塞

依旧考察`onSyncedBlockEvent`，在此处注意到这段代码：

```
BlockPos targetpos = pos.add(facing.getOffsetX() * 2, facing.getOffsetY() * 2, facing.getOffsetZ() * 2);BlockState targetState = world.getBlockState(targetpos);boolean hasMovingPistonInFront = false;// 如果目标位置已有移动活塞方块，尝试完成它的扩展if (targetState.isOf(Blocks.MOVING_PISTON)) {    BlockEntity blockEntity2 = world.getBlockEntity(targetpos);    if (blockEntity2 instanceof PistonBlockEntity) {        PistonBlockEntity pistonBlockEntity = (PistonBlockEntity)blockEntity2;if (pistonBlockEntity.getFacing() == facing && pistonBlockEntity.isExtending()) {            pistonBlockEntity.finish();hasMovingPistonInFront = true;        }    }}
```

拉下拉杆时，前方的活塞先在面前位置创建一个 b36 (注意活塞底座并未变为 b36)，然后执行上述操作，调用了前一个活塞的活塞臂的`finish`方法。我在前面提到，该方法对于活塞臂的 b36 会有特判，直接将其变为空气。因此前一个活塞的活塞臂就被移除，变为无头活塞。后一个位置的粘性活塞收回，变为`Extended=False`的活塞。

1.14.6 案例 6

拉下拉杆会发生什么?

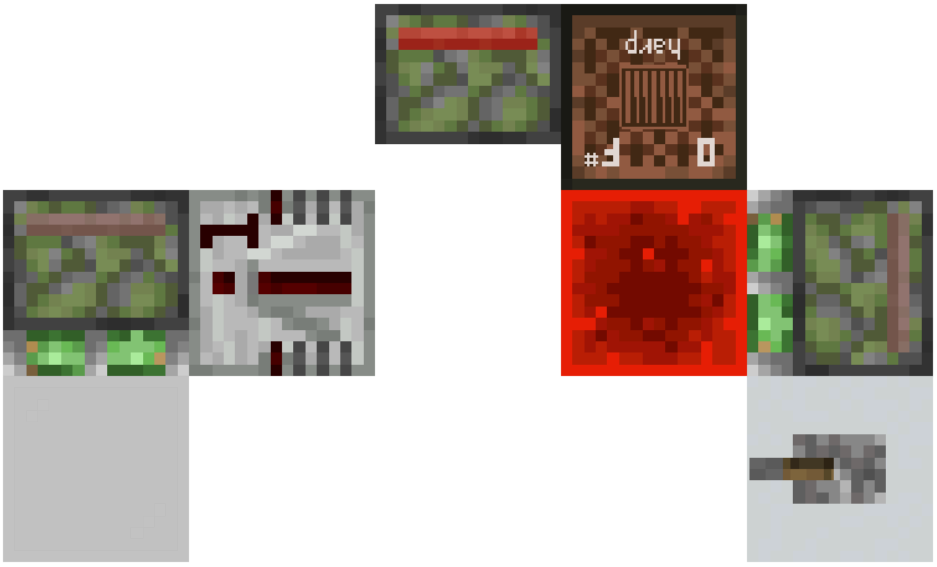
两个活塞换头

原理和案例 5 类似，但是调用的是这里的代码：

```
BlockEntity blockEntityInFront = world.getBlockEntity(pos.offset(facing));// 如果该位置有活塞方块实体，完成收缩操作if
(blockEntityInFront instanceof PistonBlockEntity) {    ((PistonBlockEntity)blockEntityInFront).finish();}// 创建移动的活塞方块
状态BlockState pistonHeadState = Blocks.MOVING_PISTON.getDefaultState().with(PistonExtensionBlock.FACING, direction)
.with(PistonExtensionBlock.TYPE, this.sticky ? PistonType.STICKY : PistonType.DEFAULT);// 设置新的活塞方块状态，并创建新的方块实体
world.setBlockState(pos, pistonHeadState, 20);world.addBlockEntity(PistonExtensionBlock.createBlockEntityPiston(pos,
pistonHeadState, (BlockState)this.getDefaultState().with(FACING, Direction.byId(directionData & 7)), direction, false,
true));world.updateNeighbors(pos, pistonHeadState.getBlock());pistonHeadState.updateNeighbors(world, pos, 2);
```

面前是 b36 就直接调用`finish`方法，将其变为空气。

1.14.7 案例 7



拉下拉杆会发生什么?

红石块消失，最左侧粘性活塞丢方块

拉下拉杆，无头活塞面前方块为 b36，并被更新。红石块瞬间到位，造成 NC 更新，更新到中继器，添加 TT 事件

```
BlockEntity blockEntityInFront = world.getBlockEntity(pos.offset(facing));// 如果该位置有活塞方块实体，完成收缩操作if
(blockEntityInFront instanceof PistonBlockEntity) {    ((PistonBlockEntity)blockEntityInFront).finish();}
```

但注意，这里`onSyncedBlockEvent`还没被 return，继续执行如下操作:

```
if (!hasMovingPistonInFront) {    if (actionType != 1 || targetState.isAir() || !isMovable(targetState, world, targetpos,
facing.getOpposite(), false, direction)            || targetState.getPistonBehavior() != PistonBehavior.NORMAL &&
!targetState.isOf(Blocks.PISTON) && !targetState.isOf(Blocks.STICKY_PISTON)) {        world.removeBlock(pos.offset(facing),
false);    } else {        this.move(world, pos, facing, false);    }}
```

于是红石块就被移除掉了。发出 NC 更新。添加 TT 事件。

中继器亮起 2gt，于是最左侧粘性活塞丢方块。

2 应用

2.1 复制

2.1.1 亮起侦测器的移除运算

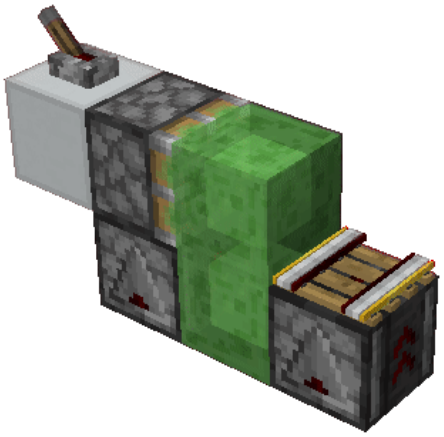
亮起侦测器被移除时会向后方发出 NC 更新并熄灭自己。

2.1.2 基于侦测器的复制

复制的本质实际上是这个方块添加进了移动列表，但是这个时候方块还没转化为 b36。我以某种迅雷不及掩耳盗铃的姿势插入一个更新，这个时候这个更新导致了某些东西被激活 / 掉落，但是因为这个方块本身已经滚进移动方块列表，到位的时候又被以 b36 的形式放下来了。

我们将复制分为两种，基于侦测器和基于附着性方块的。其中，基于侦测器的复制正利用了上述只要侦测器被移除（换成了 b36）的时候就会瞬时产生 NC 更新。

这里给出基于侦测器的铁轨复制：

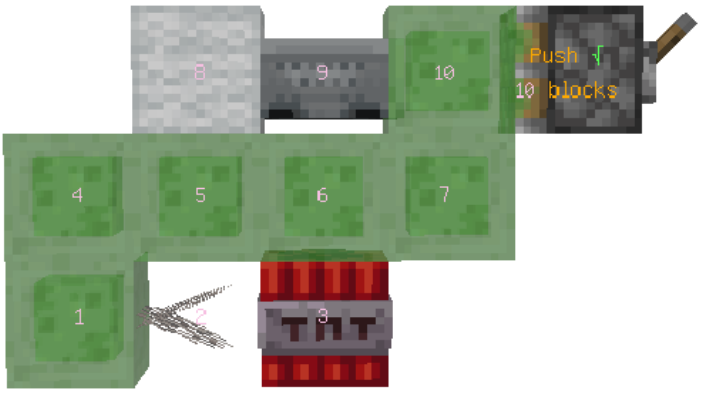


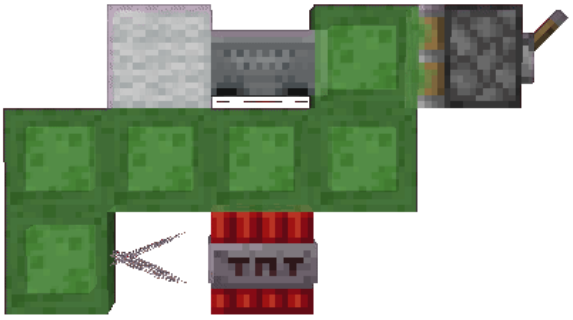
在移动方块列表中，当粘性活塞推出时，会先移动左侧侦测器，在目标位置创建 b36，再移动粘液块，覆盖了亮起侦测器，更新，铁轨发现下面居然是 b36，掉落。移动铁轨，在目标位置创建 b36。

这样我们就完成了一次复制

2.1.3 基于附着方块破碎的复制

2.1.3.1 基于 BUD 的 TNT 复制



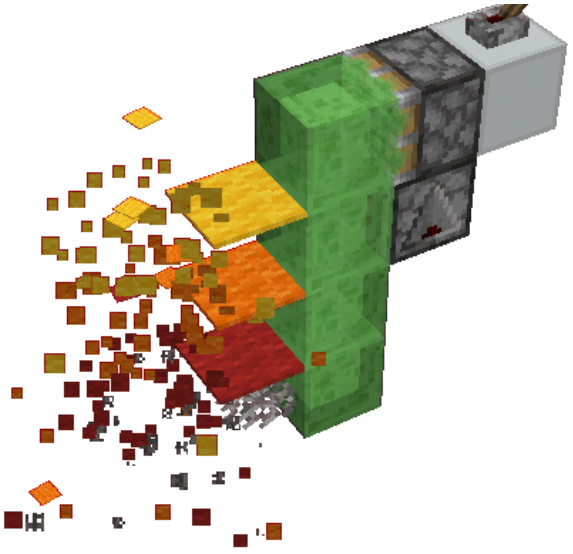


附着方块的掉落同样会产生更新，但是由于附着方块通常会直接掉落而难以二次利用。例如：

2.1.3.2 基于粉导向的极简 TNT 复制

2.1.3.3 地毯复制

下图中是一种极为流行的地毯复制:



当活塞收回时其完成一次复制，我们在这里对其稍做分析：

移动方块列表：

1. 移动最下方的粘液块，在目标位置创建 b36
2. 移动失活的珊瑚扇，在目标位置创建 b36，发出 PP 更新，自检发现自己应该掉落，发出 PP 更新和 NC 更新，引导上方地毯连锁掉落
3. 移动其余粘液块和地毯

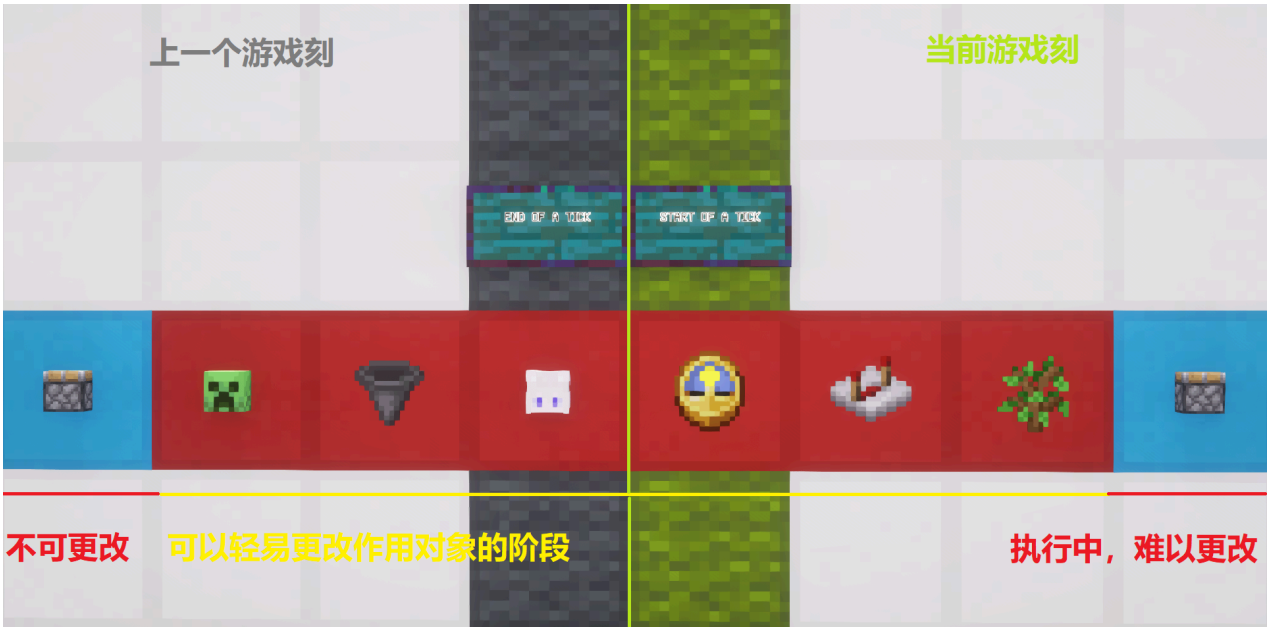
这就完成了复制。

2.2 信息不一致的方块事件

根据 5.1 章节、2.1.5 章节等，我们可以发现：活塞在执行方块事件时只对其 `方块位置` 和 `方块类型` 进行校对。尽管记录了 `活塞朝向`，但它只管辖 b36 的方向。

而活塞执行方块事件时的 `真实方向` 是即时采样的。

这意味着，在一个活塞的方块事件实例被在方块事件阶段中执行前，存在一个时段：此时，我们可以随意改变其对应位置的活塞的属性，甚至对此活塞本生进行破坏，更换。



如图所示，从上一游戏刻的 EU 到当前游戏刻的 BE，这段时间内我们可以修改某一方块事件的作用对象。
同理，从当前游戏刻的 EU 到下一游戏刻的 BE，也存在这样一段时间。
但只要执行到这个活塞的方块事件实例时，如果执行位置有活塞，且活塞类型没有改变，那么活塞尝试推出 / 拉回的朝向就会由执行时的朝向决定，尽管这个朝向可能并不是方块事件添加时的朝向。

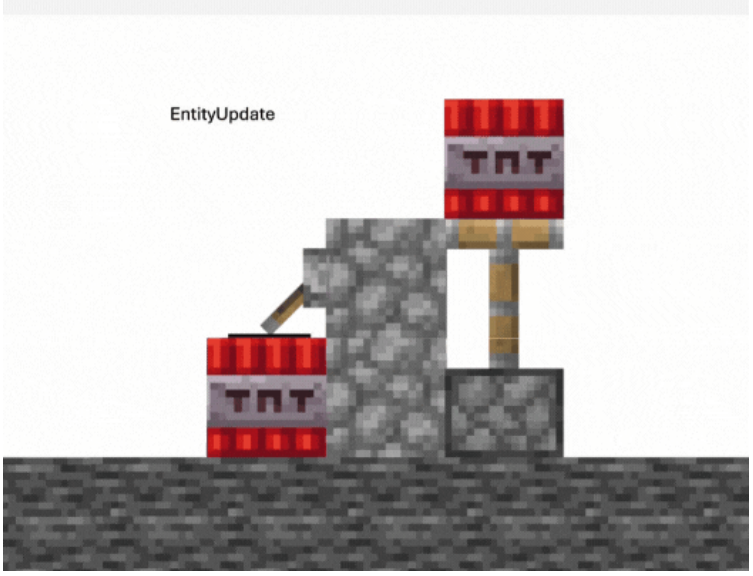
> *有趣的是，b36 的收回时的动画仍然以方块事件传入的 data 参数为准，*
> *所以，尽管方块事件执行时活塞朝向与添加时不同，活塞头的动画仍会面向方向方块事件添加时的活塞朝向（但碰撞箱是准确的）。*
>
>

2.2.1 破基岩

2.2.1.1 跨越游戏刻的破基岩

> *活塞的逆袭之旅*

正如前面所提到的，我们可以在活塞的方块事件阶段前，偷偷摸摸地修改活塞的属性，包括朝向。
这意味着我们可以在活塞推出时，将其朝向改为朝向基岩，然后在活塞收回时，就会将基岩当作活塞头摧毁。
下面的展示动画是一个典型, 在方块事件阶段前更改活塞朝向的破基岩例子。



在整个过程中，发生了如下重要事件

CECECE {gt0}-EU 阶段

- 左侧 TNT 爆炸。
  - 爆炸破坏拉杆。
  - 爆炸破坏圆石。
  - 右侧 TNT 爆炸。
    - 爆炸破坏活塞头, 发出 nc 更新。
    - 活塞基座被更新，由于拉杆已经被炸掉，活塞给自己添加**收回**的方块事件。

      > *可惜的是，它已经错过了这个游戏刻的 BE 阶段，所以只能在 gt1 的 BE 阶段默默等待了。。*

    - 活塞头破坏活塞基座。

CECECE {gt0}-AT 阶段

- 玩家放置朝下的活塞新活塞。

CECECE {gt0}-BE 阶段

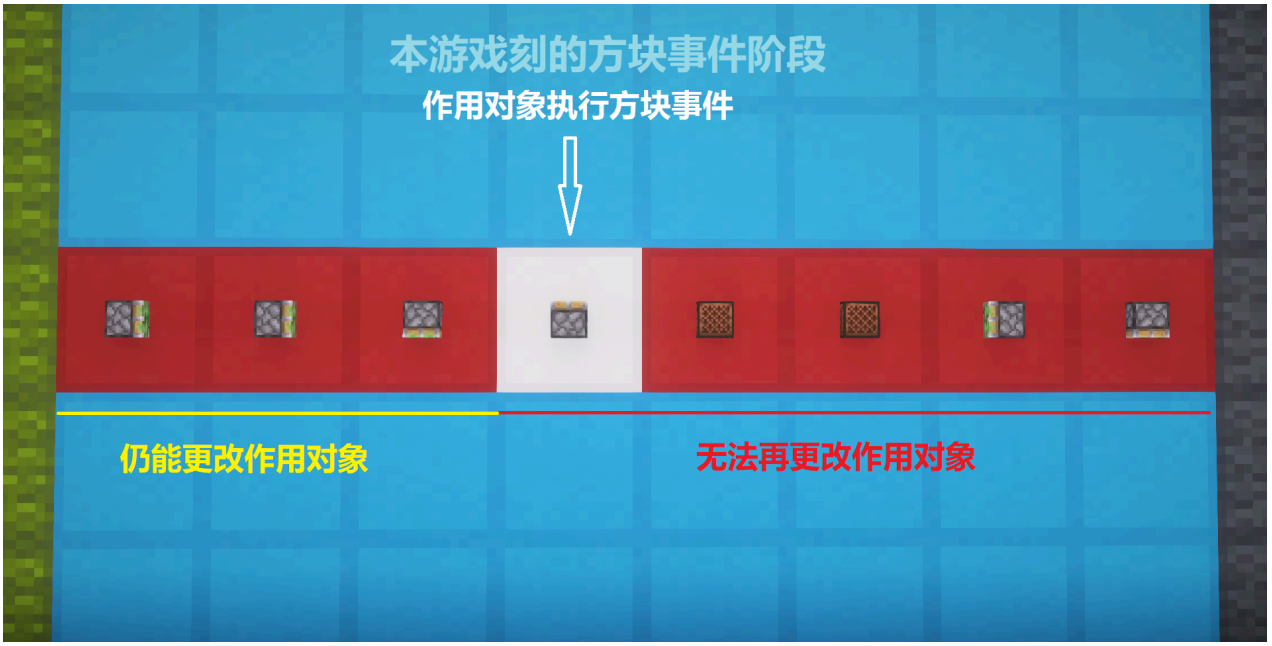
- 活塞按照现有朝向收回，破坏基岩。

以上便是破基岩的全部过程。

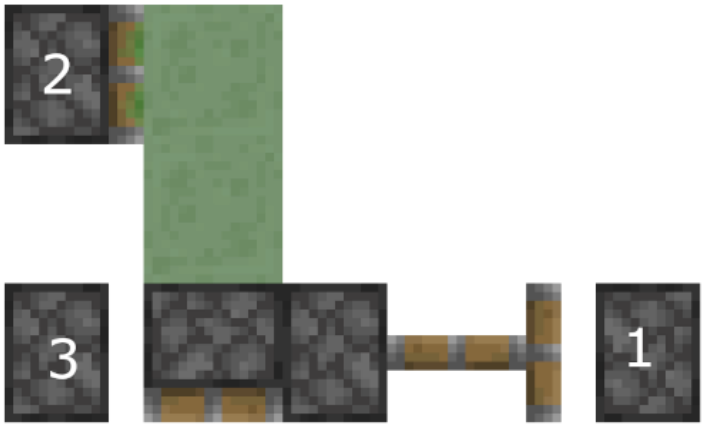
但是，在破基岩机等结构内，由玩家放置活塞是不可能的，我们必须通过其他方式来实现这个效果。

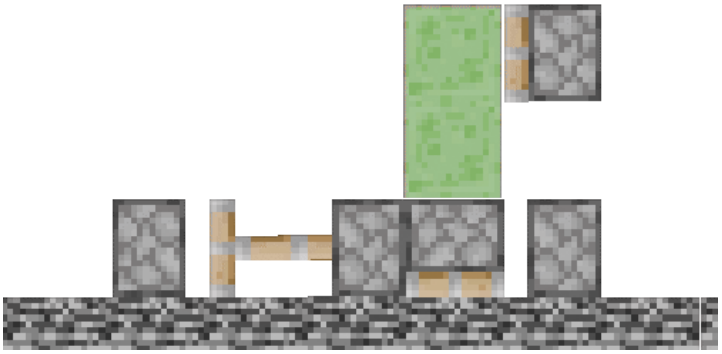
2.2.1.2 在方块事件阶段完成的破基岩

"在活塞的方块事件阶段前" 的 "前" 不仅仅适用于其他的游戏阶段。就算在方块事件阶段内，你仍然可以在某个方块事件之前更改它对应的方块。



如你所见，下图是部分破基岩机中的破基岩结构的简化版本..





如动图所示，在整个过程中，发生了如下重要事件

> *前提 -- 上图面标记的数字是各个活塞的深度，他们按照这个顺序在 BE 阶段尝试伸出或收回。*

CECECE {gt0}-BE 阶段

- 1 号活塞收回，破坏活塞头, 发出 nc 更新
  - 1 号活塞前方的活塞基座被更新，由于处于 BUD 状态，活塞给自己添加**收回**的方块事件。

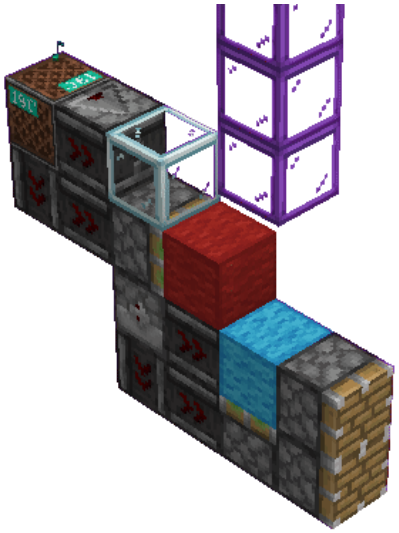
    > *由于深度 1，2，3 已被占用，它将会被安排到 4 号，也就是目前情况下的 "最后执行"。*

  - 活塞头破坏活塞基座。
- 2 号活塞伸出，创建了 3 个 b36。
- 3 号粘性活塞收回，将正前方的朝下的活塞的 b36 强制瞬间到位。
- 朝下的活塞现在的位置拥有先前 1 号活塞导致的方块事件，按照现有朝向收回，破坏基岩。

2.3 欺骗活塞添加 / 取消添加方块事件

2.3.1 欺骗活塞取消添加方块事件：推动上限检测

方块事件也会出现活塞觉得自己要执行的事件与实际状况不一致的情况，最典型的便是推动上限检测。



推动上限检测机制最初由玩家_Kayleigh 和 Landmining 发现，其核心在于 "欺骗" 活塞。这里展示的是 Bright_Observer 制作的简化版本，便于分析理解。

该装置虽具有方向性设计，但这并不影响其检测功能，因此可自由选择建造方向。

运行现象：

- 放下紫色羊毛 (羊毛连接了一根很长很长的柱子...) 后敲击音符盒，普通活塞会启动
- 破坏紫色羊毛后敲击，普通活塞则保持不变

原理分析：

1. 上推活塞在被更新时自检
2. 系统检查推动上限条件
3. 若羊毛放下，因上方玻璃柱子已超出推动上限，活塞不会添加方块事件
4. 此时激活两个普通活塞
5. 粘性活塞被普通活塞更新，自检后伸出
6. 羊毛不存在时：
   粘性活塞正常运作
   上方普通活塞的推出状态取决于朝向
   下方普通活塞因深度关系保持原位

关键发现：
粘性活塞的计划动作与实际执行时推动结构可能存在差异，这种特性被巧妙地用于检测机制。

2.3.2 欺骗活塞添加方块事件

遗憾的是，如果你试图复刻***5.10.1 破基岩***的布局和激活顺序

FOOTNOTES

1. 这里的`tryMove`是需要消歧义的，在`Java`中，两个在不同类的方法是可以被赋予相同名字。这里的`tryMove`是在`PistonHandler.java`里的 ↩

slug: rails
title: 铁轨
title-en: Rails
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- Gudu-Z
  date: '2025-10-01T00:26:51+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

1 铁轨的更新

除普通铁轨外，特殊铁轨（动力铁轨、激活铁轨、探测铁轨）都具有特殊的更新范围。

1.1 动力铁轨 / 激活铁轨的更新

动力铁轨和激活铁轨的更新行为是完全一致的。[1]

当动力铁轨 / 激活铁轨的激活状态改变时，它将以这些位置为更新核按序发出 NC 更新：

1. 上方一格（如果铁轨是倾斜的）
2. 自身
3. 下方一格
4. 自身
5. 下方一格
6. 上方一格（如果铁轨是倾斜的）

也就是说，对平放的动力铁轨 / 激活铁轨而言，它以**自身 -> 下方 -> 自身 -> 下方**方块为更新核发出 4 次 NC 更新；

对倾斜的动力铁轨 / 激活铁轨而言，它以**上方 -> 自身 -> 下方 -> 自身 -> 下方 -> 上方**方块为更新核发出 6 次 NC 更新。

1.2 探测铁轨的更新

暂未完成

2 铁轨激活条件检测

此段落中的所有 “铁轨” 皆指动力铁轨或激活铁轨

2.1 激活方式

有两种条件能够使铁轨激活：

- 直接与红石信号源相邻，称为**直接激活**
- 与自身**相连**的 8 个铁轨中（不含自身）有 1 个铁轨直接与红石信号源相邻，称为**间接激活**

例如，一个铁轨直接与一个红石块相邻，它就是被**直接激活**的；

而一个铁轨与另外几个铁轨相连，构成一条铁轨链，在铁轨链上有一个相邻的红石块，且这一红石块在该铁轨的 8 格内，这个铁轨就是被**间接激活**的。

2.2 “相连” 的判断条件

一个未被直接激活的铁轨会通过一种 “寻找” 机制，判断自己是否被间接激活。具体来说，它会在上查找是否有距离自身 8 格以内的、与自身**相连**的、**直接激活**的铁轨。

通俗来讲，想象你正要去一个餐厅。你会打开地图，搜索餐厅，看看自己能不能走过去；餐厅不会飞到你的面前，告诉你 “我距离你 800m，你可以走过来”。

铁轨也是这样。每个铁轨都会自己 “寻找” 符合条件的信号源 —— 也就是说，铁轨会尝试沿着铁轨链 “走” 最多 8 格，看自己能不能找到信号源 —— 也就是能不能找到和信号源直接相连的铁轨。

而一个十分朴素的想法是：如果铁轨 “走” 到一半，铁轨链 “断了”，那么它肯定就走不下去了。

铁轨在某一位置寻找相连的铁轨时，会根据自身的**形状**（即是否为倾斜的），查找**特定位置**的、与自身**方向相同**的铁轨，即铁轨判断下一个铁轨是否与自身相连，只与这三个因素有关：

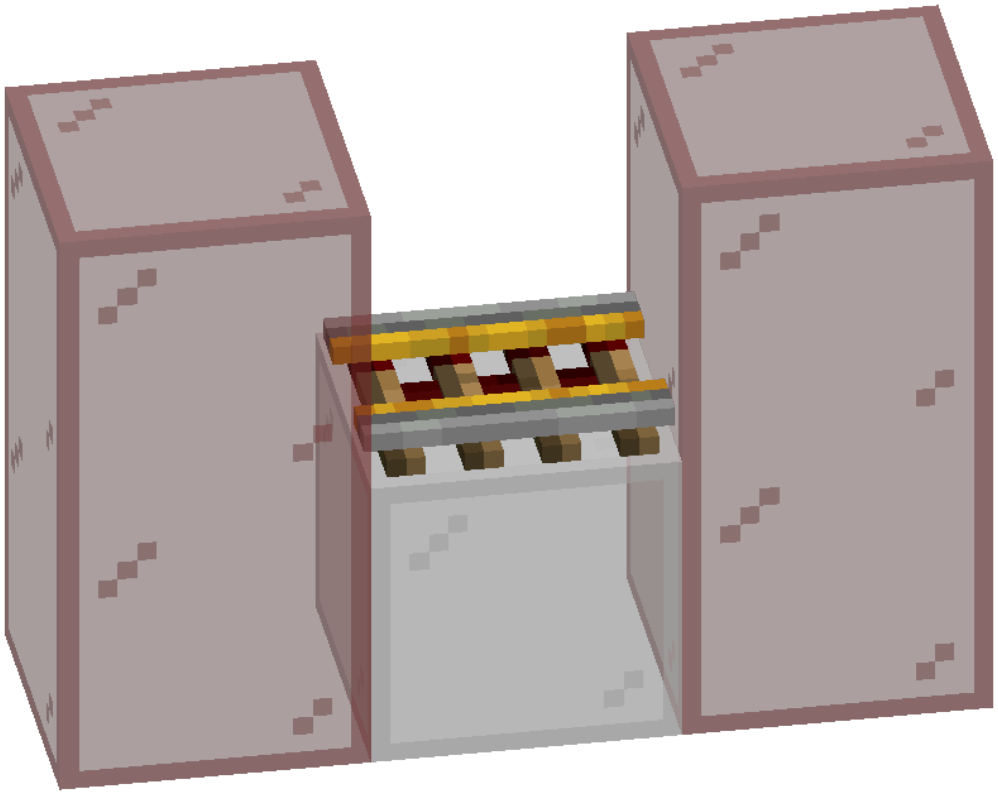
- 自身的形状（自身是否是倾斜的）
- 下一个铁轨的位置
- 下一个铁轨的方向

而与**下一个铁轨的形状**无关。

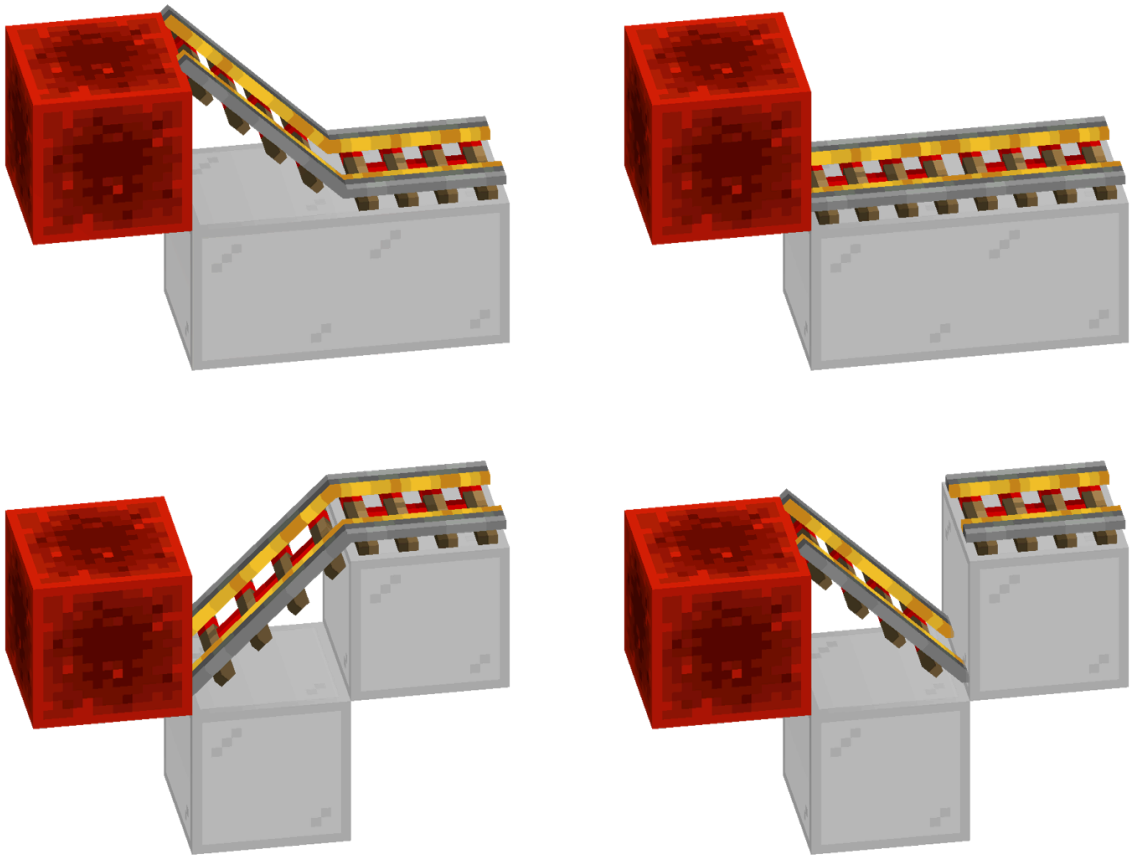
平铁轨会检查这些位置：

- 铁轨两端的、和自身相同高度的位置
- 铁轨两端的、比自身低一格的位置

范围如图中红色玻璃位置所示：



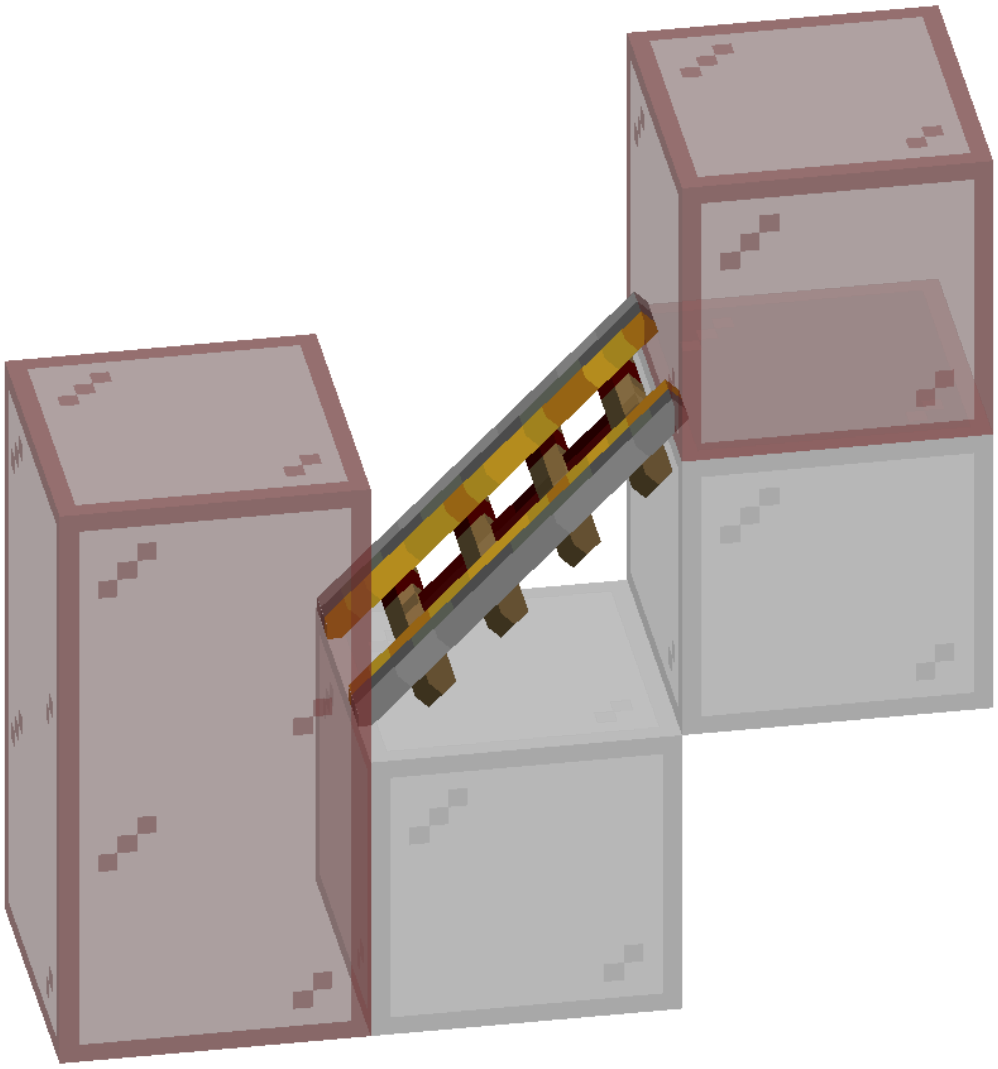
例如，图中最右侧的平铁轨在向左侧 “寻找” 直接激活的铁轨时，会判断左侧的铁轨铁轨与自身相连。图中前三种情况是 “符合直觉” 的，而最后一种则是**铁轨检查相连时不考虑下一个铁轨的形状**导致的。:



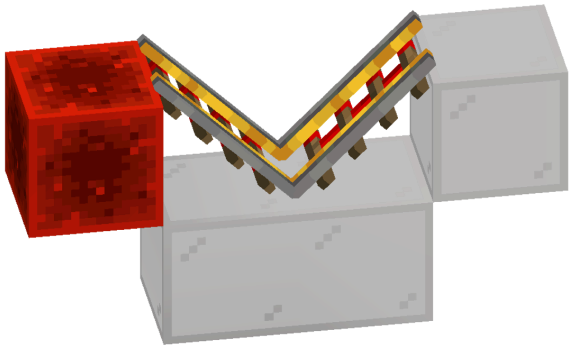
斜铁轨会检查这些位置：

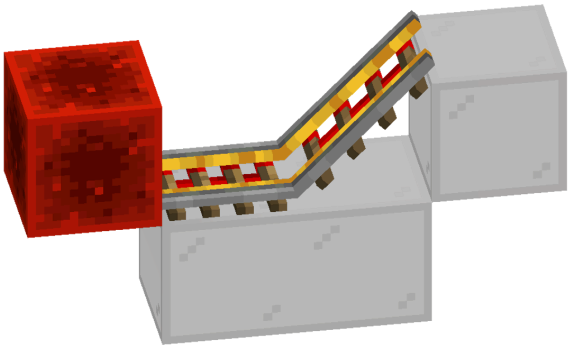
- 斜铁轨较低一端的、和自身相同高度的位置
- 斜铁轨较低一端的、比自身低一格的位置
- 斜铁轨较高一端的、比自身高一格的位置

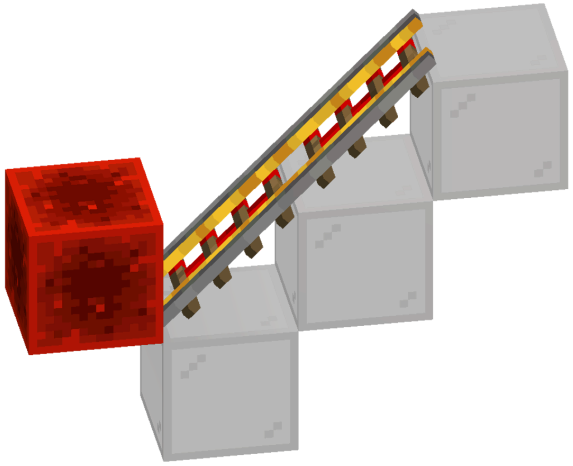
范围如图中红色玻璃位置所示：

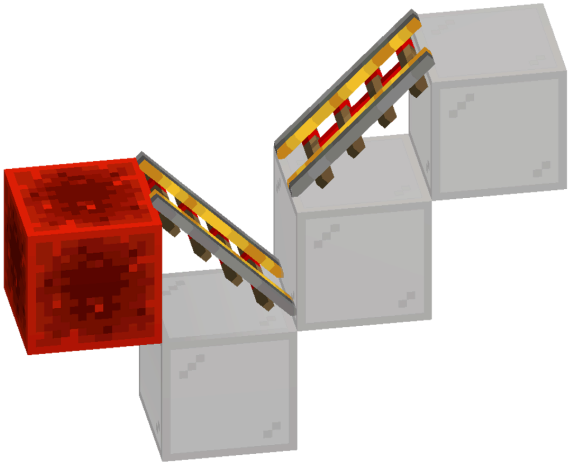


例如，图中最右侧的斜铁轨在向左侧 “寻找” 直接激活的铁轨时，会判断左侧的铁轨铁轨与自身相连。图中前三种情况是 “符合直觉” 的，而最后一种则是**铁轨检查相连时不考虑下一个铁轨的形状**导致的。

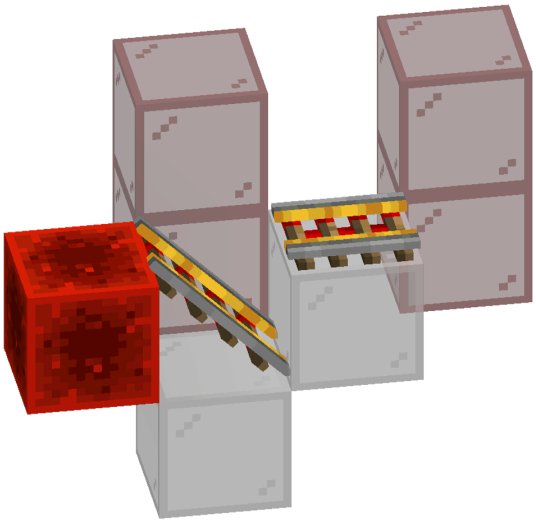


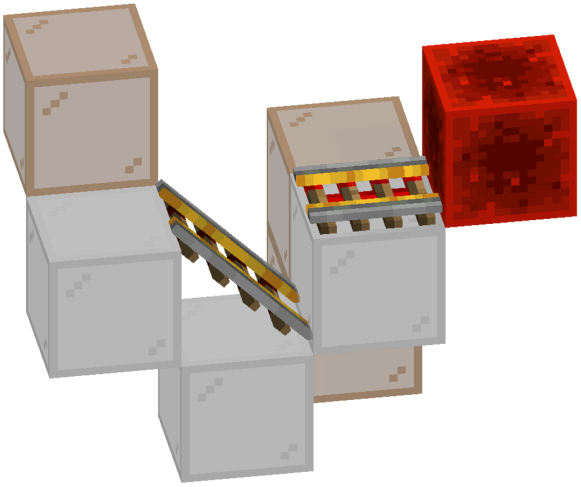






结合以上两点，我们可以发现：铁轨 “相连” 具有单向性：即铁轨 A 判断铁轨 B 和自己相连时，铁轨 B 不一定判断铁轨 A 和自己相连:





从直观上来看，图中铁轨链红石信号可以从右侧传向左侧，但不能从左侧传向右侧；从 “寻找” 的视角来看，就是：左侧的斜铁轨判断右侧的平铁轨和自身不相连，而右侧的平铁轨判断左侧的斜铁轨和自身相连。

分析这两个铁轨的各自检查相连的位置：上图中橙色玻璃对应的水平位置为斜铁轨检查相连的位置，红色玻璃对应的水平位置为平铁轨检查相连的位置。也就是说，斜铁轨在平铁轨的检查位置上；而平铁轨不在斜铁轨的检查位置上，导致了**铁轨相连具有单向性**。

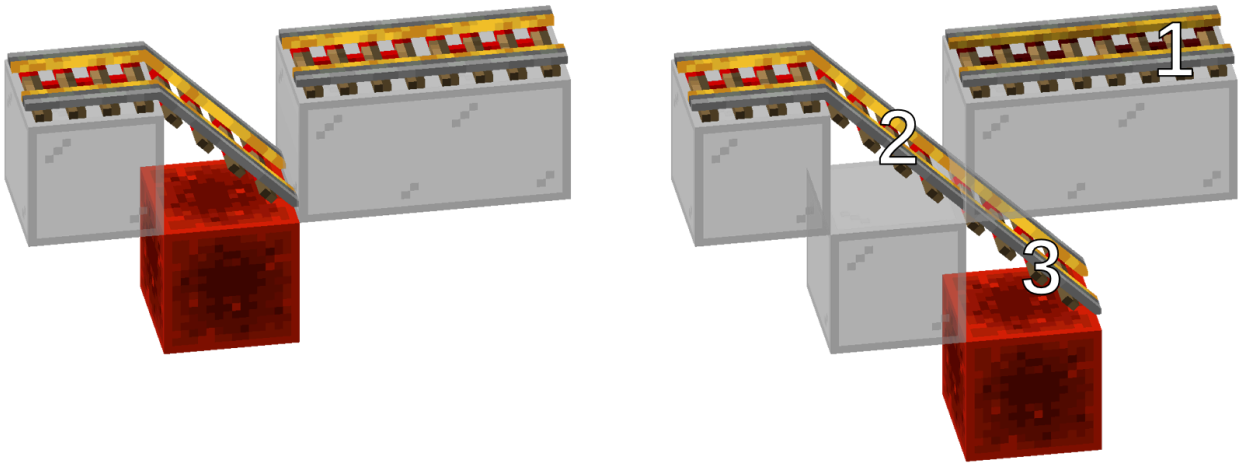
这可以用于制作一些只能单向传递铁轨信号的铁轨链。

2.3 “寻找” 的方向

铁轨在寻找相连的铁轨时遵循固定的顺序：对两端分别进行独立搜索，且每次搜索都是单向的（不会折返）。

- **搜索流程：** 先从一端沿该方向连续向外搜索，搜索过程中不会改变方向（即只 “进” 不 “退”）；该端搜索结束后，再对另一端重复相同的搜索过程。
- **方向规则：** 南北走向的铁轨总是先向南搜索，再向北；东西走向的铁轨先向西搜索，再向东。

因此，多个铁轨是否能被判定为 “相连” 不仅取决于距离，还取决于搜索的方向顺序。这就能解释图示中的差异：在第一种情况，红石块两侧的铁轨都能被找到并激活；在第二种情况，只有红石块左侧的铁轨能够被激活，因为右侧的搜索方向无法经过中间的某个铁轨去到目标位置，从而找不到直接激活的铁轨。



如图，在第一种情况中，红石块左右两侧的铁轨都可以被激活。

而在第二种情况中，只有红石块左侧的铁轨可以被激活。红石块左侧的铁轨在**向右**寻找直接激活的铁轨时，会先寻找到 2 号铁轨，然后**向右**（下）寻找到 3 号铁轨。

红石块右侧的铁轨在**向左**寻找直接激活的铁轨时，同样会先找到 2 号铁轨；但由于此时的寻找方向为**向左**，而 3 号铁轨在 2 号铁轨的**右侧**，于是就不能寻找的 3 号铁轨，也就找不到直接激活的铁轨，无法激活。

FOOTNOTES

1. 动力铁轨和激活铁轨都由 `PoweredRailBlock` 类实现，因此它们在源码层面的行为完全相同。两者对矿车产生的不同效果，是由矿车自身进行判断和处理的，而不是由铁轨决定的。 ↩

CHAPTER 3

LOADING TICKETS

slug: ticket-types
title: 加载票系统中的类型
title-en: null
author: RoyanAB
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-02-11T16:54:10+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/01Ticket.png
  alt: img.png

本章将建立对加载票这一概念的基本认识。

1 加载票 TICKET

一个加载票包含了

`net.minecraft.server.world.ChunkTicket`

- `type` 加载票类型，记录加载的原因。
- `level` 加载等级，决定区块的运算。
- `tickCreated` 创建时间，用于判断加载票是否过期。

2 加载票类型 TICKETTYPE

在 Minecraft 一共有 8 种加载票类型

`net.minecraft.server.world.ChunkTicketType`

```
    public static final ChunkTicketType<Unit> START = create("start", (a, b) -> 0);    public static final
ChunkTicketType<Unit> DRAGON = create("dragon", (a, b) -> 0);    public static final ChunkTicketType<ChunkPos> PLAYER =
create("player", Comparator.comparingLong(ChunkPos::toLong));    public static final ChunkTicketType<ChunkPos> FORCED =
create("forced", Comparator.comparingLong(ChunkPos::toLong));    public static final ChunkTicketType<ChunkPos> LIGHT =
create("light", Comparator.comparingLong(ChunkPos::toLong));    public static final ChunkTicketType<BlockPos> PORTAL =
create("portal", Vec3i::compareTo, 300);    public static final ChunkTicketType<Integer> POST_TELEPORT =
create("post_teleport", Integer::compareTo, 5);    public static final ChunkTicketType<ChunkPos> UNKNOWN = create("unknown",
Comparator.comparingLong(ChunkPos::toLong), 1);
```

- `start`：用于世界启动时加载出生点。加载距离是 11，即加载等级 22 。`net.minecraft.server.MinecraftServer#prepareStartRegion`
- `dragon`：用于进行龙战时加载末地主岛。加载 9 距离，即加载等级 24。 `net.minecraft.entity.boss.dragon.EnderDragonFight#tick`
- `player`：玩家加载。
- `forced`：用于实现 `/forceload` 指令与强加载出生点。
- `light`：用于加载需要计算光照的区块。
- `portal`：用于生物或玩家穿过传送门时加载目标维度的区块，300gt（15s）内没有使用过之后，移除与之相关的加载票，加载距离是 3 。 `net.minecraft.world.PortalForcer#getPortalRect`
- `post_teleport`：用于生物或者玩家传送时加载目标区块，比如 `/tp`，5gt 后移除。
- `unknown`：用于游戏内代码上使用了 `getChunk` 后加载区块，1gt 后移除。

加载等级对应着加载距离，算法是：

加载等级 = 33 - 加载距离

`net.minecraft.server.world.ChunkTicketManager#addTicket`

```
  ChunkTicket<T> chunkTicket = new ChunkTicket<>(type, ChunkLevels.getLevelFromType(ChunkLevelType.FULL) - radius, argument);
//ChunkLevelType.FULL = 33
```

需要注意的是，加载距离并不由 `ChunkTicketType` 指定，比如 `post_teleport` 在 `/tp` 指令使用时加载距离为 1，而其他情况下为 0。

3 加载等级 LEVEL

加载等级被限制在 [22-45]，至于为什么上限是这个数：

```
public static final int INACCESSIBLE = 33 + ChunkStatus.getMaxDistanceFromFull();
```

在 1.20.1 中 ChunkStatus.getMaxDistanceFromFull() 会返回 12，具体请看 1.6 小结。

4 加载等级类型 LEVELTYPE

在加载等级决定了这个区块能运算什么, 一共有四中类型

`net.minecraft.server.world.ChunkLevelType`

```
public enum ChunkLevelType {    INACCESSIBLE,    FULL,    BLOCK_TICKING,    ENTITY_TICKING;}
```

加载等级与加载等级类型的关系:

`net.minecraft.server.world.ChunkLevels#getType`

```
    public static ChunkLevelType getType(int level) {        if (level <= 31) {            return
ChunkLevelType.ENTITY_TICKING;        } else if (level <= 32) {            return ChunkLevelType.BLOCK_TICKING;        } else
{            return level <= 33 ? ChunkLevelType.FULL : ChunkLevelType.INACCESSIBLE;        }    }
```

加载等级类型决定了区块运算。

- 运算实体 (强加载) `ENTITY_TICKING` ： 当加载等级小于等于 31 时为运算实体，会计算全部的游戏流程，包括实体等等。
- 基础运算 (弱加载) `BLOCK_TICKING` : 当加载等级小于等于 32 时为基础运算，不会运算实体，其他正常运算，如红石原件。
- 加载边界 `FULL` ： 当加载等级为 33 时为加载边界，几乎不会有游戏流程运算，但是会记录实体，也就是说在加载边界的里的实体会被记录上刷怪上线。
- 不可访问 `INACCESSIBLE` ： 当加载等级在大于 33 时为不可访问，这些区块并没有真正地加载，只是进行了部分的部分的世界生成。

5 区块实例 CHUNKHOLDER

不在本文的讨论范围，但会简略的介绍 ChunkHolder 的作用。

`ChunkHolder` 是区块管理系统底层中区块的容器，存储了区块本身、区块加载等级、区块生成状态和区块的允许用途。大约可以看成区块在游戏内部的实例。

6 区块状态 `ChunkStatus`

不在本文的讨论范围，但会简略的介绍 ChunkStatus。

`ChunkStatus` 对应了区块生成阶段，1.20.1 中有：

- `empty`：空区块；
- `structure_starts`：获取区块中可能起始生成器结构；
- `structure_reference`：用于世界生成；
- `biomes`：确定生物群系；
- `noise`：生成地形的大致轮廓与基岩层；
- `surface`：替换地表附近的方块；
- `carvers`：生成洞穴和峡谷；
- `features`：生成海底洞穴和峡谷；
- `initialize_light`：计算区块的初始光照?；
- `light`：计算区块的光照；
- `full`：区块加载完成 `ProtoChunk` 转换为 `WorldChunk`。

加载等级对应的区块状态：

- 34-：`full`；
- 35：`initialize_light`；
- 36：`carvers`；
- 37：`biomes`；
- 38~45：`structure_starts`。

---

slug: ticket-mechanics
title: 加载票系统的运作细节
title-en: null
author: RoyanAB
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-02-11T16:54:10+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 2
  is-advanced: false
  banner:
  src: img/02ChunkTicketManager.png
  alt: img.png

本章围绕 ChunkTicketManager `net.minecraft.server.world.ChunkTicketManager` 展开，了解其的运作。

1 CHUNKTICKETMANAGER 的基本架构和其作用

2 对于玩家的处理

2.1 玩家视野与模拟距离

2.2 玩家加载票的更新

---

slug: glossary
title: 中英术语表
title-en: Glossary
author: BFladderbeanawa
co-authors:

- Arcadi4
  date: '2026-04-05T22:16:03+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

一些跟本文有关的译名，大部分取自于 wiki。

全称	译名
Lazy Processing Chunk/Lazy Load/Block Ticking	弱加载
Entity Processing Chunk/Entity Ticking	强加载
Loading Border Chunk/Border Load	加载边界
Inaccessible	不可访问
RenderDistance/ViewDistance	渲染距离
SimulationDistance	模拟距离
Ticket	加载票
Level	加载等级

---

slug: helper-mods
title: 辅助 mod
title-en: Helper Mods
author: RoyanAB
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-01-24T19:25:44+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 2
  is-advanced: false
  banner:

src: ./img/chunkdebug.png
alt: 演示

---

1 CHUNKDEBUG

链接:Modrinth

一款可以查看服务器上个个地方区块加载情况。

2 INTRICARPET

链接:Modrinth

一个 carpet 扩展，可以关闭玩家区块加载，方便调试。

`/interaction chunkloading false`

3 MESSMOD

链接:Modrinth

一个有多种功能的工具箱。

CHAPTER 4

MASA MOD

---

slug: masa-gadget
title: Masa Gadget
title-en: null
author: SDU-Little-River
co-authors:

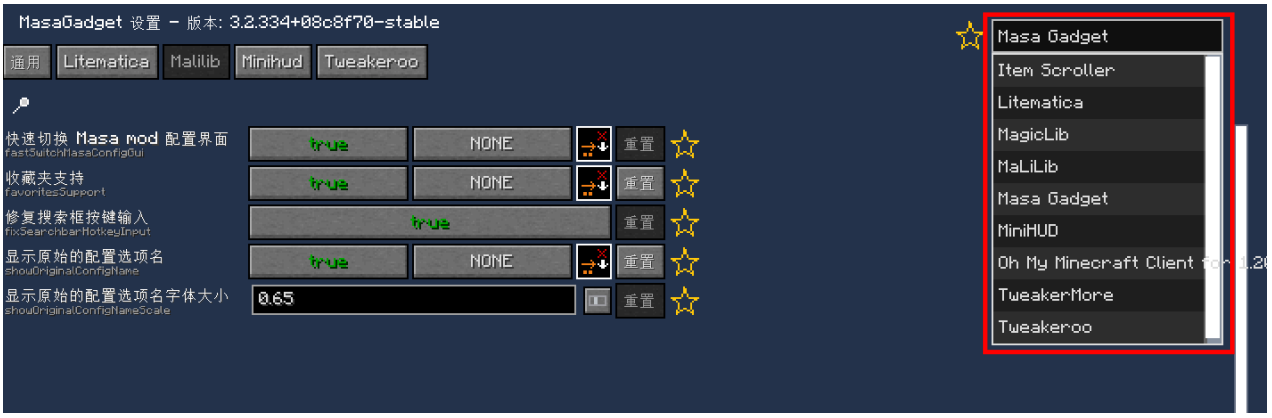
- Arcadi4
- BFladderbeanawa
  date: '2025-01-23T16:21:48+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

Masa Gadget 为各个 masamod 添加了一些额外的功能，同时本身也提供了村民信息渲染，搜寻可刷怪位置等额外功能

1 通用功能

1.1 快速切换 MASAMOD 界面

安装 Masa Gadget 后默认开启，在任何一个 masamod 配置界面的右上角可以快速切换到其他的 masamod 配置界面中
再也不会为了忘记某个模组配置界面的快捷键而烦恼



1.2 收藏夹功能

默认关闭，开启后可以将 masamod 的条目添加到收藏夹中，方便下次更改, 效果如下

2 LITEMATICA

2.1 更好的轻松放置模式 & 修复轻松放置协议

默认开启，在开启轻松放置后仍然可以打开容器 (不一定有效),修复了一些投影放置时方块方向错误的 bug (还是有错)

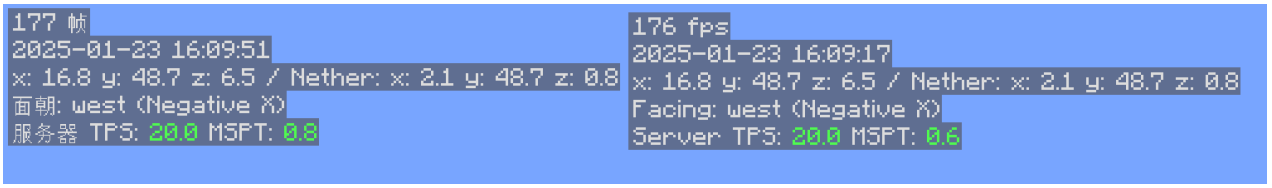
2.2 在服务器中保存投影时保存容器数据

默认开启，需要plusls-carpet-addition的支持

3 MINIHUD

3.1 MINIHUD 翻译

Minihud 显示的文本可以被翻译, 效果如下



3.2 PCA 同步协议同步蜂巢数据

按下 §6 容器预览 §r 的快捷键将会使用 PCA 同步协议来 同步蜂巢，蜂箱数据

---

slug: litematica
title: 投影模组教程
title-en: null
author: SDU-Little-River
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-01-23T11:50:01+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 2
  is-advanced: false
  banner:
  src: https://www.xhbsh.top/img/mod/litematica/img.png
  alt: ''

---

1 投影模组简要介绍

默认情况下, 投影主菜单的快捷键位是 M, 打开后即可进入投影的主菜单

在配置菜单中可以更改快捷键及开关投影的各种功能
在主手或副手中持有投影工具时, 左下角会显示投影菜单栏

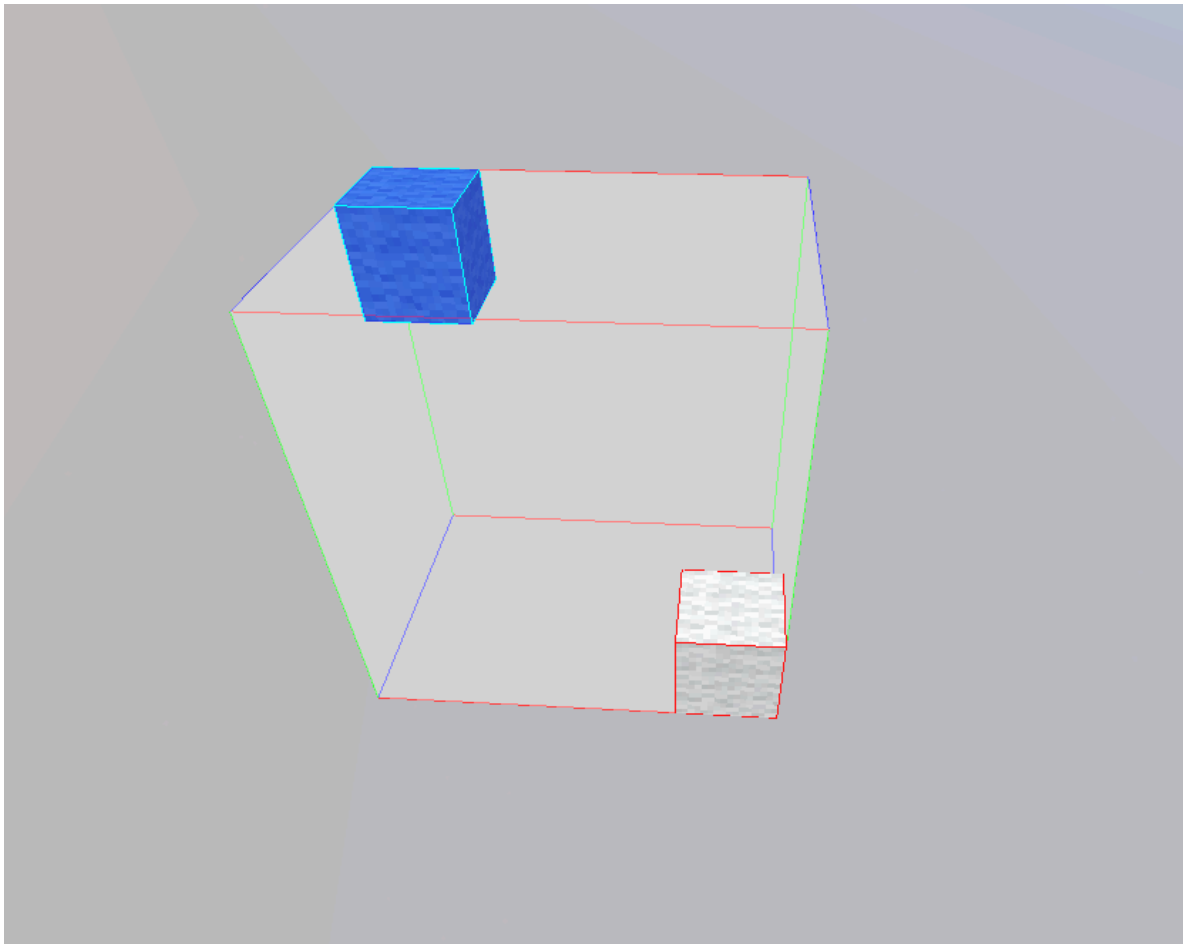
使用 ctrl + 滑动滚轮即可切换模式.

1.1 模式 1-- 区域选择

区域选择共有两种模式, 可以在主菜单中切换, 推荐新手使用简单模式, 并且在区域编辑器中将角点模式设置为角点

1.1.1 简单模式

在简单模式下, 仅有一个选区, 使用左键和右键分别确定两个投影原点即可将投影的选区确认
使用鼠标中间点击角点 (选区其他位置) 可以选中角点 (选区),选中后按住`alt`滚动鼠标滚轮可以移动角点 (选区)

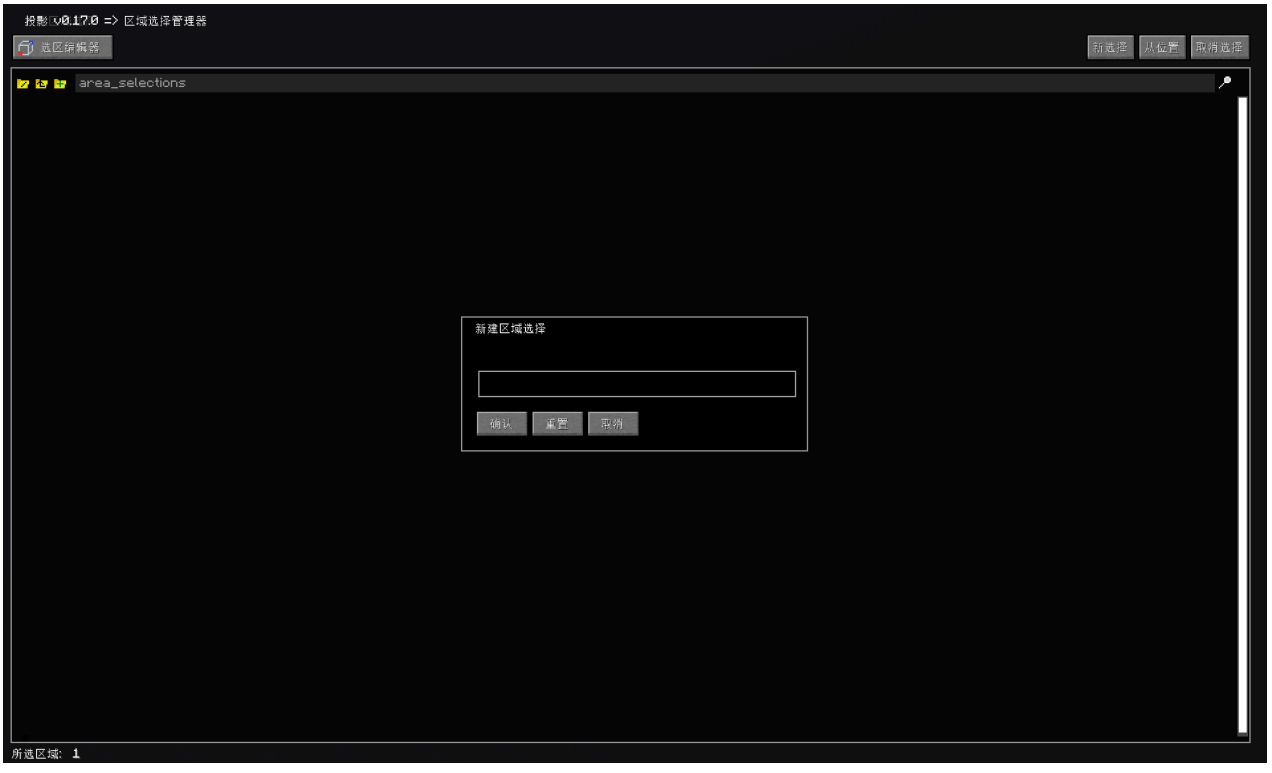


在选区编辑器中可以更改选区的名称或这保存原理图

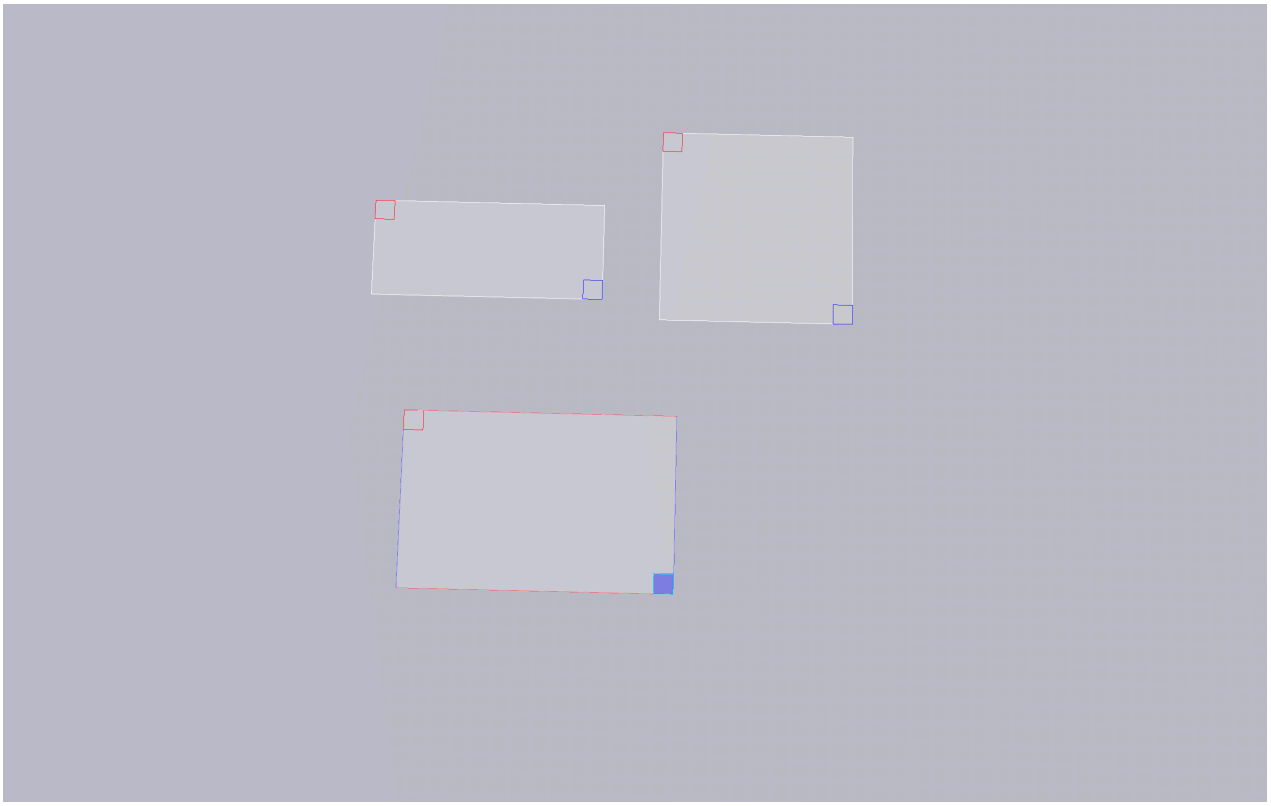
1.1.2 普通模式

对于大型的机器, 如果使用简单模式进行保存, 会导致建造时的种种不便, 使用普通模式可以很好的解决这一问题
在普通模式下, 选区并不会被自动创建, 需要玩家手动创建选区, 步骤如下

1. 打开 `投影主菜单-选区目录-新选择`



2. 如果需要新建子选区, 在 `投影主菜单-区域编辑器` 中选择新建子区域, 也可以在 配置菜单 - 热键 中修改 `添加选区` 的快捷键以方便快捷添加一个选区
使用鼠标中键可以选中子选区角点 (子选区)
最终结果如图



保存原理图的步骤和简单模式一样

1.2 模式 2 原理图放置 & 模式 5 原理图粘贴

导入外部的原理图 (没有则跳过)
建议开启投影设置中的自定义原理图目录, 这样可以在所有版本共用一个保存原理图的地址

然后将网上下载到的投影看那个放入这个文件夹即可

1. 在游戏中加载原理图

`投影主菜单-加载原理图` 选中所需的原理图后在左下角点击加载原理图即可

1. 调整原理图的位置, 旋转原理图

使用鼠标中键选中原理图, alt + 鼠标滚轮移动位置

`投影主菜单-配置原理图` 选中所需的原理图, 点击配置按钮进入配置界面

在配置界面右侧可以调整投影原点的位置, 旋转 / 镜像原理图

注意! 除非作者说明, 否则不建议在建造机器的时旋转 / 镜像, 这极有可能造成机器无法正常运行

1. 粘贴原理图 (仅创造)

CHAPTER 5

MICRO TIMING

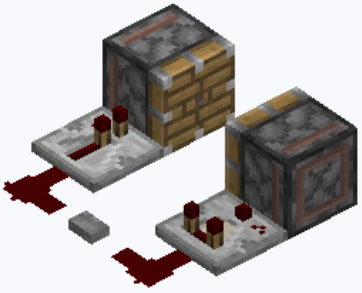
---

slug: intra-tick-timing
title: 初窥刻内时序
title-en: null
author: hotpad100c
co-authors:

- Arcadi4
- BFladderbeanawa
- Gudu-Z
- tanhHeng
- SDU-Little-River
  date: '2024-12-21T23:48:41+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 2
  is-advanced: false
  banner:
  src: ./img/TickPhase.jpg
  alt: 各个阶段

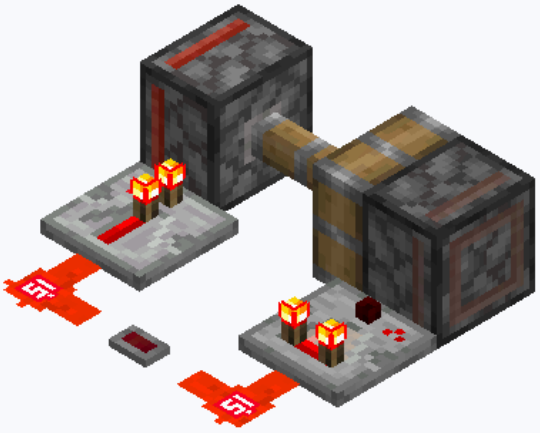
---

1 刻内时序的观测

> *这天小 B 做了一个装置：*
>
> 

请各位猜猜按下按钮会发生什么？

1. 什么都不会发生
2. 左边活塞伸出了
3. 右边活塞伸出了
4. 两个活塞同时推出
5. 游戏崩溃

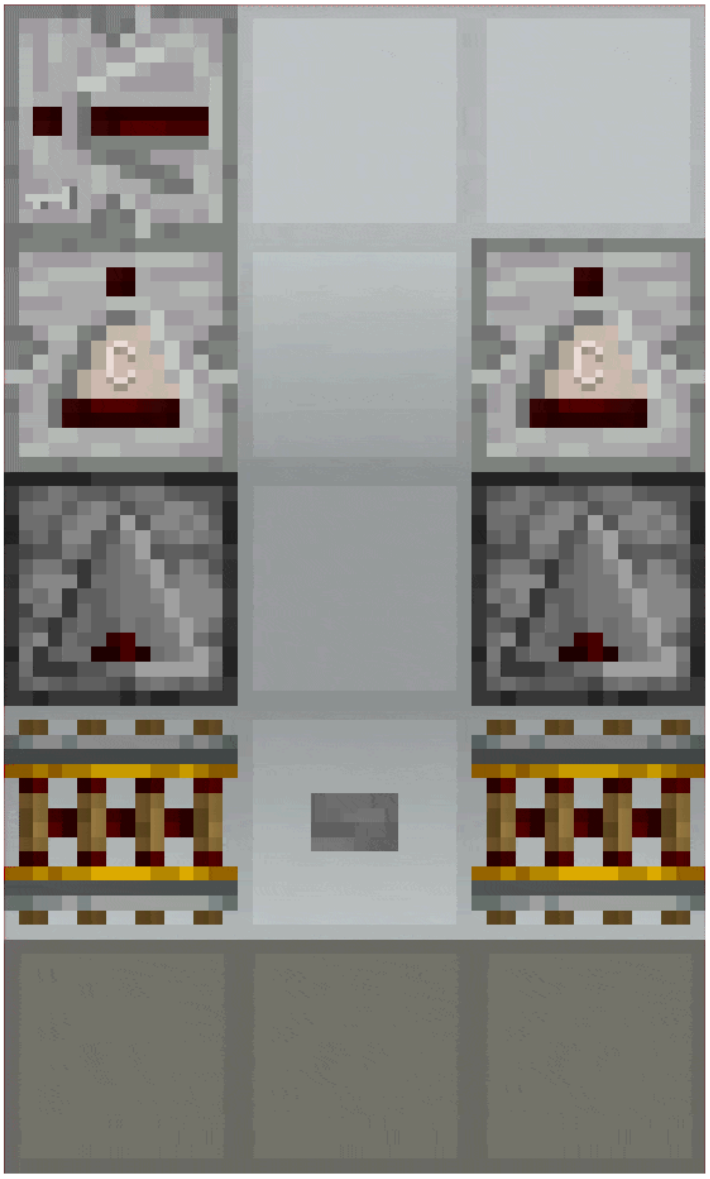
> *于是小 B 按下了按钮*
>
> 
>
> *左…… 左边的活塞伸出了！！！！！！*
>
> *根据前两天看01 - 刻与刻间时序学到的知识：他进行了以下计算：*

> *这个中继器延迟 2gt，那个比较器延迟 2gt……？哇两个活塞应该同时推出！那为什么只推出了一个活塞呢？明明是同一 gt 却有个先后顺序，gt 不是最小单位了？*
>
> *面对这样的疑问，聪慧至极的小 B 给出了惊人的结论：~~游戏出 bug 了！~~*

当然不是这样的！所有的玩家都应该建立一个这样的一个观念：在 Minecraft 中，大多数游戏逻辑是运行在单线程中的，这意味着从逻辑层面上看，事件的执行必定是有顺序的，而非严格意义上的同时！

于是一切都解释的通了，在同 1gt 内必定存在某种更加精细的时序，我们称在 1 刻内的精细时序为**刻内时序**。正如前文所提到的，进行如上的~~明明理论上是同一 gt 发生但偏偏有个先后顺序~~的测试，便完成了一次**刻内时序的观测**。

这里还有个例子：
当按下按钮，两个装置分别出现了以下症状：



明明在同一 gt 内，加个中继器就大有不同，怎么有这么诡异的事情！为什么呢？本篇就将简要的探讨以上现象！

什么你问这玩意有什么用？

多的是呢！上到极为复杂的红石装置，下至最最基本的精确时序分析，都用得上。刻内时序理论的研究也在很大程度上促进了红石研究！包括但不限于推动上限检测，BED 等听起来**高大上**的红石装置和理论。

这里选取一段 Void 先生写的文字：

> *...*
> *因此，刻内时序的学习之所以必要，不仅仅是为了解决设计中遇到的问题、解释以前无法解释的经验结论。更重要的是，有很多人轻视刻内时序的学习，不愿意学习刻内时序而将其仅仅视为一个问题的来源，在红石设计中常常遇到刻内时序导致的问题，却殊不知刻内时序是红石设计中最强大的工具之一，善用刻内时序的知识可以大大精简红石电路的设计，并提高红石机器的性能。刻内时序知识的缺乏使得许多人从一开始就不知道这件工具是可以利用的，这个情景从社区的良性发展的角度，是需要尽力避免的。*

2 微时序理论

2.1 微时序的阶段划分

1gt就像现实世界中的**一天**，而在一天之内，我们又会有一定顺序的安排：起床、早饭、午饭、晚饭、睡觉…… 在 Minecraft 中，以 1gt 为最小单位的时序称为宏观时序，而某 1gt 内的更加细分的时序就称为微时序。

在前文中我们知道，MC 执行任何东西总是有优先顺序的，我们通过源码、实验等途径发现 ——Minecraft 在 1gt 内总是大致按照以下顺序执行固定的事件：

1. **世界时间更新 World Tick Update**，简称WTU。游戏内存在一个和世界绑定的「计时器」，在世界创建时「计时器」被初始化为 0。在**本游戏阶段**内，该「计时器」自增 1。我们称第 n 刻就是：使这个「计时器」增加到 n 的那个**世界事件更新阶段**的刻。
2. **计划刻 Schedule Tick / Tile Tick / Next Tick Entry***，简称TT/NTE*。大部分具有延迟执行行为的元件都是由「计划刻」控制，在**计划刻阶段**执行的的。*（在此处提及 NTE 和 Next Tick Entry 的**错误**译名旨在帮助各位读者看懂部分早期的其他文档，不希望各位继续使用这两个名字。）*
3. **区块刻 Chunk Tick**，简称CT。区块刻由分为**雷，雪**和**随机刻**（Random Tick）。在此阶段内，游戏先处理**雷、雪**等事件；然后处理**随机刻**：当在存在玩家在一定距离内时，会发「生作物生长」、「草方块蔓延」、「水结冰」等随机刻事件。在每个**区块刻阶段**，游戏遍历玩家附近的所有区块，然后在这些区块里面随机选取方块，执行这些事件。
4. **方块事件 Block Event**，简称BE。像是**活塞推拉，音符盒发出声音**都是在这个阶段。拿活塞举例，当活塞发现自己的**【实际状态】**和**【供能的状态】**不符时，就会添加一个**「方块事件」**，在**方块事件阶段**内添加的**「方块事件」**会在本**方块事件阶段**执行，而在**方块事件阶段**之外的阶段添加的**「方块事件」**会被放到未来最近的一个**方块事件阶段**执行。继续上面活塞的例子，活塞会在**此阶段**在要推 / 拉的方块的预定位置摆上移动中的方块 (b36) 。
5. **实体运算 Entity Update**，简称EU。实体需要每刻都会主动**进行运动、实体 AI 等行为**。所有的**实体行为**，比如「生物运动」、「TNT 爆炸」、「怪物攻击」，都发生在**本阶段**。非玩家「激活」压力板、绊线，使得它们产生红石信号，也发生在这个阶段。
6. **方块实体 Block Entity / Tile Entity**，简称TE。有部分方块需要每刻**运行自己相关的逻辑**，这些事情发生在**方块实体阶段**。在**方块实体阶段**，漏斗会「吸取」、「传输」物品；被活塞推动的方块，会变成移动中的方块（b36），它们会在自己创建后的前两次**方块实体阶段**进行「推动实体」的逻辑，并且在接下来的第三个**方块实体阶段**变回普通的方块。
7. **异步事件 Async Task / Network Update**，简称AT/NU，也可称为 Player Action / 玩家操作。「玩家操作」实际上是从客户端发往服务端的网络数据包。在**本阶段**，也就是每一刻的结尾，服务端会统一执行所有在这一刻内收到的「玩家操作」数据包。

这些是在平常分析时主要涉及的游戏阶段，后文中我们也会围绕这些事件展开叙述。

2.2 瞬时

在上文中，我们认识到 1gt 内又划分成了不同的阶段。就像早饭只能在早上吃、午饭只能在中午吃一样，Minecraft 中的许多事件只能遵循一定的顺序，发生在特定的阶段。但还有一些 “立刻响应” 的事件，就像我们渴了就可以立刻去喝水，而不关心早上、中午、还是晚上。

如果一个方块的行为可以在任意阶段发生的、只由**方块更新**触发，那么这个方块就称为**瞬时元件**。

举例来说，红石粉亮灭是瞬时的。如果玩家关闭了一个拉杆，那么可以导致红石粉在玩家操作阶段熄灭；如果活塞推走了一个红石块，就可以导致红石粉在方块事件阶段熄灭。

2.3 延时

既然有瞬时事件，就必然有 “延时” 事件。举例来说，如果玩家放置了一个红石块，红石块激活了一个中继器，这个中继器就会在 2gt 后亮起。中继器亮起的这一行为具有**延时**，且**由游戏控制**（由计划刻控制），在**特定阶段发生**（计划刻阶段），中继器就不是瞬时事件。

在接下来的几部分中，我们将详细地展开这些**不同的游戏阶段**。

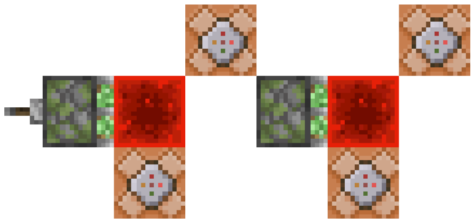
2.4 常见元件的运行阶段

元件种类	运行阶段
命令方块运行指令	计划刻 TT
中继器、比较器、红石火把、侦测器的亮灭	计划刻 TT
红石粉、铁轨改变状态	瞬时
栅栏门、活板门改变状态	瞬时
漏斗因红石信号改变状态	瞬时
漏斗吸收、传递物品	方块实体 TE
音符盒、钟因红石信号改变状态	瞬时
音符盒、钟发出声音	方块事件 BE
发射器、投掷器因红石信号改变状态	瞬时
发射器发射 / 投掷器投出物品	计划刻 TT
红石灯的亮起	瞬时
红石灯的熄灭	计划刻 TT
按钮、压力板、绊线的亮起	瞬时[1]
按钮、压力板、绊线的熄灭	计划刻 TT
重力方块判定下落、创建实体	计划刻 TT
重力方块下落、到位	实体运算 EU
活塞推出或收回	方块事件 BE
b36[2] 推动实体	方块实体 TE
b36 自然到位	方块实体 TE
b36 被粘性活塞收回到位[3]	方块事件 BE

2.5 刻内时序基础分析

在这里，我们给出一个基本的例子。

下图中，所有的命令方块内指令都为 time query gametime



上升沿 *（拉下拉杆，粘性活塞推出）*：

刻数	阶段	事件
0	AT	玩家拉下拉杆
0	AT	第一个活塞添加方块事件
1	BE	第一个活塞推出
3	TE	第一个红石块到位
3	TE	命令方块计划在 4 刻（延迟 1 刻）运行
3	TE	第二个活塞添加方块事件
4	TT	第一个命令方块读数 4
4	BE	第二个活塞推出
6	TE	第二个红石块到位
7	TT	第二个命令方块读数 7

下降沿 *（拉回拉杆，粘性活塞收回）*：

刻数	阶段	事件
0	AT	玩家关闭拉杆
0	AT	第一个活塞添加方块事件
1	BE	第一个活塞收回
1	BE	第一个红石被撤去
1	BE	第二个活塞添加方块事件
1	BE	第二个活塞收回
3	TE	两个红石块到位
4	TT	两个命令方块都读数 4

（改自 void 的例子）

FOOTNOTES

1. 但是压力板的亮起只能由实体运动触发，所以实际上只能在实体运算或玩家操作；类似地，按钮只能被玩家或者箭矢按下，所以也限制在这两个阶段。 ↩
2. b36 : minecraft:moving_piston, 移动中的活塞。 ↩
3. 当粘性活塞收回的时候前方是 b36，则会立刻将其到位，也就是所说的粘性活塞短脉冲。 ↩

---

slug: tick-timing
title: 刻与刻间时序
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- SDU-Little-River
- hotpad100c
  date: '2024-12-14T20:46:48+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 1
  is-advanced: false
  banner:
  src: ./img/OrderBetweenTicks.jpg
  alt: 不同时间亮起的灯

刻与刻间时序

> *我们在前文中已经学习了更新理论了，但是好奇的小 B 决定不止于此。他拿出了创造物品栏的~~雷石东直放站~~红石中继器，拉下了拉杆。*
>
> *也许是小 B 太过敏感，当他拉下拉杆的时候，他发现红石中继器好像没有立刻亮起，他以为游戏出现了虫，便多尝试了几次。他很快意识到这是个稳定的，可复现的现象。很显然，与前文中我们探讨的大量瞬时现象不同，这里出现了所谓**异步**的现象，他称这种现象为**延迟***

1 游戏刻，红石刻与延迟

在 Minecraft 中有大量的事件需要处理，这些事件被按顺序放在一个大循环里，我们称这个循环为游戏主循环，在不使用 */tick* 命令，游戏正常运行的情况下，这个主循环总是以约 **每秒 20 次** 的间隔循环，我们称这个最短间隔为 **GameTick**，简称 **gt**。由于历史遗留问题，当前部分情况下我们仍采用**红石刻 RedstoneTick**的记号，简称**rt**。其与 gt 有如下换算关系：$1rt = 2gt$。

这意味着：

1. 1gt = $\frac{1}{20}$秒 = 0.05秒
2. 即使 Minecraft 算力充足，在不到 0.05 秒内计算完了游戏在本 gt 需要计算的全部内容，也会停止计算直到下一 gt 到来。

因此，玩家定义了两个指标以衡量游戏是否卡顿，分别是 **TPS** 和 **mspt**。TPS 全称 Tick Per Second 指的是每秒钟游戏执行了多少游戏刻。通常而言，这个数值是 20，但当游戏卡顿时，其将会低于 20。而 mspt，全称 Millisecond Per Tick，指的是当前游戏执行每一刻所需要花费的毫秒数，数值越低则游戏卡顿越低。

值得注意的是，mspt 总是与 TPS 保持着如下关系：

- 当 mspt 低于 50 时，TPS 为 20
- 当 mspt 大于 50 时，TPS = $\frac{1000}{mspt}$

在游戏中，绝大部分红石元件的响应是有**延迟**的。元件的**宏观**延迟我们在此处将其单位定义为**gt**，部分元件的延迟如下：

- 中继器：2-8gt
- 比较器：2gt
- 侦测器：2gt
- ...

2 中继器，比较器初步

中继器和比较器具有宏观延迟。中继器的延迟和其设定档位有关，而比较器的延迟总是 2gt。

中继器的基本作用是增强信号。即将任何强度不为 0 的红石信号在延迟指定游戏刻后变为强度为 15 的信号。

当中继器侧面被中继器或比较器直接贴脸输入时，则会进入 “锁存” 状态，此时，其输入端的任何改变均不影响其亮起或熄灭状态，直到其侧面输入结束，等待延迟对应 gt 后，若状态需要更改，则更改其状态。

比较器总共有 3 个输入端，分别为前面和两个侧面输入。当侧面无任何输入时，其总是会直接输出等同于前面输入的信号强度的信号。当其侧面存在输入时，比较器的行为取决于其模式。比较器有两种模式，分别为：

- 比较模式
- 减法模式

其默认为**比较模式**。当比较器处于**比较模式**时，其总是遵循如下逻辑进行输出：

> *取左侧输入和右侧输入的最大值，若其最大值大于其前端输入，则不进行任何输出，否则输出其前端输入。*

当比较器处于**减法模式**时，其总是遵循如下逻辑进行输出：

> *取左侧输入和右侧输入的最大值，若其最大值大于其前端输入，则不进行任何输出，否则输出其 (前端输入 - 侧面输入的最大值)。*

比较器与容器的互动在此处不做讨论。

3 充能理论

我们建立充能理论以解释一种发生在中继器与比较器等元件的现象，其重要特征为隔着方块充能红石粉或其它中继器 / 比较器。

我们定义对于任意**可充能**方块，其有两种充能状态：**强充能** 和 **弱充能**。

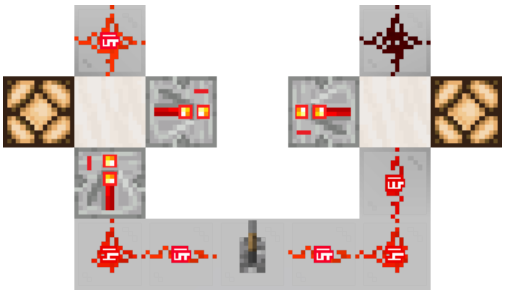
当可充能方块被强充能时，其标志性特征为可以激活其周围 6 个方向的红石粉，无论其指向。而弱充能时仅能激活输入端对着可充能方块的中继器，比较器和部分其他红石元件。这也被作为辨别两种充能的重要办法。

以下方法可以使得方块被强充能：

- 中继器，比较器，红石火把，侦测器直接对着可充能方块发出信号。

以下方法可以使得方块被弱充能：

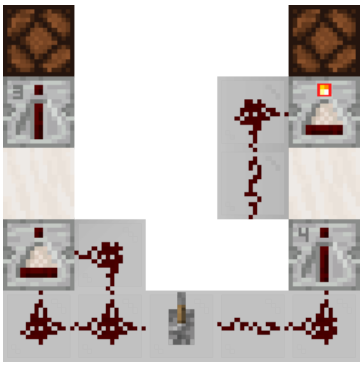
- 可充能方块被信号强度不为 0 的红石粉指向。



在 MC 中，可充能方块是大多数，因此我们在此以不可充能方块的显著特征 —— 不压线，来区分两种方块。压线指的是一种现象，绝大多数方块在会切断上下连接的红石粉，我们称这种现象为压线。但部分方块，如玻璃，台阶，半砖，箱子等并不具备此现象，这也意味着他们是不可充能方块。我们在这里定义活塞 / 粘性活塞也为不可充能方块。

4 刻间时序

> *此部分重点用于讲解基础时序分析。*



在这里我们给出一个例子来使用 gt 的概念来分析最简单的时序。

如图所示，当拉下拉杆时，右侧中继器会在延迟 8gt 后被激活，接下来强充能方块，此时处于减法模式的比较器侧面的红石信号强度为 14，而正面输入的信号强度为 15，15-14 = 1 > 0，在延迟 2gt 后激活红石灯，共延迟 10gt。

左侧的比较器处于比较模式，其正面和侧面的信号强度都是 14，两侧强度相等，延迟 2gt 后输出信号强度为 14 的红石信号，强充能了面前的方块，随后中继器在延迟 6gt 后激活红石灯，共计延迟 8gt。

---

slug: block-events
title: 方块事件
title-en: null
author: SDU-Little-River
co-authors:

- Arcadi4
- BFladderbeanawa
- Gudu-Z
- hotpad100c
- tanhHeng
  date: '2024-12-28T22:09:13+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 5
  is-advanced: false
  banner:
  src: ./img/BlockEvent.jpg
  alt: BEHeroImg

方块事件

本篇讨论的是方块事件 (Block Event, BE) 的实现原理。

基础部分

- 方块事件的概念与存在意义
- 方块事件的信息结构与相关元件
- 方块事件延迟 (BED) 的通俗认识

进阶部分

- 方块事件队列与广度优先搜索机制

1 什么是方块事件?

在 Minecraft 游戏交互中，为了减少服务器与客户端之间的网络同步压力，游戏会避免把所有复杂方块的巨量数据都在每一刻同步给客户端。

相反，服务端会选择告诉客户端：“这里发生了一个特定的事件”，然后让客户端的方块自己把服务端发生的事情在本地模拟一遍。

方块事件 (Block Event) 就是为了这种高效的 “通知与模拟” 而生的。

1.1 方块事件的信息结构

当一个方块事件被创建时，它包含且仅包含以下四个核心信息：

- 位置 `pos`：事件在哪里发生
- 方块种类 `block`：是哪一种方块在执行这个事件
- 类型 `type`：是哪种具体的事件动作（对活塞而言，代表伸出或收回）
- 数据 `data`：执行该事件时附带的必要参数（对活塞而言，代表推动的朝向）

2 产生方块事件的元件

> *一部分红石元件会产生方块事件，用途各不相同，包括但不限于以下方块：*

元件种类	事件类型	附带数据
活塞	（伸出 `0`，收回 `1`，瞬推收回 `2`）	活塞朝向 (0~5)
音符盒	`0`	`0`
钟	`1`	敲击方向（0~5）

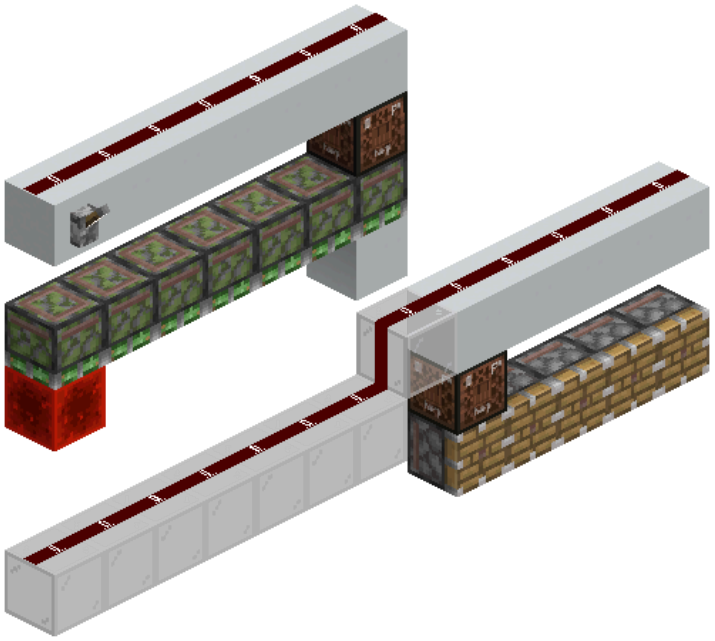
...

3 方块事件延迟 （BED）

当多个方块在同一时间被激活并产生方块事件时，它们会同时执行吗？

Minecraft 的世界观告诉我们：游戏中不存在绝对的 “同时”。即使是在同一游戏刻内，事件的执行也必须有一个先后顺序。这种由执行顺序的时间差造成的现象，就称为**方块事件延迟 (Block Event Delay, 简称 BED)**

下面展示一下方块事件延迟是怎么在游戏中表现的：



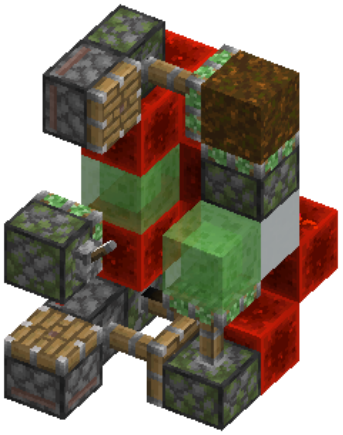
开关拉杆，我们发现，当改变红石块的位置的时候，旁边活塞推出的数量发生了变化，当红石块靠近压线方块的时候，数量减少了，而当红石块远离压线方块的时候，数量增多了。

3.1 深度与队列的通俗理解

你可以把方块事件的执行想象成人们正在排队买票：**先来的先执行，后来的后执行**。

当一个方块事件执行时，它可能会触发后续的新事件。这些新事件会被追加到队列末尾。为了追踪这些事件是由 “哪一代” 产生的，我们引入**深度**概念：初始事件深度为 0，它直接产生的事件深度为 1，以此类推，用以刻画事件间的衍生因果链

下面是一个树场 0t 下吸底座的例子，用于展示方块事件深度是如何在实际设计中应用的:



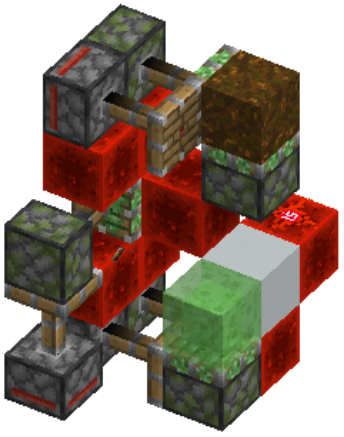
0gt AT：拉杆被拉下

1gt BE 深度 0：粘性活塞开始收回

1gt BE 深度 1：粘性活塞将灰化土拉回，转向粉转向

1gt BE 深度 2：下吸活塞被更新，自检，发现自己被激活，伸出

1gt BE 深度 3：充能方块被移除，更新下吸活塞，自检，发现自己应该收回，添加收回的方块事件，发现自己正在伸出，0t 发生。



3gt TE：全部方块均到位。

4gt BE 深度 0：上下两个普通活塞伸出，泥土的粘性活塞添加方块事件

4gt BE 深度 1：普通活塞伸出，更新粘性活塞，回推粘性活塞伸出，底座激活活塞自检。红石块被移除。

4gt BE 深度 2：上下两个普通活塞收回，发出更新，更新泥土的粘性活塞，粘性活塞自检，发现自己不应该推出，添加收回的方块事件，下方粘性活塞同理

4gt BE 深度 3：红石块粘性活塞和灰化土粘性活塞的 0t 发生，灰化土瞬间到位。红石块到位，控制下吸活塞充能方块的粘性活塞伸出。

6gt TE：红石粉被转向，充能方块到位，底座完成重置。

---

4 方块事件队列与广度优先搜索

在源码层面，方块事件的 “排队” 本质上是一个广度优先搜索的队列机制。

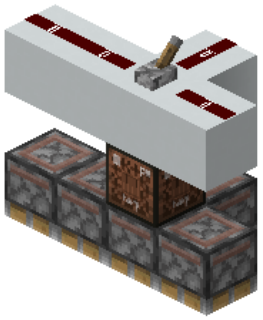
在游戏的主循环中，方块事件被存放于一个先进先出的队列中。当某一源点（例如被拉杆激活的中心活塞）以 NC 更新的顺序向周围发出更新是，周围相应的活塞会依次将自己的方块事件压入队列末尾。

执行时，游戏会从队列头部取出一个事件执行：

1. 取出深度为 `0` 的第一个节点执行
2. 若该更新引发了新的更新，产生里新的方块事件（深度 `1`）会被按更新顺序压入当前队列的最末端。
3. 游戏继续执行队列中下一个深度为 `0` 的节点，直到深度 `0` 的事件全部出队，才会开始执行深度 `1` 的事件。

这一操作完全等价于广度优先搜索。

这里给出一个例子



其中长边从 + x 延伸至 - x 方向

当拉下拉杆时，中心活塞被激活，并以 - x、+x、+z 的顺序更新周围三个活塞，依次将它们压入队列。游戏随后按顺序执行 - x 方向和 + x 方向的活塞，然后更新 + x 方向的下一个活塞。此时会添加方块事件并将其压入队列，位于 + z 方向活塞之后。接着执行 + z 方向活塞，最后执行 + x 方向的最后一个活塞。

我们可以将每个活塞抽象为一个节点，把深度为 0 的活塞视作根节点。需要注意的是，在实际情况中，这个图可能存在多个源点。我们对这个抽象出的 "活塞图" 进行多源 BFS 模拟，其中子节点加入队列的顺序就是 NC 的更新顺序，这个顺序由子节点相对于父节点的方向决定。已被访问过的子节点将不会被重复访问。

---

slug: block-entities
title: 方块实体
title-en: null
author: Molforte
co-authors:

- Arcadi4
- BFladderbeanawa
- SDU-Little-River
- hotpad100c
- tanhHeng
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 4
  is-advanced: false
  banner:

src: ./img/BlockEntities.jpg
alt: 带有方块实体的方块们

方块实体

1 方块实体的概念与内容

方块本身通过预定义的有限 BlockState 保存数据；通过默认的方块渲染行为渲染模型；通过更新系统、计划系统等决定执行逻辑。
但是，这样的模式具有局限性，无法直接满足在游戏交互中一些方块的功能。

方块实体为此而生。它为普通方块提供了三个重要的额外功能：

- ***使用 NBT 存储数据***
- ***自定义的渲染行为***
- ***在每一次 tick 中进行更新***

1.1 方块实体的内容

方块实体是因方块而异的，但一个抽象的，基本的方块实体含有以下基本信息与功能:

- type 类型
- world 所在的世界
- pos 位置
- removed 是否已被移除
- cachedState 缓存的，对应方块的状态

1.2 方块实体与方块

方块实体与世界中的方块绑定。一个世界里，每个方块位置仅可有一个方块实体实例。

仅有经过特殊声明的方块可以合法地与指定种类的方块实体共存。
方块实体随着方块一同添加与移除。但通过**更新抑制**等特殊手段可以在方块移除时保留其方块实体。

2 重要的带有方块实体的方块

2.1 漏斗

在 Minecraft 中，漏斗是一种重要的方块实体，主要用于物品的自动化传输。

2.1.1 漏斗方块实体携带的信息

一个可爱的漏斗方块实体携带以下基本信息:

- `inventory` 储存空间（5 格）
- `transferCooldown` 冷却（默认为 - 1）
- `lastTickTime` 上一 tick 的世界时间
- `facing` 朝向

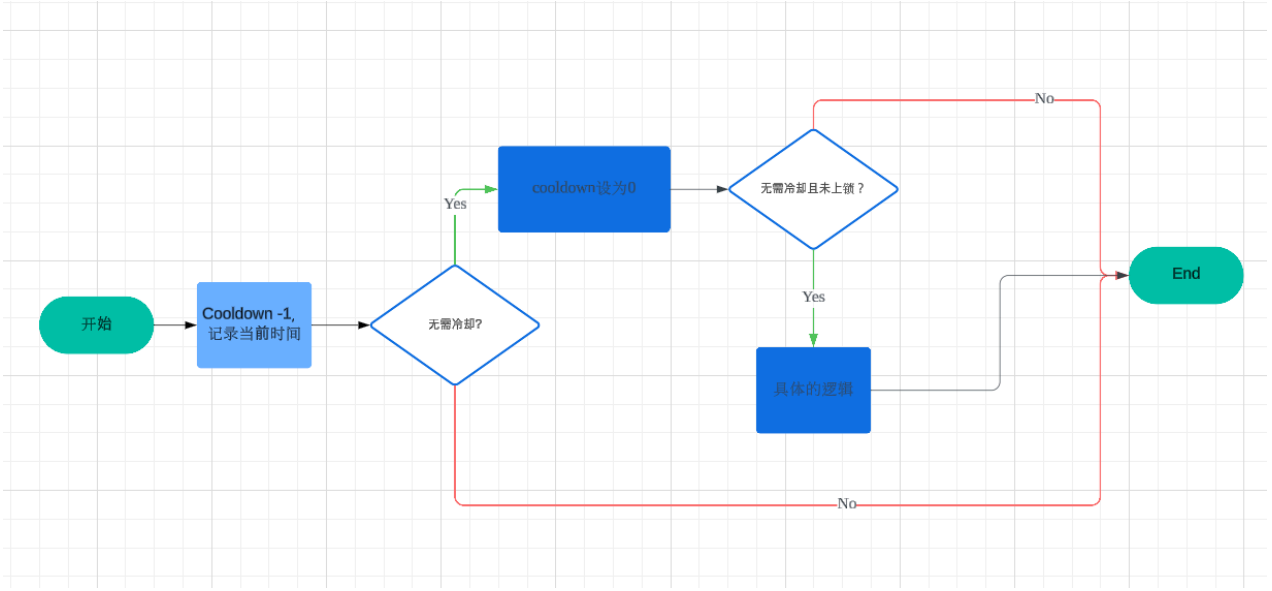
2.1.2 漏斗方块实体的功能

- **将物品传输到目标容器**
- **从容器中吸取物品**
- **尝试吸取物品实体**

2.1.3 漏斗方块实体的工作流程

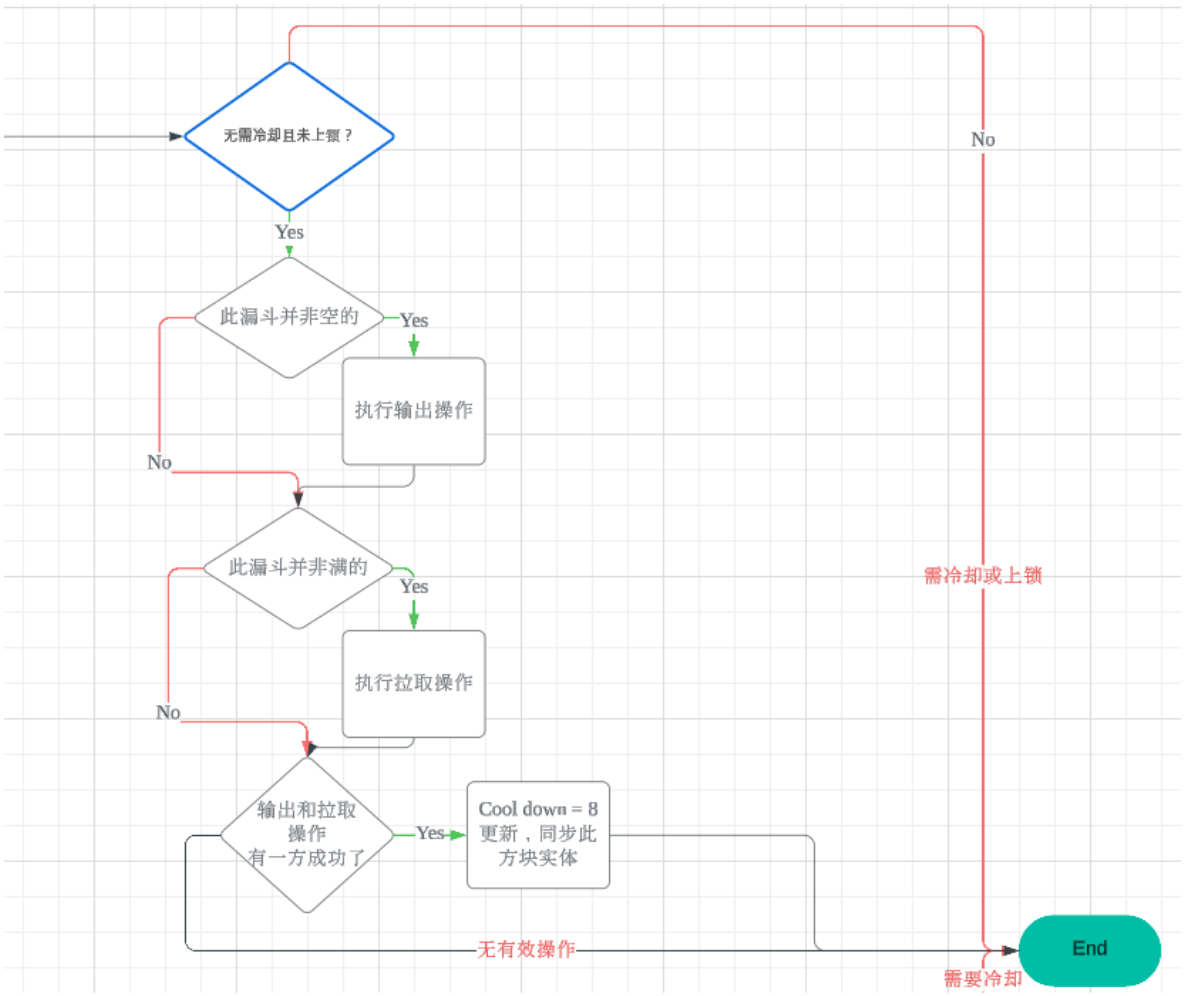
每个游戏刻（tick），漏斗都会执行以下步骤：

1. **减少冷却时间（cd, Cooldown）**
   - 若 cd 仍大于 0，漏斗不会执行任何拉取或输出操作，而是直接结束运算。
   - 若 cd 归零，漏斗会开始尝试执行**输出**和**拉取**操作。
2. **执行物品传输逻辑**
   - **首先尝试输出**（将物品放入目标容器）
   - **随后尝试拉取**（从上方容器吸取物品）
   - **如果吸取或输出成功**，则漏斗会重置冷却时间，并同步更新自身数据。



现在，让我们移步到***尝试拉取和输出的具体的逻辑***。

当漏斗确定自己需要做些什么时，它会先尝试向指向的容器输出。随后，它会尝试从上方吸取物品。最后，如果吸取和输出中有任意一方成功了，它就会设置冷却，并更新同步自己的数据。



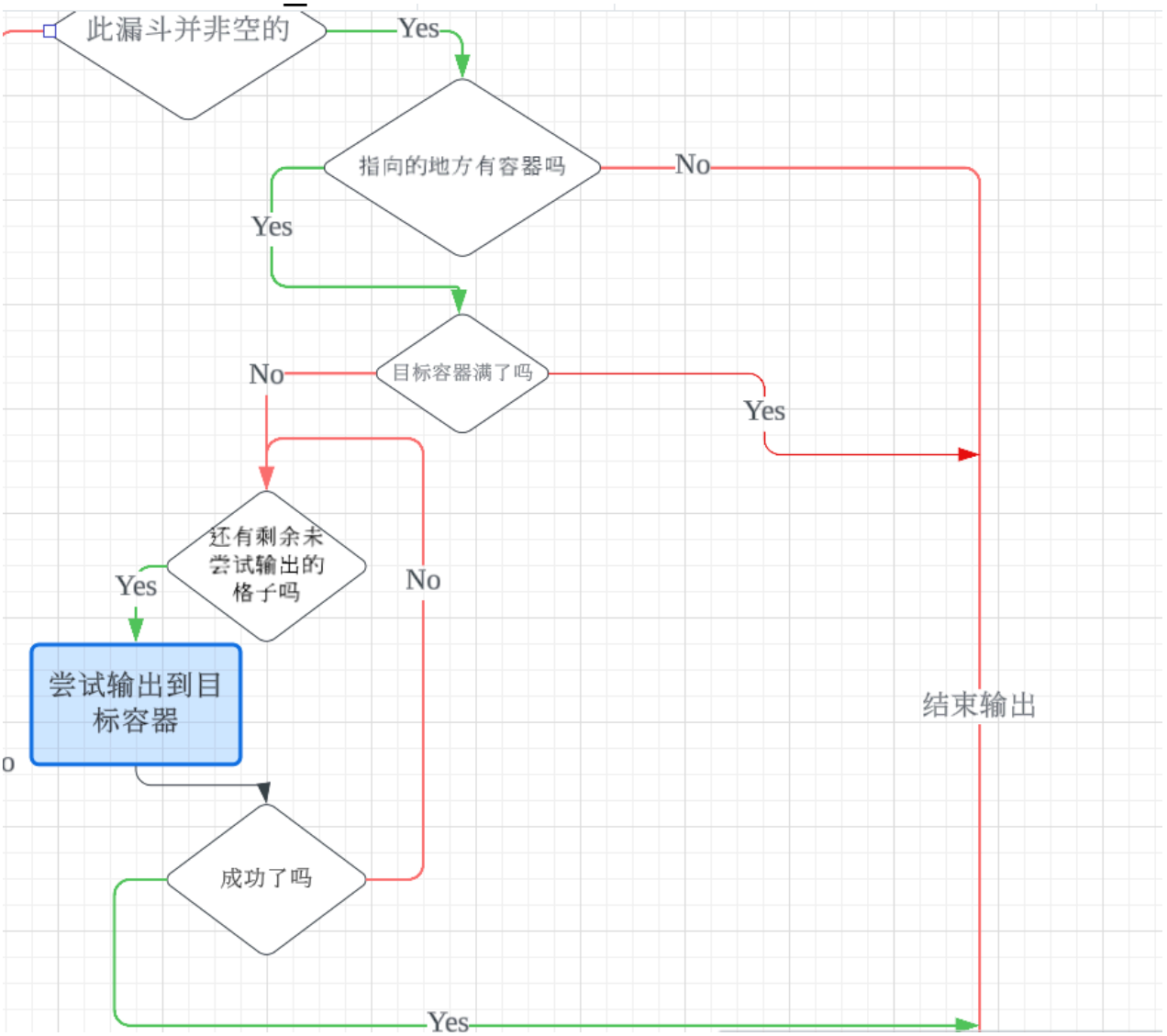
我们分别来看输出和拉取的逻辑。

先看***输出***...

输出输出操作由 `insert()` 方法完成，它的作用是将物品从漏斗传输到目标容器，并返回一个布尔值来表示是否成功传输。如果传输成功，则漏斗进入冷却状态。

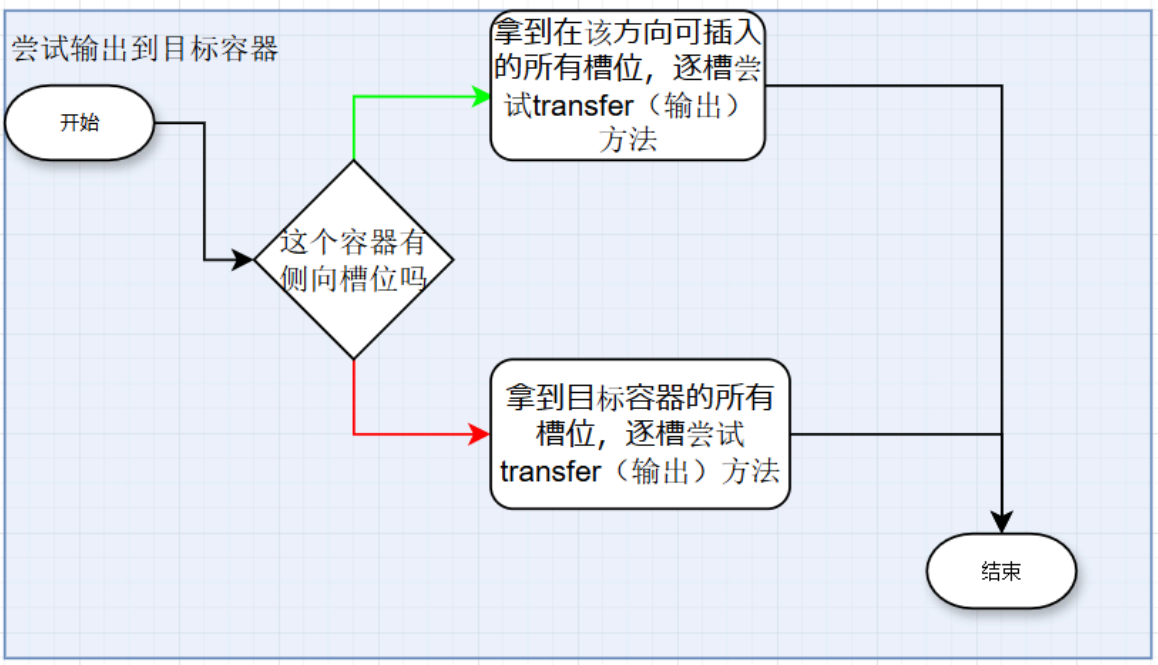
输出的基本逻辑：

1. **确认目标容器是否可用**
   - 如果目标容器已满，则输出失败。
   - 如果目标容器有可用槽位，则尝试传输物品。
2. **尝试输出物品**
   - 遍历漏斗中的物品槽，逐个尝试传输物品到目标容器。
   - 只要有一次传输成功，就返回 “成功”；否则返回 “失败”。



当调用 `transfer()` 方法尝试传输物品时，`transfer()` 方法会返回一个物品组：

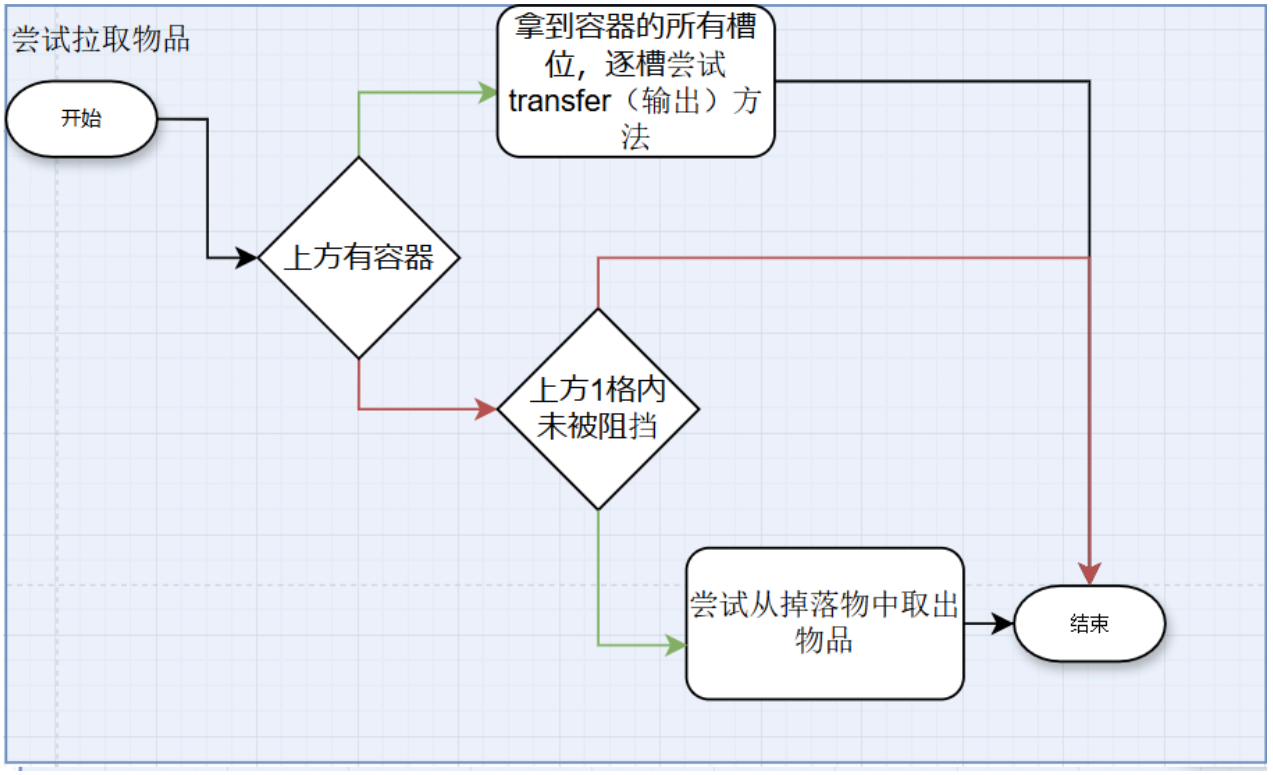
- 若返回值为空，则说明物品已完全传输，输出成功。
- 若返回值不为空，则说明物品未能全部传输，输出失败。



到此，输出的部分已经完成，接下来，我们来看看***拉取***的部分。
拉取物品的逻辑与输出类似，但方向相反：

- **输出** 时：漏斗 `transfer()` 物品至**目标容器**。
- **拉取** 时：某个**容器** `transfer()` 物品至**漏斗**。

由于 `transfer()` 方法的逻辑马上会详细解释，因此这里不再赘述。



`transfer()` 是漏斗实现物品传输的核心方法。它接收以下参数：

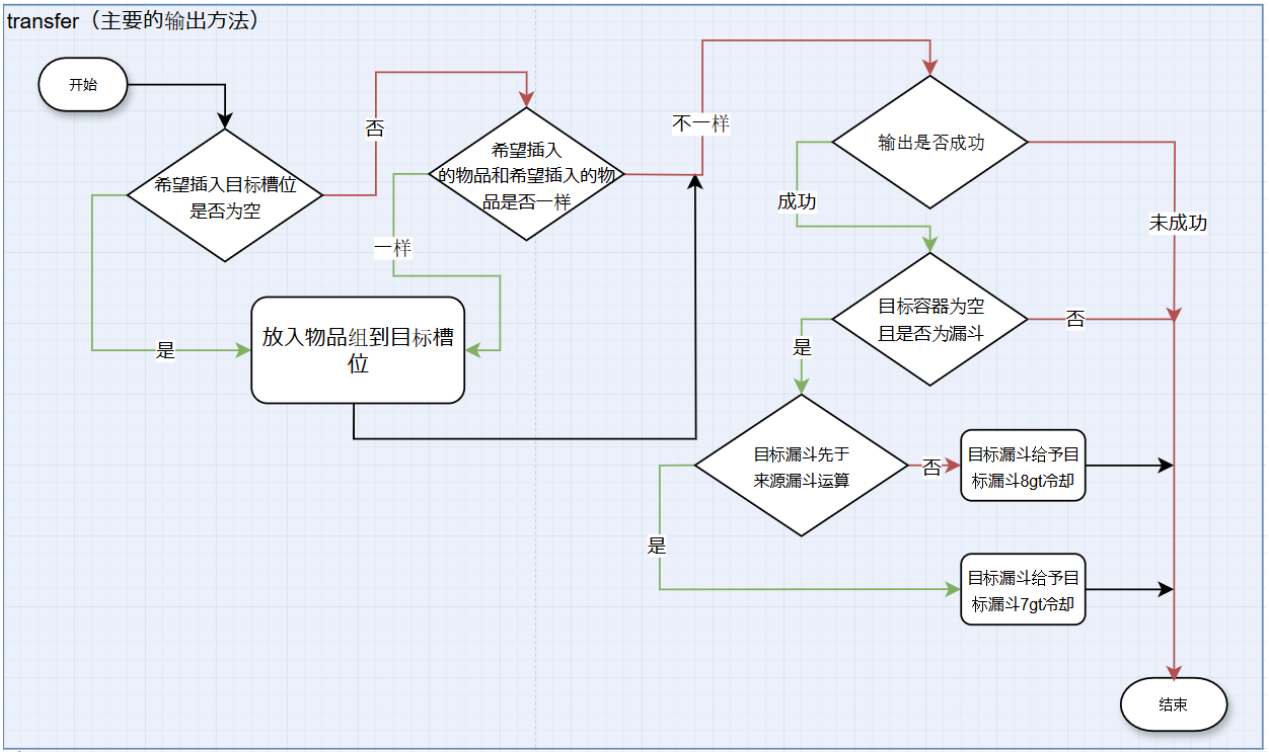
- **from**：物品来源（可以是漏斗或其他容器）
- **to**：传输目标（可以是容器或另一个漏斗）
- **itemStack**：希望传输的物品组

- **slot**：目标容器中的目标槽位

执行流程如下：

1. **检查目标槽位的状态**
   - 如果该槽为空，直接将 `itemStack` 放入，并清空 `itemStack`。
   - 如果该槽已有相同类型的物品，并且仍有剩余空间，则合并物品并清空 `itemStack`。
2. **判断是否传输成功**
   - 传输后，若 `itemStack` 为空，则表示成功传输，否则表示失败。
3. **特殊处理漏斗间传输的时序**
   - 如果目标是另一个漏斗，则会给目标漏斗增加 `8gt` 的冷却时间。
   - 若目标漏斗 `to` 已先于源漏斗 `from` 计算，则目标漏斗 `to` 仅会得到 `7gt` 的冷却时间。

> *值得注意的是，这只在目标漏斗为空时发生；*
> *此外，就算 `from` 后于 `to` 计算，并给予 `8gt` 的冷却时间，由于随后 `to` tick 时会将冷却立刻 - 1，所以结果上还是只有 `7gt` 冷却。*



到此，你已经完全了解了漏斗的基本运作机制。相信这将会对之后的篇章理解提供很大的帮助。

2.1.4 移动中的活塞

另外一个及其重要的方块叫 moving_piston，移动的活塞，俗称 b36。

2.1.5 移动的活塞方块实体携带的信息

一个移动活塞方块实体携带一下基本信息：

- `pushedBlock` 存储的方块
- `facing` 朝向
- `extending` 是否在伸出（与动画播放的方向有关）
- `source` 是否是收回中的活塞本体
- `progress` 当前的移动进度
- `savedWorldTime` 上次执行逻辑时的世界时间

2.1.6 移动的活塞方块实体的功能

- **处理活塞推送实体**
- **更新动画进度**
- **如果`progress>=1`，则将 MOVING_PISTON 替换为`pushedBlock`中存储的方块**
- **在客户端渲染动画**

2.1.7 移动的活塞方块实体的工作流程

详情见5.7。

3 方块实体间的时序

方块实体`tick`过程执行由 `LevelChunk` 中以下两个列表维护：

- `pendingBlockEntityTickers`：在`tickBlockEntities`过程中动态加入的方块实体。
- `blockEntityTickers`：将要`tick`的方块实体

这两个列表都是 ArrayList，因此它们的遍历顺序完全取决于元素的加入顺序。具体流程如下：

- `tickBlockEntities()` 会首先将 `pendingBlockEntityTickers` 中的元素添加到 `blockEntityTickers` 末尾，然后清空 `pendingBlockEntityTickers`。
- 遍历 `blockEntityTickers` 时，如果某个 `TickingBlockEntity` 已被移除（`isRemoved()` 返回 `true`），则从列表中移除；否则会调用其 `tick()` 方法进行更新。

新增方块实体时，会调用 `addBlockEntityTicker()` 方法，将其加入列表。若当前正处于 `tick` 过程中（`tickingBlockEntities == true`），则加入 `pendingBlockEntityTickers`；否则直接加入 `blockEntityTickers`。

卸载时, 同一区块内的方块实体会被保存在 `Map<BlockPos, BlockEntity>` 中，加载时遍历这个 Map，并依次调用 `addBlockEntityTicker()` 加入 `blockEntityTickers`。因此它们的执行顺序取决于 `BlockPos` 的 `hashCode` 计算顺序。

不同区块之间的方块实体加载顺序则取决于区块的加载顺序，即哪个区块先加载，它的方块实体就会先加入 `blockEntityTickers`。

一句话总结:

卸载前, 方块实体的计算顺序取决于放置的顺序. 重新加载后。一个区块内的方块实体计算取决与其 pos 的 hash, 不同区块的方块实体则取决于区块加载的顺序。

相关代码如下 (删减了部分无关内容)

```
// 用于暂存在 tickBlockEntities 过程中动态加入的方块实体private final List<TickingBlockEntity> pendingBlockEntityTickers =
Lists.newArrayList();//将要 tick 的方块实体列表protected final List<TickingBlockEntity> blockEntityTickers =
Lists.newArrayList();private void tickBlockEntities() {   this.tickingBlockEntities = true;   if
(!this.pendingBlockEntityTickers.isEmpty()) {      this.blockEntityTickers.addAll(this.pendingBlockEntityTickers);
this.pendingBlockEntityTickers.clear();   }   // 遍历正在 tick 的方块实体列表   Iterator<TickingBlockEntity> iterator =
this.blockEntityTickers.iterator();   while (iterator.hasNext()) {      TickingBlockEntity tickingBlockEntity =
iterator.next();      if (tickingBlockEntity.isRemoved()) {         // 如果方块实体已经被移除，则从列表中剔除
iterator.remove();      } else if (this.shouldTickBlocksAt(tickingBlockEntity.getPos())) {         // 如果所在位置应被 tick，则
调用 tick () 方法进行更新         tickingBlockEntity.tick();      }   }   this.tickingBlockEntities = false;}// 添加一个方块实体的
tickerpublic void addBlockEntityTicker(TickingBlockEntity tickingBlockEntity) {   (this.tickingBlockEntities ?
this.pendingBlockEntityTickers : this.blockEntityTickers)           .add(tickingBlockEntity);}// 区块内所有方块实体protected
final Map<BlockPos, BlockEntity> blockEntities = Maps.newHashMap();// 区块加载完成后，注册所有已有的方块实体（将它们加入 tick 系统）
public void registerAllBlockEntitiesAfterLevelLoad() {   this.blockEntities.values().forEach(blockEntity -> {      // 为该方
块实体添加 ticker，即加入到 blockEntityTickers 中      this.updateBlockEntityTicker(blockEntity);   });}
```

---

slug: scheduled-ticks
title: 计划刻与计划刻元件
title-en: null
author: hotpad100c
co-authors:

- Arcadi4
- BFladderbeanawa
- tanhHeng
- SDU-Little-River
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:41+08:00'
  index: 3
  is-advanced: false
  banner:
  src: ./img/ScheduledTick.jpg
  alt: 一些计划刻原件们

本部分将解析计划刻及计划刻元件是如何运行的。

基础部分

- 计划刻的概念与内容
- 常见的计划刻元件
- 简单实例分析

进阶部分

- 计划刻队列的维护
- 4gt 侦测器高频的时序解析
- 深入解析比较器的计划刻执行逻辑
- 计划刻抑制

1 计划刻的概念与内容

1.1 什么是计划刻?

在 Minecraft 中，我们常常见到许多红石元件在被触发后并不是立即变化的，例如中继器、比较器。这些元件总是在被触发后延迟一段时间，再发生变化。

让我们以一个生活中的例子来看。上午，你收到了一封邮件，于是你计划下午去处理这封邮件，然后定了一个闹钟。到了下午，闹钟响了，你想到自己要处理邮件了，于是打开邮箱开始处理邮件。

必须要注意的是：**这个 “闹钟” 并没有任何 “文字注释”。它只负责在你的手机中、在正确的时间、提醒你一个人 “有事情”。至于具体是什么事情，闹钟并不关心。** 这个闹钟包含且只包含这些内容：

- 什么时候响
- 这是第几个闹钟
- 有多重要
- 在谁的手机上响
- 提醒谁

计划刻就是这样一个 “闹钟”。红石元件在被触发后，为自己添加了一个计划刻。当计划刻执行时，就像是闹钟响了，红石元件就变化了。

由计划刻控制行为的红石元件，也就称为**计划刻元件**

1.2 计划刻的内容

像上文中的闹钟一样，**计划刻只包含这些内容**，我们称之为一个**计划刻的信息结构**：

- 执行时间`triggerTick`：在何时执行，或称延迟多久后执行[1]
- 子顺序`subTickOrder`：计划刻添加顺序
- 优先级`priority`：计划刻有多优先
- 位置`pos`：执行的坐标
- 方块种类`type`：哪一方块种类执行这个计划刻

位置和**方块种类**都是易于理解的属性。毕竟，方块自己的计划刻不能由其他方块乱执行 ~~（如果你恰好在方块执行计划刻前把它推走或破坏掉换成其他方块的话）~~、也不能在世界里到处乱跑。

执行时间，指的是宏观时序上计划刻的执行时间，也就是计划刻应该在哪一 gt 执行。例如，一个 1 挡位的中继器被触发后添加 2gt 后的计划刻，2gt 后，该计划刻就会被执行。我们通常说的 “计划刻元件的延迟” 就是指计划刻在多少 gt 后执行。

子顺序，指的是相同时间内的计划刻添加顺序。例如，在同一 gt 内，中继器 A 先被触发，中继器 B 后被触发。那么，在子序列中，中继器 A 就在中继器 B 前面。

优先级，指的是计划刻的优先程度。优先级是一个 - 3~3 的整数[2]，其中数值越小，优先级越高。也就是说，在同一 gt 内，优先级为 - 3 的计划刻总是比优先级 - 2、-1、0 的计划刻更先执行。

> *通常在讨论优先级时，“优先级更高” 和 “优先级数值更低” 的意思是相同的。为了避免由 “优先” 这一概念和 “优先级数值” 造成的歧义，我们更推荐读者在向其他人说明时使用 “某一元件的计划刻更优先” 来表述。*

同样的，就和上文中的闹钟一样：**计划刻只是一个 “提醒方块有事要做” 的闹钟，它并不关心方块实际要做什么。一切执行计划刻时的行为都由方块自身控制，“执行计划刻的行为” 并不在 “计划刻的信息结构” 中。**

而当我们说 “某一元件已存在一个计划刻时”，指的就是在当前位置、存在一个方块类型和当前方块相同的、且还没有被执行的计划刻。[3]

1.3 计划刻的执行顺序

在前两篇中，我们已经学过了宏观时序的分析，并且认识了微时序。我们知道，**宏观时序总是优于微时序的**，计划刻也是同理。对于执行时间不同的计划刻，执行时间早的计划刻总是更先执行。对于执行时间相同的计划刻，优先级更优先的计划刻总是更先执行。对于优先级相同的计划刻，子序列更小的、也就是更早添加计划刻的总是更先执行。

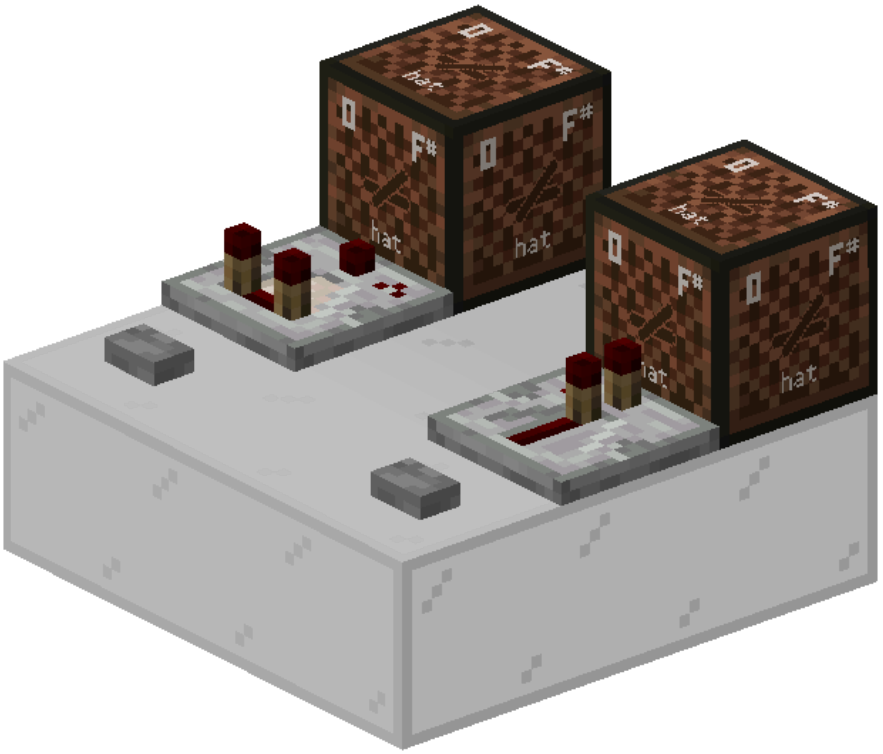
所以，在比较计划刻执行顺序时，我们可以遵循这样的逻辑：

1. **比较宏观时序**，宏观时序更早即更先执行。
2. **比较计划刻优先级**，优先级更优先即更先执行。
3. **比较计划刻添加顺序（子顺序）**，计划刻添加更早即更先执行。

这就像比数字一样，宏观时序就是百位，优先级就是十位，添加顺序就是个位。

1.4 计划刻顺序的实例

那么，让我们来看看实际的例子。



已知在图示情况下，比较器的优先级为 0，中继器的优先级为 - 1。比较器和中继器的延迟都为 2gt。

1. 如果在不同 gt 下，先按下比较器的按钮，再按下中继器的按钮，哪一个音符盒会先响起?
2. 如果在同一 gt 内，先按下比较器的按钮，再按下中继器的按钮，哪一个音符盒会先亮起?

答案:

1. 比较器先亮起。因为宏观时序上，比较器先亮起，而后间隔一定 gt 中继器才亮起。
2. 中继器先亮起。因为在计划刻上，虽然比较器比中继器先添加计划刻，但是中继器的优先级为 - 1，比比较器的优先级 0 更优先，而计划刻优先级 > 计划刻添加顺序，所以中继器先亮起。

2 常见的计划刻元件

2.1 中继器

中继器和比较器统称为**红石二极管**或**红石门**（Redstone Gate）[4]

在刻与刻间时序中，我们已经初步认识了中继器和比较器。现在，让我们深入中继器的计划刻行为。

若中继器被锁定，则不会添加计划刻，也不会在执行计划刻时改变任何状态。 若中继器未被锁定，则具有以下行为：

添加计划刻行为：

- 当中继器受到 NC 更新时，它会检查自身状态。若自身不存在计划刻，且应当改变状态 *（即自身没有亮起，但输入端有红石信号；或自身亮起，但输入端没有红石信号）*，那么添加计划刻。
- 中继器添加的所有计划刻的延迟都为 `中继器挡位*2 gt`，除了下面这种情况：
- 当中继器被放置时，它会检查自身状态，若应当改变状态，则添加 1gt 后的计划刻。

执行计划刻行为：

- 若中继器亮起，则立即熄灭。
- 若中继器未亮起，则立刻亮起。这一步不受输入端信号影响。
  - 若此时没有输入信号，则再添加计划刻（用于熄灭）。

举个例子：

从表现上来看，举例来说，如果给一个 2 挡位中继器一个 `时长<=4gt` [5]的信号，中继器的行为如下：

- 受到 NC 更新，添加计划刻
- 4gt 后，执行计划刻
  - 检测到自身未亮起，于是立即亮起
  - 检测到输入端无红石信号，添加计划刻（用于熄灭）
- 再 4gt 后，执行计划刻
  - 检测到自身亮起，于是立即熄灭

这一例子中包含了中继器的全部计划刻行为。

特殊的优先级变化：

- 若中继器指向一个横放的红石二极管或指向一个红石二极管的输入端，则计划刻的优先级为 `-3`。
- 否则，若中继器添加计划刻时处于亮起状态（即这一计划刻是用于熄灭的），则计划刻的优先级为 `-2`。
- 否则，计划刻的优先级默认为 `-1`。

2.2 比较器

添加计划刻行为：

- 当比较器受到 NC 更新时，它会检查自身状态。若自身不存在计划刻，且应当改变状态 *（即自身没有亮起，但输入端有红石信号；或自身亮起，但输入端没有红石信号）*，那么添加计划刻。
- 比较器添加的所有计划刻的延迟都为 `2gt`，除了：
- 当比较器被放置时，它会检查自身状态，若应当改变状态，则添加 1gt 后的计划刻。

执行计划刻行为：

比较器在执行计划刻时，实际上只做了一件事情：

- 根据比较器现在的输入状态和比较器模式，更新自己的输出能量等级。

这里的**输出能量等级**指的就是比较器经过计算后得出的输出能量等级 *（具体计算方法见后文）*。而这里的**更新**包括了**更新能量等级**和**发出更新**两件事。并且对于比较器来说，从表观上可能会有输出能量等级不发生变化的情况，但实际上此时比较器依然 “更新” 了自己的输出能量等级。

比较器还有相对复杂一点的更新行为 ——

比较器发出更新的行为：

1. 在执行计划刻时，若比较器的充能属性改变 *（即自身由熄灭变为亮起，或由亮起变为熄灭）*，则先发出 NC 更新，再发出 PP 更新。
2. 如果比较器为比较模式，或比较器为减法模式且自己的输出能量等级发生变化，则发出 NC 更新。

举例来说，就是：

- 首先，这段内容全部建立在比较器**确实执行了计划刻**的前提下 ——
- 只要比较器为比较器模式，就算它的输出等级不改变，包括由 0 变成 0，也会有一次 NC 更新。
- 如果比较器的输出能量等级由 0（熄灭）变为 1、2、3、4、5……14、15（亮起），或者由亮起变成熄灭，那么它会先发出一次 NC 更新，再发出一次 PP 更新，最后又发出一次 NC 更新。
- 如果比较器为减法模式，它的输出能量等级由 1、2、3、4、5…… 变成任意一个其他的能量等级，只会有一次 NC 更新。

这些行为在一些特殊的布线中可能会有所应用。如果读者对这部分的理解感到困难，可以暂时只记忆比较器最常规的用法 —— 即比较信号大小并判断是否输出、检测容器容量和减法模式，或者直接查表。

比较器模式	计划刻行为	更新行为
比较模式	充能属性改变	NC->PP->NC
比较模式	充能属性未改变	NC
减法模式	充能属性改变	NC->PP->NC
减法模式	能量等级改变，充能属性未改变	NC
减法模式	充能等级和能量属性均未改变	不更新

比较器输出能量等级计算

如果信号输入是红石粉或者其他比较器，则继承输入的能量。

如果是容器，请查看此部分内容比较器信号强度计算。

特别的，当同时有容器信号和红石信号输入时，比较器会根据情况优先选择不同的信号输入来计算输出, 这一现象也被称为容器屏蔽。

输入端	输出
比较器输入端直接与容器相连	只计算容器的输入，忽略红石信号
比较器输入端隔着实体方块检测容器	当红石信号为 15 时，输出 15 信号强度，其他情况优先计算容器输入

案例 (木桶中均没有物品)：

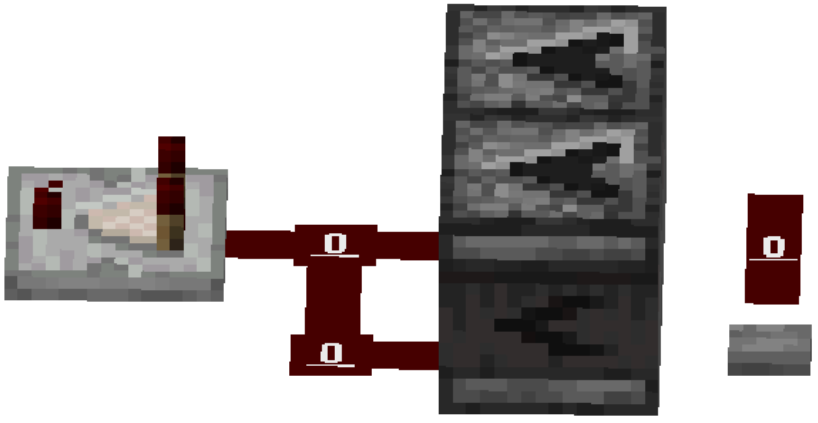
特殊的优先级变化：

- 若比较器指向一个横放的红石二极管或指向一个红石二极管的输入端，则计划刻的优先级为 `-1`
- 否则，计划刻的优先级默认为 `0`

至此，我们已经可以分析上一章中的案例

	左侧	右侧
gt0	按钮按下铁轨激活 侦测器添加 2gt 计划刻	按钮按下铁轨激活 侦测器添加 2gt 计划刻
gt2	侦测器亮起 侦测器添加 2gt 计划刻 (优先级 0) 比较器添加 2gt 计划刻 (优先级 - 1)	侦测器亮起 侦测器添加 2gt 计划刻 (优先级 0) 比较器添加 2gt 计划刻 (优先级 0)
gt4	比较器计划刻优先级 - 1, 侦测器计划刻 0, 比较器计划刻先执行， 比较器亮起 侦测器熄灭	由于侦测器和比较器优先级相同，侦测器计划刻先添加，先执行侦测器计划刻， 侦测器熄灭 比较器执行计划刻，由于此时侦测器已经熄灭，所以比较器不亮起

读者可以自行分析一下下面这个类似的案例，在按下按钮后，比较器是否会亮起。



2.3 侦测器

添加计划刻行为：

- 侦测器在受到面前的方块发出的 PP 更新后，如果自身未亮起且当前位置不存在自身的计划刻[6]，则添加 2gt 后的计划刻。

执行计划刻行为：

- 若自身未亮起，则亮起，发出 PP 更新，再添加 2gt 后的计划刻，最后发出 NC 更新。
- 若自身亮起，则熄灭。

2.4 红石火把

添加计划刻行为

- 红石火把在受到 NC 更新后，如果自身亮起但应当熄灭且当前位置不存在自身的计划刻，则添加 2gt 后的计划刻。

执行计划刻行为

- 如果自身亮起但应当熄灭，则熄灭。
  - 如果自身**燃尽**，则**熄灭**[7]，并添加 160gt 后的计划刻。
- 如果自身熄灭但应当亮起，且没有燃尽，则亮起。

应当熄灭 / 亮起：红石火把所附着的方块是否为实体方块且受到红石信号。

燃尽：如果红石火把在 60gt 内亮起了 8 次，那么自身燃尽。

红石火把燃尽后，你可以通过破坏再重新放置一个红石火把来让它 “亮起”，或者等到 160gt 后火把执行计划刻自行亮起。

3 简单的计划刻时序分析

3.1 侦测器能熄灭红石火把吗?

通常不能。 让我们来简单分析一下：

侦测器和红石火把的优先级都是**0**。并且侦测器亮起时**先发出 PP 更新**，然后**添加计划刻**，最后**发出 NC 更新**。红石火把**接受 NC 更新**。这里的**执行计划刻内的行为顺序**和**元件接受什么更新**根本上影响了接下来的分析。所以假设一个侦测器对着一个红石火把附着的实体方块，它的计划刻顺序如下：

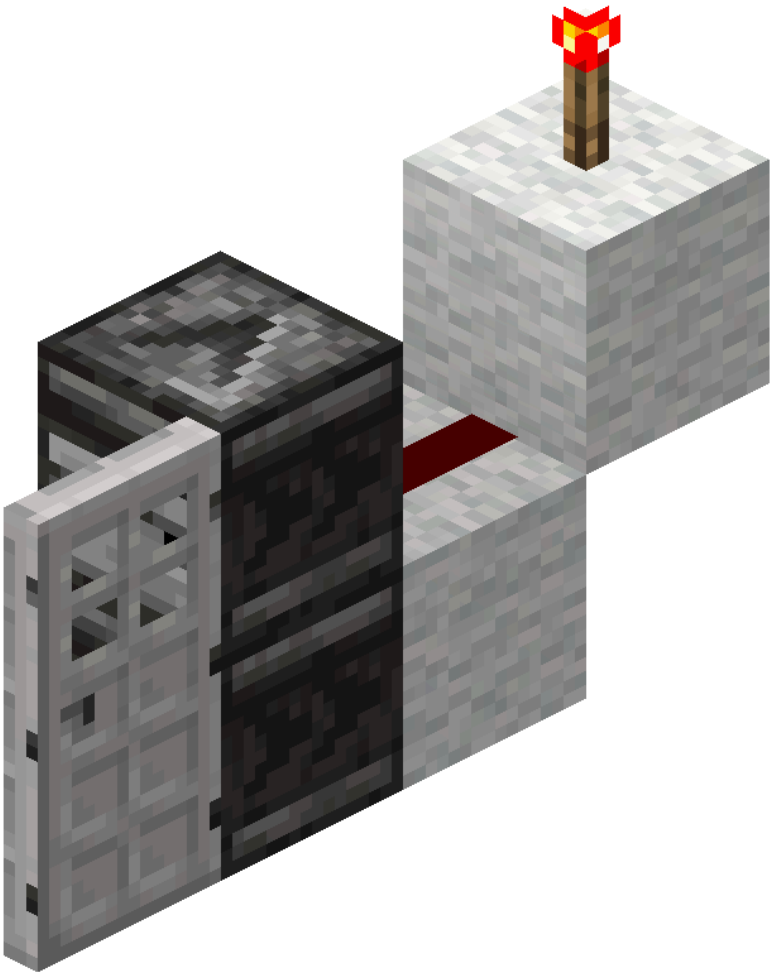
- 侦测器亮起：
  - 发出 PP 更新，无事发生。
  - 侦测器**添加计划刻**
  - 发出 NC 更新：
    - 红石火把受到 NC 更新：
      - **添加计划刻**

2gt 后：

- 侦测器**执行计划刻**，熄灭。
- 红石火把**执行计划刻**，此时检查自身是否应当熄灭，发现不应熄灭（因为侦测器熄灭了，附着方块没有受到充能），于是不执行任何行为。

所以概括来说就是：红石火把在要亮起的时候发现没充能了，遂开摆）

但是，到这里我们就可以发现一个问题：假如我放置两个侦测器，其中一个侦测器用来 **“触发” 红石火把添加计划刻**，另一个用来 **“延续” 红石信号来熄灭火把**，那么红石火把是不是就可以亮起了呢？没错。一个简易的结构如下图所示：



3.2 侦测器能使比较器亮起吗?

3.3 什么是 “末端常亮” 中继器?

4 计划刻队列的维护

在本部分的说明中，我们会省略一些已被声明但未被调用或极少涉及的变量。

为了提高读者在阅读**计划刻执行过程**中的流畅度，我们会先解释所有涉及的重要概念和方法，最后再解释计划刻的执行逻辑。在阅读时，你可以先初步了解这些概念和方法，再在理解计划执行逻辑遇到困难时回看查阅。

4.1 基础

整个世界的计划刻在实际执行时都是由一个**世界层计划器**（`WorldTickScheduler`，或称**世界层计划刻调度器**）[8]控制的；每个位置的计划刻在添加与存储时都由对应区块的一个**区块层计划器**（`ChunkTickScheduler`，或称**区块层计划刻调度器**）控制。

4.2 计划刻类

一个包含了计划刻的所有信息的对象可称为**计划刻对象**。计划刻类`OrderedTick`则定义了计划刻对象的属性和方法。它除了具有方块类型`type`、坐标`pos`，执行时间`triggerTick`、优先级`priority`、子顺序`subTickOrder`这五个基本属性外，还实现了以下四个方法：

4.2.1 执行时间比较

先比较`triggerTick`大小；若相同，则比较`priority`大小；若仍然相同，则比较`subTickOrder`大小。

此方法实现了**区块层计划器**中的**优先级队列**中的 “优先级” 比较（见下文）。

4.2.2 简单比较

不比较`triggerTick`大小。仅先比较`priority`大小；若相同，则比较`subTickOrder`大小。

此方法实现了**世界层计划器**中的**区块层计划器优先级队列**中的 “优先级” 比较（见下文）

4.2.3 哈希化

对`type`和`pos`的哈希码进行运算后得到该计划刻对象的哈希码：

```
31 * pos.hashCode() + type.hashCode()
```

在 Java 中，所有的哈希码都是整数。

4.2.4 快速实例化

除了常规的实例化方法外，计划刻类还实现了一个`create`方法，用于只基于`type`和`pos`临时创建一个计划刻对象。它的其他三个属性都为`0`。这一临时计划刻对象用于判断某一位置是否存在某一方块的计划刻。

4.3 区块层计划器

区块层计划器管理计划刻的**添加、“检查” 与存储**。它主要维护了一个**优先级队列**（`PriorityQueue`）和一个**哈希集**（`ObjectOpenCustomHashSet`）

4.3.1 添加计划刻

尝试向**哈希集**中添加**哈希化后的计划刻**，若添加成功（即集合中不存在该计划刻），则同时向**优先级队列**中添加该计划刻的**计划刻对象**。

4.3.2 计划刻存储

所有计划刻的哈希码都存储在对应区块的区块层计划器的哈希表中，所有计划刻的计划刻对象则按顺序存储在对应区块的区块层计划器的优先级队列中。

优先级队列的排序依据**触发时间比较**，即它将同时考虑`triggerTick`、`priority`和`subTickOrder`。

4.3.3 访问下一计划刻 `PEEKNEXTTICK`

或称 “访问头部计划刻”，此方法会返回当前区块下一个执行的计划刻的计划刻对象，即优先级队列中的第一个计划刻

4.3.4 取出下一计划刻 `POLLNEXTTICK`

或称 “取出头部计划刻”。此方法会尝试取出当前区块下一个执行的计划刻的计划刻对象，若取出成功，则同时删除哈希表中该计划刻对象的哈希码。

4.4 世界层计划器

世界层计划器管理计划刻的 “检查” 和执行。在计划刻的执行阶段，它统筹管理所有区块的区块层计划器，并按一定逻辑按顺序取出计划刻放入可执行计划刻列表中，最后一并执行。

详见执行计划刻。

4.5 检查计划刻是否存在

此方法即我们常说的 “某元件自身不存在计划刻” 或 “某元件当前位置不存在自身计划刻”，显而易见的，它描述了两个核心参数：“某原件” —— 方块类型 `type`，“自身” “当前位置” —— 坐标 `pos`。

4.5.1 方法 1：由区块层计划器控制的检查方法

即上文中描述的**添加计划刻**方法。向哈希集中添加哈希化后的计划刻实际上就是一种 “检查”：如果哈希集中已经存在位于某一坐标的某一方块的计划刻，此时这一方块再尝试添加位于该处的计划刻时就会失败。

4.5.2 方法 2：检查计划刻是否 “将要执行” `ISTICKING`

此方法用于检查元件自身是否会在当前 gt 执行计划刻。

它接收方块类型和坐标两个参数，先复制一份当前 gt 将执行的计划刻的队列，然后检查这一队列中是否存在方块类型、坐标都相同的计划刻。

在执行计划刻时，当前 tick 的所有可执行的计划刻（如果没有达到计划刻上限的话）都会被从各个区块层计划器的哈希集和优先级队列中取出（详见下文**执行计划刻**），也就是说 —— 如果元件将在这一 gt 执行计划刻，此时又恰好在执行前尝试添加计划刻，那么这一添加行为就无法被**方法 1** “拦截”。

中继器、比较器、红石火把等元件在添加计划刻时就会使用此方法检查计划刻是否 “将要执行”。以中继器为例，由于此方法的作用，假如中继器在第 2gt 时将亮起，那么在第 2gt 时、中继器亮起前，由于中继器检查到自身将在当前 gt 执行计划刻，那么无论信号如何变化，它都不会添加计划刻；而在第 0gt 和第 1gt 时，如果中继器输入端的充能状态改变且中继器受到更新，那么它就会尝试添加计划刻，但由于方法 1 的 “检查”，这一添加行为不会成功。所以方法 1 和方法 2 共同保证了中继器不会在执行计划刻前重复添加计划刻。

4.5.3 方法 3：检查计划刻是否在队列中 `ISQUEUED`

此方法是侦测器所调用的的 “检查计划刻是否存在” 方法，其实际功能和方法 1 完全一致。

> *我真的不知道它除了可能能减少卡顿外有什么用。*
> *——tanh_Heng*

它接收方块类型和坐标两个参数，先将坐标对应到区块，再访问该区块对应的区块层计划器，然后快速实例化一个具有对应 `type` 和 `pos` 的计划刻，最后检查哈希集中是否存在与这一临时计划刻的哈希码相同的计划刻。

显而易见的，与方法 2 的根本区别就在于此方法不会检查当前 gt 将要执行的计划刻。这是 4gt 侦测器高频的核心原理。

4.6 执行计划刻

执行计划刻分为**收集**（`collect`）、**执行**（`run`）和**清理**（`cleanup`）三个阶段。其中最重要的是收集阶段。在收集阶段，世界层计划器将按如下步骤将计划刻添加至**可执行计划刻列表**（`tickableTicks`）中：

4.6.1 描述说明

区块层计划器的头部计划刻：区块层计划器中，排在最前面的、将要最先执行的那一计划刻。

所有没有明确表述**取出计划刻**的描述都为**访问**方法。

达到了计划刻上限：游戏中每 gt 能够执行的计划刻数量是有限的，**达到了计划刻上限**即指可执行计划刻列表中计划刻的数量大于等于计划刻上限 `maxmaxTicks`。

4.6.2 第一步：筛选可执行的区块层计划器 `COLLECTTICKABLECHUNKTICKSCHEDULERS`

1. 扫描所有区块层计划器（负责管理单个区块内的计划刻）。只选择那些：
    - 内部有等待执行的计划刻 (存在计划刻)。
    - 其队列中头部计划刻的执行时间小于等于当前 gt。
    - 该计划刻对应的区块本身满足加载要求。

2. 把这些符合条件的区块层计划器放入一个优先级队列中，称为可执行的区块层计划器的优先级队列（`tickableChunkTickSchedulers`，以下简称计划器队列）。

那么，这个优先级队列如何排序？队列根据每个区块层计划器的头部计划刻进行排序。排序使用简单比较（即与区块层计划刻的**执行时间比较**相比，简单比较不会考虑`triggerTick`属性）。这个排序机制是后续计划刻抑制器实现的关键。

4.6.3 第二步：从队列中提取可执行计划刻 `COLLECTNEXTTICKS`

1. **取出第一个计划器：** 从计划器队列的最前面取出一个区块层计划器。我们称它为**当前区块层计划器**。
2. **取出它的第一个计划刻：** 从当前区块层计划器的计划刻队列中取出其头部计划刻。
3. **标记为可执行：** 把这个取出的计划刻添加到可执行计划刻列表中，等待实际执行。
4. **尝试连续提取更多计划刻：** 接下来，游戏会尝试从当前区块层计划器中连续地、尽可能多地取出后续的计划刻（只要满足特定条件）：
    1. **上限检查：** 如果已经达到了本次处理允许的计划刻上限，则停止整个连续提取过程。
    2. **访问队列下一个计划器：** 访问当前计划器队列最前面的区块层计划器（因为第一个计划器已经被取走了，所以这是原本的队列中的第二个计划器）。我们称它为**区块层计划器 2**。
    3. **检查队列是否非空：** 如果计划器队列不是空的（即成功访问到区块层计划器 2），那么访问区块层计划器 2 的头部计划刻。
    4. **开始连续取出：** 如果还没达到计划刻上限，就循环执行下面的步骤 (5 和 6)。
    5. **查看当前计划器的下一个计划刻：** 再次访问当前区块层计划器的计划刻队列，获取当前它最前面的计划刻（和步骤 2 同理，因为第一个计划刻已经被取走了，所以这是排在原先的头部计划刻的后面的那一个计划刻）。
    6. **决定是否取出 (核心条件)：**
        - 如果当前区块层计划器中确实还有计划刻（成功访问到其当前头部计划刻），
        - 并且这个计划刻也需要在当前游戏刻（gt）执行，
        - 并且使用**简单比较**规则判断，当前区块层计划器 的这个计划刻的执行时间早于区块层计划器 2 的头部计划刻的执行时间，
        - 那么就把当前区块层计划器的这个计划刻也取出来，加入到可执行计划刻列表中。
        - 如果以上任何一个条件不满足（没有计划刻了、不是当前 gt 执行、或者不比队列里下一个计划器的计划刻早），就退出这个连续提取的循环，即**不再尝试连续提取更多计划刻**。
5. **处理未完成计划器：** 当步骤 4 的连续提取循环结束后，意味着暂时无法再从当前区块层计划器中连续取出更多符合条件的计划刻了。此时检查：
    - 如果当前区块层计划器里还有未被取出的计划刻（队列没空），
    - 并且它现在队列最前面的计划刻（新的头部计划刻）仍然需要在当前游戏刻（gt）执行，
    - 并且整个处理过程还没有达到计划刻上限，
    - 那么就把当前区块层计划器重新加入到计划器队列中（根据其新头部计划刻的执行优先级重新排序），等待后续再次处理。
6. 循环以上步骤（步骤 1 至步骤 5），直到达到计划刻上限或计划器队列被取空。

概括来讲，上述步骤可以这样描述：

游戏首先筛选出所有在当前 gt 有计划刻要执行、且区块已加载的区块层计划器，并按它们最优先计划刻（头部计划刻）的简单比较顺序排队。然后，它从队头取一个区块层计划器，将它最优先的计划刻加入准备执行的队列中，并尝试从这个计划器里 “连续” 提取更多同样在当前 gt 执行的、且比队列里下一个计划器最优先的计划刻还要优先的任务来一起加入准备执行队列中（目的是提高效率）。如果连续提取被打断（比如下一个计划刻不够优先、或者达到执行上限），而这个区块还有当前 gt 的计划刻未被添加且未超上限，就把当前计划器按照头部计划刻的优先顺序放回计划器队列中，以便稍后继续处理它剩下的任务。

4.6.4 第三步：延迟未被添加的区块层计划器 `DELAYALLTICKS`

将所有计划器队列中还没有被取出的区块层计划器加入一个`区块坐标 -> 下一执行时间`的映射表（`nextTriggerTickByChunkPos`）中，以待后续执行。

4.7 计划刻抑制器

由执行计划刻的逻辑我们可以注意到：区块层计划器的排序是不考虑`triggerTick`的。也就是说，如果某个或某几个区块层计划器的头部计划刻优先级足够高、且该计划器中的计划刻数量足够多，就可以使世界层计划器不断地只从这几个计划器中取出计划刻直到达到计划刻上限，而不会取出其他计划器中的计划刻。并且这一抑制器只能对抑制器所在区块外的其他区块生效。

例如，如果在某几个区块以一定高频激活足够多的中继器，在其他区块的优先级为 0 的元件就会在高频期间被 “抑制”，如比较器不亮起、侦测器高频不工作，直到高频结束这些计划刻才会重新执行。

FOOTNOTES

1. “延迟多久后执行” 并不严谨。实际上，执行时间记录的是世界刻，“延迟 `x` gt” 就是 `执行时间的世界刻=当前世界刻+x` 。`当前世界刻>=执行时间的世界刻` 时，计划刻执行。但这一区别并不影响大多数情况下的计划刻分析，区块卸载的情况除外。 ↩
2. 但是目前并没有优先级为 1、2、3 的计划刻。它们只是在代码中被声明了。 ↩
3. 由于后文说明需要，基础部分仅作简单表述。具体内容涉及计划刻队列的维护，参见进阶部分。 ↩
4. 因为在源码中，它们继承自同一个抽象基类 `AbstractRedstoneGateBlock` ↩
5. 准确来说，应为 “在中继器执行计划刻（进行亮起）前熄灭的信号” ↩
6. 由于计划刻的执行逻辑，在当前 gt 即将执行的计划刻不会视为 “存在计划刻”。也就是说，如果在某一 gt 侦测器将会亮起，而在侦测器亮起前对侦测器发出 PP 更新，此时侦测器也会添加计划刻。该逻辑是 4gt 侦测器高频的核心原理。具体行为和实例详见进阶部分。 ↩
7. 此处熄灭通过 `WorldEvent` （世界事件）控制，而非直接调用 `setblockstate` 。 ↩
8. 本部分（3.4）所使用的这些译名均为编者自行拟定，目前尚未在其他相关文献中发现对应译法。如果读者有更好的译名或修改意见，请直接提交 issue 与我们联系。 ↩

CHAPTER 6

MOLFORTE

---

slug: practical-machines
title: 实战篇：入门机器的快速上手
title-en: null
author: Molforte
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-08-03T21:37:47+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 3
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

本篇的实战性很强，基本需要玩家着手操作。

1 挂载

2 出发站、返回站、停靠站

3 三向轰炸机

4 老版本盾构机

5 特训篇 1 无沟世吞教程

6 特训篇 2 无沟地吞教程

7 * 特训篇 3 基岩机原理

slug: core-modules-explained
title: 沉淀篇：基础模块的举例讲解
title-en: null
author: Molforte
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-08-03T21:37:47+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 2
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

1 概述

本篇主要讲解基本的绿萌模块，比如飞行器，活塞虫，延时模块等。本篇只着重讲述它具体的原理，至于一些骚气的布线，详见：

2 飞行器

2.1 最经典的侦测器飞行器

飞行器是大多数玩家接触到的第一个绿萌结构，不仅是绿萌中最基础的模块，还应用于各种其他的械电机器中。

图 全树种中的飞行器

这里需要简单理解一下黏性活塞 “丢方块” 的特性：首先，我们要知道侦测器在检测到更新时，会发生信号，由于其信号较短，侦测器在第二 gt 时就结束了信号，活塞也便失去了信号，这就造成了你可能有所耳闻的 “丢方块” 的特性。

正是因为此项丢方块的特性，图中飞行器运动时，负责推出的活塞便可视为普通活塞。(并不是真正的普通活塞，这里只是为了新手理解，提供的一种简便的理解思路）推出的活塞把对应的另一部分 “丢出去”，被丢出去的普通活塞受到侦测器的充能拉回，被拉回的活塞被侦测器激活后又把另一部分给 “丢出去”…… 循环往复，飞行器便可以 “左脚踩右脚” 一般飞行了。

2.2 活塞虫

理解了上一小节飞行器左脚踩右脚的原理，理解活塞虫也就不难了。毕竟也只是丢掉了黏液块这条 “红石线” 嘛。

图 两种活塞虫的简图

在大多数情况下，我们一般使用图 2 所示的活塞虫，读者应该能想到这个活塞虫的方向。由左至右 这种活塞虫需要外部的信号才能运动。

图 挖沟机底座

这种活塞虫能推动其运动方向为轴的方块，同时，相比于上一小节的飞行器，还具有侧面防粘的优势，因此，它常被用在绿萌机器的外部，或是在一些狭窄区域布线。

需要提醒读者的是，活塞虫的历史比上一小节中的飞行器要早得多，如图是莫尔在萌新时期，在麦块中下载到的活塞虫飞行器。这个飞行器甚至比下一小节中的 1.8 版本出现的黏液块飞行器还要早。

图 简易活塞虫飞行器

2.3 BUD 飞行器

这个飞行器是 1.11 之前最经典的飞行器，它的后半部分是

2.4 引擎

我需要理清楚的是，引擎的与上节中的 “飞行器” 的概念几乎一致，不过更偏向于非 10gt 周期的飞行器。本小节会讲大多数非 10gt 的飞行器，并着重讲述其概念。

2.4.1 GT 双向飞行器引擎

图 9gt 双向飞行器引擎

还记得吗？我们在上一篇讲解过如何制造 3gt 的延时结构，这个引擎就是将上小节的 10gt 飞行器的 4gt 推出部分改为了 3gt，$3gt + 6gt = 9gt$ 它不仅具有比 10gt 飞行器快 1gt 的优势，并且可以挂载一条活塞链，其运输的重力方块可以被这个特殊的时序顶掉。当然，运维过挖沟机的玩家早就被这玩意整麻了。

图 挖沟机上的 9gt 双向飞行器

2.4.2 GT 引擎

图 11gt 飞行器结构

你叕记得吗？上一篇我们也讲过 7gt 延时结构，这也是以此脱胎而来，这里就不细讲了。

2.4.3 GT 引擎

2.5 自返回飞行器

说到自返回飞行器，我想一部分玩家会首先想到的是基于红石灯的自返回飞行器，如图，在原有飞行器的基础上，一侧的侦测器前方加上了红石灯，并使用了一个侦测器作为信号源，飞行器此侧碰壁时，时序关系，红石灯会额外让侦测器产生一次信号，因此达到自返回的效果。

图 红石灯自返回飞行器

当然，现版本的无沟世吞也是同理，有的是利用活塞，有的则是利用栅栏门。自返回的实现方式多样，通常只需要给碰壁的一侧给一次额外的红石信号即可。

图 无沟世吞上的自返回飞行器

3 连杆

3.1 无延迟连杆

3.2 BUD 连杆

3.3 安全连杆

4 TNT 复制简述

本小节讲讲解一些可移动的 TNT 复制原理，包括有矿车 TNT 复制和无矿车 TNT 复制。

4.1 有矿车 TNT 复制

4.2 无矿车 TNT 复制

5 延时模块

5.1 活塞链

5.2 飞行器延时

6 开关

6.1 简易开关

6.2 防呆开关

7 杂项

7.1 二步进模块

是的，常见的绿萌机器就由这几部分组成，你可以从大部分玩家作品中拆分出这些模块，或是找到以上模块衍生品。当然，仅仅学会这些，就像是学会了教材中空泛的公式，想要进一步上手，还是得接下来的实战篇。

slug: setup-and-basics
title: 预备篇：环境配置和基础知识
title-en: null
author: Molforte
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-07-31T21:48:49+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

1 概述

本篇将一步步教你配置绿萌使用环境，并简单地讲解相关的基础知识。

以下是涉及到的模组汇总：Carpet mod、WorldEdit（创世神）、Tweakeroo、pistorder（或 Tweakmore，本篇用 pistorder 演示。）、Replay mod（可选）。

2 环境配置

你大可以跳过本节，但是我依旧建议你阅读本节内容，使用较为先进的工具总归对你的学习和设计有好处的。

2.1 世界环境

打开单人游戏，选择**创造模式**，创建一个**虚空存档**。

`创建新的世界→世界→世界类型：超平坦→自定义→预设→虚空`

如果你出生在一片石头平台上，证明你成功了。

它日后会变成这样子：

当然，鉴于玩家对破基岩科技树的研究，我们可以将预设改为：`minecraft:bedrock;minecraft:the_void`

如果你想要多层，只要在方块名称后面加上$*x$即可（其中$0 < x <= 324$,且$x \in N^*$）如：`minecraft:bedrock*5;minecraft:the_void` 便可以模拟五层破基岩环境。

当然，为保证生存实装的稳定性，最终都要创建一个正常的世界进行实地测试，一切不进行实地测试就发布的机器，都是对自己和他人不负责任的表现。

2.2 结构方块

在自己的作品经历了一次次炸膛、解体、粘连后，我们找到了比备份更为简单的局部存档利器：**结构方块**。我们主要使用两个模式：**保存模式**和**加载模式**。

2.2.1 保存模式

2.2.2 加载模式

2.2.3 必要说明

- 结构名称不能使用中文命名。
- 相对位置和结构大小都有三个框，该三个框从左到右分别对应 x 轴，y 轴，z 轴，可通过按下键盘上的 “F3”，屏幕中间的指针来对照。
- **保存模式仅能保存 对应加载模式中角度为 0° 的结构框**。
- 包括实体：在保存模式中，为保存结构框内的实体；在加载模式中，为加载保存模式已保存过的实体。

2.2.4 小技巧

我们可以对迭代中的结构多次留档，便于以后回退和 debug，以下是示例：

在上面保存模式和加载模式的示意图中，保存模式负责留档：上图留档示意的图中，结构方块都是完全一致的保存模式的结构方块 文件名 bedrock_breakers 也可以看出来，是一系列的基岩机，以此作为留档区，工作区仅有一个加载模式的结构方块（图中未显示）。

以上仅是对我来说比较舒服的方式，读者可创新思路。

2.3 CARPET 指令

下面会简述 Carpet 的配置，如需更多选择，请参考其他章节，或观看 Carpet 使用教程。

2.3.1 CARPET 配置

打开你的测试存档，在对话框中输入指令。本小节内容的一经设置，基本不会再更改。

`/carpet tntDoNotUpdate true`：放置 TNT 时，TNT 不会被红石激活。这使得我们在制作 TNT 复制机时，省去推矿车，或者其他的” 上弹 “方式。

`/carpet fillUpdates false`：使用 fill、clone、setblock 指令和结构方块时不会造成更新。尤其是解决加载结构后，造成的机器” 乱动 “的困扰。

`/carpet creativeNoClip true`：创造模式无碰撞，便于我们愉快地穿梭在各种机器结构中，同时解决被粘液块弹走的麻烦。

`/carpet structureBlockLimit 48/96/192/256`：其中，数值选项用 “/” 隔开。当你需要的结构范围大于结构方块默认的框选范围时，可以考虑此选项。**需要注意的是，在实际使用过程中，过大的结构框会造成一定的渲染问题和卡顿，如非必要，默认选项为 48 即可。**

`/carpet explosionNoBlockDamage true`：当你设计和 TNT 有关的结构时，可以把该调试打开， 这使得 TNT 爆炸不会炸坏任何方块；当你需要测试 TNT 是否会破坏预期的范围时，可以设置为 false。

2.3.2 游戏刻指令

- `/tick freeze`：冻结游戏刻，通常结合下面的`/tick step x`指令，来观察设计的结构是否按照预期运作。类似于编程中打的调试断点。
- `/tick step x`：冻结游戏刻后，该指令可以使游戏前进 x 游戏刻，其中 x 是要前进的游戏刻数。
- `/tick sprint x`：类似于`/tick warp`通常用来长时间测试机器稳定性，或者加速飞行器的到位（便于调试）。
- `/tick unfreeze`：解除冻结游戏刻，部分版本使用`/tick freeze`同时兼顾开关冻结游戏刻。吐槽一下，不知道为什么要把一条指令分开，每次打指令都麻烦得要死。

2.3.3 TNT 监测指令

- `/log tnt`：使用此指令可以订阅 TNT 有关的选项。
- `/log tnt full`和`/log tnt brief`：当 TNT 爆炸的时候，告诉你 TNT 的生成位置和爆点位置，在控制 TNT 的爆点时非常有用，我们将在实战篇讲到。
- `/log tnt clear`：取消订阅 TNT 有关的选项。

2.3.4 假人指令

- `/player <Name> spawn`：利用假人的第一步，最基本的是测试机器时用来强加载区块。如使用`/player yu_xian520 spawn`召唤一个名为余弦的假人。哔哩哔哩关注叁日余音喵~
- `/player <name> use`：可以让假人使用物品、放置方块或与方块互动，相当于鼠标右键。相同的指令还有`/player <name> use once`。将假人对准音符盒/栅栏门，可以实现远程开机/步进效果。如使用`/player Twisuki use`可让 Twisuki 点击一下音符盒 / 栅栏门。
- `player <name> use continous`：可以让假人持续使用物品、放置方块或与方块互动。通常用来放基岩机的活塞。使用`/player <name> use`可以结束放置。

2.4 WORLDEDIT 指令

本节内容主要涉及`//set`，`//undo`，`//redo`，`//stack`，`//move`等指令的使用，提前掌握了的玩家可以跳到下一节。
***本小节只讲绿萌设计中最常用的部分**。*

1. //set、//undo、//redo
   //set 是很基础的指令了对吧？使用小木斧，和投影 mod 一样，左右键框选一定的范围，输入`//set 0`

不出意外的话，你框选的这片区域已经被替换成了空气。那么，如何恢复这个平台呢?

- 使用`//undo`便可以撤销这一步操作；
- 使用`//redo`便可以复原刚刚撤销的操作。

1. `//move <Num>`
   这个也是很基础的指令，在我们框选区域后，对着某一方向（或者用`//set <Num> <direction>`便可让移动该区域的方块，*如使用//move 1 up 来向上移动一格*，不使用方向便默认玩家朝向为移动方向。使用`//move <Num> -e`可以保留实体。

可以使用这个指令来将制作的模块移动到合适的位置。

1. `//stack <Num>`
   在框选区域后，你可以对区域向着某一方向批量复制，非常方便拓绿萌单元。
   这条指令用法与`//move`基本类似，只是由” 移动多少格 “的功能转变为” 复制多少次 “。便不再过多赘述。

受限于我的笔力，以上内容肯定有点抽象，你可以框选后输入相关的指令试一试，或者搜索相关的 WE 教程。

2.5 TWEAKEROO 配置

1. **禁用侦测器选项**：
   在游戏界面中，同时按下 `x+c`，在禁用类选项中，可以找到禁用侦测器选项，我建议你在键盘的右侧设置一个快捷键，方便我们在 debug 时随时打开。

打开此选项后，侦测器就变成了 “石头”，不再响应更新。当我们使用 WE 或者其他工具，对绿萌结构进行操作时，可避免侦测器激活造成的不可预知的结果。也可以防止我们手残对侦测器造成更新。

1. **Flyspeed 飞行速度控制 飞行速度预设**：
   挖沟机，矿机等往往都有超过 200 格的高度，我们游走在机器各处是非常麻烦的，使用 Flyspeed 功能可以大幅提高玩家的飞行速度，在各个部分之间快速通勤。

2.6 PISTORDER 使用方法

这是设计绿萌的必备模组之一，前面的章节也讲过 pistorder 和活塞到位的相关规律，这里只对绿萌相关的做相关简述。

空手右击普通活塞，活塞上会显示 能否推动（如图上的 推动√，不能推动则显示 推动 ×），而被推动的方块，则**从小到大**显示方块到位的先后顺序。有助于我们压缩机器和快速 debug。

- 对于普通活塞：空手右击一次为直接推动（活塞臂直接接触），右击两次为隔一个方块推动（如下图）。
- 对于黏性活塞：空手右击一次为直接推动，右击两次为隔一个方块拉回。

2.7 REPLAY MOD 的 DEBUG 小技巧

Replay mod 对于主播或者录过 mc 视频的玩家而言并不罕见 骂谁罕见！，它能够记录所加载区块，在录制所在的时间段内发生的事。

正是因为 Replay mod 的情景复现功能，我们可以通过回放后拉取进度条的方式 debug。通常在问题复现时间太长，或使用 `/tick freeze` debug 比较困难的情况下使用这个小妙招。

3 基础知识

3.1 时序科普

3.1.1 游戏刻

前面的章节我们已经了解了游戏刻的基本内容，本节更倾向于介绍游戏刻在绿萌中的应用。

为了照顾未学习过时序章节的读者，这里简述下比较重要的基本内容：

- Minecraft 时间的基本单位是游戏刻（简称**gt**）。
- 不同红石原件有不同的游戏刻延迟，比如侦测器延时 2gt，中继器每档分别延迟 2gt（故中继器延时 2~8gt）。

本章所有内容都将抛弃**红石刻（RedTick，简称 rt，1rt = 2gt）** 的使用， 因为我们后面将涉及到一定的奇数刻的时序关系，使用游戏刻方便计算。

对于绿萌来说，游戏刻可以用来指示各个布线的先后运动关系。对于大部分情况来讲，我们仅仅知道它们运动的先后顺序即可。如下，是我复现的我的第一台绿萌机器，一台以活塞链条作为延迟的盾构机。

图 活塞延迟式盾构机

读者可以顺带记住的是，上方黏性活塞一小节延迟为 6gt，下方普通活塞延迟为 4gt。

了解过绿萌的玩家都知道，信号的传导从上方的黏性活塞链开始，至下方的普通活塞链结束；*此结构被用来作为盾构机等 TNT 延迟；* 熟悉了上方的信号延迟后，我们可以计算出上图的总延迟为多个黏性活塞和普通活塞相加，即多个 6gt 和多个 4gt 相加。

当然，经过简单的计算我们也知道，以上的活塞延迟排列组合，我们能创造出 4gt 及以上，所有偶数 gt 的延迟。**那么，我们如何创造出 2gt 的延迟呢？** 这似乎有些麻烦，我们通常只能把 6gt 的延时改为 4gt 的延时来提早 2gt，反之亦然。

既然已经有了偶数 gt 的延迟，那么，**如何创造出奇数 gt 的延迟？** 这看似困难的事，实际上简单得多！我们回到上图 4gt 延时的活塞结构中，我们可以将其改造为基于红石块的传动结构，红石块的关系，活塞会到位后的下一 gt 就触发，这样，我们就创造了 3gt 的信号延迟。

图 两种活塞延迟的示意图（拼图，用两张图将其二合一）

分析一下具体的情况：对于上图侦测器的结构，gt0 活塞和侦测器到位后，gt1 侦测器激活，gt2 活塞伸出，gt3 活塞伸出中，gt4 被推动的方块到位；对于下图红石块的结构，gt0 活塞到位后，gt1 活塞伸出，gt2 活塞伸出中，gt3 被推动的方块到位。

结合以上 4gt 和 6gt 的延迟单元，我们可以创造出 3gt 和 7gt 及以上的延迟单元。

下面我们讨论一种奇妙的 7gt 延迟技巧：我们回到上图中 6gt 延时的结构，我们如图在斜上方放一个可充能方块，并再加一个侦测器对其 qc，我们就创造出了一个简单巧妙的 7gt 延时，这个在 kades 的三向破基岩中亦有应用：

图 7gt 延时结构，附 kades 的基岩机应用部分。

我们仍然可以分析一下具体的情况：先欠着。

当然，根据上段原理，我们可以创造出 11gt 周期的飞行器，或是 11gt 的二步进结构，应用在无沟地吞上简直是点睛之笔。

图 11gt 周期的飞行器和二步进结构

当然，在很多情况下，我们不需要如此精确地计算延迟，很多绿萌结构需要下一次启动前复位即可，正是因为很多结构所谓的 “钝感”，便有了较为巧妙的布线风格：**借力**。

3.1.2 借力

自我入坑绿萌之初，就听说过类似于这样一句话：

> *厉害的布线往往应用着大量的借力。*

正如上一节所言，借力就是把对时间不敏感的结构 A，通过其他结构 B 甚至毫不相关的结构 C 复位甚至直接替代结构 A 的布线手段。很拗口是吧？让我们直接看图：

图 借力示意图，最好 GIF

由图可知，上面的无延迟连杆的活塞，在触发后，他的黏性活塞由下面一个 可能作为其他用处的无延迟连杆复位，这就是借力， 这里可以作为一个比较浅显的例子。

再结合上面的游戏刻部分，我们可以以机器运行的某一刻作为时间原点（类似于电路分析中设定的电位的零参考点。电子相关的学生并不陌生），每一个模块甚至每一个活塞都能得到一个 gt 数标记，这样我们能准确分析每一个部分的运动先后顺序。

图 我的 14m 高前弹无沟地吞

如图是我的 14m 前弹无沟地吞出发站部分，我们以 TNT 缓存输出的信号部分作为 0gt 点，可以测出弹射飞行器的时间间隔为 7gt。（当然，标记点不同，测出的时间也不同）。

除了对机器的大小或者时间有极端要求的情况以外，大部分时候并不需要如此理性地计算时序，本小节只是提供了一个理性的分析思路，以满足极端情况下的需求。

图 无沟地吞的借力单元，GIF

3.1.3 * B36 到位时间分析

本小节可直接参考前面章节的 b36 到位时间分析，本小节主要讲我们如何利用 b36 的到位时间来压缩机器。直观起见，这里依然用 pistorder 来辅助讲解。

（咕咕咕）

---

slug: appendix
title: 0a - 附录
title-en: null
author: Molforte
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-08-03T21:37:47+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: -1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

1 术语解释对照表

由于不同的绿萌玩家对于术语有不同的解释，为了保证整个章节的准确性，这里添置一张术语解释对照表。另外，以下的术语也会在文章中以脚标形式展现（未完成）。

- 飞行器：广义上飞行器指代绿萌，狭义上是能自己移动的模块。本文按狭义行文。
- 引擎：指飞行器，通常指非 10gt 周期移动的飞行器。
- 活塞虫：广义上指飞行器，狭义上指仅活塞头和活塞头直接相对的可移动的飞行器。本章按狭义行文。
- 自返回飞行器：受到阻碍（如撞上黑曜石）后自动返回的飞行器
- 自动：指飞行器可以自己移动，即左脚踩右脚移动。
- 步进：被激活后做有限移动的模块，如单步进：即活一次运动一次；双步进：即活一次运动两次或两格。
- 挂载：在一个模块上通过活塞等接上另一个模块称为挂载。被挂载的模块通常不是自动的，或者被挂载的模块不会对挂载的模块产生反应。
- 停靠站：可停靠飞行器，并能发射飞行器的绿萌结构。可以步进后发射飞行器，也可以 0 步进（即飞行器不移动）
- 出发站：世界吞噬着等机器默认开机的停靠站。
- 返回站：默认出发站的另一头为返回站。也可作为停靠站，也可以不能停靠飞行器。
- 拓展：本文仅指拓展如世界吞噬着、矿机、盾构机等重复的单元，也可等同于堆叠。
- 活塞链：通常是由粘性活塞间隔粘液块而成的链形装置，通常用来作为延迟或者方块传送带。

CHAPTER 7

TREE FARM

---

slug: dustless-wiring
title: 为什么你的树场卡 —— 无粉布线简介
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 5
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

相比不少人已经等候多时了。无粉树场近些年来似乎是相当好的一个 “装逼” 途径。然而，星河一直强调：

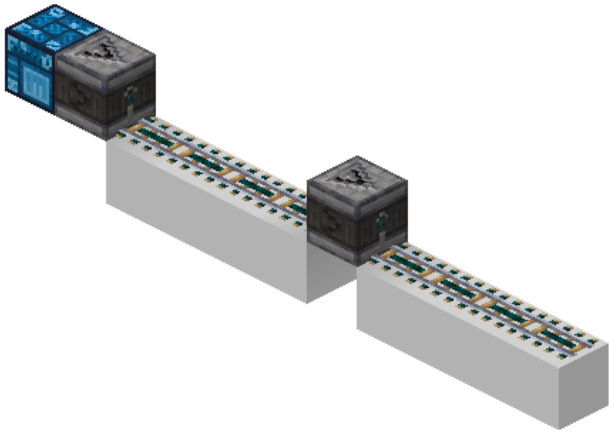
> *无粉从来都不是目的，而是手段。最终目标应当是减少卡顿，而不是强行无粉。强行无粉很容易导致卡顿上升，这与无粉的根本理念是背道而驰的*

明白这一点，我们才能真正用好无粉布线

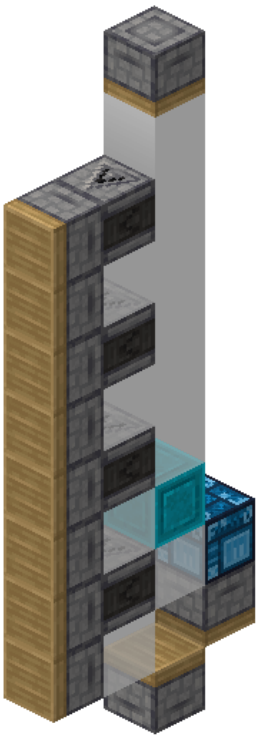
接下来，我们一点一点阐述无粉布线的主要方式

1 铁轨与侦测器

众所周知，铁轨在激活状态改变的时候会**同时发出 PP 更新**，并可以被侦测器检测。铁轨与侦测器就能构成最基础的无粉信号传递方式

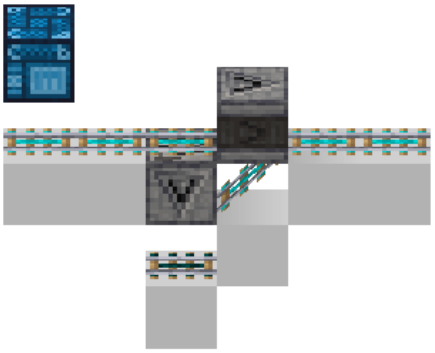


树场中，活塞一般是纵向排布的，我们只需要同时激活纵向的一排侦测器，就可以同时激活整排的活塞。最基本的有两种方式：通过活塞**推动侦测器本身**或**推动侦测器面对的方块**~~这一排东西的归位随便做一下就好了~~



由于侦测器自带 2gt 延迟，我们可以利用它构造宏观时序。这个方向的例子已经有很多，不再进行具体分析

同时也可以使用 bud 铁轨来无延迟传输信号：



2 树电/脚电/墙电

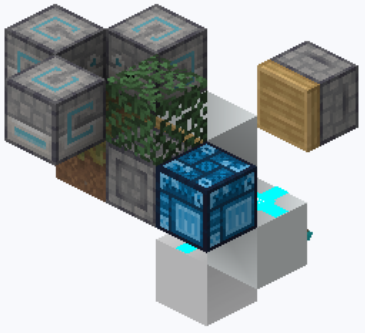
其实还有个水电但因为没人用我们这里就忽略它好了

2.1 树电

众所周知，从 1.14 开始，在接收到 PP 更新后，树叶会添加一个计划刻（如果在 TT，则立刻执行，否则在**1gt 后的 TT**执行）检查自己到原木的距离。如果距离发生改变，则发出**NC 和 PP 更新**。在树场中，我们一般通过活塞或粘液块推拉原木，实现距离的改变

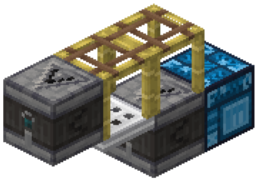
> *事实上树电的更新方式有点像红石粉：它检查四周的原木和树叶的 distance，如果毗邻有原木，则将自己的 distance 设置为 0；如果四周没有原木，则将自己的 distance 设置为毗邻的树叶中最小的 distance + 1*

通过树电，我们可以造出非常多好用的**奇数 gt 延迟**装置，比如让上推柱子在 3gt 开始下推

> *树电检测，就是利用树电在 TT 内无延迟的特性。树场的催熟模块是**运行在 TT 内**的，因此，树必然在 TT 内长成，并放置原木。因此，树叶会在 TT 内立刻执行检查，然后发出 NC 和 PP 更新，从而触发 BUD，实现生长检测。不过由于空间问题，现在几乎不再使用。*
>
> 

2.2 脚电

众所周知，当一个脚手架接收到 PP 更新后，会添加计划刻，检测自身与支撑方块的**水平距离**，若距离改变（且不大于 6，大于 6 会掉落），发出 NC 和 PP 更新。脚电会在**每一个脚手架上增加 1gt**的延迟。 一般而言我们通过脚手架下**活版门的状态变化**触发脚电

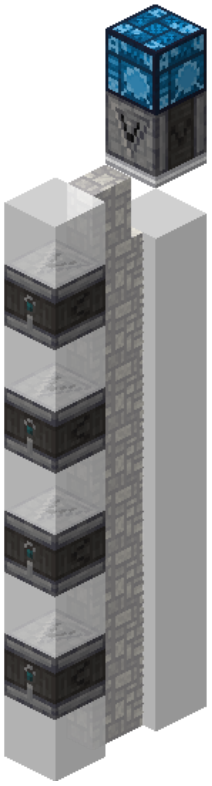


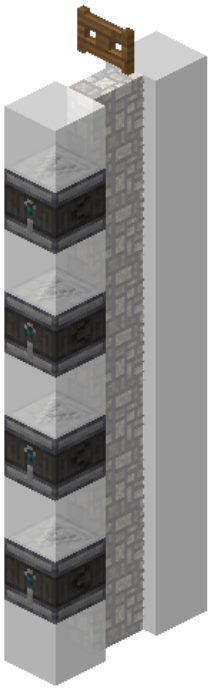
一般而言我们用脚电制造**奇数 gt 延迟**和~~树电看着像就对子，使用场景是几乎一样的~~

> *脚电和树电一样运行在 TT 内，如果能在 TT 内确认自身与支撑方块距离的改变，理论上是可以实现无延迟的。不过脚电在目前的实际应用中还显得比较玄学（*

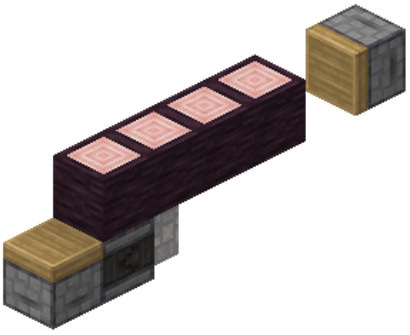
2.3 墙电

众所周知，从 1.16 开始，x 或 z 轴上均有连接方块，则石墙为低墙，否则为高墙。若一整列石墙中有一个高墙，则从其开始，其下所有墙都为高墙（低墙不会覆盖）。我们一般通过最上方墙**上方的栅栏门的开关**或**侧面的门 / 活版门的开关**实现墙的状态变化。





另外，墙会主动连接**上方的某些方块**，是否连接也是一种墙的一种状态，可以用于将方块流转向上方向



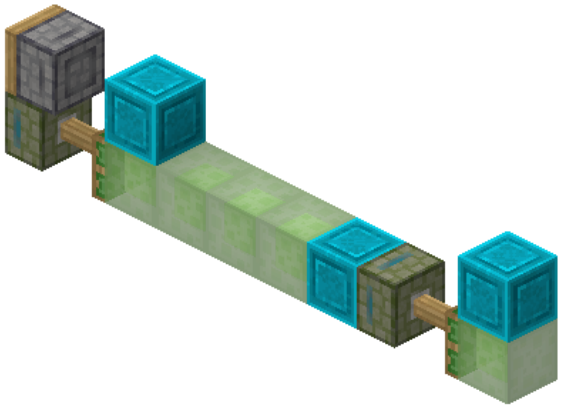
墙是**瞬时原件**，所有墙的状态变化都会发出**PP 更新**，并被侦测器侦测。

> *推动墙的连接方块甚至墙本身的设计也是有的，特别用于单边沿墙电。以上更详细的内容可以参考东东的视频*

3 无宏观延迟的信号传递

上面说的信号传递方式都是**有宏观延迟的**。如果你在上一章之后尝试了布线，你应该会发现**我们很需要无宏观延迟的信号传递方式**。这里，我们简单了解无粉布线中最重要的两种无宏观延迟延迟的信号传递方式

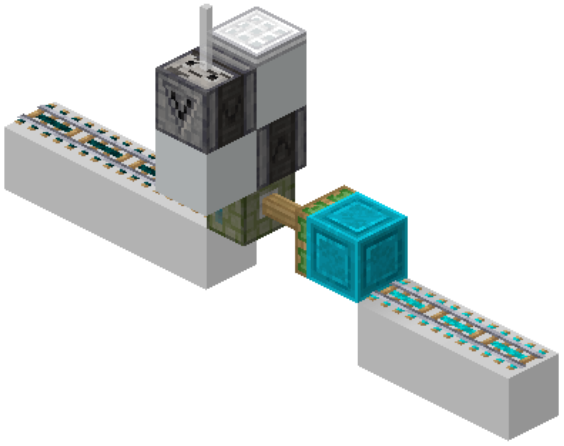
3.1 粘液棒子



粘液棒子顾名思义，就是用粘液块拉动一些东西完成信号传递。众所周知，活塞在推拉一串方块的时候，**会在 BE 阶段内就删除原本位置的方块**，因此**可以在同 gt 内改变另一个活塞的激活状态**，从而消除宏观延迟

这样串出来的棒子，对于其中的粘性活塞，**传递下降沿无宏观延迟，而上升沿会有每个活塞 3gt 的延迟**，怎么办呢？一般而言，我们会为每一根棒子添加一个自复位装置，在 3gt 的时候自动复位，这样就可以让它们同步复位，**同步输出上升沿信号**

3.2 铁轨 + BUD



还记得铁轨是瞬时原件吗？利用铁轨更新 bud，bud 再改变另一串铁轨的激活状态，我们就可以让信号一直在 BE 阶段内传递

这样的结构一般而言同样是下降沿无延迟（*因为在无粉环境下，我们唯一一个可以无延迟输出上升沿的布线方式较为复杂*）而自复位输出上升沿的

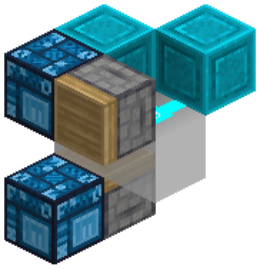
> *其实对于 BUD 而言，它需要的只是更新，而与上升沿或下降沿信号无关，这里的表述只是为了与前文统一*

4 红石粉重定向

由于它的英文是 Redstone Redirection，我们不使用常见的 “转向” 作为翻译

这就是刚才所提到的**无粉环境下唯一可以无延迟输出上升沿信号的方式**

众所周知，红石粉**只会向指向的方向充能**，因此只要创造一个**有能量，不指向某个方向**的红石粉，在输入上升沿信号时，**使它重定向而指向某个方向**，就可以向那个方向无延迟输出上升沿信号。



众所周知，红石粉会被以下方块重定向：

- 中继器、比较器、绊线钩 *不可推动*
- **红石块、侦测器、标靶、避雷针、探测铁轨** *可推动*

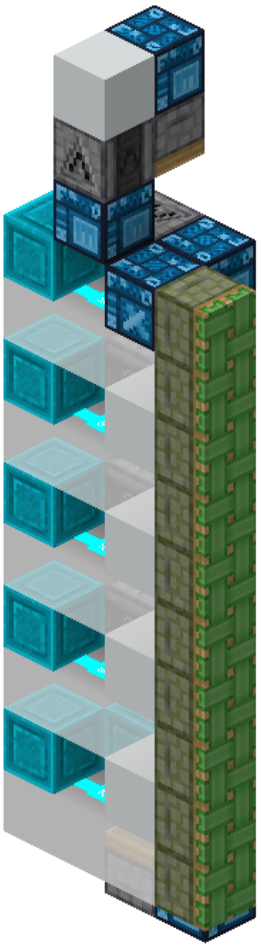
其中**可推动的**就是我们最重要的实际使用红石粉重定向的方式

需要注意的是，红石粉的重定向**只包含 PP 更新**，要用于激活某些部件，我们需要**额外提供 NC 更新**

下面我们简单介绍几个基于红石粉重定向的在树场中被**大量使用**的部件

4.1 柱子

非常简单，我们将若干红石粉重定向纵向排列，你能很自然的想到将重定向用的方块纵向排列，并在纵向上移动



这样我们就轻松获得了一整列**同步**的红石粉重定向，从而轻松地激活一整排活塞。

4.2 3GT 发生器

这是一个标准的 3gt 发生器

`0gt BE`，更新上推红石块柱子的活塞，红石粉**重定向并指向需要充能的方块**；上方的铁轨受到柱子推动提供的 NC 更新，被激活，向活塞柱子发出 NC 更新；活塞响应 NC 更新，推出

`2gt TE`，红石块柱子到位，被充能的活塞完成推出，**自检**，发现**不被充能**，创建计划收回

`3gt BE`，被充能的活塞开始收回，红石块柱子在略深一点的位置开始下推，**此时虽然充能了目标活塞，但目标活塞会优先完成收回动作**

`5gt TE`，复位，目标活塞再次自检，发现不被充能，不再推出

这样我们就完成了一次**3gt 推，3gt 拉**的活塞动作，这在现代蜜绿树场尤其重要

> *想必你能注意到，如果我们在 0gt BE 不给目标活塞 NC 更新，而在 3gt BE 给，我们就能让目标活塞的动作延迟 3gt。这是目前完美时序全树种采用的时序切换方案：在 0gt BE 和 3gt BE 都提供更新，由于目标活塞是二次递归的二级，由一级推动到位，它就不会受到第一次充能而推出，转为被第二次充能激活*

4.3 0T 发生器

这是一个简单的基于红石粉重定向的 0t 发生器

总之就是在红石粉重定向充能了被充能方块，并给到 NC 更新使活塞推出之后，**移除被充能方块**，从而达成 0t

你可以设计很多很多类似的装置，其中也包括像 [5.4.1] 提及的那样纵向堆叠

> *事实上，由于这里只有一个单独的红石粉，直接移除红石粉的充能源 —— 背后的红石块 —— 来制造 0t 并不会带来多少卡顿（约等于 3 个铁轨）。如果你不追求完全无粉，这也是相当不错的布线方式*

以上就是基本的无粉布线方式。再说一遍：**你自己去多扒几台 / 布几台比什么都强**，请务必重视自己的亲手实践

---

slug: high-speed-intro
title: 为什么你的树场慢 —— 高速树场入门
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 4

is-advanced: false
banner:
src: img/banner.png
alt: Graduate Text in Minecraft

如果不出意外的话，你在上一章设计的树场效率只有一万出头，处理周期轻松突破 30 甚至 40gt

是什么造成了这些？我们一点点分析

1 开始之前 ——0t

在1.1.2中，我们提到，mc 的时序分为**刻间时序**与**刻内时序**。如果我们能将原本通过刻间时序实现的功能**使用刻内时序实现**，我们就成功地对一台树场提速了。其中，最重要的实现方式是被称为**0t**的**活塞动作和布线模式**。

1.1 0t 的概念

在1.1.2中，我们同样提到：

> *BE 原件（活塞与音符盒）在 BE 中会按**受到更新，确认需要变更状态**的顺序执行运算。这个顺序有时被称为**深度**。音符盒不会增加深度（即延后其更新的对象在 BE 内的运算顺序）*

那么想象一个活塞：

在`0gt BE`，某个深度接收到**上升沿**信号，在其伸出后接收到一个更深（深度更深，即在 BE 执行队列中更靠后）的**下降沿**信号

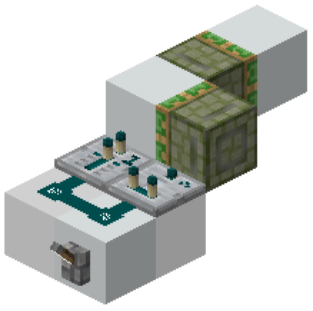
显然，它应当在被充能的时候推出，退充能的时候收回。这就完成了一次**0t 动作**。特别的，对于粘性活塞，它推动的第一个方块会在**比下降沿信号“深一个活塞”的深度**到位，拉动则与平常一致

> *事实上以活塞添加的深度作为深度单位并不严谨，由于活塞推动顺序的差异，事实上在推出的时候可以通过创建 b36 的顺序造出小于“一个活塞”的深度差异，有时候会以“珊瑚”作为单位（因为最广泛的使用似乎是在基于珊瑚的 TNT 复制上）。这有时候有点用，但并不会对整体布线造成太大的影响*

1.2 常用的 0t 发生器

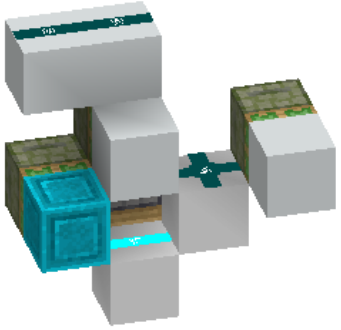
思路非常简单：在**同一个 gt 内的 BE 结束前**，在一个稍早的位置给活塞上升沿，稍晚的位置给下降沿，我们就可以获得一个 0t 动作

比如下图这个非常经典的基于**比较器和中继器在 TT 内执行顺序差异**的 0t 发生器：



由于一般而言中继器优先于比较器，在 2gt TT，会优先为 0t 活塞添加 BE，再为移除充能的活塞添加，因此移除充能的活塞必定晚于 0t 活塞动作，由此创造出了**晚于 0t 活塞伸出的下降沿信号**，从而完成 0t

再比如这个非常经典的基于**红石粉**（*希望你还记得红石粉时瞬时的*）的 0t 发生器：



在`0gt NU`，红石块被放置，`1gt BE`，移除压线方块的粘性活塞运动，由于移除红石块的粘性活塞被 QC，**在移除压线方块的粘性活塞更新它之后才添加 BE**，由此创造了一个在`1gt BE`，位于**1 活塞深度**的上升沿，**2 活塞深度**的下降沿。这样的信号足够被充能的活塞完成 0t *了当然你也可以通过更多的一路更新过去的 QC 活塞添加下降沿的深度，从而激活在输出端只能 QC 而被其他活塞更新的活塞*

> *值得注意的是，红石粉转向只有 PP 更新，我们需要额外的操作 NC 更新充能的目标活塞，从而完成上升沿信号（也就是对于活塞，上升沿和下降沿信号事实上是指其受到 NC 更新，确认需要推出或收回。*
>
>
>
> *实际上这里的 0t 指的是那个普通活塞更新粘性活塞的深度，而不是红石粉转向的深度（虽然这里没有大差别）*

2 “底座”

对于高速树场，我们很大程度上必须将**催熟**、**检测**、**树根处理**与**树苗循环**视为一体设计，称为**底座**

2.1 树根处理

这次我们必须先选择树根处理的模式了

在1.2.2中，我们提到：

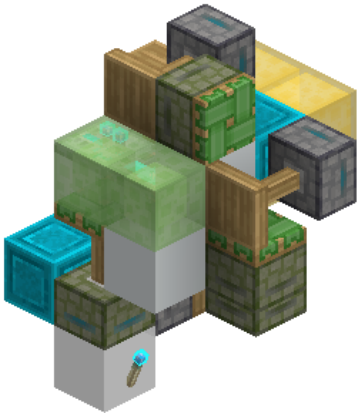
> *树根处理一般而言，有与主干一同处理、**上推**、**下吸**和**侧吸**四种方式*

如果你仔细思考，就会发现，与主干一同处理，我们就**只能在催熟上使用一个发射器**，**上推**我们至少在树场运行的**第 6gt**才能种树，**下吸**则是**第 3gt**，**侧吸**最快可以做到**0gt**。在高速树场中，我们一般选择**下吸**或者**侧吸**

下吸：

@Scorpio 天蝎君

侧吸：



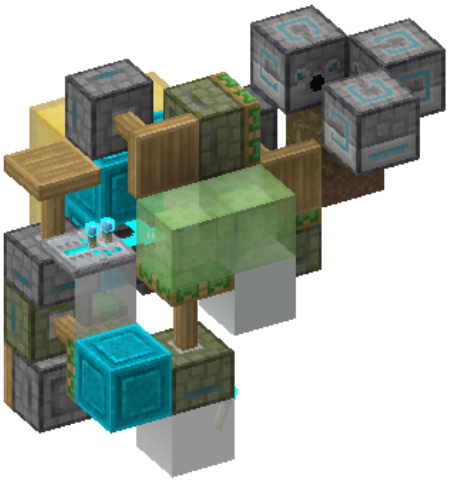
@Qonctrol, Dreaming_Galaxy, BFladderbean, Feng_Bl PUTF std2

> *因为玩家操作在 NU 阶段，在 gt 最后，泥土归位在 BE 或者 TE 阶段，因此在泥土归位的那一个 gt，我们就可以种树了*

> *事实上上推的问题不止慢，还记得丛林木最高可以长 12 格吗？算上泥土，就超过活塞的推动上限了。还记得金合欢的侧枝吗？上推会导致同 y 最多有三个原木，极大增加处理的难度，因此上推现在基本上是被废弃的*

2.2 催熟与树苗循环

~~虽然蝎子很想讲催熟机制但我觉得没必要，因为~~高速树场的催熟事实上约等于**堆发射器**。简单而言，**不要在乎骨粉消耗**，尽可能多地放置催熟用的发射器，增加玩家种下树苗到树苗可以长成前**预催熟**的次数，和树苗可以长成之后的**生长窗口**（因此我们会引入**交叉催熟**，即错开发射器发射骨粉的时间。一般而言会错开 2gt）



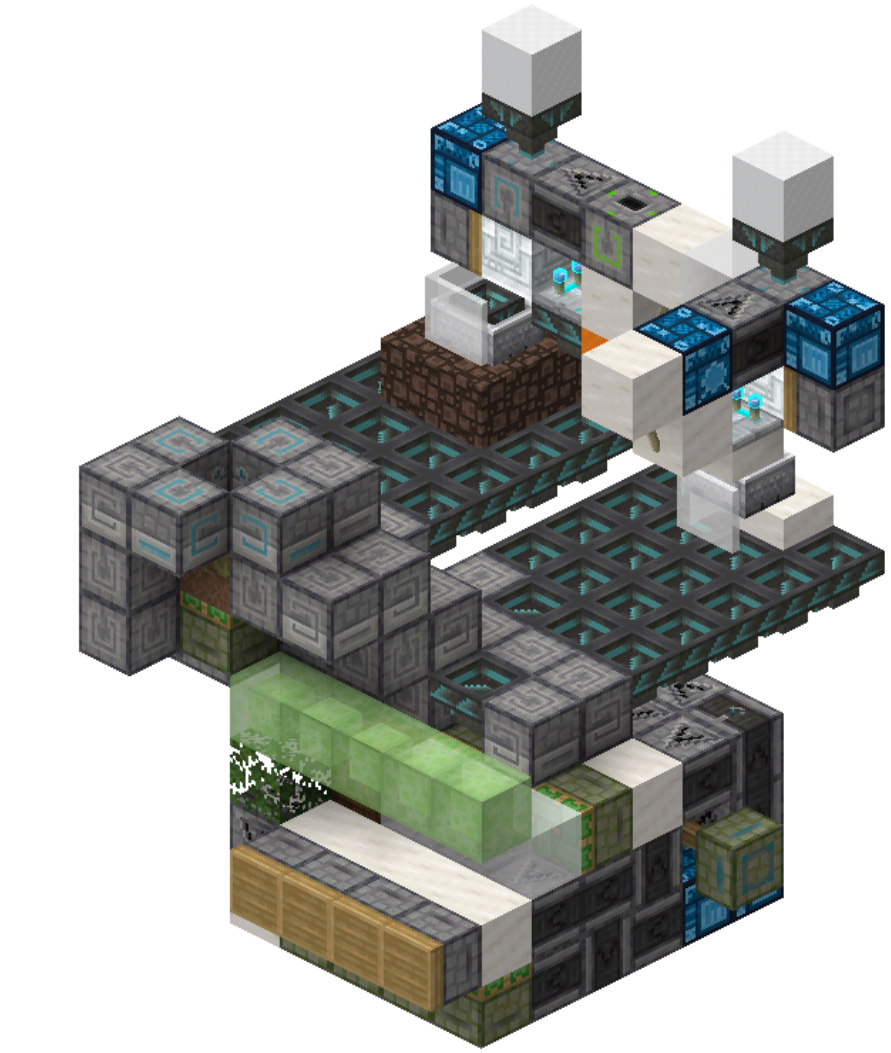
@PUTF std2

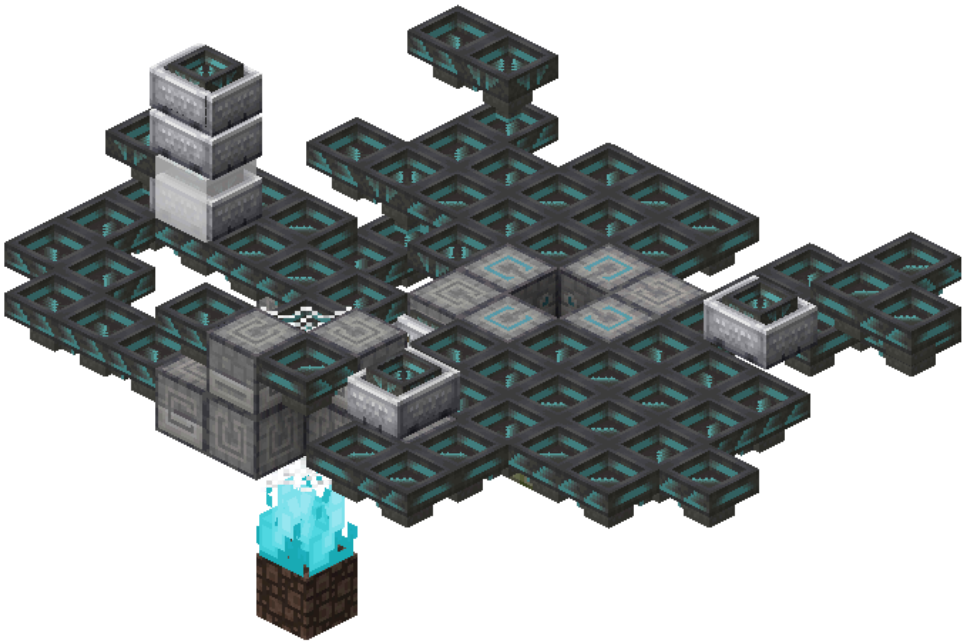
@Scorpio 天蝎君

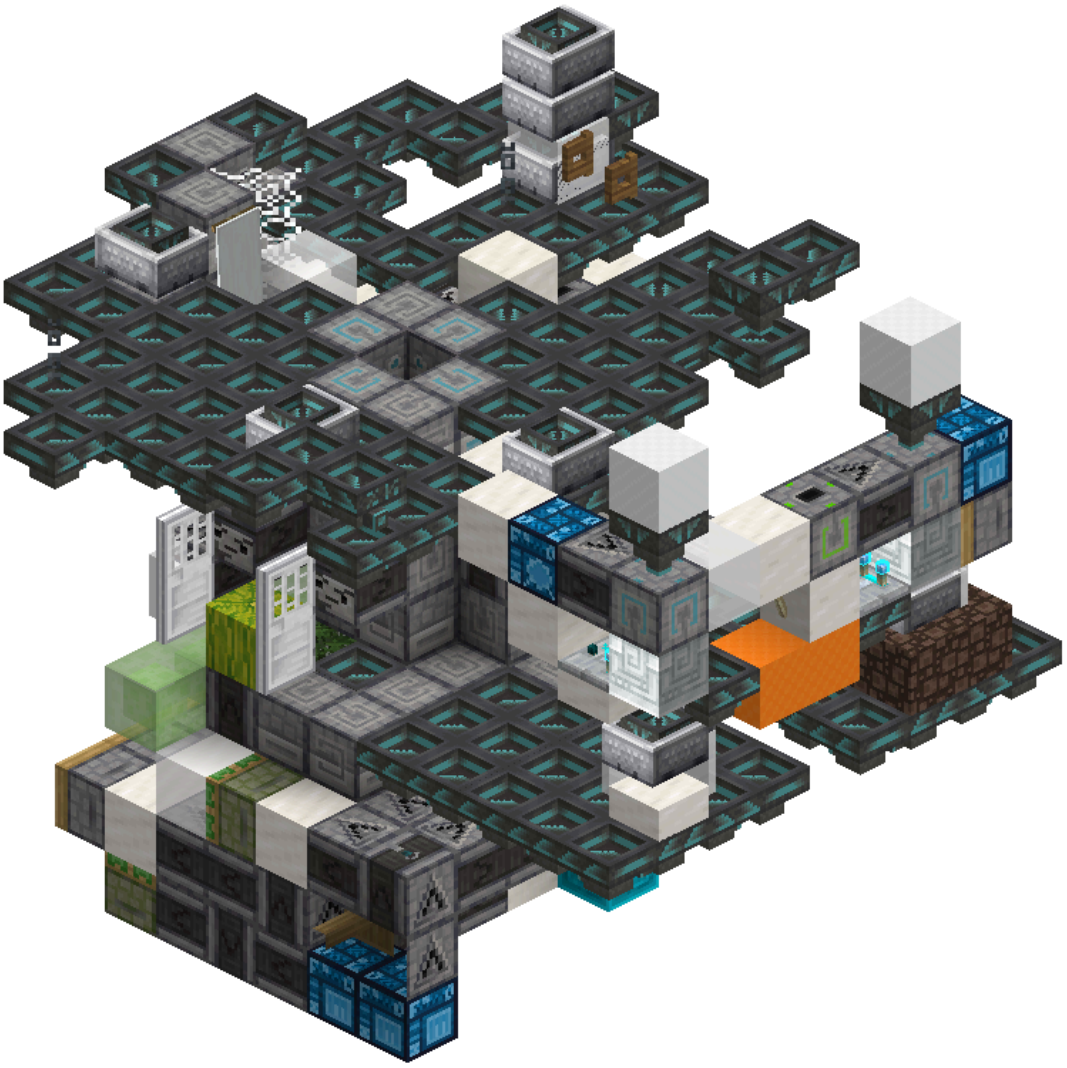
意义明确的交叉催熟 / 齐喷催熟切换装置配图.jpg

高速树场催熟结构需要面临最大的问题事实上来自于**树苗循环**。拥挤的底座空间会导致催熟的骨粉供应链和树苗循环漏斗**各种打架**。一般而言我们会将骨粉供应链设计的尽可能规整和紧凑，为树苗循环留更多的空间。~~事实上没啥真的可以写到课本里的，只能自己多做几次~~

一个非常紧凑的骨粉链例子：



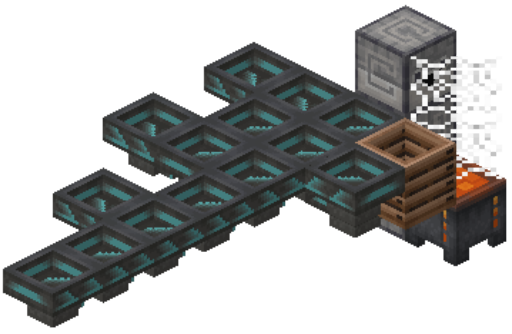


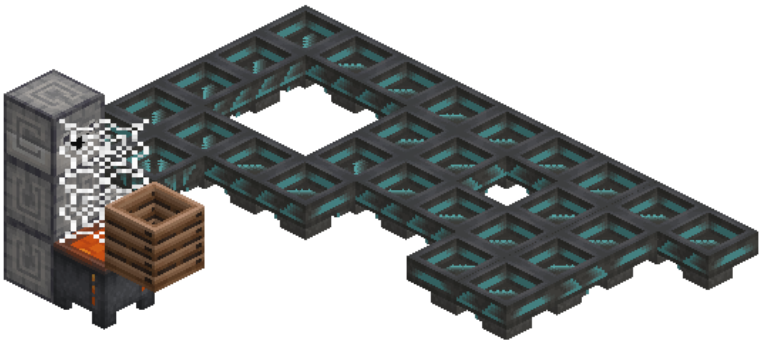


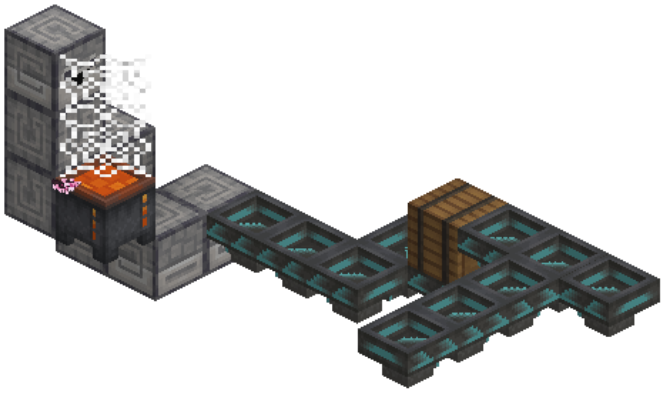
@Islyric, Qonctrol, BFladderbean PUTF+

可以看到，即使非常紧凑的骨粉链依然会被其它东西塞得满满当当

对于树苗循环，由于~~沟槽的麻将乱改树叶掉落导致~~树叶会掉落木棍，我们需要更多的投掷器处理大量的物品，另外我们也需要尽可能 “拉直” 漏斗链，并采取分区域收集的策略，以最大化物品传输效率：



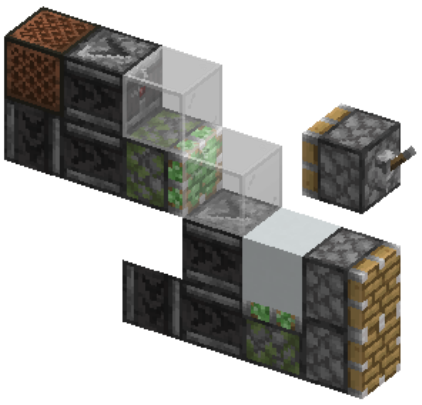




@Chuo_di, Islyric, Qonctrol, BFladderbean, Sarumi_QAQ PUTF std2c

> *事实上树苗循环还会受到主干和树叶处理架构的影响，对于高速树场而言，有时还要在主干和树叶处理布线中加入用于收集树苗的漏斗和漏斗矿车*

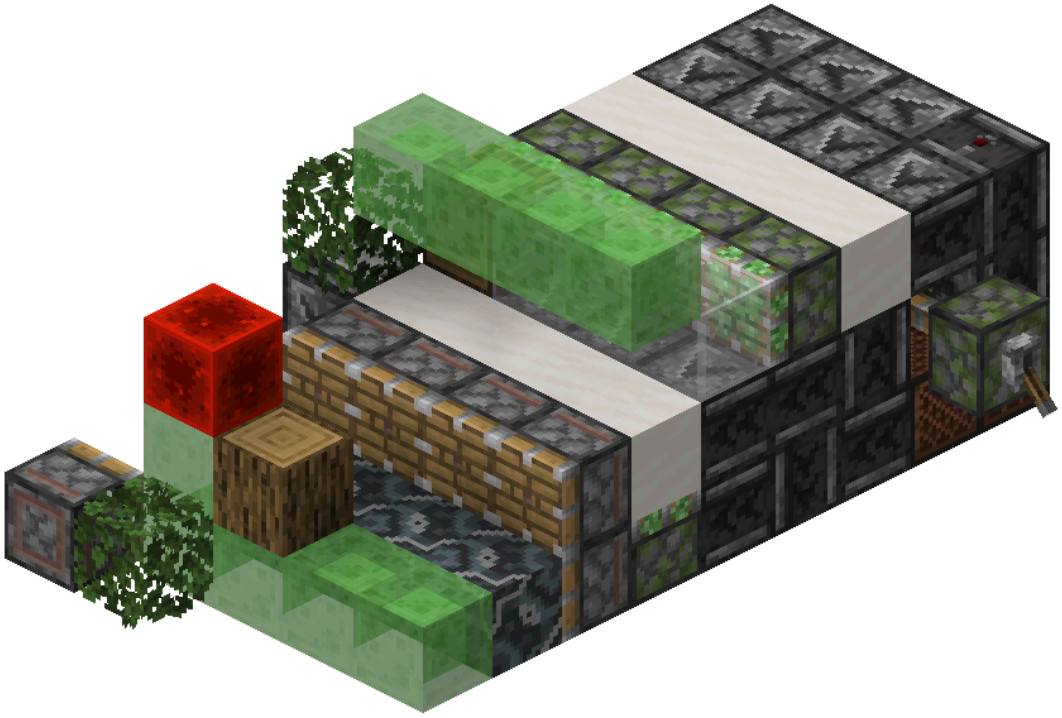
2.3 推动上限检测



这是一个由 Bright_Observer 制作的推动上限检测单元

> *当上推活塞在 TT 阶段被更新时，它会计划推动。活塞计划推动的时候会检查其是否达到推动上限，而此时上方的玻璃没有收回，若拉杆拉下，活塞伸出，则会无法推动。因此得以激活两个普通活塞，在 BE 阶段更新到粘性活塞。粘性活塞再次自检，伸出。若拉杆松开，则一切正常，粘性活塞正常推出，两个普通活塞靠上方的一个是否推出取决于朝向，而由深度影响，下方活塞总是不会推出。这也就完成了检测。*
> *——GTMC 时序理论 5.10.1*

总而言之，这个单元可以以**8gt 周期**运行。为了对齐交叉催熟时钟，我们可以堆叠四片这个单元，每个错开 2gt 运行，放到泥土下，用粘液块连杆连接。这样我们就拥有了一个几乎不占用发射器和其他布线空间的检测模块。~~\魔法检测/~~



@Islyric

3 核心架构 —— 主干与树叶处理

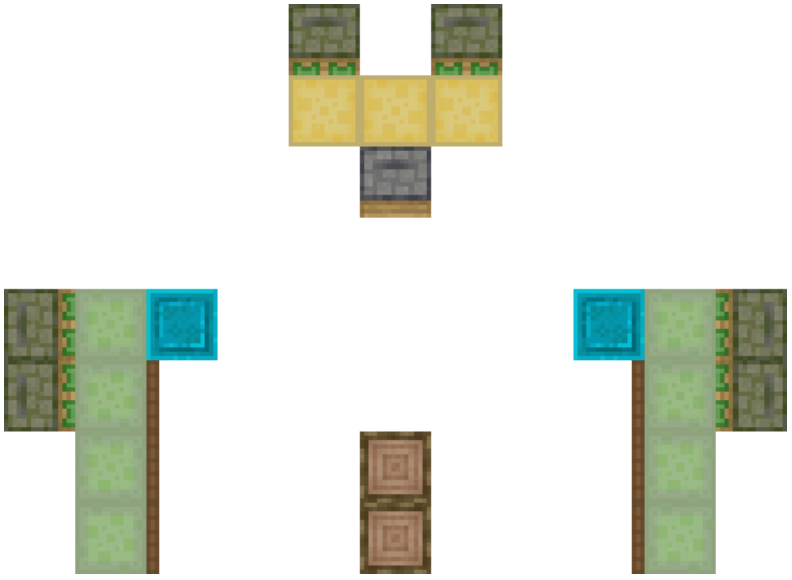
> *这是决定一个树场最小运行周期的部分，请务必仔细考量*

在设计架构之前，我们需要先了解如何分析一个架构的时序

3.1 时序设计

对于一个既定的架构，我们需要分析它处理树木的方式，并设计出最短的运行时序

以 PTHSUTF 的架构为例，我们尝试设计对于金合欢最短的处理时序（处理原则请回顾3.1.2：



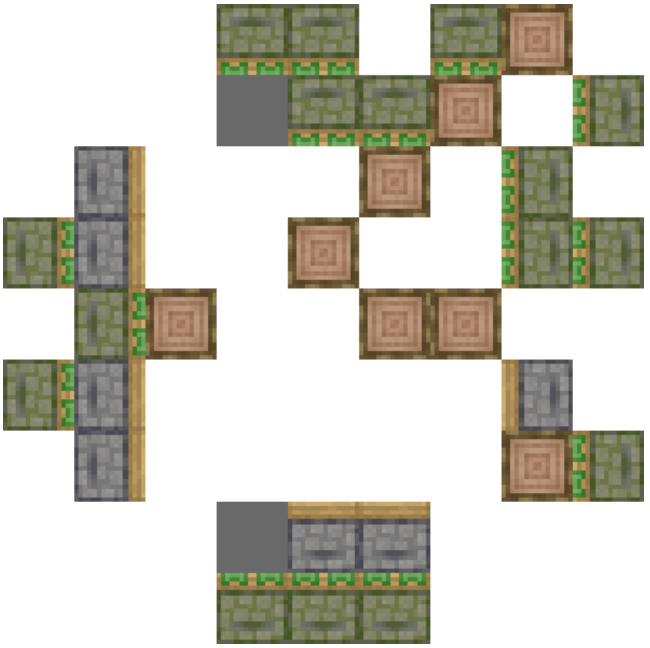
@Fallen_Breath, CCS_Convenant

`0gt` 主推和侧墙一级推出 *对所有侧枝进行一级归中*
`3gt` 主推和侧墙一级到位，主推二级推出 *让主干方向侧枝优先归中，避免打架*
`6gt` 主推二级到位，左侧墙二级推出 *将仍未被推出的侧枝归中到中央和右侧一格内*
`9gt` 左侧墙二级到位，激活主推三级，并开始收回，使主推三级 0t；随后，右侧墙二级推出 *清除位于中央的侧枝，并将右侧一格内的侧枝归中到中央*
`12gt` 右侧墙二级到位，激活主推三级，并开始收回，使主推三级 0t *完成处理，开始复位*
`15gt` 主推二级开始收回，侧墙一级开始收回
`18gt` 主推一级开始收回
`21gt` 复位

> *在树场中，我们一般可以认为时间是以 3gt 为单位的，尽管它是错误的。事实上，这个架构的金合欢时序只要满足 6gt 第一个侧墙二级推出，9gt 一侧墙收回后另一侧墙推出，都可以完成 21gt 复位，并不只有这一种时序设计方式*

各位可以尝试设计一下 TT1998 的 6gt 丛林架构的时序 *（注：下图是 @大尸凶一只修改过的架构，但不影响时序分析）*

第一层：



第二层：

（这里只分析主干处理和 “红石块轮” 的时序）：

`0gt` 侧面伪二次递归一级 0t，随后二级 0t，抽走正面伪二次递归面前的原木；随后正面的伪二次递归拉走主干位置的原木；红石块轮第一个活塞推出，激活另一侧的树叶处理活塞，随后第二个活塞推出，0t 树叶处理活塞并激活主干处理活塞，最后第三个活塞 0t 收回红石块，0t 主干处理活塞
`3gt` 两个伪二次递归一级 0t 收回；红石块轮最后一个活塞 0t 收回红石块
`6gt` 复位

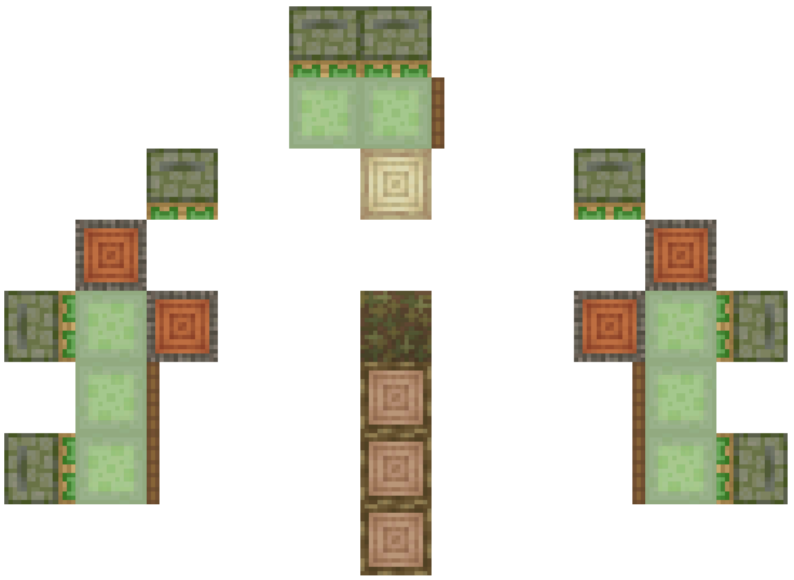
对既定的架构进行时序设计，对实际布线有很强的参考意义。同时，也有助于理解导致某个架构运行周期长的**罪魁祸首**在哪里，从而进行针对性优化

3.2 架构设计

如果对现有的架构进行时序设计后，发现还是不足以达到心目中预期的速度，就需要设计**更快的架构**

这里展示两个经典的提速方案：

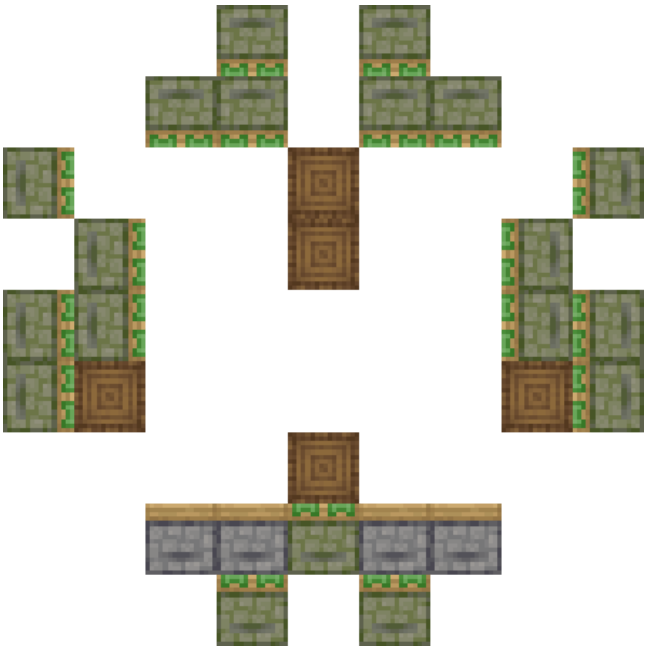
- Nutstory 架构：在3.1.2我们提到，金合欢侧枝除了可以归中处理，还可以通过**增加原木输出**处理掉。现在我们学习了时序分析，你应该可以注意到：这个架构不需要为侧枝归中设计时序，因此可以运行在 12gt 下，比 PTHSUTF 的架构快上许多。



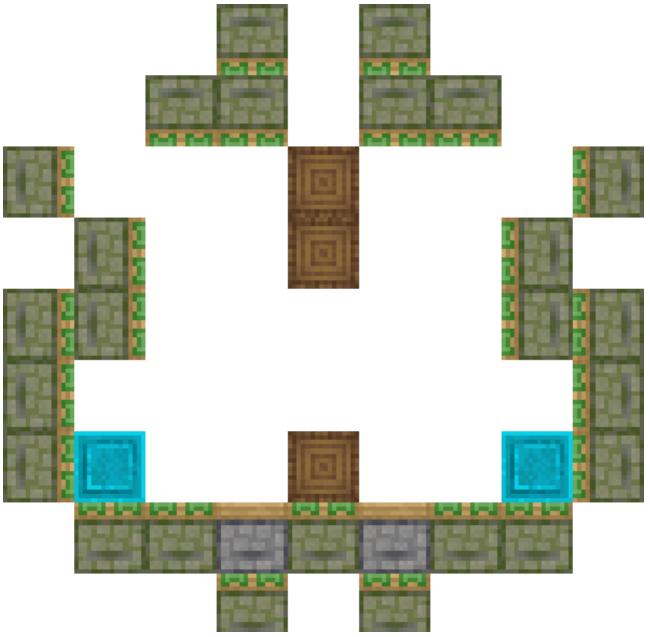
@Nutstory

- TT1998 的 6gt / 尸凶的 4gt 架构：它们事实上是一个架构。在上面我们已经做了时序分析。另外由于 4gt 的时候，**伪二次递归的二级活塞已经被移除了，并在目标位置创建了 b36**，树苗可以正常生长，因此只要在另一侧也做一套为二次递归，并解决树苗问题，我们就可以轻松地获得一个 4gt 树场

第一层：



第二层：



其实要设计更快的架构，思路非常简单：**减少必须发生在不同宏观时刻的活塞动作**。请务必在设计架构的同时思考架构对应的时序，**不然你很容易陷入不知道自己在做什么的境地**

> *事实上时序和架构设计也有特例，yunj 的 STFU 是 8gt 的全树种树场，有一部分架构并不跑在 3ngt 的时序下。~~不过由于现在已经有 4gt 树场了我们不需要太关心这方面的问题~~*
>
>

> *对于侧枝树，我们并不需要死磕最小运行周期，可以转而增加侧枝的处理量。~~你完全可以用做出樱花效率暴打某些 6gt 的 12gt 树场~~*

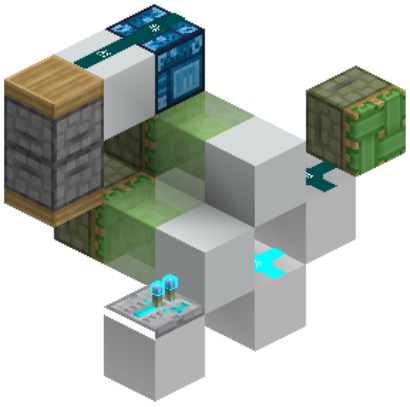
总之，一个优秀的架构是做出一台好树场的**最重要前提**。在开始布线之前，请**尽可能优化你的架构**

4 布线

4.1 布线方式

这里我们提一下最简单好用的高速树场布线方式：

这是一个非常经典的基于红石粉和深度的**双边沿 0t 发生器**（即输入信号的上升沿和下降沿都会被转化为一次 0t 信号）。我们可以将一个粘性活塞连接到它的输出，推拉一个红石块，这样就可以得到**不增加宏观延迟，但上升沿更深**的与输入**拥有相同宏观时序**的信号。利用这点，我们可以轻松地控制布线的深度：**只需要按照设计的深度顺序，将激活活塞的红石粉依次以相同的方式连接就可以了**



基于这种布线方式的经典例子

4.2 布线检查

虽然我们一直在说 “经济基础决定上层建筑”，但是也不能设计出来一个 6gt 的架构最后布线跑了个几十 gt 不是？分析一下自己的布线，看看**到底是哪里的时序有问题，拖慢了树场整体的运行速度**

不过对于高速树场的布线，我们无法给你一个更具体的办法。**你自己去多扒几台 / 布几台比什么都强**

---

slug: simple-design
title: 从零开始设计一台最简单的树场
title-en: null
author: jslskd
co-authors:

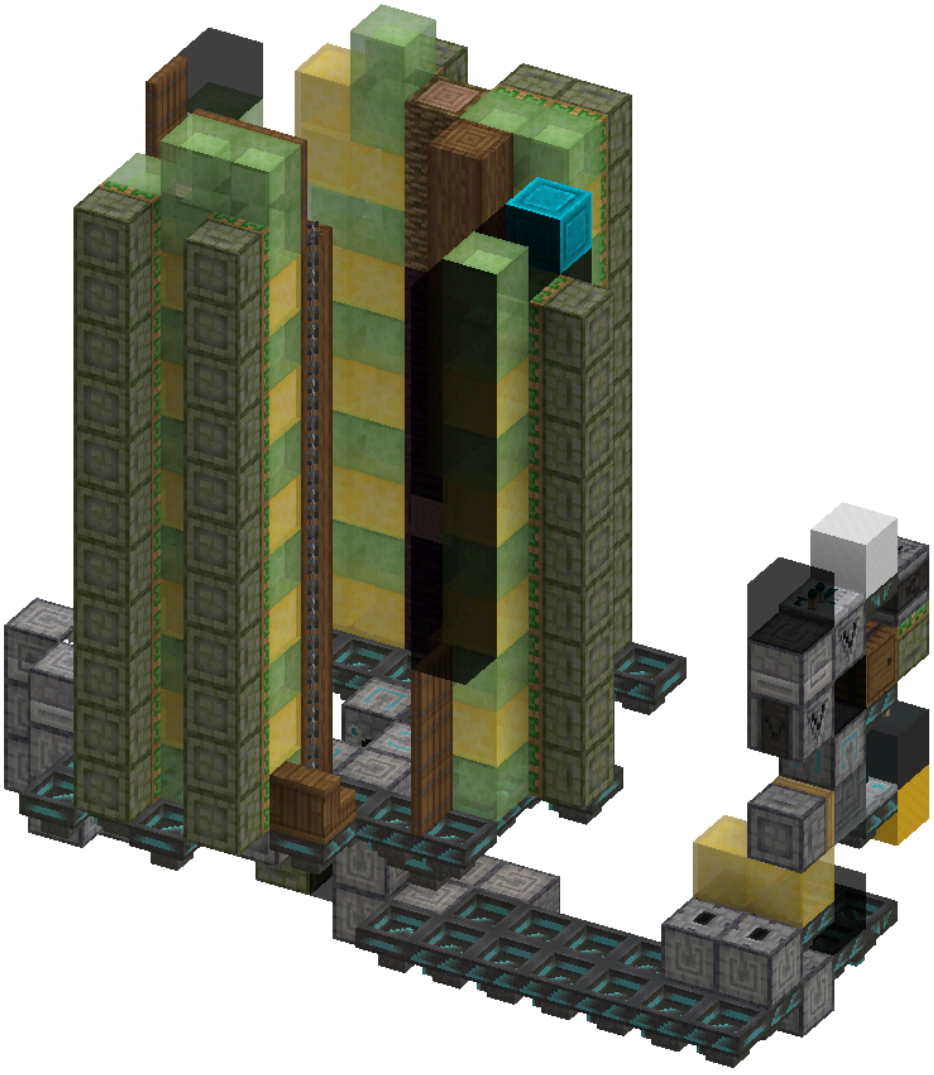
- Arcadi4
- BFladderbeanawa
- SDU-Little-River
  date: '2025-02-03T00:47:44+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 2
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

树场的设计流程一般分为**架构设计**和**布线**。这里我们以一台非常简单的白桦树场为例，带大家从零开始设计一台树场。

1 架构设计

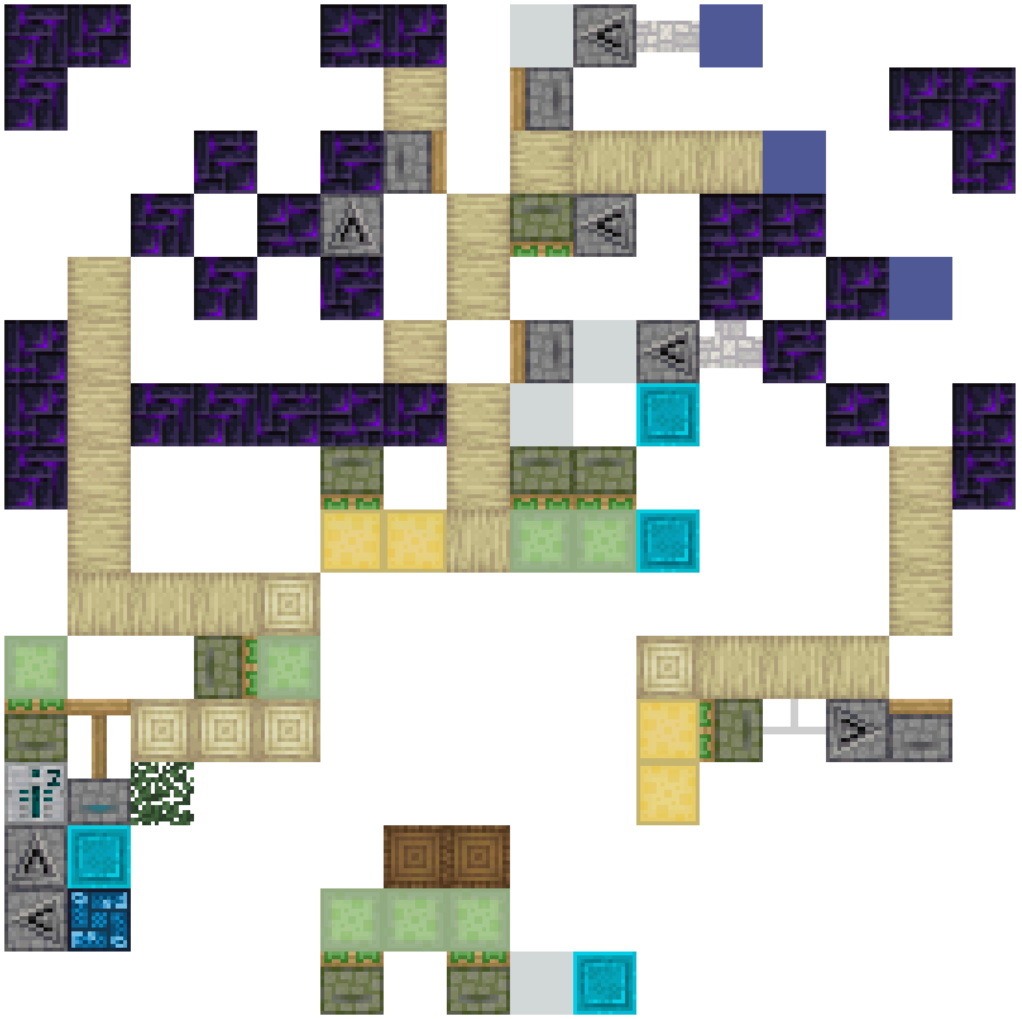
架构，在树场中一般指代树场结构的**特定排布方式**，一般而言可以分为**核心架构**和**处理架构** *即方块流转掉落物模块的架构设计 *

核心架构：



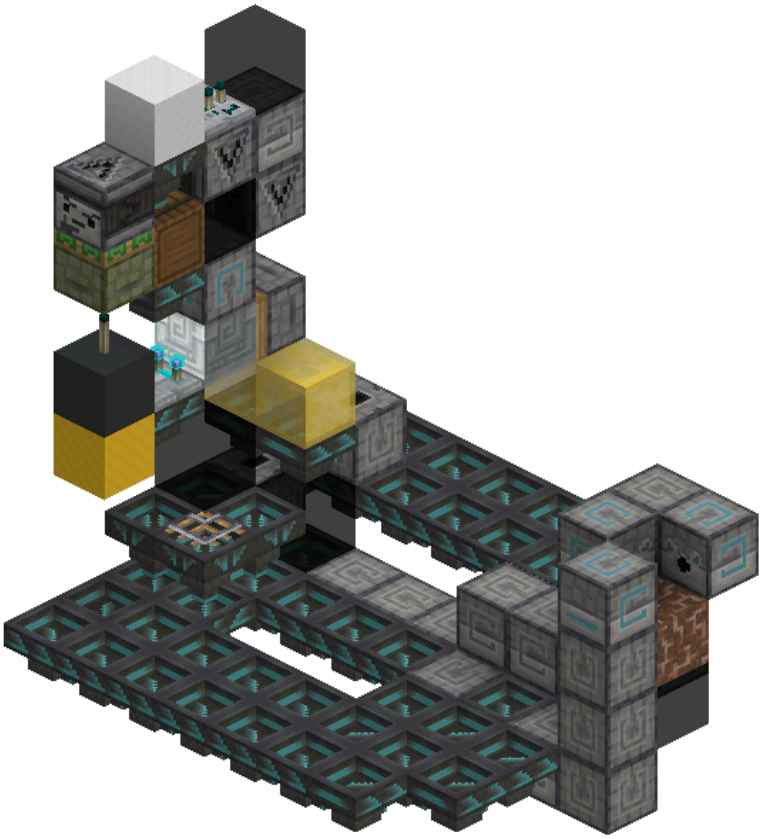
@Chuo_di

处理架构:

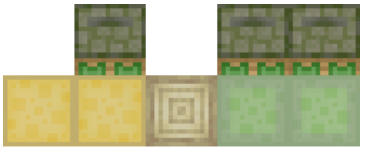


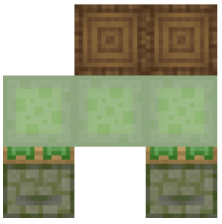
@Chuo_di

核心架构一般包含**催熟架构**,

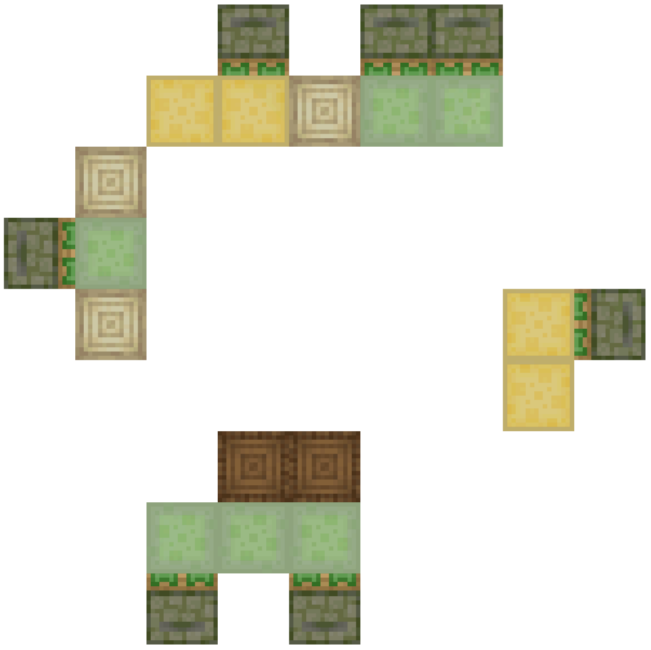


树干处理架构,

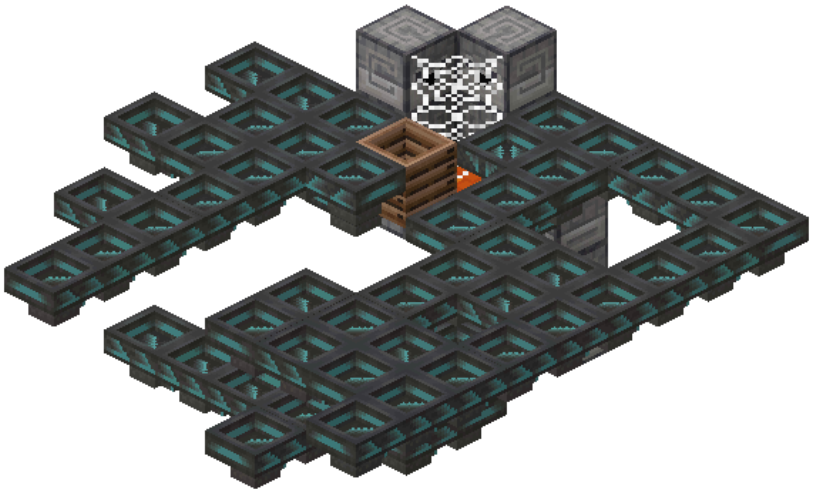




树叶处理架构,

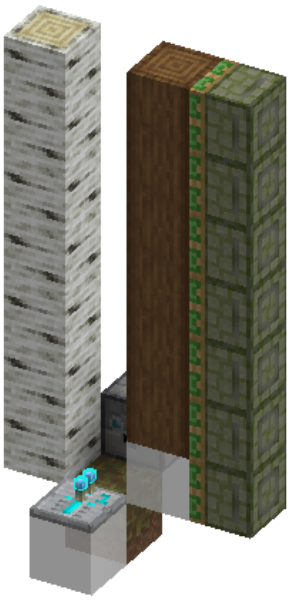


和**树苗循环架构**，



一般先设计**催熟结构**或**树干处理架构**，再设计**树叶处理架构**和**树苗循环架构**

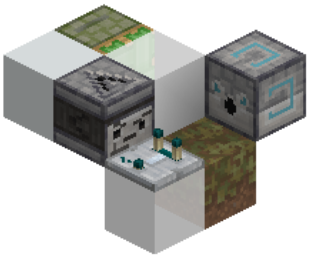
树干处理架构的基本形式已经在上一章阐述，这里我们主干处理选用**直推**（*采用推出结构的主干处理会被称为“主推”*），根部与主干一同处理。催熟架构选用**单发射器**。



一般而言，我们不希望处理周期长于 4gt 的树场使用时钟激活。因此我们需要引入**检测**结构，用于检测树木的生长，并触发树场的机械结构

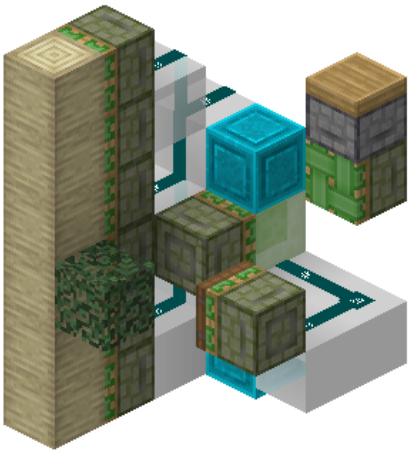
检测的基本方式有：**比较器检测、QC 检测、BUD 检测、推动上限检测**

比较器检测：

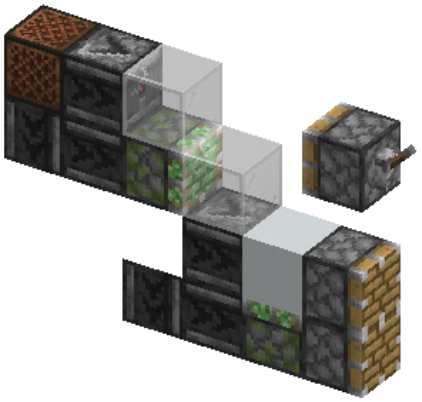


QC 检测：

BUD 检测（这里 BF 偷懒了直接使用树叶检测了）：



推动上限检测：



这里我们采用 QC 检测。QC 检测的基本原理是，由于树干是可充能的，树苗长成后，树根被充能，于是侧下方的活塞被 QC。只需要给它更新，它就会推出，从而触发树场的处理结构

> *虽然暂时我们还不会引入，但推动上限检测是目前最重要的检测方式。在这一章之后，几乎所有我们分析的检测式树场设计都使用了推动上限检测，值得读者注意*

树叶处理架构而言，白桦只需要 20 个树叶，因此简单的活塞墙就能满足需求。由于活塞很卡，这里我们改为粘有活版门和铁链的**蜜绿墙 ***这样可以增加树苗飞溅的空间，从而保持较少的树叶处理量并很大程度上避免漏斗矿车的使用 *

> *未经优化的械电设计带来的卡顿相当大，其中的最大头之一就是活塞，希望读者随时注意能否减少树场中的活塞（另一个最大头是红石粉，我们会在另外的章节叙述减少红石粉的方法）*

> *对于更复杂的树场，设计核心架构的同时也要同时设计架构运行的时序。这里我们暂时忽略它，留到侧枝树处理再做具体讨论*

树苗循环架构而言，我们用漏斗铺满树场的底部，并连接到一个投掷器上。玩家挂机时站在这个投掷器旁边，就可以拿到树场回收的树苗。另外，我们将核心周围围起来，避免树苗飞溅范围过大

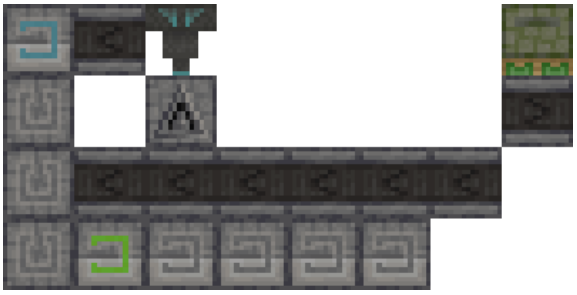
处理架构，我们采用一个简单的**TNT 爆炸室**

> *值得注意的是，由于目前的高速全树种有多个原木方块流输出，处理架构也会非常复杂，我们会在对 PUTF std2 的分析中讨论对这些情况一般的设计思路。*

2 布线

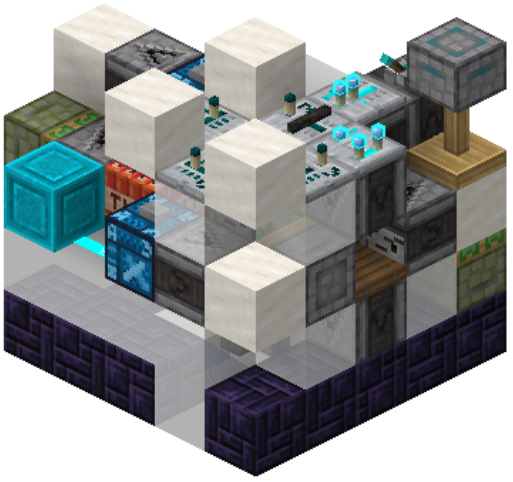
基本思路就是在不影响树场时序的前提下，将树场的各个部分连接起来

首先，我们为催熟发射器和树苗投掷器接上**4gt 时钟**



接着使用红石粉激活各部分的活塞

对于 TNT 爆炸室，我们随便放一个**TNT 复制**，为了减少空间的占用，我们使用水流**缓存 TNT**，并用水流汇聚掉落物



到这里，一台最基本的树场就设计完成了。如果你没有参考插图，这可以说是你设计的第一台树场。现在尝试分析一下它的时序？

> *时序分析是对树场最基本的分析方式。通过架构与布线读出树场的运行时序是每个树场人必备的基本素养。*

基于以上阐述的设计过程，你可以设计出最快 6gt 周期的白桦树场。这是一个参考设计~~懒狗星河没做卸货器好孩子不要学哦~~

---

slug: 4gt-farms
title: 关于 4gt 树场的一切
title-en: null
author: jslskd
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-02-05T17:10:25+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 7
  is-advanced: true
  banner:

src: img/banner.png
alt: Graduate Text in Minecraft

关于 4GT 树场的一切

这一章需要将前面的知识最大限度地综合起来，对于过往提到的部分，我们都会写出第一次提到的地方，~~hope you enjoy overbraining~~

开始之前，我们必须提一嘴：虽然检测式的 4gt 树场确实存在，但是较为复杂，**我们最后简单讨论**。这里其余的树场都是**以 4gt 时钟驱动运行的**

> *以及，由于一个活塞动作是 3gt，所以你完全让树场跑在 3gt 时钟下，但很显然这就需要我们用来连点器催熟。传统意义上，任何使用连点器的树场都是不被认可的。不过现在 12gt 和 8gt 大云杉已经成为特例了*

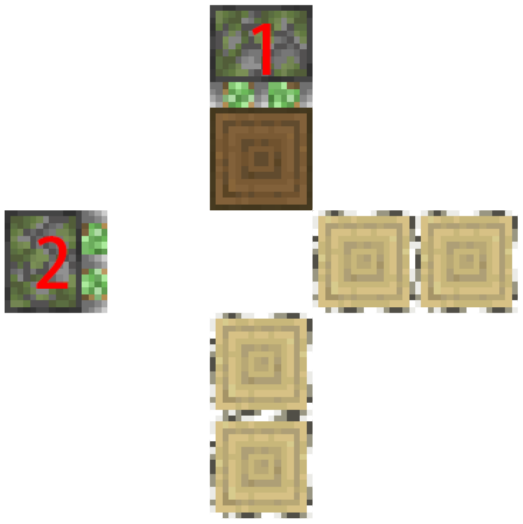
1 4GT 白桦

无论怎么说，还是得先从最简单的开始：4gt 白桦

由于 4gt 主干与底座独立性较强~~且底座几乎可以套公式~~，这里我们先讲主干 —— 核心架构设计

1.1 核心架构与时序设计

这里我们讲两个典型的比较简单的架构：



这是一个非常古早的架构，某种意义上似乎已经无从考证最初的设计者了，不过它也是非常好的用于分析 4gt 设计的实例之一

首先分析一下时序（建议先自己尝试设计一下，再看我们给出的时序）：

- 第一个周期 首先 1 号活塞 0t，将处理用的原木推到中心，将原木向下推走；然后 2 号活塞 0t，将原木收回
- 第二个周期 首先 2 号活塞 0t，将处理用的原木推到中心，将原木向右推走；然后 1 号活塞 0t，将原木收回，**复位**

显然每一个周期都只包含 “一个活塞动作”（*所有活塞都只进行一次同步的动作*），只需要 3gt 就可以完成，因此这显然可以作为 4gt 树场的架构

> *为了架构清晰，我们移除了树叶处理。你可以尝试自己设计一个。*

这个架构就没这么古早了，是星河做的，时序会略微复杂一点

时序：（都在`0gt BE`） 1 号活塞 0t 推出上方原木；随后 3 号活塞 0t 收回下方原木，4 号活塞向下 0t 推出原木；随后 2 号活塞 0t 横向收回原木，**复位**

这样我们同样完成了 “一个活塞动作” 内处理原木的任务

> *这里我没有引入收回式的架构设计：它的复杂度对于白桦而言完全没有必要。待会在 4gt 全树种的部分我们要解决关于它很多很多很多的问题*

1.2 4GT 底座与树苗循环系统

如4.2.1所述，一般而言我们使用下吸或侧吸处理树根。对于 4gt 而言，**三发 + 侧吸**是比较方便的选择

由于侧吸活塞在 2gt te 就完成了收回动作，我们只需要让横向推出的活塞立刻推出，**并在 4gt BE 结束前收回**，我们就可以轻松地设计一个 4gt 侧吸根部处理。

> *事实上，下吸也是可以用的。传统上我们会使用两个分别处在 x 轴和 z 轴上的 3gt 归位泥土的下吸模块；后来，hampter 发表了通过粘液块推拉实现整体 4gt 复位的下吸架构，如下图*

侧吸架构给我们留了一个非常美妙的挂机位置：站在侧吸下方，蹲下（潜行），瞄准左右其中一个发射器的侧上角，我们就可以开始种树了。

接下来的问题就是树苗循环和骨粉供应。我们一般**先做骨粉供应~~因为有公式~~**

像这样用投掷器向上投掷骨粉，在横向拉到树场外围，接上拆包机，你就获得了一个非常标准的骨粉供应模块。需要注意的是，**由于基于时钟的 4gt 树场每 4gt 才有一个长树的窗口，我们不使用交叉催熟，而使用同步催熟**

树苗循环而言，由于 4gt 会带来大量的需要处理的物品，我们一般使用**3-4 个投掷器**向玩家投掷树苗。由于 4gt 常用的布线方式是无粉的（*尤其是侦测器 0t，我们待会会讲*），且内部没有太多树苗飞溅的空间，**我们经常需要收集飞到树场核心之外的树苗**。常规做法是做一个围墙把核心围起来，然后在其中**铺满漏斗**。有时我们还需要收集卡在核心上方的树苗，一般而言我们会**用水冲下来**或者**用漏斗矿车收集**

在这些之后，只需要进行布线，你既可以获得一台 4gt 白桦树场了。不过显然，如果使用红石粉，这台树场会变得非常卡。因此我们需要了解对于 4gt 树场，无粉布线的方式

2 无粉 4GT 布线 —— 无粉 0T 发生器

对于无粉布线的整体思路，大致可以这么说：

- 首先我们要明白，不是所有的东西都必须跑在不同的深度上，对于 “必须跑在不同深度上” 的东西，事实上我们也对**上升沿**和**下降沿**有**不同的操作空间**。
- 其次我们要明白，**4gt 树场是依靠时钟运作的**，我们不需要严格按照运行时序设计启动时钟的布线，**只要全部启动之后能够维持正确的时序**，剩下就只是开关机复位的事情。这两件事情对于无粉布线是尤其重要的，它们可以帮助我们极大地**节省 BED 装置**，从而削减卡顿

理解这两件事之后，就会理解，**基于无粉 4gt 布线设计出来的树场必然是模块化的**，越大型的树场，越是这样

> *对于以红石粉为基础的布线而言，之前提到的 “按照深度顺序一个一个串过去” 的方式依然是最简单粗暴且好用的，这与无粉布线的情况是不一样的*

不过由于 4gt 全树种是由时钟驱动的，**将各个部分连接起来**实际上是非常简单的。我们真正需要了解的部分，是**可以纵向堆叠无粉 0t 发生器**，拥有了它们，我们才能真正让我们的无粉 4gt 树场按照我们想要的时序运行

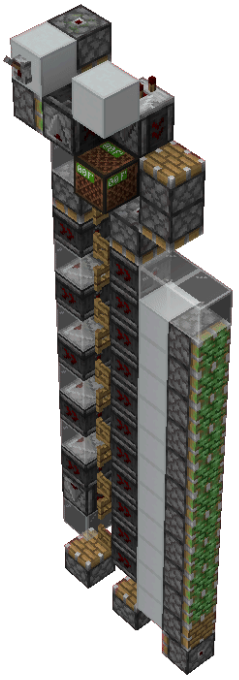
2.1 侦测器相关的 0T 发生器

这是一个基于侦测器的，8gt 周期的 “单边沿” 0t 发生器（*即只在向下推动（这里一般称为上升沿 [^1]）时会产生 0t，原理请参考 GTMC 的时序理论部分*）。我们这里直接接入 8gt 时钟驱动，之后的分析都在 8gt 时钟驱动下进行

[^1]：这里的上升沿概念与之前提到的不同，这里明确：上升沿的本意是输入信号从无到有（0 到 1）的 “边沿”（反之，下降沿就是从有到无（1 到 0））

反正 4gt 树场跑在时钟下，我们可以给**上方受时钟激活下推的活塞一定的深度**，这样理论上向上推动侦测器之后，如果侦测器可以提供充能，移除侦测器之前的时间足够目标活塞开始推动，从而完成向上推动的 0t

下一个问题就是，显然侦测器在上推之后是不会激活的，怎么办呢？既然侦测器只能在一个边沿产生 0t，那么我们就让侦测器在**第一次下推时被激活**，这样他就只会在上推之后被激活，**并在下推时产生 0t 了**



> *事实上我们可以在侦测器的脸上贴一排方块，并随着侦测器运动一同运动，但那玩意非常卡，我不希望你们有谁滥用它。它只有在需要填满周围空间以方便树苗回收的时候优于上述设计*

由于这个 0t 发生器产生的 0t 信号**上升沿在 TT**，因此我们**只能控制它的下降沿信号**。方法非常简单，串一大堆互相更新的活塞就好了

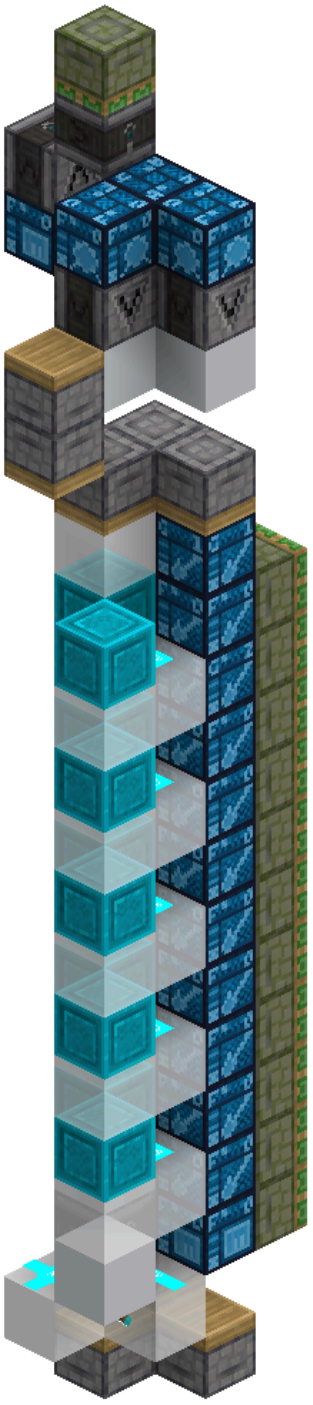
> *其实我们可以结合树场本体的布线做一些更优化的处理，反正给上推活塞更深的 NC 更新就好了*

> *事实上上升沿深度也不是不能控制，只需要在 BE 阶段 0t 推入被充能方块，我们就控制了充能的上升沿深度。但这个方法有个严重的问题：太大了。很多时候如果不是架构设计刚好可以在中间塞一坨这个结构，我们都会使用红石粉重定向*

另外一种方式是直接使用粘性活塞（或者粘液块）推拉侦测器，只需要让给粘性活塞的 NC 更新带有一定的深度就可以让这个 0t 跑起来。它一般只用于上升沿 0t（可以做双边沿但一般而言没必要，你可以自己尝试一下）

2.2 红石粉重定向相关的 0T 发生器

如图



由于下推重定向用柱子的活塞旁边的活塞会在下推的时候发出 NC 更新，**同时更新音符盒和下推音符盒的活塞**，众所周知，**音符盒不增加深度**，因此音符盒会在下推开始之前**更新所有目标活塞使它们开始推出**，从而完成 0t

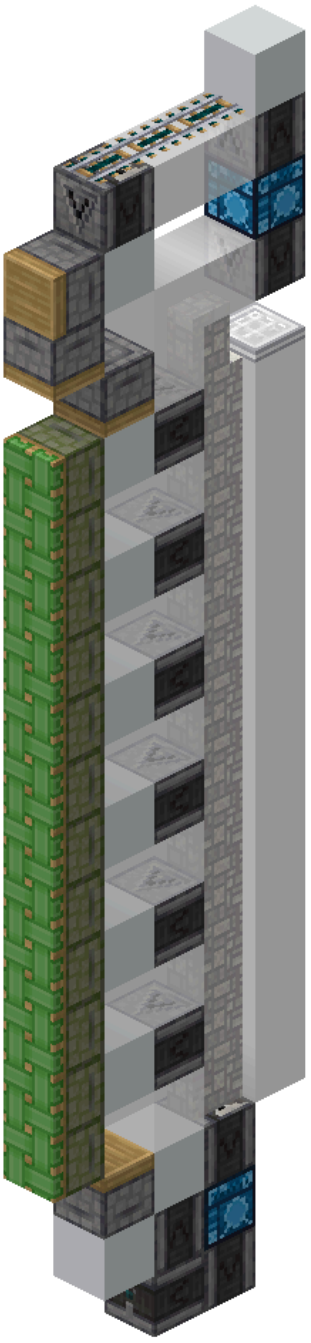
红石粉重定向是我们在无粉 4gt 布线中**最方便的控制 0t 的上升沿信号深度的 0t 方式**

> *事实上不是所有地方你都非要控深度，通过在 TT 中侦测器不同的激活顺序来控制 BE 中活塞的激活顺序也是可以的*

> *红石粉重定向 0t 发生器最大的问题在于要卡矿车，不然中间会囤积大量的树苗，所以请务必谨慎使用（但其实用起来还是比推拉被充能方块的侦测器 0t 方便一点*

2.3 基于墙电的 0t 发生器

如图



在通过墙电激活侦测器之后用活塞移除被充能方块，非常简单的设计

> *其实由于体积问题似乎用的并不是很多*

到这里，你只需要自己进行布线组装，就可以完成这一台无粉 4gt 白桦了。这里我们推荐蝎子的设计作为参考

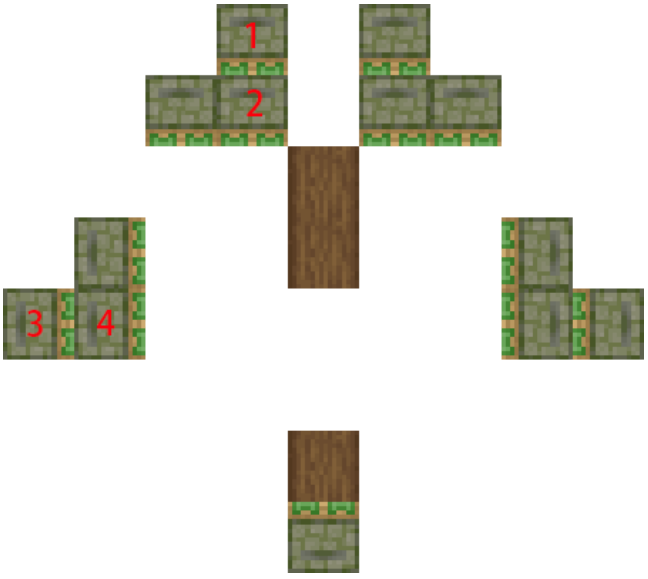
> *其实你仔细看蝎子这台的线会发现：我们都不一定非要用 0t，侦测器给出的 2gt 信号也是可用的。这也是 4gt 树场中一个相当重要的压缩卡顿的技巧。佛冷的 4gt 全树种因为全都用了 0t，即便是和用 kay 和 land 更卡的架构做出来的东西比卡顿依旧吃亏*

3 4GT 全树种

由于 4gt 全树种的内容实在是比较庞杂，我们一点点来

3.1 侧枝树 —— 金合欢/杜鹃/樱花的基础时序

4.3.2我们提到，只需要将 TT 设计的斜向收回主干处理在另一侧再做一遍，我们就可以轻松地得到一个 4gt 架构。在这里，由于全树种的生长检测要求，**我们无法使用白桦的架构**，这种由尸凶首倡的伪二次递归架构就成了我们唯一的选择



这里我们暂且以尸凶的原始架构为例，但暂时忽略用于处理树叶的一些额外部分

如果你还记得3.1.2的内容，你应该能看出来这个架构是可以处理侧枝树的，问题就在于时序设计

目前~~除了 plexi 的那个异形架构~~所有 4gt 全树种使用的侧枝树处理时序都来自于繁花千目，以下是他设计的时序：

1 号活塞 0t 推出 2 号活塞，2 号活塞 0t 收回主干侧面的原木；其后，3 号活塞 0t 推出，4 号活塞 0t 推出（如果在 3 推出前，4 面前有原木，则 4 不会推出）

这样就成功地**先将一部分侧枝转移出处理位置**，随后再进行归中和常规的处理。关于剩下一个处在下方的活塞，随便给他个 0t 就好了~~反正推不出去也不会坏~~

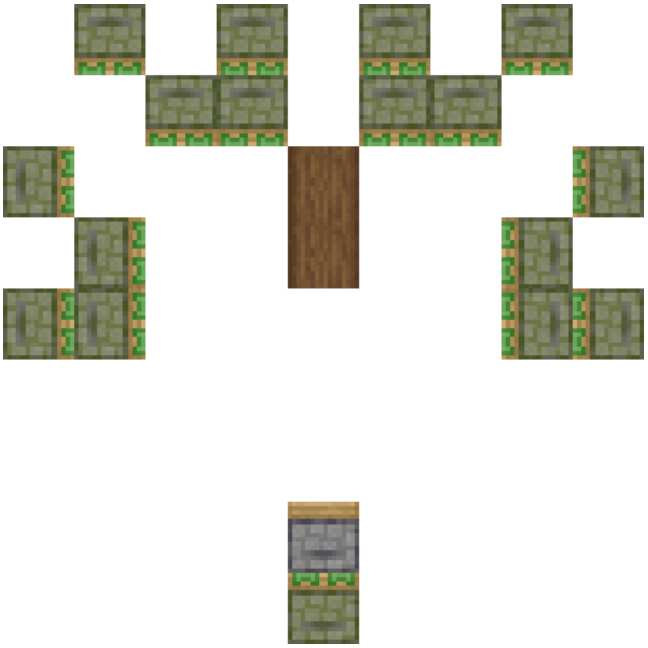
> *其实你还记得的话就会发现：这就是 TT 一开始为 6gt 丛林设计的时序，但是最早的 4gt 丛林树场并没有使用这个时序，原因我们不久之后就会提到*

你应该能想到这个时序甚至可以处理核心内沿 x 和 z 轴全部塞满原木的情况

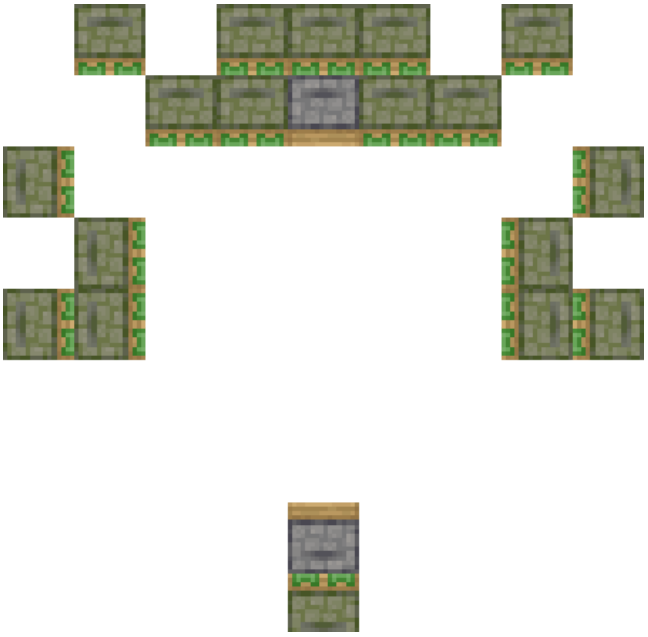
> *为了提升侧枝处理量，事实上我们可以底下那个可怜的活塞换成二次递推（其他架构设计可以自己想想，标准答案是 plexi 的设计）*

> *当然，我们还有更极端的处理：在用于激活侧面伪二次递归的那一整排原木中每隔一格塞入一个二次递推，并让整个侧面伪二次递归按 8gt 周期同步，只让直接抽走主干的伪二次递归交替运行（同样是 plexi 的设计）*

Layer1:



Layer2:



3.2 【PLEXI 架构的特例】杜鹃与樱花的特殊处理

其实如果你用的是尸凶原始架构，按道理你不需要担心这个事情。但如果是将下方的粘性活塞推动原木换成二次递推的架构，事情就会复杂许多

假设由于杜鹃和樱花的特殊生长机制，我们在上方吸墙二级激活用原木和下方二次递推之间获得了三个侧枝，此时，如果不给二次递推的上升沿 0t 加延迟，**就会出现主干还没吸走，二次递推推不动**的情况，3gt 后，二次递推被推出去，你就要掉效率了。对于杜鹃，这个事情还会更糟，**杜鹃的生长检测是 3x3**，因此，在二次递推的二级活塞被甩出去之后，**杜鹃不会停止长树，反而会在一级和二级之间生成原木**，然后把你的二级活塞送走

plexi 在吸墙激活原木中间插入二次递推之后，我们需要对那边也进行处理。但很显然，**那边没地方给你放上升沿控制**，怎么办呢?

> *事实上没有办法控制上升沿的最主要原因是，dpe 二级斜上方的原木在推出前 1gt 就被侦测器充能了。如果我们不希望它在这个时候被充能，我们就需要让它 4gt 才开始回推，就会和对面的二次递推打架，要么放弃这次激活，要么就会变成上述的送走活塞的情况。于是，如果我们控制一级推出的上升沿，我们就无法与两侧没有上升沿的吸墙使用相同的二级下降沿信号，很显然这里没地方放另一个不同的激活*

> *了。同时，如果我们控制一级的上升沿（事实上，只要提供的信号开始时间晚于侦测器，就会完蛋），树长出来之后会防止树叶，NC 更新二级活塞，让它提前伸出，你的一级就推不动了，无论如何都不是理想的情况*

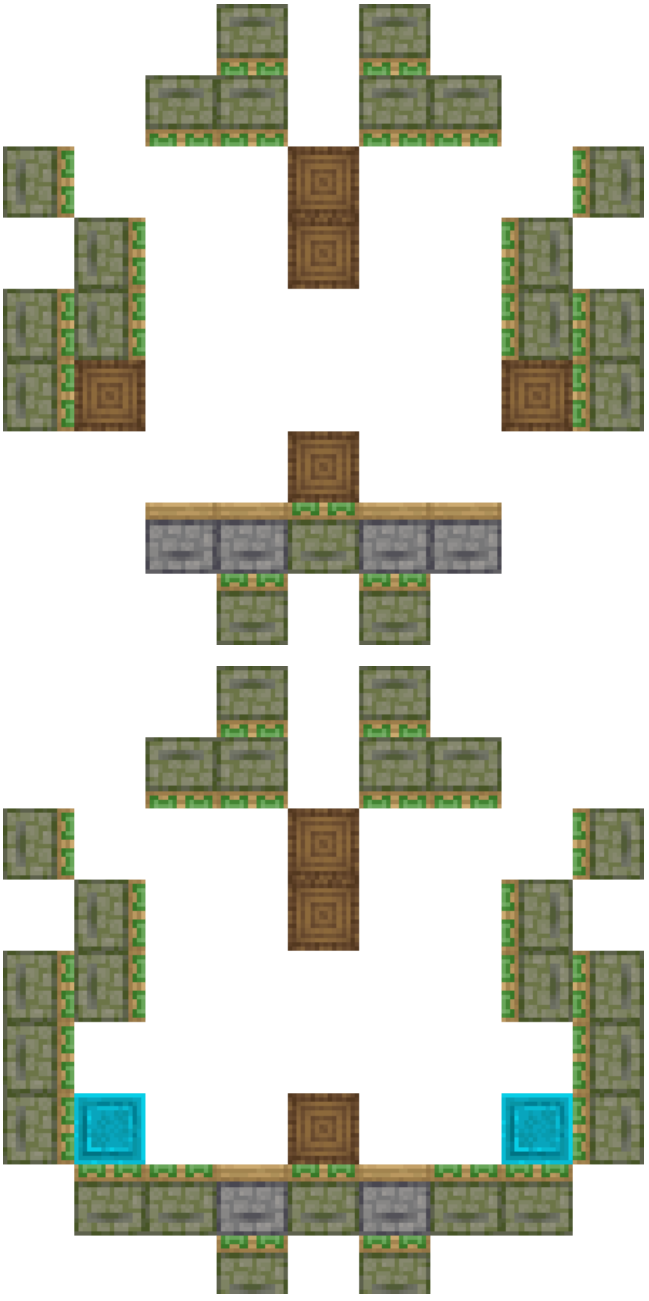
plexi 想到了一个很好的解决方案：**推不动，就把活塞直接拉走**，从而移除 3gt 的那次推动，这样就避免了把普通活塞扔出去送走的情况。理论上会有挺多实现方式的，你可以自己多尝试一下，~~虽然星河和 plexi,qontrol,lintex 论证了半天都只有 plexi 之前做过的布线架构能用~~

3.3 丛林木

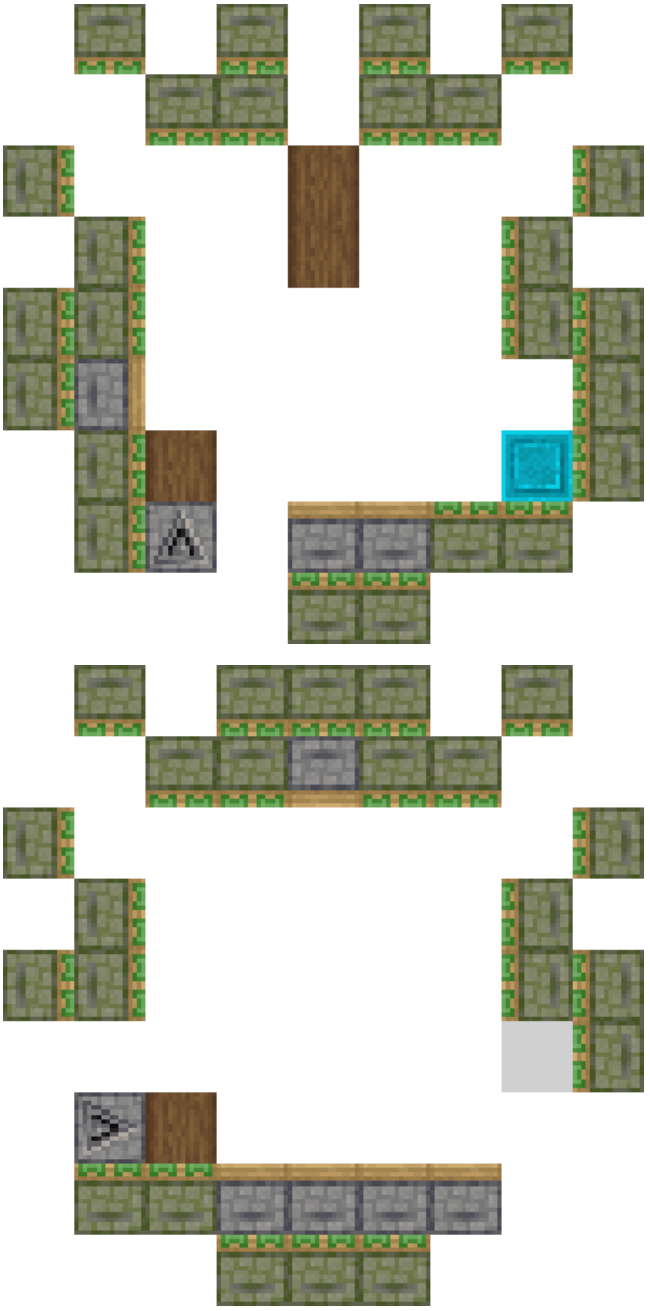
欢迎来到 4gt 树场中事实上最大 boss。由于丛林 1 / 40 的树苗掉率，4gt 树场事实上每个周期只有一半在跑，丛林树苗循环成为了 4gt 全树种最大的阻碍~~想必你还记得落息当年 100.5% 的极限回收率~~

上面我们提到，在 4gt 全树种中，我们对于丛林会弃 TT 的原始时序不用，而**将某一侧用于抽走主干的伪二次递归和对侧侧面的伪二次递归同步**，这样可以在同一个周期内处理更多的树叶，从而获取更多的树苗

我们能不能再处理更多一点树叶呢？我们先补全尸凶的原始架构



看上去 plexi 增加侧枝处理的同时，也处理了更多有更高概率长出树叶的位置，我们补全看看



很明显，plexi 的架构处理了多不少的树叶，这很大程度上缓解了树苗循环系统的压力 ~~但这并不意味着你可以随便找漏斗链子，还是得进行一些设计的~~

> *考虑到丛林树苗掉率较小，这会导致每一棵树掉落的树苗量有较大的方差，很不稳定。我们需要某个方式，缓存多余的树苗，并在树苗不足时重新投掷给玩家，被称为主动树苗循环。不过由于现代 4gt 树场的树叶处理量较大，我们已经不再需要它的辅助来得到稳定充足的丛林树苗补给了*

4.2.2中我们提到，高速树场中经常有大量的掉落物需要处理，显然，为了丛林增加的树叶处理量对其他树种是没有用处的；反而，为了减轻漏斗链的压力，**我们需要减少树叶处理量**。因此，一般而言我们会让其他所有树种都跑在侧枝树的时序下

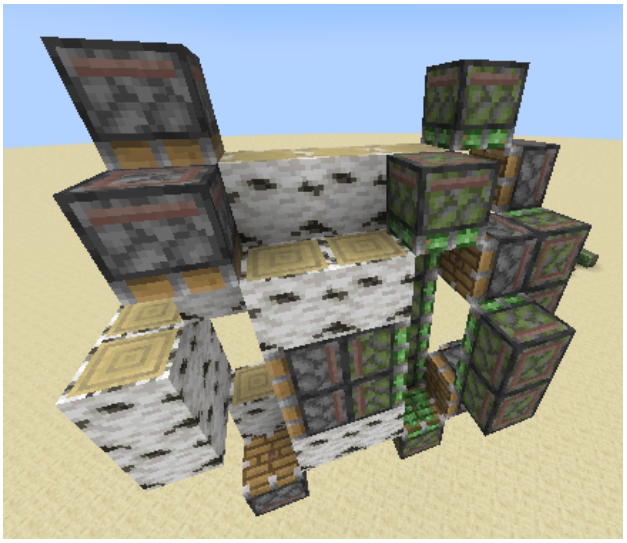
> *事实上你也可以模仿完美时序树场关停一部分结构，但由于现代 4gt 架构整体性比较强，~~关了也约等于没关~~，所以干脆就做的轻量化一点了*

3.4 原木输出 —— 吸转推

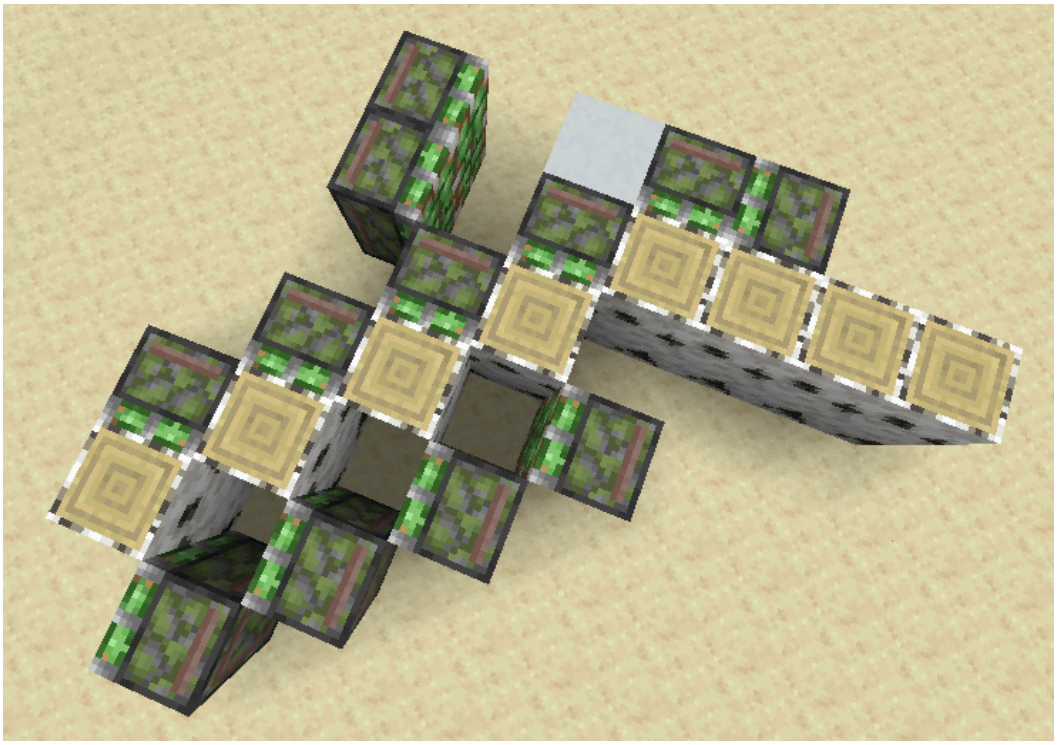
仔细看看现在的树场，你会发现一个很严重的问题：**方块流每走一格都是由一整排粘性活塞拉动的**。这样的方块流显然是无法处理的。所以这里我们要填一个坑：纯收回式伪二次递归架构的原木输出

> *关于为什么不在 #4 讲，是因为星河认为对于现代高速树场，除了 4gt 相关和 8gt 大云杉，这个部分都可以通过架构设计规避。如果你说对于某些架构这种输出方式能够增加侧枝处理量，那么你要知道，世界上还有一个叫 PUTF + 的东西，用 12gt 的时序，做出了超过 6gt 全树种的樱花效率。所以这种处理方式仍然只在 4gt 相关有必要性*

第一种方式，我们可以注意到原木流的上下方是空的。我们可以**从上方拉出原木，然后横向推出**，这样就完成了最上下方两个原木的吸转推，如此循环对原木流进行操作，我们能得到一个阶梯状的方块流。**这种方块流尤其适合制作凋零处理**，因为我们只需要把上下两组纵向推到一起然后再横向塞入凋零笼子就好了



第二种方式，我们可以在方块流的末端加入一个这样的二次递归模块，从而将方块流**一次性向后移动两格**，并顺路把方块流推出去



其实你仔细想想就会发现，需要这些本质上都是因为**我们没有地方放推出用的活塞**，所以我们找个地方把推出用的活塞放下来，然后想办法把原木挪过去就好了。一个常见的做法是使用蜜绿，**它们可以把原木粘到它们的侧面**，这样就有了额外的空间放置一排类似侧吸的模块，从而完成吸转推

显然蜜绿流能够减少 0t 而降低卡顿，同时缩短方块流的长度，是我们现在最优先考虑的方案

3.5 布线 —— 整体化/独立化/模块化?

虽然之前我们说，4gt 是由时钟驱动的，所以只要保证跑起来的时序对了就好，但是**我们仍然要考虑设计的稳定性和性能开销**

出于稳定性考虑，我们使用同一个时钟，在时钟之后接入随树场运行的时序线肯定是更稳妥的。比如上述的蜜绿方块流，**如果蜜绿墙和其他方块流活塞的时序不对，那些活塞就可能被蜜绿墙粘走而破坏树场结构**，因此例如 plexi 的设计，一部分方块流活塞的时序就是从激活蜜绿墙的时钟引出去的。这就是**整体化**

然而如果强行将所有的部分连接起来，这很显然会造成很大的卡顿，因此**我们也需要将一定范围外的部分接入另一个时钟**，从而减少信号传递导致的性能开销。这就是**独立化**

> *事实上我们拥有非常非常简洁的 4gt 时钟设计，这导致甩掉纵向的信号传递直接增加时钟在有些时候反而可以减少卡顿。你可以在 plexi 的蜜绿方块流输出部分见到这种设计*

一片区域内的活塞接入同一个时钟，这就会导致我们之前提到过的**模块化**的倾向。我们可以分别设计各个部分的布线，最后再像搭积木一样将它们拼装起来。以上面展示的两个 4gt 全树种架构图为例，模块化在上方的伪二次递归墙尤为明显，**因为它不需要与剩下的部分连接，而且只要布线都在其背后，就不会造成任何额外的空间占用而影响组装**

到这里，你只需要简单设计一下时序布线，把一切连接起来，就大功告成了。显然，你第一次只是读教材，并不能完全理解 4gt 全树种的时序和布线方式。我再次推荐：**去把别人的布线抄一遍**。这里尤其推荐 plexi 的 4gt 全树种，只要你稍微认真点拆线，在我们之前的知识基础上，你绝对能完全明白它的运作方式。唯一美中不足的是发布页在Tree Hungers Discord，你得挂梯子，而且在加入这个 server 之前你根本就没办法打开发布页的链接，所以只能你们自己翻了（

最后，我们只剩下一个有些偏门的东西没有讨论了：检测式 4gt 树场

4 检测式 4GT 树场

其实说起来，检测式 4gt 树场最大的问题在于：**我们唯一可以使用的无粉的，可以提供无延迟上升沿 0t 的方式**，是**红石粉重定向**。如果你有尝试扒几台 4gt 全树种的线或者自己搓一台，你就会发现：**真 tmd 占地方**

所以现在最大的问题就是，怎么把这一大堆红石粉重定向全部塞进去。不过关于这点我只能说：**架构比什么都重要，架构不足够优秀的话你是没有塞进去的可能的**（当然你也可以把一部分东西挂在时钟上~~不过我是认为这样的“检测式树场”没有什么实际意义~~）

> *如果你不怕死的话，当然可以尝试一下基于红石粉的检测式 4gt 全树种。~~星河的电脑反正是跑不起来的~~*

另一个问题在于，我们需要通过某种方式，让我们的检测单元**能够跑 4gt 周期**。最简单粗暴的方案当然是**让输出信号的结构 4gt 之内复位**，另一个不那么优雅的方式式**另外做一套，每一套都做成 8gt 之内复位**

对于这种时序极端紧凑的检测式无粉树场，可以看 Qontrol 的无粉检测式 4gt 白桦和无粉检测式 6gt 全树种 PUTF#（没办法了现在还没有已经完成的无粉检测式 4gt 全树种，只能拿 6gt 凑数了）

> *不过如果你还记得的话，我们之前说过 4gt 树场也可以使用下吸。显然，由于下吸会移动泥土，如果树苗没有长成，就会被破坏，所以我们不能让下吸接入时钟。这个时候，你还是需要用到一些无粉检测式 4gt 树场的思路。不过另外一个问题是，你真的需要给 4gt 树场上下吸吗？~~其实还是要的回头我们会在双核的部分再次提到这个问题~~*

---

slug: basics-and-structure
title: 前置知识与树场的基本结构
title-en: null
author: jslskd
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-02-02T23:57:52+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

1 前置知识

> *这里是开始树场研究的一些基本内容的简述。出于简化考虑，这部分会包含一些 “错误” 的内容，关于更详细的说明，请参考 GTMC 的**更新理论**与**时序理论**部分。*

1.1 更新

mc 中更新分为**NC 更新**和**PP 更新**

NC 更新在树场中的主要来源是**TT（Tile Tick）原件**（一般而言：红石中继器、比较器、侦测器），**红石粉**与**动力或激活铁轨** *以下部分将用 “铁轨” 指代它们，因为树场中几乎不涉及另外两种铁轨 *、**活塞**与其推拉的方块、**音符盒**

一般可以认为以上原件的某些状态（如是否充能、激活，方块的状态）发生变化时，向：

- 红石粉（能量等级变化），二阶毗邻
- 中继器、比较器、侦测器，先它们输出端指向的方块，再输出端指向的方块的毗邻，除去自身
- 平放的铁轨，先自身的毗邻，再下方方块的毗邻；斜放的铁轨，先自身的毗邻，再下方方块的毗邻，最后上方方块的毗邻。
- 活塞，本身与其受推拉方块原位置的毗邻
- 音符盒，毗邻

发出**NC 更新**，并可由**BUD**响应

由于树场的主体结构一般由活塞构成，一般可以认为树场中所有**NC 更新**都同时带有**PP 更新**。侦测器检测的是**PP 更新**

在树场中，纯 PP 更新（不带有 NC 更新）一般由：

- 活版门 / 栅栏门开关
- 发射器/投掷器/漏斗激活状态改变
- 玻璃板/栅栏/铁栅栏/石墙的连接状态改变

提供

> *在无粉布线阶段，我们会涉及三种很重要的信号传递方式：树电（通过树叶，传递 NC 和 PP 更新）、脚电（通过脚手架，传递 NC 和 PP 更新）、墙电（通过墙，纵向传递 PP 更新）。利用好它们的特性，我们能极大地简化树场中的布线。更细节的内容，我们会在无粉布线专题讲述。*

1.2 时序

mc 中时序分为**刻间时序**与**刻内时序**

mc 中，每秒会被分为 20gt（*这里不阐述其概念*），部分原件会带有固定的**宏观的**延迟，如 TT 原件：中继器（2-8gt）、比较器和侦测器（2gt）
可以认为除了 TT 原件以外，其他树场涉及的原件都**不带有**宏观延迟

每 gt 会被分成多个阶段，分别执行不同的运算。与树场相关性较强的阶段（按运算顺序）有：

- **Tile Tick**（TT），TT 原件的运算在这里执行
- **Block Event**（BE），活塞与音符盒的运算在这里执行
- **Entity Update**（EU），实体（如载具或者生物）的相关计算在这里执行
- **Tile Entity**（TE），漏斗与被活塞推拉的方块相关的运算在这里执行
- **Async Task**（AT / NU），玩家的相关计算在这里执行

除了以上提到的原件以外，**瞬时原件**相关运算会在接受更新后立刻执行（无论更新处在以上哪个阶段）。常见的瞬时原件有**红石粉和铁轨、栅栏门或活版门、音符盒、投掷器和发射器**

TT 原件的执行顺序由**宏观时序、TT 优先级、子顺序**决定，可以认为，TT 原件的执行顺序为：
任意中继器 >= 指向比较器的比较器 > 一般比较器或侦测器

BE 原件（活塞与音符盒）在 BE 中会按**受到更新，确认需要变更状态**的顺序执行运算。这个顺序有时被称为**深度**。音符盒不会增加深度（即延后其更新的对象在 BE 内的运算顺序）

分析时序时，优先分析**宏观时序**，其次分析**运算所处的阶段**，最后分析**运算在其阶段内执行的顺序**

树场中，可以认为每一个活塞动作（推/拉/0t）的时间都是**3gt，1gt 和 2gt**的充能导致的活塞动作分别为**4gt 和 5gt**

2 树场的基本结构

在 mc1.15 及以上，树场的基本结构有：

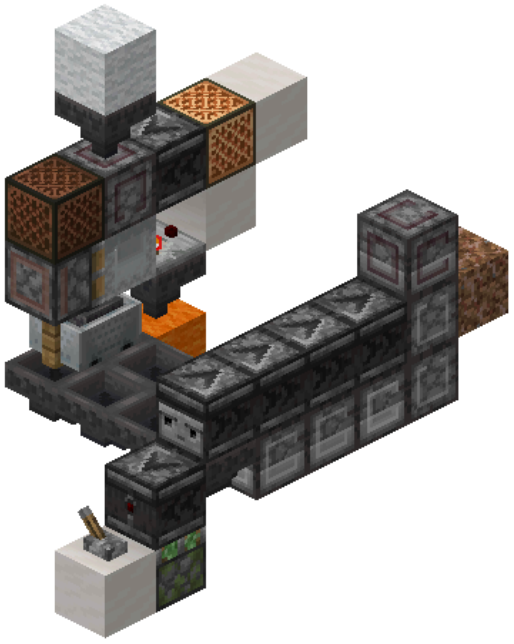
- 催熟
- 树干处理（有时分为**主干处理**和**根部处理**）
- 树叶处理
- 树苗回收
- 方块流转掉落物

> *大部分树场还有**检测**模块，用于检测树苗生长并触发树场，但这不是树场必须的，我们会在下一章接触它*
>
> *在 mc1.15 以下，由于从林木和金合欢特殊的生长检测方式，有时候会为树场增加**增高**模块；由于云杉生长检测要求 5x5 内不能有原木，需要增加**收回墙**。本文主要讨论 1.15 及以上的树场，我们会暂时忽略这些结构*

2.1 催熟

一般而言为**发射器**对树苗喷骨粉

由于发射器能储存的骨粉很少，一般而言会通过**漏斗或者投掷器**向发射器输送骨粉，并用**卸货器**将骨粉由潜影盒转移到漏斗或投掷器链中。

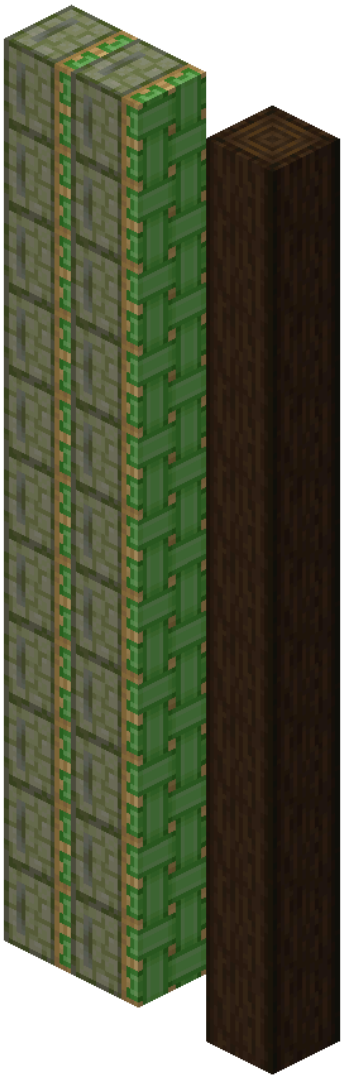


2.2 树干处理

主干处理的思路非常简单：将主干从原本的位置移走。基本的实现方式有**三次递推**和**伪二次递归**；对于白桦和橡树，我们可以采用直接推拉的形式

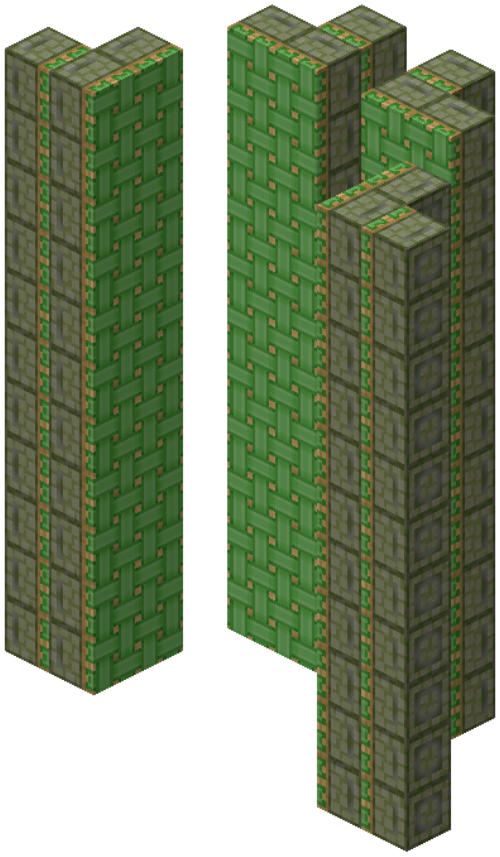
三次递归：

伪二次递归：



蜜绿墙二次递归:

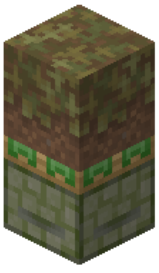
收回式伪二次递归（通过让二级活塞只执行收回动作（0t）压缩一个活塞动作时序的伪二次递归架构）：



> *现在使用最多的形式是蜜绿墙二次递归*

树根处理一般而言，有与主干一同处理、**上推**、**下吸**和**侧吸**四种方式

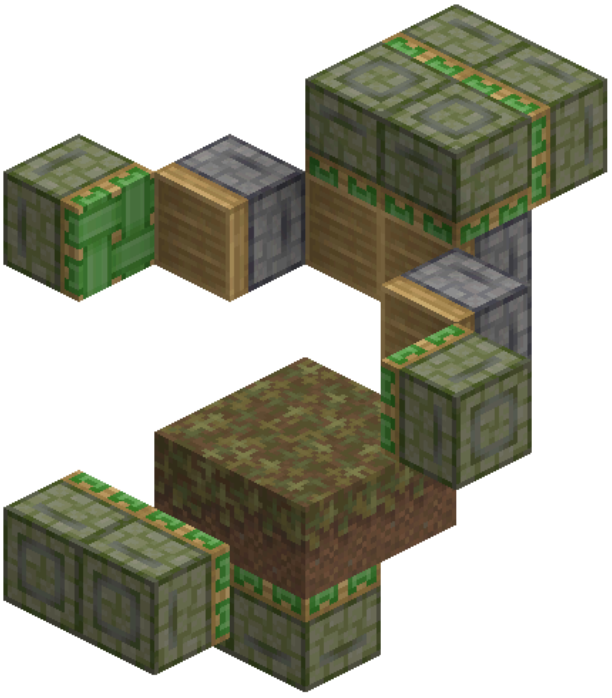
上推：



下吸：

侧吸：

对于深色橡木，有时会采用**上吸**的处理方式

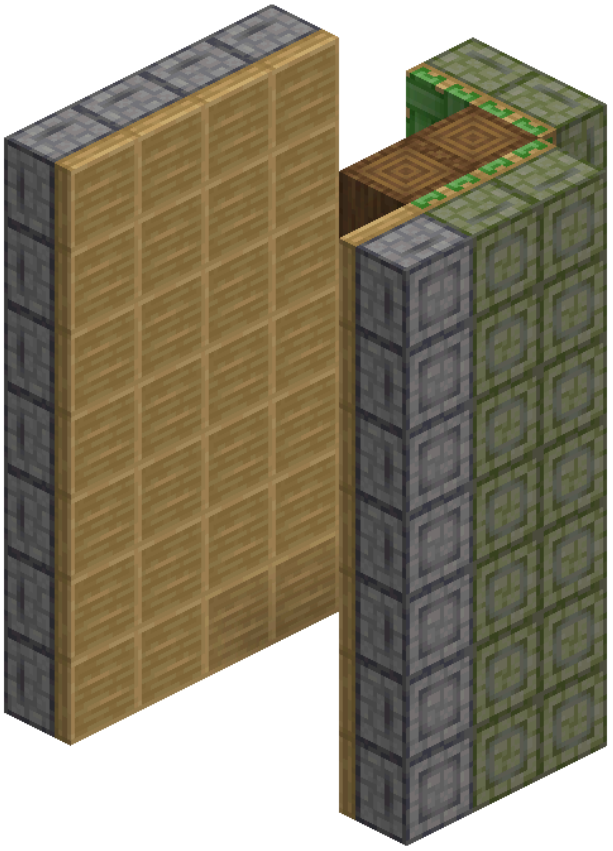


2.3 树叶处理

同样非常简单：用活塞 / 蜜绿墙将**足够数量的树叶**推掉，从而获得足够的树苗

蜜绿墙：

活塞:

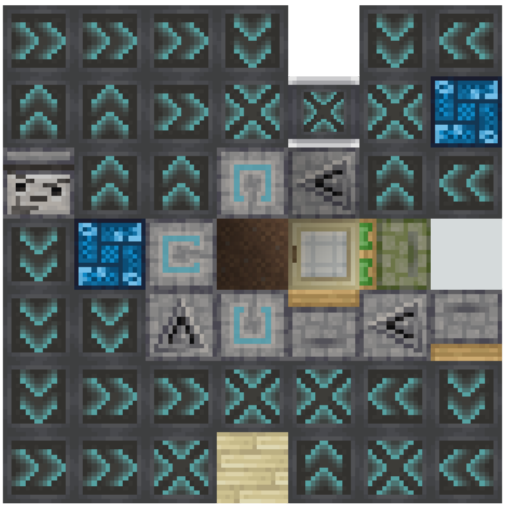


每个树叶会以固定概率掉落树苗：丛林 1/40，其他树种 1/20

2.4 树苗回收

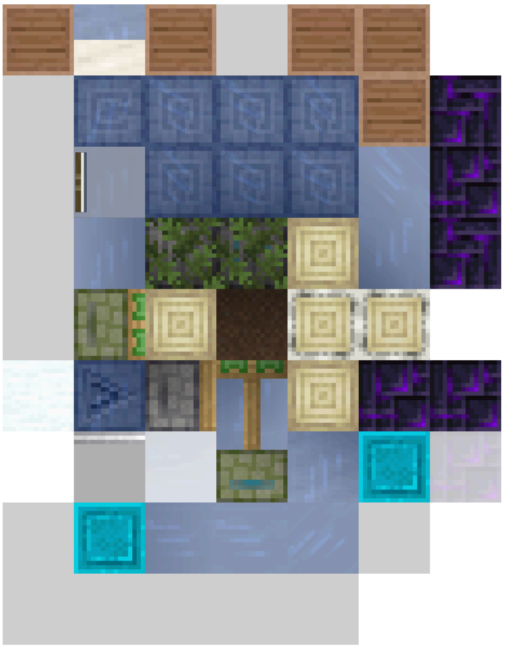
通过**漏斗**、**水流**等方式将树叶处理模块中掉落的树苗集中到玩家附近

漏斗：



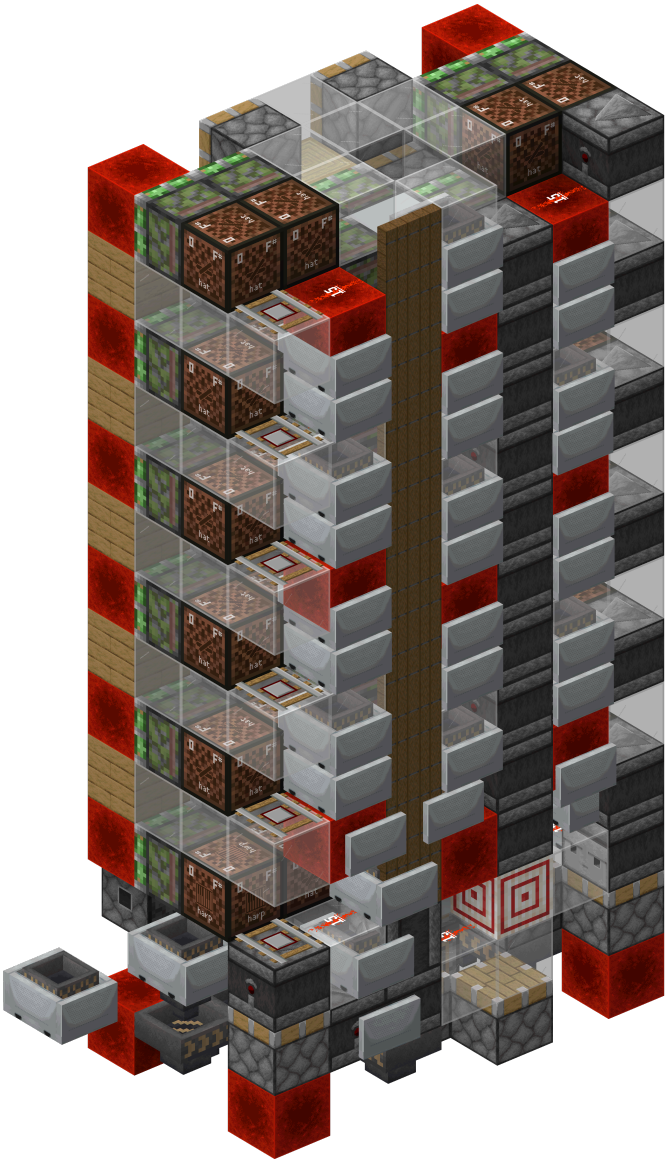
@Scorpio 天蝎君 4gt 白桦

水流：



@Feng_Bl, Sunflower_Lin 6gt 白桦

在处理周期较短，且内部空间较狭窄的树场内，有时会使用**漏斗矿车**回收树苗

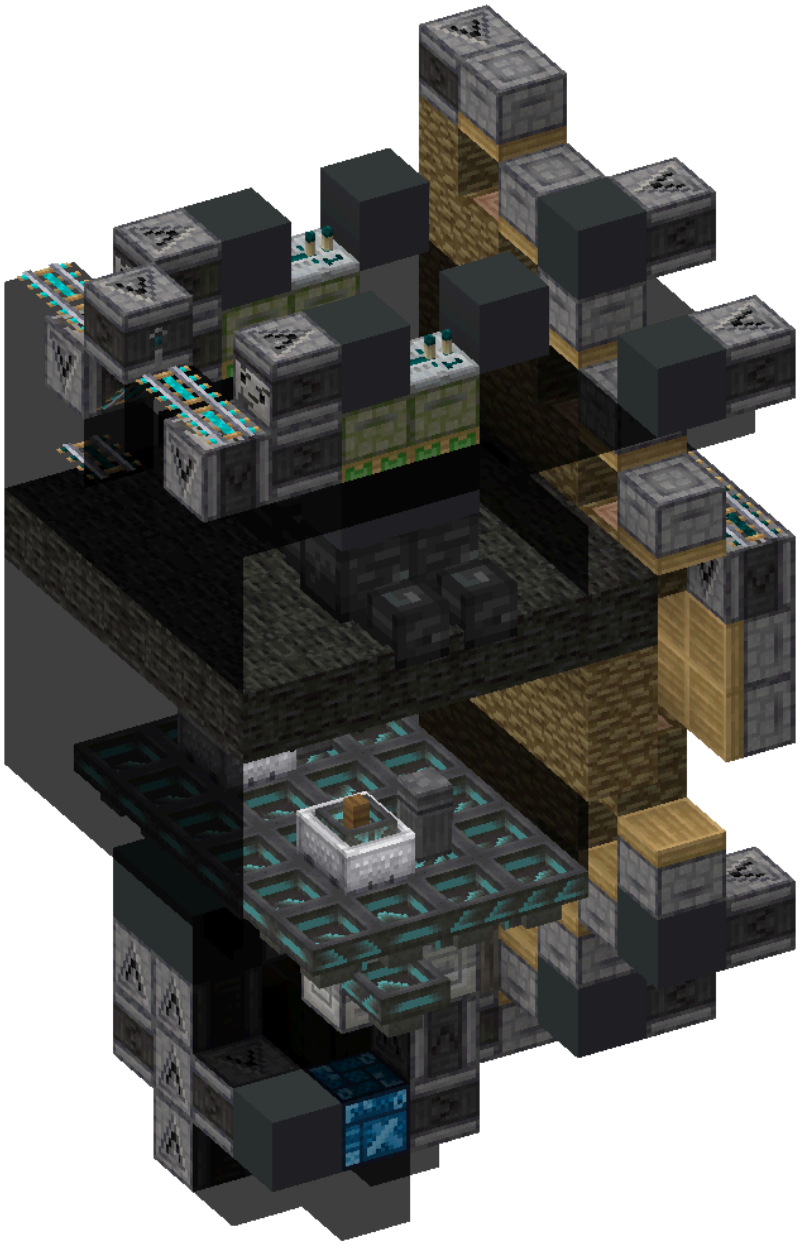


@Qonctrol，PUTF sharp

2.5 方块流转掉落物

一般而言有**凋零**和**TNT**两种方式

凋零：



@Scorpio 天蝎君，PUTF flat wither

TNT：

@Feng_Bl

---

slug: multi-core-farms
title: 双核、三核、四核树场
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'

lastmod: '2026-04-05T22:16:03+08:00'
index: 13
is-advanced: false
banner:
src: img/banner.png
alt: Graduate Text in Minecraft

---

slug: nether-fungus-farm
title: 地狱菌场
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 11
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

slug: large-spruce-farm
title: 大云杉树场
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 8
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

> *欢迎来到中后期工程用木材的最主要来源，这里我们将逐渐带你设计一台 mc 中原木效率最高的树场*

1 大云杉树场入门

1.1 大云杉生长机制与架构设计

大云杉的生长检测范围为：**以西北角树苗为中心，树苗层 3x3，以上 5x5**

大云杉最大树高为 28 格（西北角会多一个，所以是 29）~~一般而言不会有人做了大云杉树场还不处理全高~~

简单而言，我们需要一个二次递归来处理主干原木，然后简单地用蜜绿墙推掉树叶就好了

1.2 纵向信号传递

用棒子 bud 拉上去就好了，对于主干和侧墙激活，可以考虑转向粉或者墙电

1.3 处理

从核心推出来之后左右分流，然后正常重组即可

到这里，你已经可以独立设计一台能用的大云杉树场了。但是不出意外的话，效率就只有 10w 左右，不尽人意。接下来我们就要来讲，怎么做一台更快的大云杉树场。对于处理时序，相信你们都在之前的学习中有了足够的理解，这里我们只讲如何尽可能凹预催熟

> *当然，这里我也希望你们能够体验一下过去的探索历程，所以我会加入很多有关大云杉树场的历史，当然也有一些旧版本内容，hope u enjoy it*

2 高速大云杉 PART1：角落下吸

2.1 角落下吸的源头

2016 年，老仙发布了他的 12gt 大云杉树场，采用上推架构，首次突破了 30w 大关。然而随着时间推移，老仙的时序在 1.12 不再可用，同时玩家逐渐发现了更优的架构和时序设计。

在 2020 年 2 月 12 日，gpw 发布了 sprucemacy v1，用侦测器充能树根检测和角落下吸，正式开启了现代高速大云杉树场的时代，成为了之后相当一段时间内大云杉的设计标准，直到 2021 年 4 月 15 日 gpw 发布 sprucemacy v2，带来了角落侧吸，我们稍微晚些讲解。现在，让我们回到 2020 年

2.2 角落下吸的思想

如果你在 8.1 尝试做过一台大云杉树场，想必你可以注意到这东西想要加发射器比较困难。如果不想办法修改底座架构，即便使用推动上限检测，我们也只能拥有两个发射器

* 因为我们还在 2020 年，我们暂时还没有推动上限检测 / 树电检测，所以暂时忽略它们。* 我们需要在可以放置发射器的四个位置中选择一个，放置充能树根检测。显然，如果想增加到三个发射器，我们就必然面临一个问题：无论如何都无法通过侧吸处理角落原木

解决方案是显然的，下吸它就好了。但是由于下吸的泥土归位速度较慢，我们需要另外设计一个种植时序

gpw 的方案是，让玩家在 8 / 10gt（1.12 中玩家的右键放置是跑在简单的 4gt interval 下的）种植角落树苗，剩下的正常种植就好了

对于玩家移动，gpw 采用了矿车

2.3 角落下吸的局限

在 1.14 中，mojang 修改了玩家右键放置的行为。如果因为一些问题，玩家先放置了靠近玩家这一侧的树苗，那么远离玩家的树苗就无法放置了。目前在不修改种植时序的情况下，唯一的解决方案是羊驼船~~（但是在 1.21 由于 mojang 又改了生物在船上的行为，羊驼船也坏了）~~

角落下吸在 1.12 即便是 3gt 泥土归位也无法保证 4gt 前种植角落树苗（原因是上述的右键放置逻辑），因此无法达到极限的效率

> *不过可以提前剧透的是，在使用连点器的大云杉中，角落下吸因为更简单的时序和布线设计，焕发了第二春（笑*

3 高速大云杉 PART2：角落侧吸（CORNER RETRACTION）

3.1 角落侧吸的源头

2021 年 4 月 15 日，gpw 发布了 sprucemacy v2，第一次为我们带来了角落侧吸。2022 年 2 月 8 日，floppy 发布的 12gt Spruce v2 带来了 3gt 完成受催熟树苗种植的角落侧吸时序。2024 年 8 月，Qontrol 开发了有实用意义的 6gt 复位的角落侧吸底座。

> *其实在 12gt 角落侧吸刚出来没多久星河就随便做了一个全粉 6gt，~~但是完全不像能用的样子~~*

~~由于 mojang 在 1.21 改了羊驼船，角落侧吸的意义也被摧毁了~~

3.2 角落侧吸的思想

如果你想要先种植角落树苗，你就必须侧吸它。角落树干的侧面有两个发射器和两个原木，如果我们能以某种方式移除侧面的一个原木，我们就可以塞入一个侧吸活塞，这就是角落侧吸

现在的实现方式是下吸其中一个原木，另一侧用常规的并排侧吸处理就好

对于角落侧吸，很显然我们要用二次递归。对于 16gt 非连点大云杉，只需要用正常的二次递归就好了；对于 12gt，我们采用二级活塞从侧面推入的二次递归；对于 6gt，我们采用类似吸转推 “标准流” 的设计

一般而言，玩家移动采用羊驼船，种植时序角落优先；或者猪船，先种并排侧吸侧的一个，再种角落，再种并排侧吸侧的第二个，最后种植下吸位

> *猪船的具体来源我们会在下一个部分叙述*

3.3 太难做了

如果你还记得 8gt 大云杉的话，你就会知道，角落侧吸并没有出现在那台树场里。其实原因很简单：我和 lintex 都没有想到足够好的布线方式，来处理 8gt 角落侧吸。于是我们转向了角落下吸并更改种植时序（反正没人会给 8gt 大云杉用真人不带连点挂的

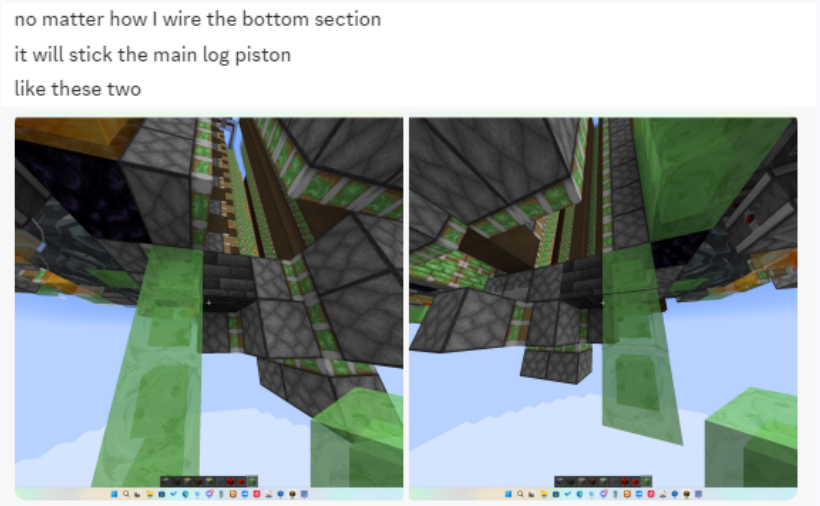
> *本来我们立项的时候 hampter 说要做一个角落侧吸给 8gt 大云杉用的，结果中途他电脑坏了，~~然后现在他的单元还没发出来~~*

这就是角落侧吸最大的缺点：复杂

~~当然还有更大的缺点：羊驼船坏了所以你只能用猪船和猪船种植时序，那就完全打不过角落下吸了~~

4 高速大云杉 PART3：角落下吸的回归 / 猪船种植时序

4.1 《无论怎么做都会粘到主吸活塞》



~~其实事后看不会粘到的但整到子更简单的底座还是不错的~~

2022 年底，lintex 开了 8gt 大云杉，很快就把主干弄了个七七八八，~~然后我们就卡死在底座上了~~

以当时的视角看，我们无法做出一个足够好使的角落侧吸底座，于是我提出角落用下吸，然后将种植时序倒转过来，这样就可以直接扔掉角落侧吸和对应的种植时序了

Dreaming Galaxy 2022-12-27 16:28

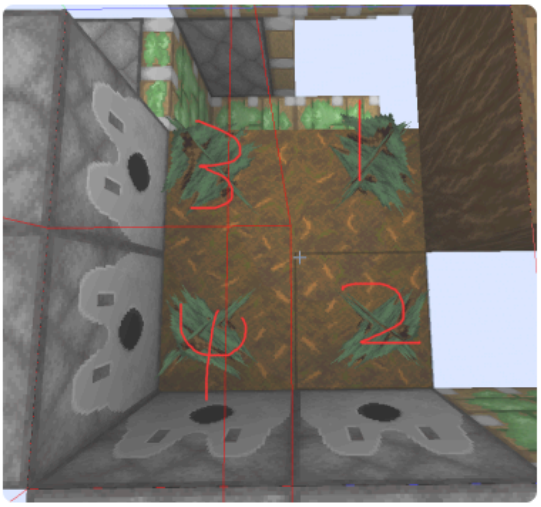
I have another layout

but can't plant the corner sapling first

do a downward retraction for the corner

16:29 and turn around the player moving thing

then plant in this sequence



should be much much much much much easier to wire

should be alright for a clicker farm

这就是角落下吸的连点种植时序

由此，你应该可以独立设计一个角落下吸的连点底座了。还是不会的话可以参考 8gt 大云杉或者 gushen 的大云杉

4.2 为什么是猪船?

其实我不是很喜欢矿车，原因很简单：随便推一下就跑了，然后就要重新放置

羊驼船坏了（躺

对于单一的船，这玩意是在客户端和服务端分别运算的，容易造成客户端和服务端不同步，然后就毁了

至此，你应该可以独立设计一台足够快的大云杉树场了。不过，在这里我们还想提一下 1.12 的一个天顶星科技：ITT / IF 6gt 大云杉

5 ITT / IF 6GT 大云杉

---

slug: perfect-timing-cherry
title: 完美时序，与樱花全树种
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 6
  is-advanced: true
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

slug: multi-species-design
title: 尝试设计一台全树种树场
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 3
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

在上一节中，我们已经完成了一台最简单的树场。但这台树场有个很严重的问题：只能处理白桦树，而对于其他树种无能为力。为了解决这个问题，这一节我们尝试设计一台全树种树场。

1 核心架构设计

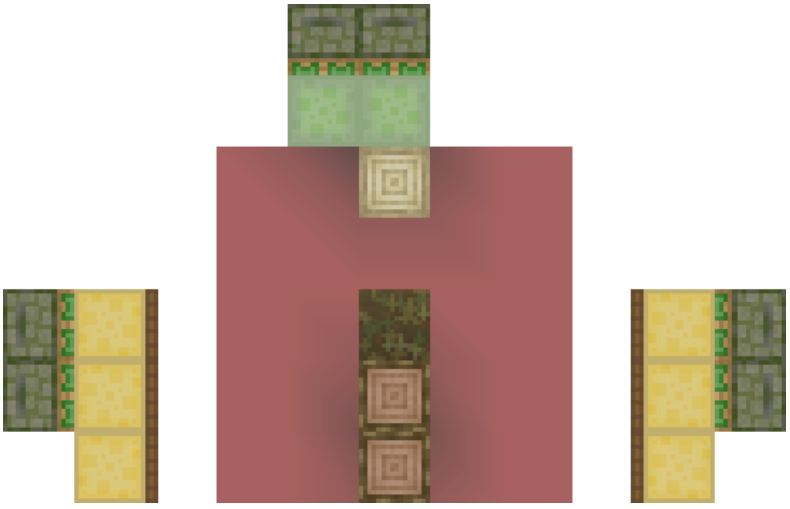
对于全树种树场，我们需要架构能够满足**所有需要处理的树种**的生长需求。这意味着**取并集**，我们需要在架构设计的某个方面上满足**要求最高的树种**的要求。这里我们暂时只涉及最传统的五个树种：**白桦 橡树 丛林 云杉 金合欢**。我们按照设计的一般顺序，逐一讲解。

其中白桦是生长限制最小的树种，我们从云杉开始。

1.1 云杉架构要求

云杉要求以树苗为中心，5x5 的范围内没有阻挡生长的方块。对此，我们可以在主干处理上选择**三次递推**，**伪二次递归**，和**蜜绿二次递归**等方式

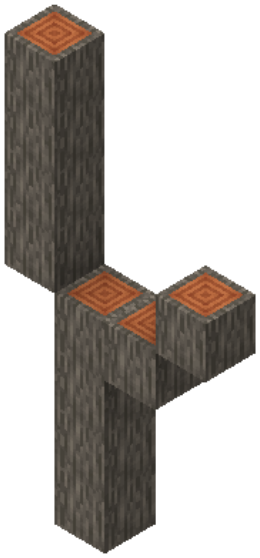
示例架构：

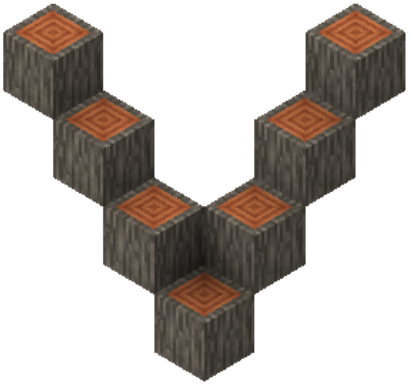


> *主推设计会极大影响橡树限高的方案，请谨慎选择*

1.2 金合欢架构要求

金合欢会在同一 y 值上，沿**x 或 z 轴**，距离树苗**至多 4 格**，长出至多**2 个**原木，其中包含直线、拐角等各种情况，如图。

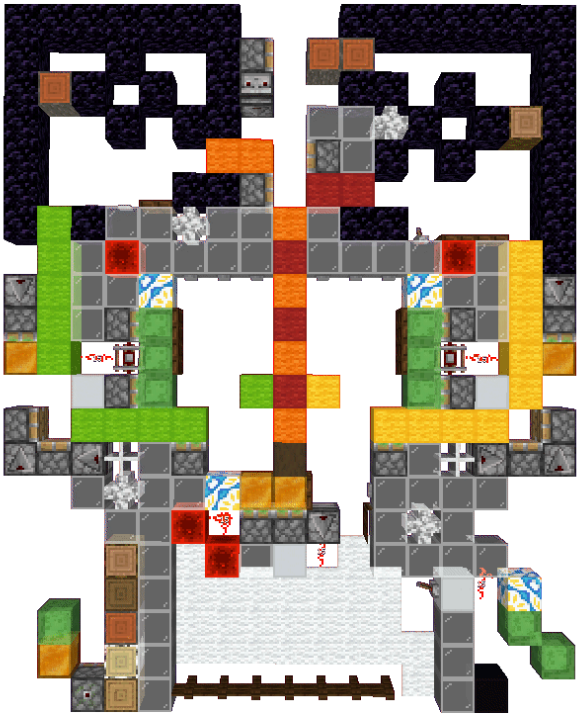




这意味着我们无法通过主推移走所有的原木，对此我们需要增加**侧枝处理**。其有两种设计方案：**归中处理**、**增加原木输出**

归中处理而言，我们需要先将主推侧的原木推到中间，再将左侧的原木推到中间，由主推移出中心，最后将右侧原木推到中间，由主推移出中心

增加原木输出的典型案例是 nutstory 的架构，相信你们能通过动图理解它的工作方式。



@Dreaming_Galaxy, XJH_Jorhai, Feng_Bl, Bright_Observer PUTF standard

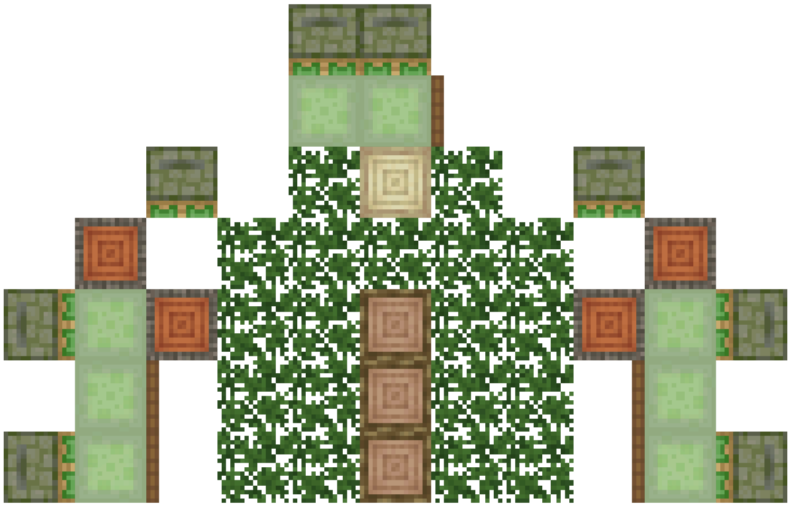
> *在一些架构中，我们会同时见到这两种处理方式，如 PUTF -2。另外 PUTF//给出了另一种处理思路：增加一格主推，将侧枝归中到中间 2 格后直接推出。但这种方式不能用于樱花树，在 GTMC 所处的 1.20.1 并没有显著的价值*

1.3 丛林架构要求

丛林的树干高度最高为**12 格**，我们需要有足够高度的主推，以尽可能处理所有的原木

> *事实上存在不处理全高的全树种树场，一般有 8 高（方便橡树限高）和 10 高（方便布线），但我们不提倡这种行为*

丛林的树苗掉落概率为**1/40**，因此我们需要为丛林木设计额外的树叶处理。最简单的方式是使用**二次递推**，或者**蜜绿墙**尽可能处理所有可能长出的位置。一般而言，俯视情况下，5x5 范围内**不超过 5 列**树叶无法处理，则这台树场可以获得足够的丛林树苗。



@nutstory 5 列无法处理

> *其实这种估算方式并不严谨，丛林树叶在靠近主干的位置分布的更密集。严格意义上讲需要计算每种树叶生长情况下处理量的平均值，一般而言大于 42 即可获得足够的丛林树苗*

1.4 橡树架构要求

由于橡树会随机地长出**大橡树 *俗称鸡腿树 *，我们需要对橡树进行限高，以避免这种情况。限高方块位于**泥土上第 9 格**。

> *事实上能直接处理鸡腿树的架构和树场是存在的，但是目前为止我们无法使其获取足够的丛林树苗，因而不在全树种树场的讨论范围内*

这里我们举例采用主推**蜜绿三次递推**、侧枝**归中处理**、树叶处理**蜜绿墙**、限高手放黑曜石

> *熟悉老树场的同学应该会发现，这几乎就是 PTHSUTF 的架构。事实上，由于械电设计方式繁杂，最好的学习某种设计的方式就是 “模仿”。某种意义上，可以说在械电的学习阶段，不要忌讳 “抄”，能够抄一遍优秀作品的布线，你都可以学到很多*

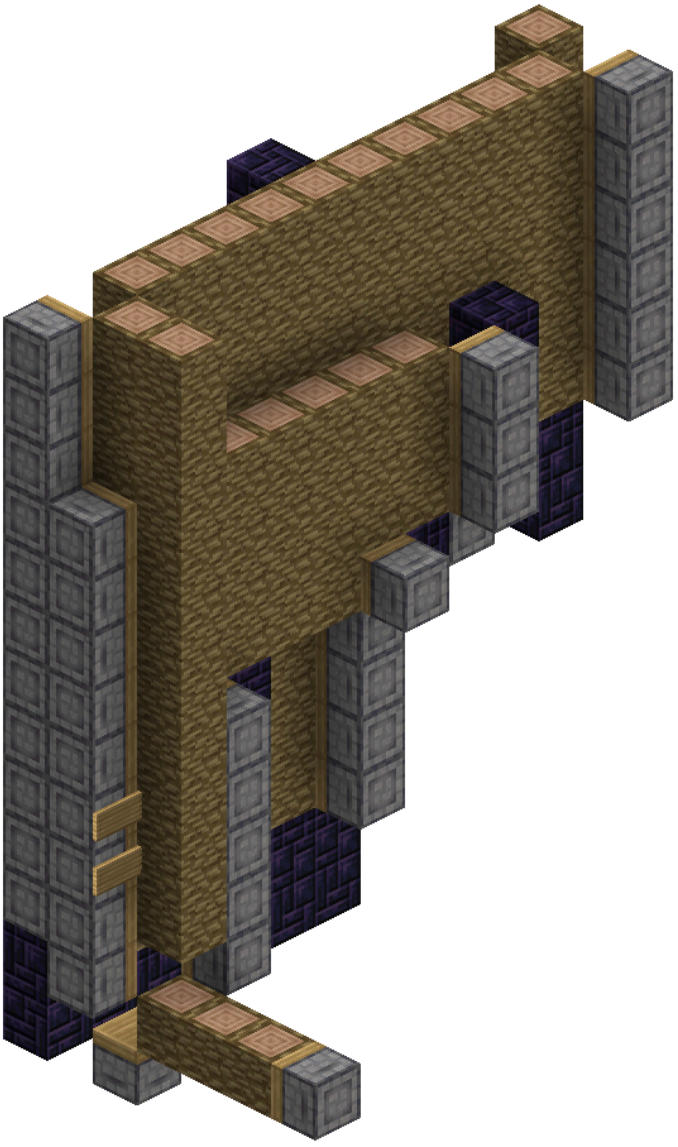
2 方块流转掉落物架构设计

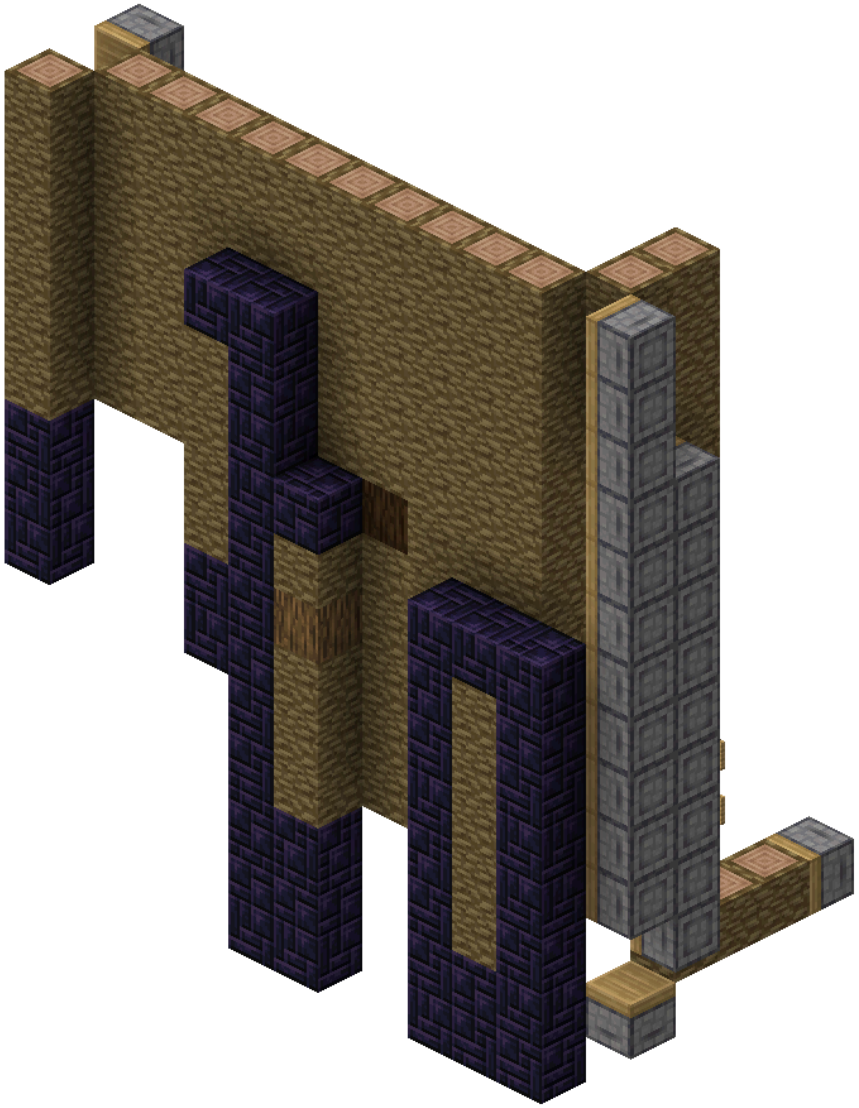
由于全树种树场较为特殊的方块流输出，我们需要对方块流进行**重组**，再送入特殊设计的爆炸室

2.1 重组

我们举例的设计而言，所有的原木都会被输出到主干正面，每一个运行周期至多输出两格。

熟悉 DLHSUTF/PTHSUTF/DLPTUTF 的同学应该能立刻想到直接使用**两排并排的活塞**，将目前还不知道间隔的两次原木输出转化为一个 2 宽的原木输出。另外我们可以用一个类似**侧吸底座**的结构，将不确定间隔的两次原木输出转化为确定间隔的两次原木输出（*一般为 6gt*）





@Dawnemo DLPTUTF

> *由于金合欢同 y 的两个原木可以分布在中心与其周围 1 格（如下图），即便增加原木输出，主干也需要使用这类重组。对于额外的原木输出，我们可以简单的添加爆炸室解决它们的方块流转掉落物问题*

2.2 爆炸室

这里我们先随便挂一些爆炸室处理方块流，不过我们尽量不同时推出两格厚的方块流，因为**这样会极大地影响爆炸室的处理效率和最后的掉落物回收率**

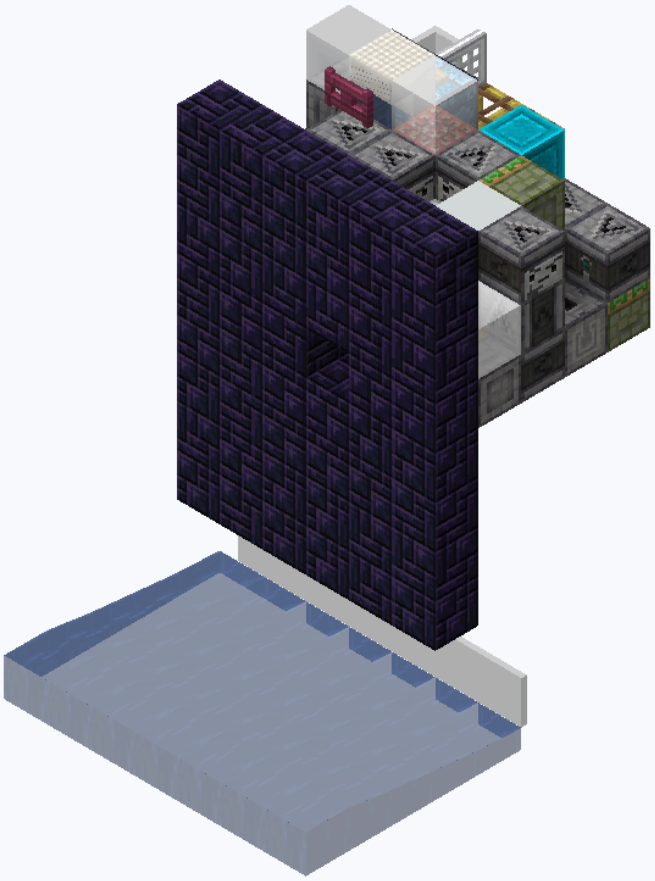
> *事实上对于全树种，我们可以引入**纯牛奶爆炸室**（俗称 “奶爆”）和**b36 爆炸室**（俗称 “推爆”）两种较为特殊的爆炸结构，以应付大量不甚规则的方块流。这种方式已经在现代高速全树种得到了广泛应用，但是由于没人给我们的例子改爆炸室，我们在提速章节统一讲解（*

> *奶爆：*

@从来只喝纯牛奶

@Qonctrol, Dreaming_Galaxy, BFladderbean, Feng_Bl PUTF std2

推爆:



@嘟了个嘟噜

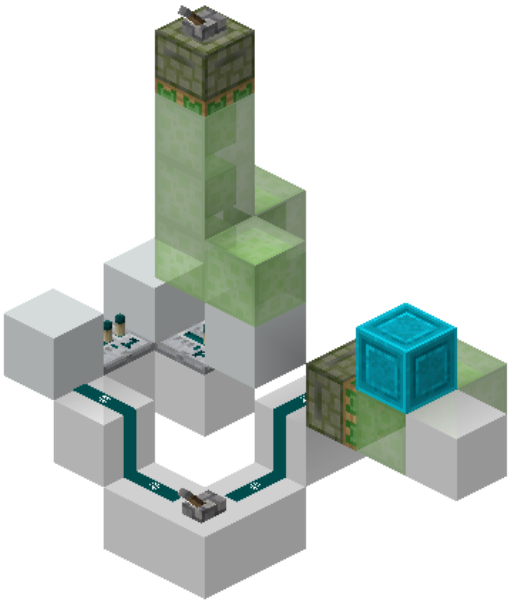
3 布线 —— 模式切换与限速器

其实在设计好时序之后，你只需要尽可能严格地按照你设计的时序激活各组活塞就可以了，这里我们想特别讨论**模式切换**的相关布线

3.1 金合欢时序切换

如果你有认真观察，一定可以发现：**除了金合欢，对于其他所有树种，我们都可以使两个侧面蜜绿墙（*后简称侧墙*）同时推出，从而缩短处理周期**，更进一步，我们可以让主推也同时推出，进一步缩短处理周期。这时，就产生了不同的运行时序

处理方案并不算复杂。你可以注意到，从常规的运行时序切换到金合欢运行时序，**实质上等同于为两个侧墙增加延迟**，因此我们只需要另外连一条延迟更大的线激活侧墙，并设计一个在两条线路之间切换的装置，就可以了

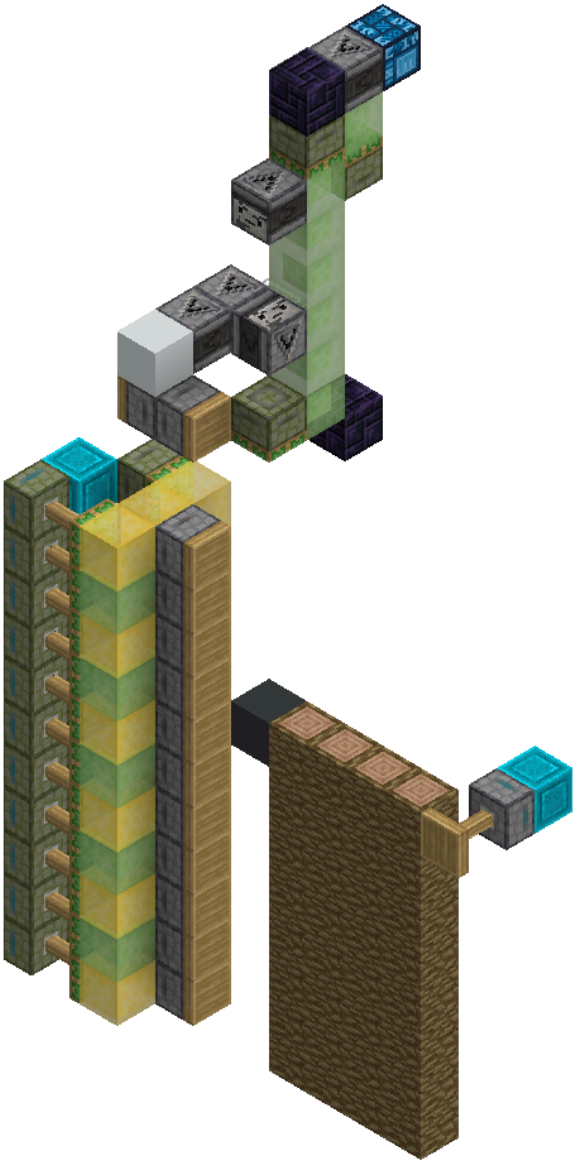


3.2 自动橡树限高

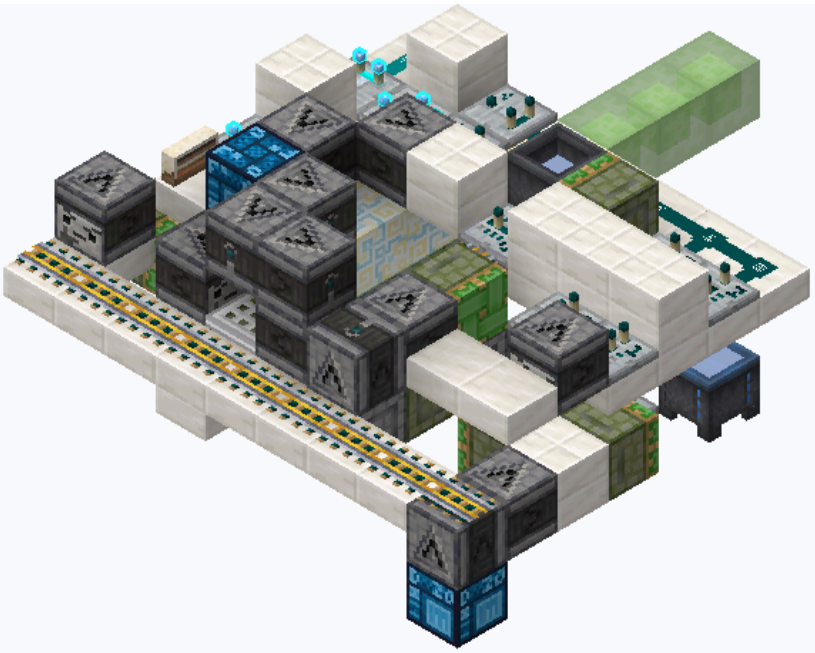
很多人会说手放黑曜石太麻烦了。这当然是有解决方案的：自动限高

由于不同树场自动限高的方式有很大差别，这里只讲解我们举例的架构使用的自动限高

可以发现，我们只需要将一个**不能被主推推动的方块**放到对应的位置，就可以达成限高的效果，但为了通过某种方式（一般是飞行器）将这个方块送下去，我们只能使用可推动的方块。解决方案是，在主推的推动方向上填满原木，在重组那边塞入一个**激活的活塞**从而阻碍主推方向上这个方块的移动。



当然也有一些其它方案，在这里举个例子：
切史莱姆棒子的：

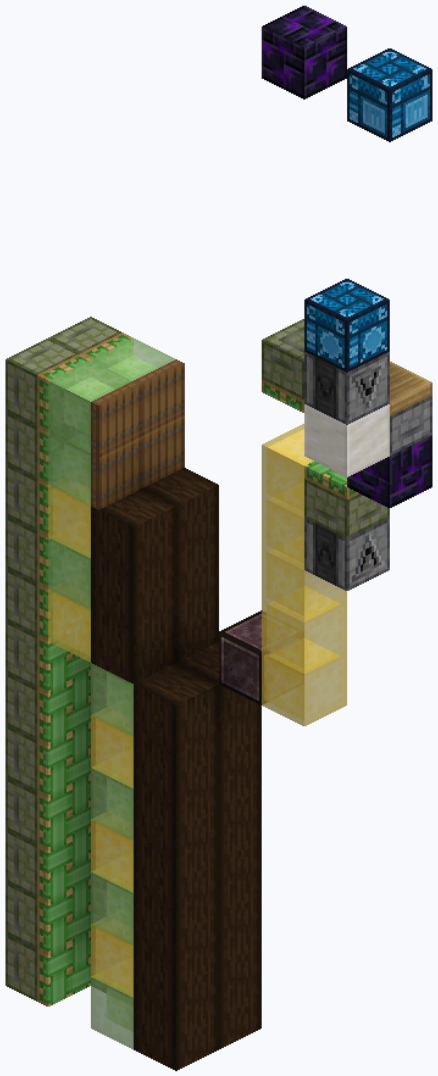


*@Jellie_*

飞陶瓦的：

@XJH_Jorhai

切墙主推飞飞行器下去直接塞的：



@Feng_Bl

3.3 * 丛林与金合欢增高（1.14 及以下内容）

在 1.14 及以下版本中，丛林与金合欢的生长检测为**树干 3x3，树冠 5x5**，因此我们可以在某个高度以下（*一般是泥土以上 5 格*），向 5x5 以内塞入一些方块，从而使低于一定高度的丛林和金合欢被**强制增高**到一定高度，从而提升这两种树的效率

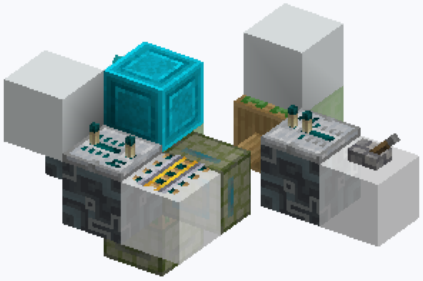
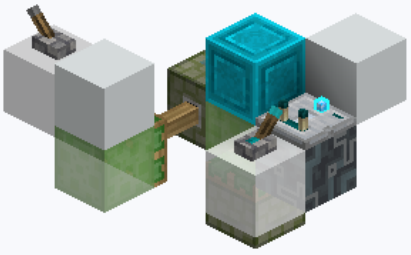
> *由于增高的机制事实上是禁止低于一定高度的丛林或金合欢长出，所以它会增加骨粉消耗。另外，即便在 1.20.1 中，深色橡木也保留了相同的生长要求（以西北角树苗为中心），因此我们依然可以 “增高” 深色橡木*

3.4 限速器

其实到这里还有个问题，因为树有可能在我们的架构完全复位之前就长出来，然后触发检测结构，这样会把我们的树场搞坏，所以我们需要给检测结构添加限速器，同时，限速器的工作时间也要依据每个模式树场其他部分的运行时序（复位时间）来确定

对于 qc 检测，我们可以直接充能回推活塞，或者在检测后推出的结构上粘住些什么，让检测结构在某个时刻之前无法推出，比如 PTHSUTF 的方案

考虑到 qc 检测的限速很难模块化，你自己做一个比什么都强（

> *对于比较器检测，我们可以采用类似的方案，即在某个节点之前切断信号输出*
> *对于推动上限检测，我们可以切断检测结构与泥土的连接*
>
> 
>
> 
>
> *对于 bud 检测，如果你要保证稳定，你必须采用 4gt 复位的 bud，然后在信号输出端做延迟。如果采用 6gt 复位的 bud，就有可能出现提前在 4gt 长树无法触发的情况，~~比如莱逼星河早年的凡台无粉完美时序橡树模式和 TT 的 6gt 丛林树场就是这么炸的~~*

至此，我们就完成了一台相对简单的全树种树场。

这里是我们做的例子！

---

slug: sapling-growth-mechanics
title: 树苗生长机制：树形与催熟
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- Huang He
- SDU-Little-River
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 12
  is-advanced: true
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

1 树苗催熟

树苗生长有两个阶段 (蘑菇 / 下届菌),随机刻和骨粉 (45% 概率) 让树苗增加一个阶段

第二阶段的树苗再次尝试生长时就会尝试生成树

> *注意树苗最开始就已经在第一阶段, 所以最少只需要两个骨粉即可催熟*

不同骨粉数量的催熟成功率如下图

骨粉数量	催熟成功率		骨粉数量	催熟成功率
1	0.000%		11	98.611%
2	20.250%		12	99.174%
3	42.525%		13	99.513%
4	60.902%		14	99.714%
5	74.378%		15	99.832%
6	83.643%		16	99.901%
7	89.758%		17	99.943%
8	93.682%		18	99.967%
9	96.148%		19	99.981%
10	97.677%		20	99.989%

对于蘑菇/下届菌, 使用骨粉有 40% 概率尝试生成巨型蘑菇/巨型下届菌

2 树的生成

所有树的生成都可以分为几个阶段, 其中准备阶段几乎一样, 其余阶段将在各自生成过程中解释

- 准备阶段:
  - 计算树的高度 (或者说树干的长度):基础高度 + (0 到随机高度 1 的随机数) + (0 到随机高度 2 的随机数)
  - 如果有树根生成, 那么将树干起点向上移动
  - 检查空间与高度限制
    1. 确保树的最低点和最高点都在世界的有效高度范围内
    2. 从树干起点向上直到树干顶部往上一层，一层层地检查是否有足够的空间容纳树。有障碍物则生成失败
       - 仅限奇异橡树：如果遇到障碍物, 返回障碍物的 y-2 作为最高可生长的高度, 尝试剪裁部分高度 (最多为 4) 然后生成, 裁剪后仍然不能生成则本次生成失败
- 树根生成 (目前仅红树有):生成树根
- 树干生成: 生成主干和分支以及对应的树叶附着点
- 树叶生成: 在每个树叶附着点按一定的规则生成树叶
- 装饰: 生成藤蔓, 蜂巢等等装饰物

	基础高度	随机高度 1	随机高度 2
橡树	4	2	0
白桦	5	2	0
云杉	5	2	1
丛林	8	4	0
金合欢	5	2	2
樱花树	7	1	0
杜鹃	4	2	0
红树	2	1	4
高红树	4	1	9
深色橡树	6	2	1
巨型丛林	10	2	19
巨型云杉	13	2	14

2.1 橡树, 白桦树, 丛林树, 云杉树, 丛林树的生成

- 检查空间与高度限制 除了云杉木检测 5×5 范围以外, 其余的都是每层检测 3×3,
- 树干生成: 这几种树都使用了`StraightTrunkPlacer`这个树干生成器, 会垂直放置相应高度的原木
- 树叶生成:

2.2 樱花树的生成

2.3 金合欢树的生成

2.4 红树的生成

- 准备阶段: 将树干生成起点向上移动 1-3 格 (高红树 3 - 7 格)
- 树根生成
  1. 检查从树苗生长位置到树干底部下方一格能否满足树根放置条件 (空气或者有`REPLACEABLE_BY_TREES`或`MANGROVE_ROOTS_CAN_GROW_THROUGH`标签)
  2. 以树干生成起点往外一格为起点开始生成树根 (四个方向, 顺序为北东南西)

- 树干生成
  1. 依次向上垂直放置原木直到达到高度 树干顶端为树叶附着点
  2. 每次成功放置非顶端原木 (如果目标位置不是空气或者有`REPLACEABLE_BY_TREES`标签则放置失败), 有 50% 概率生成一个分支 (分支长度 1-4, 高红树 1-6)
  3. 对于每个分支, 斜着 45 度放置原木, 分支上的每一个原木以及分支末端上下方一个方块为树叶附着点
- 树叶生成
  1. 对于每个树叶附着点重复 70 次以下内容
  2. 从附着点开始, 将 x 和 z 坐标 + (两个 0-2 的随机数相减)
  3. y 坐标 + (两个 0-1 随机数相减)
  4. 相当于 x 和 z 在距离附着点 2 格内, y 距离 1 格内随机
  5. 在得到的坐标处尝试放置树叶
- 装饰
- 蜂巢装饰 (红树有 1% 的概率触发蜂巢装饰)
  1. 如果树有树叶，选择最低树叶下方一格的 y 值用于生成蜂巢, 如果没有树叶, 最低树干的 y + 1 + (0 到 2 的随机数) 用于生成蜂巢
  2. 筛选出本次生成中所有位于目标高度的原木方块
  3. 获取这些原木水平相邻的所有位置（不包括北方, 因为蜂巢默认朝南）作为候选位置
  4. 将这些候选位置随机打乱
  5. 找一个满足以下两个条件的位置放置蜂巢 (~~没有就不放~~)
     - 该位置必须是空气
     - 该位置的南方必须是空气
  6. 往蜂巢中放入 2-3 只蜜蜂
- 苔藓地毯装饰
- 红树胎生苗

---

slug: dark-oak-advanced
title: 深色橡木树场
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 9
  is-advanced: true
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

slug: naked-tnt-design
title: 生炸树场
title-en: null
author: tanhHeng
co-authors:

- Arcadi4
- BFladderbeanawa
- jslskd
- RoyanAB
- Twisuki
- Molforte
  date: '2024-12-28T18:08:00+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 10
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

APPENDIX 8

APPENDIX

---

slug: glossary
title: 术语表
title-en: Glossary
author: hotpad100c
co-authors:

- Arcadi4
- BFladderbeanawa
  date: '2025-08-27T13:06:47+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 2
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

---

1 概述

本章旨在详细讲解以帮助读者理解 GTMC 中出现的各种专用词汇。

---

slug: stack
title: 栈的概念
title-en: null
author: Twisuki
co-authors:

- Arcadi4
- BFladderbeanawa
- hotpad100c
- Molforte
  date: '2025-01-21T20:09:13+08:00'
  lastmod: '2026-04-05T22:16:03+08:00'
  index: 1
  is-advanced: false
  banner:
  src: img/banner.png
  alt: Graduate Text in Minecraft

本章将介绍栈与调用栈的概念，这将帮助读者更好地理解方块更新。

1 理解栈

> *栈（Stack）是一种 **先进后出（FILO, First In Last Out）** 的数据结构，它的特点是：元素只能在表的一端被操作。*

1.1 先进后出

想象一个 **薯片桶**：

- 薯片只能从桶口放入，也只能从桶口拿出。
- 因此，**最先放入桶底的那片薯片，将会是最后才能拿出来的**。

这就是 “先进后出” 的概念。

1.2 栈

栈即是一个薯片桶。
而在数据结构的术语里：

- “往桶里放薯片” 对应 **压入栈（Push）**；
- “从桶口取薯片” 对应 **弹出栈（Pop）**。

此外：

- **桶口** 就相当于 **栈顶（Top）**
- **桶底** 则相当于 **栈底（Bottom）**

这便是栈的概念。

2 调用栈的概念

> ***调用栈（Call Stack）** 是程序在运行时管理函数调用的一种机制，它依赖栈的 “先进后出” 规则来保证函数能够正确返回。*

3 调用栈

想象你打电话问问题：

1. 你打电话给朋友 A，询问一个问题。
2. A 不知道答案，于是他打电话询问他的朋友 B。
3. B 也不清楚，再去询问问朋友 C。

这次，C 找到了答案，他先告诉 B；
B 再将答案告诉 A；
最后 A 将答案告诉你。

这里的顺序就是 **调用栈的行为**：

- 每一次电话都向 “调用栈” 中压入一个新的 “调用任务”。
- C 寻找答案时，**栈顶的任务**就是 “C 寻找答案”。
- C 找到答案后，**最后被调用的任务最先完成**，然后逐层返回给调用者（B → A → 你）。
- 而你最初的电话，是最先压入调用栈的任务，因此它位于**栈底**。

而在程序中：

- “打电话” 相当于 **调用函数**；
- “得到答案并回传” 相当于 **函数返回结果**。

这样，程序便可以在函数嵌套调用时，确保每个函数都能完成并正确返回到调用点。

CODE OF CONDUCT

To ensure the efficient operation and high content quality of the GraduateTextsInTechnicalMC (GTMC) community, and to uphold our core focus on educational and practical value, we have established this Code of Conduct. All participants must strictly adhere to these guidelines when contributing content, conducting reviews, and interacting within the community.

1 EXPECTATIONS AND STANDARDS

In our community collaboration and content contributions, we expect and require the following behaviors:

1. Expressions in discussions must be clear and unambiguous. For unconfirmed technical phenomena, please initiate discussions and verifications in the Features section; do not include them in formal articles until they are verified.
2. Community discussions should remain focused on technical topics. We permit and actively encourage rational academic and technical debates.
3. Always respect the original work of others. Prominently and clearly cite the original authors when referencing their content.

2 REVIEW AND QUALITY ASSURANCE

The following rules apply to submission reviews and quality control:

1. Factual errors must be strictly avoided. Fabricating unverified game mechanics or making severe logical errors in derivations will result in harsh point deductions during contribution assessments. Minor formatting, layout, or soft guideline errors will only result in a reminder to revise.
2. Academic fraud and copyright infringement are strictly prohibited. Fabricating unverified game mechanics, plagiarism, or unauthorized copying of others' content is strictly forbidden.
3. We strictly enforce a recusal policy. Reviewers are forbidden from approving content they have submitted or primarily contributed to. Such content must be cross-reviewed by other members in the reviewer pool to ensure fairness.
4. Maintain an objective and professional tone. Writing should be restrained, prioritizing neutral phrasing such as "results indicate" or "experiments show."
5. Avoid using subjective vocabulary such as "I," "we," "feels like," or "probably." Internet memes, cutesy/colloquial particles, and overly emotional expressions are strictly prohibited. *(Exceptions apply for experience-based or personal-driven content.)*
6. All abbreviations must be spelled out and explained upon their first appearance. Terminology must remain consistent throughout the entire document without arbitrary switching.

3 ENFORCEMENT AND APPEALS

For behaviors that violate this Code of Conduct, the community management team will take corresponding disciplinary actions based on the severity of the offense:

1. For minor formatting or tone violations (Level 1), a simple reminder will be issued. Contributors will be asked to revise the content (or assisted in doing so) to meet the guidelines, with no point deductions applied.
2. For spamming or unauthorized copying (Level 2), the violating content will be deleted immediately. The involved individuals will face a short-term ban, and corresponding contribution points will be deducted based on the specific circumstances.
3. For malicious fabrication of mechanics or personal attacks (Level 3), the management team will treat this as the most severe violation. The incident will be publicly announced site-wide and the offending individuals will be permanently banned.

If a contributor disagrees with a reviewer's rejection or a penalty for factual errors, core permanent members of the management team will intervene to make the final ruling and arbitration.

GTMC 社区行为准则 (CODE OF CONDUCT)

为保障 GraduateTextsInTechnicalMC (GTMC) 社区的高效运转与内容质量，基于本社区教学性与实用性的核心定位，特制定本行为准则。所有参与者在进行内容贡献、评审与社区互动时，均须严格遵守以下规范。

1 期望与行为规范

我们在社区协作与内容贡献中，倡导并要求以下行为：

1. 正是讨论中的表达应当清晰明确，杜绝歧义。对于尚未确定的技术现象，请在 Features 板块中发起探讨与查证，确认前不要写入正式文章。
2. 社区内的探讨应该聚焦技术相关主题。我们允许并鼓励理性的学术与技术论辩。
3. 尊重他人的原创成果。显著、清晰地标注引用内容的原作者信息。

2 审核与质量保证

以下是关于投稿审核与质量控制的具体规定：

1. 事实性错误需要被极力避免。编造未经证实的机制或出现严重的逻辑推导错误，将在贡献考核中面临严厉的扣分处罚。一般的排版、格式或软性规范错误则以提醒修改为主。
2. 严厉禁止学术造假与侵权；严禁编造未经证实的游戏机制；严禁洗稿或未经授权擅自搬运他人内容。
3. 严格遵守利益回避原则。文章编辑严禁审批自己参与提交或作为主要贡献者的内容，必须交由池内其他成员进行交叉评审，以确保公正。
4. 保持学术中立和语言的专业性。行文需克制，优先采用 “结果显示”、“实验表明” 等中性、客观的表述方式。
5. 避免主观性语言。行文中请避免使用 “我/我们/感觉是/大概” 等主观词汇；严禁出现网络梗、语气词（如 “呀/捏/呜/了呢”）或任何过度丰富的情感表达。*经验性 / 个人主导的内容除外*。
6. 规范的使用术语。所有缩写在首次出现时须注明全称并加以解释，且在全文中保持术语的统一，避免交替切换。

3 违规处理与申诉流程

对于违反本准则的行为，编辑组将根据严重程度采取相应等级的处罚措施：

1. 一级违规（排版 / 语气违规）：仅予以提醒。要求或着者协助修改以符合规范，且不计入扣分。
2. 二级违规（灌水 / 未经授权搬运）：违规内容将被直接删除，并对涉事人员处以短期封禁，同时根据具体情况扣除相应的贡献分。
3. 三级违规（恶意造谣机制 / 人身攻击）：属于最严重违规，管理组将全站公示该行为，并对涉事人员予以永久封禁。

争议仲裁：当贡献者对文章编辑的驳回意见、事实性错误的判罚存在异议时，将由管理组的核心常驻人员介入，进行最终的裁定与仲裁。

EDITORIAL BOARD STRUCTURE

1 TERMINOLOGY

- The sole administrative body of GTMC is the Editorial Board.
- Members of the Editorial Board are collectively referred to as "Editors" or "Article Editors."

2 SEAT COMPOSITION

The editorial board operates on a hybrid model that combines a stable core with a rotating exterior.

1. **Permanent Seats**
    - These seats are held by core personnel such as Petris and void.
    - They are responsible for underlying architectural decisions, arbitration of major disputes, and system revisions.
    - Promotions to these seats are currently by invitation only.
    - Note that permanent seats currently only have a promotion path and do not feature a standard exit mechanism.
2. **Rotating Seats**
    - These seats are filled by active editors selected periodically from the candidate pool.
    - These members handle the specific approval processes for all PRs, Submissions, and Features during their active term.
    - Promotions to these seats are by invitation: anyone who has contributed at least one PR may be invited to join the rotation candidate pool. Quantitative conditions may be introduced later if necessary.

3 ROTATION MECHANISM

Each rotation cycle lasts for one month, though this duration can be flexibly adjusted.

- The number of active editors per cycle is tentatively set to three.
- Selection is conducted using a weighted random process.
- Participation is non-mandatory: members can choose whether to participate in the current cycle based on their personal schedules. In principle, participation is not strictly required of every member.
- We adhere to a resting principle to prevent editors from serving consecutive cycles. Members who served in the previous cycle will automatically have their selection priority lowered to the minimum for the current cycle, unless there is an extreme shortage of personnel in the relevant expertise pool.
- We enforce a strict recusal policy where any content involving an active editor must be cross-reviewed by other members of the board.

4 DISPUTE RESOLUTION

- [TBD]
  (When an author disagrees with an editor's rejection or a penalty for factual errors, permanent seat members will intervene to make the final ruling.)

编辑组构成

1 称呼

- GTMC 的唯一管理性团队是编辑组。
- 编辑组成员统一称为 “编辑” 或者 “文章编辑”。

2 席位构成（组织架构）

编辑组采用 “核心稳定 + 外围流动” 的组合形式：

1. 常驻席位 (Permanent Seats)：
   - 成员：如 Petris、虚空等核心人员。
   - 职能：负责底层架构决策、重大争议仲裁及制度修订。
   - 晋升机制：仅邀请制。
   - *注：常驻席位目前仅设晋升路径，不设常规退出机制。*
2. 轮值席位 (Rotating Seats)：
   - 成员：从轮值候选中按周期选拔出的 “当期文章编辑”。
   - 职能：负责该阶段所有的 PR、Submission 及 Feature 的具体审批。
   - 晋升机制：邀请制，只要你至少贡献过一次 PR，就有可能被邀请进入轮值候选池。若有必要则开放量化条件。

3 轮值机制

每一周期为 1 个月，可以弹性增减。

- 每期值班人数暂定为 3 人。
- 采用加权随机的方式选择。
- 非强制性参与：成员可根据个人时间安排选择是否参与当期值班，原则上不强制要求每位成员都必须参与。
- 强制调休原则 (Resting Principle)：尽量避免文章编辑连续周期审核。在上一个周期参与过值班的成员，在当前周期的选拔优先级自动降至最低，除非对应领域人才库出现人手不足的极端情况。
- 回避原则：文章编辑参与的内容，须由组内其他成员进行交叉评审。

4 争议解决机制

- [待补全]
  (当作者对文章编辑的驳回意见或事实性错误判罚有异议时，由常驻席位介入进行最终裁定。)

GTMC ARTICLES ROADMAP

2026/04/12 @Arcadi4

我（@Arcadi4）严格来说并不是 Minecraft 研究的专家。因此，我将从作者编辑的角度出发，仅维持对课程结构和可读性的基本把控，具体内容将由编辑组的各位完成。

1 BETA

以下为 @BFladderbeanawa 的计划原文：

> *《时序》课程大纲*
> *Chapter 1. 更新理论*
> *1.1. 方块更新理论的世界观；*
> *1.2. NC 更新；红石粉、铁轨、音符盒；红石灯；方块充能理论；*
> *1.3. 刻、延迟；刻间时序；中继器、比较器；*
> *1.4. 投掷器、发射器；半连接性与 bud；*
> *1.5. PP 更新；侦测器*
> *Chapter 2. 计划刻*
> *2.1. 刻内时序；刻内时序的观测；*
> *2.2. 游戏阶段理论；计划刻阶段；*
> *2.3. 计划刻阶段的段内时序；*
> *2.4. setBlockState 序列；*
> *2.5. NC、PP 更新序列；*
> *2.6. 中继器、比较器、侦测器与铁轨的更新序列；*
> *Chapter 3. 方块实体*
> *3.1. 方块实体；容器、比较器 2；*
> *3.2. 方块实体阶段；漏斗；*
> *3.3. 方块实体阶段的段内时序；*
> *Chapter 4. 方块事件*
> *4.1. 方块事件阶段；活塞；*
> *4.2. 活塞移动开始的更新序列；*
> *4.3. b36 方块实体的行为；*
> *4.4. 活塞收回的特殊行为；*
> *4.5. 方块事件阶段的段内时序；*
> *4.6. 活塞相关的特性；*
> *Chapter 5. 认识红石元件*
> *旧版：https://shimo.im/docs/m5kvdbbzorfgEW3X*
> *新版：https://shimo.im/docs/25q5XrmpjEtWKz3D*

至少根据我对 Minecraft 的认识，以上内容足以构成一个技术 Minecraft 的合理入门内容核心。但是显然，现有内容并没有遵循以上大纲。但是，这并不代表这些内容应该被重写 —— 实际上现有内容的结构可能是更理想的。因此，首先我们可以先提取以上大纲涵盖的 scope：

1. 刻内时序
2. 方块更新

是的，如果要以最粗粒度的形式总结，只有两部分。方块更新决定一个事件的影响范围，而刻内时序则决定了这些更新发生的时间点和顺序。在核心 scope 不变的前提下，我认为以下是更合理的核心内容结构：

1. 更新理论
2. 刻内时序的具体阶段
3. 具体元件的行为

总之，核心内容的原则是*允许玩家以系统性的方式分析在通常的技术性场景出现的元件行为*，而不是无限制地挖掘游戏的底层机制。因此，例如世界生成和实体 AI 等内容虽然也非常有趣，但并不适合放在核心内容中。世界生成几乎与机器设计无关，仅限非常边缘的 hack 会对其进行利用，大部分关于世界生成知识的应用场景也只是生存中的实用性目的。而对实体 AI 的利用虽然存在但是不够普遍，并且可分析性远低于方块和方块实体，更适合作为深入内容。

从协助读者顺利构建认知上讲，玩家更加能够获取直观经验的更新理论部分是更好的切入点。而刻内时序实际上在通常游玩中能被经验的部分仅限少数边界条件，而且其结果在没有进行代码挖掘辅助理解的前提下极其反直觉。

实际上，现有内容已经基本上按照以上结构进行编排了。区块加载实际上是一个比较边缘的内容。虽然在模拟距离和渲染距离拆分之后，边界情况增加，分析变得更为复杂。但是实际上对于大部分设计，通常只需要考虑抗卸载性，或者在卸载时的损坏风险，并且很多时候，运行中强加载基本已经是默认的了。区块加载虽然较为基础，但是大部分情况具体分析并不必要，即使需要，结论也可以由微时序理论推出。因此，其更多是

在技术生存中的一种实用性知识。关于区块加载的进阶内容，我认为可以拆分融合到世界生成和实体行为。

2 POST BETA

在核心内容完成后，我们将会聚焦更加实操和经验性的内容。例如：

- 树场教程
- 绿萌（粘液块机械）教程（目前需要在不调整内容的前提下重排）
- 储电教程，by OpenST
- Masa Mod
- etc.

以及或许可以增加一些更具时效性的内容：

- 某些快照的特别分析
- 最新版本值得注意的函数级变动
- 新物品 / 新实体的分析

以及更为边缘的进阶内容：

- 世界生成
- 实体 AI，重点关注村民
- 区块加载